Prácticas Profesionalizantes III - 3ro 2da
ISFT 172

Proyecto: Gestorix – Sistema de Control de Inventario para Maxikioscos
3°2°

Integrantes:
Cristian Janza.
Emiliano Rodriguez Lemiña.
Emanuel Rivas.
Tobias Calafatello.
Marcelo Ferreyra.

Índice

Palabras Clave, Definiciones, Acrónimos, y Abreviaciones.....6
3.1 Descripción de la Propuesta.....11
3.1.1 Alcance.....12
3.1.2 Límites.....13
3.1.4 Presentación del Negocio.....13
3.1.4.1 Propósito del Negocio.....14
3.1.4.2 Propósito de Gestorix.....14

3.1.4.3 Alcance del Negocio....14
3.1.5 Idea de Negocio....14
3.1.5.1 Misión y Visión....15
Misión....15
Visión....15
3.1.5.2 Análisis Estratégico....16
3.1.5.3 Análisis de mercado....16
3.1.5.4 Segmentación del mercado.... 18
3.1.5.5 FODA....20
Fortalezas....20
Oportunidades....20
Debilidades.... 20
Amenazas....21
3.1.5.6 Análisis de factibilidad....21
Factibilidad técnica....21
Factibilidad económica....21
Factibilidad operativa....22
3.1.6 Descripción general de Stakeholders y Usuarios....22
3.1.6.1 Resumen de Stakeholders....22
Usuarios clave....23
Resumen de Usuarios....23
3.1.6.2 Entorno de Usuario....24
3.1.6.3 Perfil de los Stakeholders....25
3.1.6.3.1 Dueño.... 25 **3.1.6.3.2 Empleado**.... 26 **3.1.6.3.3 Equipo de desarrollo**....26 **3.1.6.3.4 Cliente del Kiosco**.... 27
3.2 Descripción Global del Producto....28
3.2.1 Perspectiva del Producto.... 28
3.2.2 Resumen de Características.... 28
3.2.3 Suposiciones y Dependencias....30
3.3 Descripción Detallada del Producto....32
3.3.1 Módulo de Gestión de Usuarios....32

3.3.2 Módulo de Productos.....33
3.3.3 Módulo de Ventas.....34
3.3.4 Módulo de Compras.....34
3.3.5 Módulo de Inventario..... 35
3.3.6 Módulo de Reportes y Balances..... 36
3.3.7 Módulo de Autenticación y Seguridad..... 36
3.4 Restricciones.....37
3.4.1 Otros Requisitos del Producto.....38
3.4.2 Estándares Aplicables..... 39
3.4.3 Requisitos de Sistema..... 39
3.4.4 Requisitos de Desempeño.....40
3.4.5 Requisitos de Entorno..... 41
3.5 Casos de Uso.....42
Casos de Uso de Negocio..... 42
CU01 - Registrar Producto.....42
CU02 - Registrar Venta..... 46
CU03 - Carga Empleado..... 50
CU04 - Crear Rol.....54
CU05 - Control de Caja Diaria.....58
CU06 - Registrar Proveedor.....62
CU07 - Devolución..... 66
CU08 - Buscar Artículo.....70
CU09 - Productos Próximos a Vencer.....74
CU10 - Modificar Alerta Stock Mínimo.....77
CU11 - Modificar / dar de baja producto..... 81
CU12 - Cargar stock a artículo existente.....86
CU13 - Gestionar rubros.....89
CU14 - Gestionar marcas..... 94
CU15 - Importar artículos desde Excel..... 99
CU16 - Exportar artículos a Excel.....104
CU17 - Consultar historial de ventas..... 109

CU18 - Consultar historial de devoluciones....113
CU19 - Crear promoción....117
CU20 - Dar de baja promoción....121
CU21 - Registrar compra a proveedor.... 125
CU22 - Consultar / anular compra....129
CU23 - Recepción de mercadería....133
CU24 - Modificar / dar de baja proveedor....137
CU25 - Registrar pago a proveedor (cuenta corriente).... 142
CU26 - Consultar listado de precios.... 146
CU27 - Consultar listado de reposición....150
CU28 - Generar informe de ventas....154
CU29 - Generar balance de ingresos y egresos....158
CU30 - Consultar estado e historial de caja....162
CU31 - Abrir caja....166
CU32 - Registrar ingreso manual de caja....171
CU33 - Registrar egreso manual de caja.... 175
CU34 - Consultar alertas....179
CU35 - Anular movimiento de caja....182
Casos de Uso de Seguridad....186
CUS01 - Cambiar Idioma.... 186
CUS02 - Consultar Bitácora....190
CUS03 - Realizar Backup....194
CUS04 - Gestión de Dígitos Verificadores....198
CUS05 - Gestión de Encriptado....203
3.5.9 Especificaciones Adicionales.... 207
3.5.10 Mapa de Navegación....209
3.6 Prototipos de Interfaces del Usuario....210
3.7 Diagrama Global de Clases Del Sistema.... 217
Capa Datos....218
Capa Entidades....218
Capa Lógica.... 219
Capa Vista.... 219

3.9 DER del Sistema.....220

4.0 Estructura principal del software con interfaces.....220

4.1 Estructura de capas del Software.....220

4.2 Estructura principal del Login del software con interfaces..... 223

4.3 Estructura principal del menú principal del software con interfaces..... 226

4.4 Estructura principal de un caso de uso de negocio del software con interfaces y código: ABM Marca.....226

Capa Vista // frmCategoriaMarca..... 228

Capa Lógica // ClsGestorProductoLogica.....237

Capa Entidades // ClsGestorEntidadProducto.....238

Capa Datos // ClsGestorProductoDatos..... 238

4.5 Estructura principal de un caso de uso de negocio del software con interfaces y código: Venta.....241

Capa Vista // frmmenuPres.....243

Capa Vista // frmInformeCobro..... 274

Capa Lógica // ClsGestorProductoLogica.....282

Capa Datos // ClsGestorVentaDatos.....288

4.6 Estructura principal de un caso de uso de negocio del software con interfaces y código: Importar y Exportar Excel.....294

Capa Vista // frmExportarArchivos.....295

Capa Vista // frmImportarArticulos.....300

4.7 Estructura principal de un caso de uso de negocio del software con interfaces y código: Compras.....317

Capa Vista // frmCompras..... 319

Capa Vista // frmNuevaCompra.....326

Capa Vista // frmDetalleCompra.....341

Capa Lógica // ClsCompraLogica.....343

Capa Datos // ClsCompraDatos.....345

5.0 Roles y Permisos..... 350

Roles Base..... 354

Administrador.....354

Empleado General.....354

Empleado Compras..... 354

Empleado Ventas..... 354

Palabras Clave, Definiciones, Acrónimos, y Abreviaciones

Término	Descripción
ABM	Alta, Baja y Modificación. Operaciones básicas de gestión de datos para crear, eliminar o actualizar registros en un sistema.
Arquitectura en capas	Patrón de diseño estructural que organiza el código en niveles independientes con responsabilidades delimitadas (ej. Presentación, Negocio, Datos).
ARS	Peso Argentino. Código ISO para la moneda oficial de la República Argentina.
Backup	Copia de seguridad de datos que permite restaurar la información en caso de pérdidas o fallos.
Bitácora	Registro cronológico detallado de los eventos, acciones o errores ocurridos dentro de un sistema.
bool	Tipo de dato booleano que solo puede almacenar dos valores lógicos: True o False.
Buenas prácticas OWASP	Guías y estándares de la Open Web Application Security Project para identificar y mitigar vulnerabilidades críticas en el software.
btn / Boton	Control de interfaz gráfica de usuario que desencadena una acción al ser presionado.
C#	Lenguaje de programación orientado a objetos desarrollado por Microsoft para la plataforma .NET.
cls / clase	Plantilla o modelo en programación orientada a objetos que define los atributos y métodos de un tipo de objeto.
cmb / Combo Box	Control de interfaz gráfica que despliega una lista desplegable para seleccionar una o más opciones.
Control de Caja	Proceso administrativo y contable de verificación de las entradas y salidas de dinero en efectivo u otros medios de pago.
CU / Caso de Uso	Descripción de una secuencia de interacciones entre un actor y el sistema para alcanzar un objetivo específico.
CUIL	Código Único de Identificación Laboral. Clave utilizada en Argentina para identificar a las personas físicas en la seguridad social y el sistema tributario.

CUS / Caso de Uso de Seguridad	Caso de uso enfocado en los requisitos, restricciones o flujos de seguridad del sistema.
Cta. Cte. (Cuenta Corriente)	Si el proveedor te deja la mercadería y se la pagas la semana que viene.

.bak	Archivo que contiene una Base de datos de SQL Server.
DataSet / DataTable	Objeto en memoria que representa una estructura de datos tabular compuesta por filas y columnas.
dateTime	Tipo de dato o estructura utilizado para almacenar información sobre una fecha y una hora simultáneamente.
dgv / datagridview	Control de interfaz de usuario diseñado para mostrar y editar datos representados en forma de tabla.
Digito Verificador	Mecanismo de control de integridad de datos consistente en uno o más dígitos derivados matemáticamente de un conjunto numérico.
DNI	Documento Nacional de Identidad. Documento legal para la identificación de los ciudadanos en Argentina y otros países.
dtp / dataTimePicker	Control de interfaz gráfica que permite al usuario seleccionar una fecha y una hora de un calendario desplegable.
DVH / Digito Verificador Horizontal	Algoritmo de integridad que calcula un código validador sumando o procesando los valores de una sola fila o registro.
DVV / Digito Verificador Vertical	Algoritmo de integridad que calcula un código validador procesando los valores de una columna completa entre múltiples registros de una tabla.
Egreso	Salida o flujo negativo de dinero o recursos de una entidad o sistema contable.
Encriptación	Proceso de transformación de datos de texto plano a un formato cifrado e ilegible mediante algoritmos criptográficos para garantizar la confidencialidad.
Exclude	Relación entre casos de uso en la que un flujo secundario u opcional no es indispensable para la ejecución del caso de uso base.
Faltante	Productos anotados como presentes pero que finalmente no están físicamente.
False	Valor booleano que representa la condición lógica de falso.

float	Tipo de dato numérico que representa números reales con coma flotante de precisión simple.
Flujo Alternativo	Secuencia de pasos en un caso de uso que describe desviaciones, excepciones o caminos secundarios respecto al proceso habitual.
Flujo Normal	Secuencia principal de pasos dentro de un caso de uso que se ejecuta exitosamente sin errores ni desviaciones.
Franco	Día de descanso del empleado.
frm / Formulario	Ventana o contenedor de interfaz gráfica sobre el cual se disponen otros controles e interactúa el usuario.
Fondo	Plata dejada al iniciar al día con billetes chicos.

Ganancia	Diferencia entre el costo de compra y el precio de venta de los productos.
IEEE 830	Estándar internacional que especifica las mejores prácticas para la redacción y organización de las Especificaciones de Requisitos de Software (SRS).
Include	Relación entre casos de uso donde un caso de uso base incorpora obligatoriamente el comportamiento de otro caso de uso secundario.
Ingreso	Entrada o flujo positivo de dinero o recursos en una entidad o sistema contable.
Insert	Sentencia de lenguaje SQL utilizada para añadir uno o varios registros nuevos dentro de una tabla de base de datos.
int	Tipo de dato entero que representa números sin parte decimal.
Inventario	Conjunto total de productos almacenados en el kiosco.
ISO 9001	Norma internacional sobre Sistemas de Gestión de Calidad (SGC) enfocada en la mejora continua y satisfacción del cliente.
ISO/IEC 12207	Estándar que establece el marco de trabajo general para los procesos del ciclo de vida del software.
ISO/IEC 25000	Familia de normas (SQuaRE) dedicada a la evaluación y especificación de la calidad del producto de software y datos.

ISO/IEC 27001	Estándar internacional para la gestión de la seguridad de la información mediante un SGSI (Sistema de Gestión de la Seguridad de la Información).
IVA (Impuesto al Valor Agregado)	Impuesto indirecto sobre el consumo derivado de la compra de bienes o de servicios profesionales.
lbl / label	Control de interfaz gráfica pasivo que muestra texto no editable por el usuario en pantalla.
list	Colección ordenada de elementos a los que se puede acceder por índice o recorrer secuencialmente.
Lote	Conjunto agrupado de artículos, datos o transacciones que se procesan o producen como una unidad individual.
lnk / LinkLabel	Control de interfaz gráfica similar a un Label que se comporta como un hipervínculo navegable.
Mayorista	Persona, empresa u organización que vende productos en grandes cantidades a negocios.
MensageBox	Ventana emergente predefinida para mostrar mensajes informativos, de advertencia o confirmación al usuario.
Módulo	Unidad lógica o componente independiente dentro de una aplicación que agrupa funcionalidades específicas.
Monto	Suma final o cantidad total de dinero acumulada en una transacción.

Pack	Se usa para compras cerradas.
Patrón Observer	Patrón de diseño de software de comportamiento donde un objeto (sujeto) notifica automáticamente sus cambios de estado a sus dependientes (observadores).
Pedido	La lista de los productos que se van a comprar al distribuidor.
Post-Condición	Estado o conjunto de garantías que el sistema debe cumplir obligatoriamente al finalizar con éxito un caso de uso.
Pre-Condición	Requisitos o estado previo necesario que debe cumplirse en el sistema antes de que se pueda iniciar la ejecución de un caso de uso.
Precio Unitario	Costo o valor monetario asignado a una sola unidad de un producto o servicio.

PP (Precio público)	El precio sugerido de venta al cliente.
Proveedor	Persona, empresa u organización que suministra productos, materias primas o servicios a otra empresa o negocio.
Razón Social	Nombre legal e identificatorio con el que una empresa o persona jurídica se encuentra registrada formalmente ante el estado.
Reparto	El camión que viene a traer la mercadería.
Retiro	Cuando sacas plata de la caja durante el día.
Rotación de stock	Frecuencia con la que los productos son vendidos y reemplazados.
Rubro	Categoría, sector o actividad comercial a la que pertenece una empresa o negocio.
SHA256	Función hash criptográfica que genera una huella digital única e irreversible de 256 bits para cualquier conjunto de datos.
SKU	Stock Keeping Unit. Código alfanumérico único asignado a un producto para realizar el seguimiento e inventario.
sqlconection	Objeto encargado de establecer y gestionar la comunicación física y la autenticación entre la aplicación y el servidor de SQL Server.
SQL Server	Sistema de gestión de bases de datos relacionales (RDBMS) desarrollado por Microsoft.
Stakeholder	Cualquier persona, grupo u organización que tiene un interés directo o indirecto en el desarrollo o resultado de
Stock	Cantidad de productos disponibles para la venta.
Stock Crítico	Nivel mínimo de unidades de un producto en el inventario, a partir del cual se considera urgente su reposición para evitar quiebres de stock y posibles pérdidas de ventas.
string	Cadena de caracteres o secuencia de texto utilizada como tipo de dato en programación.
Subtotal	Suma de los importes de una venta o factura antes de aplicar impuestos, recargos o descuentos.

txt / textbox	Control de interfaz gráfica de usuario que permite la entrada y edición de texto libre.
Turno	La división del tiempo durante la cual se trabaja. Ej: Mañana/Tarde.
USD	Dólar estadounidense. Código ISO de la moneda oficial de los Estados Unidos.
Vale	Papel que firma el empleado si pide plata adelantada o si saca algún producto del Kiosco.
Vencimiento	Fecha límite de consumo de un producto.
void	Palabra clave en programación que indica que un método o función no retorna ningún valor al finalizar su ejecución.
Vuelto	Cantidad de dinero devuelta al comprador cuando el monto entregado supera el total a pagar.
Windows Forms	Biblioteca de clases de la plataforma .NET para construir interfaces gráficas de usuario en aplicaciones de escritorio para Windows.
.bak	Extensión de archivo estándar utilizada para los archivos de copia de seguridad (backup) de bases de datos como SQL Server.
.NET 8	Versión LTS (soporte a largo plazo) del entorno de ejecución y desarrollo multiplataforma de software desarrollado por Microsoft.
.NET Runtime	Entorno de ejecución que proporciona las herramientas, memoria y servicios necesarios para correr aplicaciones .NET.

3.1 Descripción de la Propuesta

El proyecto consiste en el desarrollo de un sistema informático para la gestión de stock de un maxikiosco, un comercio minorista que maneja una gran variedad de productos con alta rotación diaria, como alimentos, bebidas y artículos de higiene. Actualmente, muchos de estos negocios llevan el control de mercadería de forma manual o con herramientas poco eficientes, lo que genera errores en el inventario, pérdidas por productos vencidos y faltantes de stock.
La propuesta busca informatizar la administración del inventario mediante un software sencillo, intuitivo y accesible para usuarios sin conocimientos técnicos. El sistema permitirá registrar movimientos de mercadería, controlar existencias en tiempo real y generar alertas para facilitar la reposición de productos y reducir pérdidas.
De esta manera, el maxikiosco podrá optimizar su gestión interna, mejorar la organización del stock y tomar decisiones comerciales basadas en información precisa y actualizada.

3.1.1 Alcance

Tareas/Acciones Incluidas en el Desarrollo del Sistema

- Desarrollo de módulo de inicio de sesión con autenticación de usuarios.
- Gestión de roles y permisos de usuario, permitiendo asignar permisos fijos en diferentes roles según se requiera, dando libertad de creación de roles personalizables.
- Registro, modificación, consulta y baja lógica de usuarios (ABM).
- Registro, modificación, consulta y baja de Proveedores, Marcas y Rubros.
- Registro de productos disponibles para la venta.
- Actualización de datos de productos y modificación de precios.
- Registro de compras a proveedores con ingreso automático al stock.
- Registro de ventas con descuento automático de stock.
- Visualización de historial de ventas por fecha o producto.
- Control de inventario en tiempo real.
- Organización del inventario por categorías de productos.
- Configuración de stock ideal por producto.
- Configuración de punto de reposición.
- Configuración de stock crítico.
- Generación de alertas visuales por bajo stock.
- Generación de alertas por productos próximos a vencer.
- Envío opcional de notificaciones por correo electrónico.
- Recuperación y cambio de contraseñas de usuarios con envío de email y preguntas de seguridad integradas en el programa.
- Seguimiento básico de vencimientos por fecha de ingreso al sistema utilizando una lógica mixta entre la aproximación base de Primero en Entrar, Primero en Salir y el control manual de los artículos.
- Incorporación de filtros de búsqueda para facilitar la localización de productos, proveedores, compras y ventas dentro del sistema.
- Generación de informes de venta para el control del rendimiento. (Por fecha, rubro, artículo, marca o medio de pago.
- Registro y visualización de un historial completo de movimientos de mercadería, contemplando entradas y salidas de stock.
- Creación y gestión de promociones de productos, permitiendo definir ofertas y condiciones especiales de venta.
- Gestión de cobros y medios de pago, permitiendo cobros mixtos, con múltiples métodos de pago.
- Control de caja diaria.
- Integración con lectores de código de barras externos.

- Gestión de devoluciones o notas de crédito con los respectivos detalles e historial para el control de las mismas.
- Gestión de Backup, respaldo programable y restauración por sistema.
- Gestión de Bitácora, guardando datos de modificaciones críticas que puedan ser realizadas junto a la fecha, detalles y usuario que realizó la acción.
- Importación y Exportacion de Articulos a Excel.
- Ingresos y Egresos manuales de caja pidiendo justificación y notificando al administrador por mail.

3.1.2 Límites

Tareas/Acciones Excluidas del Desarrollo del Sistema

- Reportes contables o financieros avanzados.
- Gestión de clientes o programas de fidelización.
- Trazabilidad física por número de lote de productos.
- Ubicación individual de cada unidad de mercadería.
- Gestión logística de distribución o envíos.
- El sistema funcionará solo en Windows 10 en adelante en computadoras de escritorio.
- No habrá conexión con plataformas de entrega/delivery.
- No multi-sucursal.
- Sin integración automática con terminales de cobro.
- No permite realizar facturas oficiales relacionadas con la AFIP.
- No calcula ni almacena sueldos.
- Sin conexión con balanzas.
- Permite cambiar el idioma de las interfaces, pero esto no cambia el idioma de los datos que están dentro de la Base de Datos.

3.1.4 Presentación del Negocio

A continuación se presenta la información obtenida en el relevamiento realizado sobre el MaxiKiosco “MTK (MiniTiendaKiosco)”, ubicado en AV. Colombres 746 de la localidad de Lomas de Zamora.
Dicho establecimiento se dedica a la venta de productos de consumo cotidiano, tales como Snacks y galletitas, con el fin de satisfacer a los clientes de la zona, conformados en su mayoría por vecinos o gente que se moviliza a diario por la avenida.
Su operativa se basa en la adquisición de mercadería a mayoristas y proveedores y su posterior comercialización al consumidor final. No incluye actividades de producción ni distribución a gran escala.

3.1.4.1 Propósito del Negocio

El propósito de MTK es ofrecer a los clientes productos de consumo cotidiano de manera rápida, accesible y eficiente, satisfaciendo las necesidades inmediatas de los mismos.

Asimismo, el kiosco tiene como objetivo brindar una atención cercana y constante, fomentando la fidelización de clientes habituales y adaptándose a la demanda del entorno en el que se encuentra.

3.1.4.2 Propósito de Gestorix

El propósito del sistema es mejorar la gestión interna del kiosco mediante la digitalización del control de stock y el registro de productos.

El sistema permitirá conocer en todo momento la cantidad disponible de cada producto, reducir errores asociados al manejo manual de la información y facilitar la actualización de datos de manera rápida y precisa.

Asimismo, contribuirá a optimizar la toma de decisiones del negocio, evitando faltantes de mercadería y mejorando la organización general de las operaciones diarias.

3.1.4.3 Alcance del Negocio

El alcance de MTK comprende la adquisición de mercadería a distintos proveedores y mayoristas, la gestión básica de dichos productos y su comercialización a consumidores dentro del local. Asimismo, incluye la atención al cliente, la reposición de productos y el control general de la disponibilidad de stock.
El kiosco interactúa principalmente con dos actores externos: los proveedores, encargados del suministro de mercadería, y los clientes, quienes adquieren los productos ofrecidos. No se contemplan actividades de producción ni distribución a gran escala, centrándose exclusivamente en la venta minorista.

3.1.5 Idea de Negocio

La idea de negocio del maxikiosco se basa en la comercialización de productos de consumo masivo (snacks, galletitas, bebidas, tabaquería, artículos de almacén, perfumería y librería) y prestaciones de servicio (carga de transporte y celular). Todo esto orientado a satisfacer las

necesidades inmediatas de los clientes. El comercio se caracteriza por contar con amplios horarios de atención, adaptándose a la demanda del público y funcionando durante gran parte del día, e incluso en horarios extendidos.

El funcionamiento del kiosco se apoya en dos empleados, quienes se encargan de la atención al cliente, la reposición de mercadería, el control de stock y la verificación de productos. Estas tareas suelen realizarse de manera manual, mediante anotaciones o controles informales, lo que puede generar desorganización, errores en el registro de ventas o compras y dificultades para mantener un control preciso del inventario.

Asimismo, la gestión del negocio incluye la relación con proveedores y mayoristas, la compra de productos, la organización del stock y la implementación de promociones para incentivar las ventas.

3.1.5.1 Misión y Visión

Misión

Brindar a nuestros clientes una atención rápida, cercana y confiable, ofreciendo una amplia variedad de productos de calidad, mientras impulsamos la digitalización de nuestra gestión mediante herramientas tecnológicas que mejoren la organización, el control de ventas y la eficiencia operativa del negocio.

Visión

Expandir su negocio mediante la apertura de una cadena de sucursales, consolidándose como un kiosco moderno y organizado, apoyado en sistemas de gestión eficientes que nos permitan crecer de manera sostenida y ofrecer una mejor experiencia a nuestros clientes.

3.1.5.2 Análisis Estratégico

A partir del relevamiento realizado en distintos comercios ubicados en la zona de influencia del maxikiosco MTK, se identificaron patrones comunes en la forma de operación del rubro.

La mayoría de los negocios relevados presentan una gestión tradicional, con bajo nivel de digitalización, donde el control de stock y ventas no se encuentra sistematizado. Esto evidencia una fuerte dependencia de métodos manuales y de la experiencia de los empleados.

Asimismo, se observa que muchos comercios combinan el kiosco con otros rubros (como almacén o carnicería), ampliando la oferta de productos para competir en el mercado.

En cuanto a la operación diaria, varios locales no cuentan con un flujo constante superior a 100 clientes por día, lo que indica diferencias en el nivel de actividad comercial dentro de la zona.

También se identificó que no todos los negocios poseen horario corrido, lo que puede representar una ventaja frente a nosotros que mantenemos disponibilidad continua.

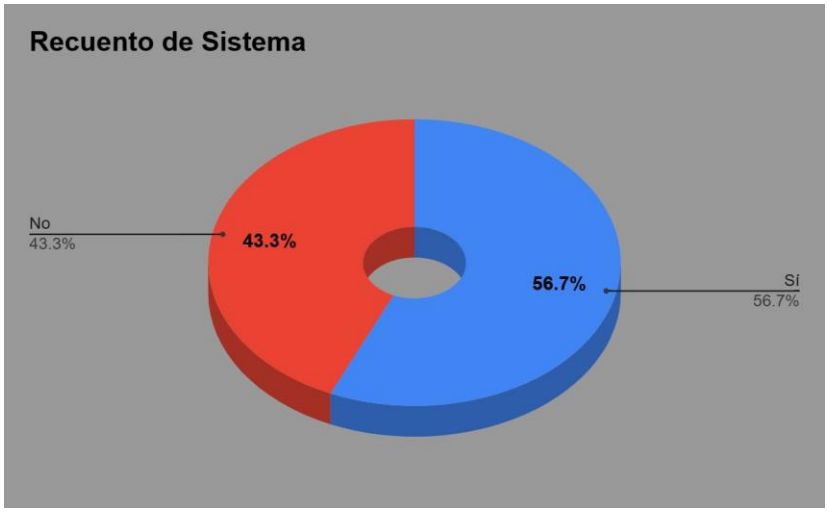
Este contexto refleja un mercado competitivo pero con oportunidades de mejora en la organización y eficiencia operativa.

3.1.5.3 Análisis de mercado

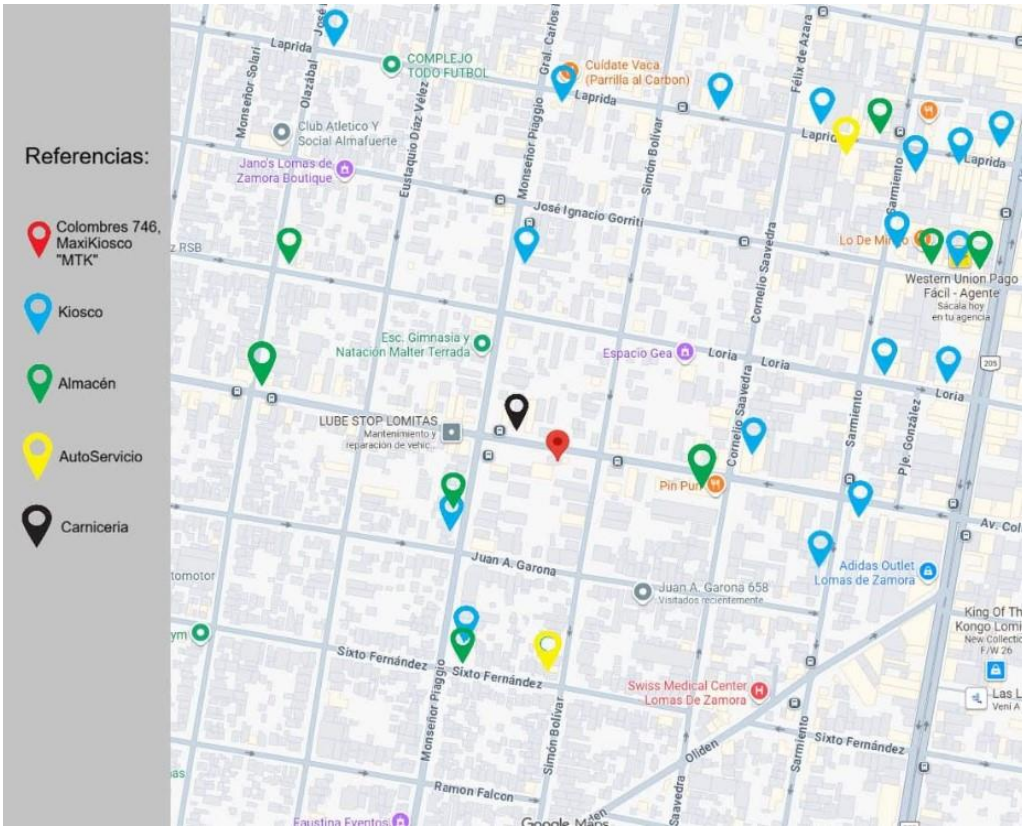
El análisis de los comercios cercanos al maxikiosco MTK permitió identificar aproximadamente 30 locales dentro de un radio de cuatro cuadras, lo que confirma un alto nivel de competencia.

A partir de los datos relevados, se pueden destacar las siguientes características:

- Una parte significativa de los competidores utiliza sistemas de gestión básicos, lo que limita el control del stock y de las ventas.



- La mayoría ofrece variedad de productos, lo que incrementa la competencia directa.
- Solo algunos comercios alcanzan un flujo superior a 100 clientes diarios, lo que evidencia diferencias en el posicionamiento.
- No todos los locales trabajan con horario corrido, lo que puede afectar la disponibilidad para el cliente.
- Ubicación y posicionamiento: se identifican competidores con ubicaciones estratégicas, como cercanía directa a puntos de alto tránsito o zonas comerciales consolidadas. Sin embargo, el maxikiosco MTK se encuentra en una ubicación intermedia con características favorables, ya que está próximo a instituciones escolares y a un edificio residencial de aproximadamente 90 departamentos, lo que representa una fuente constante de potenciales clientes tanto en horarios escolares como en la vida cotidiana de los residentes.



En este contexto, la competencia no solo se da en precios o productos, sino también en aspectos como la organización interna, la disponibilidad horaria y la capacidad de atención.

3.1.5.4 Segmentación del mercado

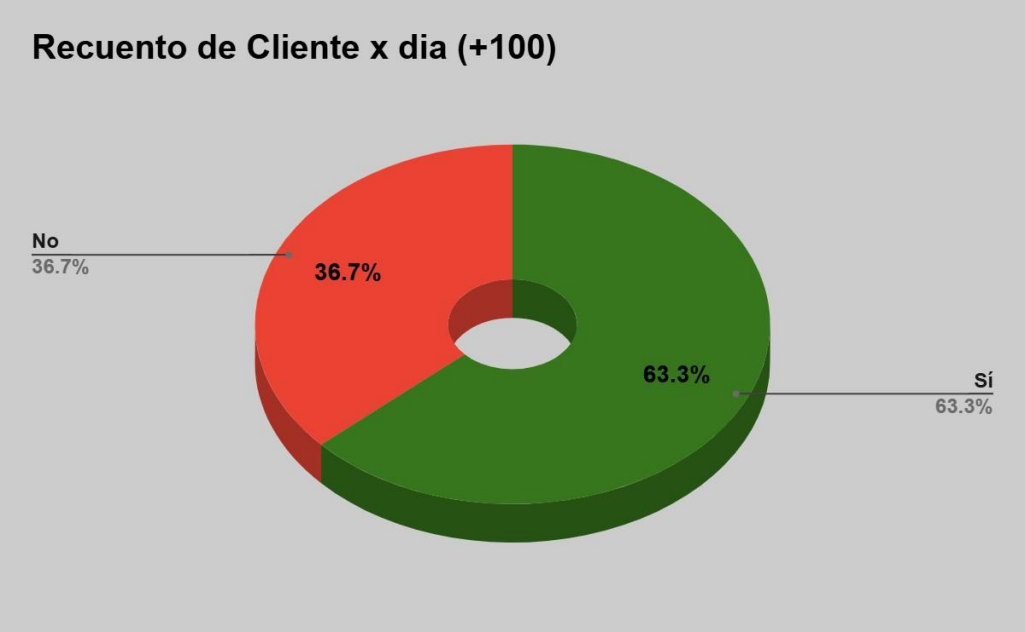
A partir del análisis del entorno y del comportamiento de los comercios relevados, se identifican distintos tipos de clientes:

- Clientes de alta frecuencia: realizan compras diarias o recurrentes, generalmente vecinos de la zona.
- Clientes de paso: personas que circulan por la avenida y realizan compras rápidas.
- Clientes ocasionales: compran de manera esporádica según necesidad.

Dirección	Numeración	Rubro	Sistema	Cliente x dia (+100)	Horario de corrido
Colombres	494	Kiosco	Sí	Sí	Si
Colombres	759	Carniceria/Almacen	Sí	No	Sí
Colombres	617	Carniceria/Almacen	No	No	No
Colombres	999	Carniceria/Almacen	No	No	No
Saavedra	290	Kiosco	Sí	Sí	Sí
Loria	488	Kiosco/Almacen	No	Sí	Sí
Loria	423	Kiosco	Sí	Sí	Sí
Gorriti	401	Almacen/Fiambreria	No	Sí	Sí
Gorriti	417	Kiosco	Sí	Sí	Sí
Gorriti	435	Almacen/Fiambreria	No	No	Sí
Sarmiento	322	Kiosco	Sí	No	Sí
Sarmiento	99	Almacen/Fiambreria	No	No	Sí
Sarmiento	28	Kiosco	Sí	No	Sí
Laprida	180	AutoServicio	Sí	Sí	No
Laprida	417	Kiosco	Sí	Sí	Sí
Laprida	593	Kiosco	No	Sí	Sí
Laprida	440	Kiosco	No	No	Sí
Laprida	519	Almacen/Fiambreria	Sí	Sí	Sí

Dirección	Numeración	Rubro	Sistema	Cliente x dia (+100)	Horario de corrido
Laprida	548	AutoServicio	Sí	Sí	Sí
Colombres	652	Fiambreria/Almacen	No	No	No
Laprida	780	AutoServicio	Sí	Sí	Sí
Laprida	796	Kiosco	No	Sí	Sí
Laprida	999	Kiosco	Sí	Sí	Sí
Loria	997	Almacen	No	No	No
Piaggio	383	Kiosco	No	Sí	Sí
Piaggio	377	Almacen	No	No	No
Piaggio	150	Kiosco	Sí	Sí	No
Piaggio	460	Kiosco	Sí	Sí	Sí
Piaggio	494	Almacen/Kiosco	Sí	Sí	Sí
Bolivar	489	AutoServicio	Sí	Sí	Sí

Además, el flujo de clientes varía según el comercio, observándose que no todos alcanzan altos niveles de demanda diaria, lo que refleja diferencias en la competitividad de cada negocio.



3.1.5.5 FODA

Fortalezas

- Ubicación en zona de alto tránsito.
- Variedad de productos similar o competitiva respecto a otros comercios.
- Posibilidad de captar clientes de comercios con menor organización.

Oportunidades

- Bajo nivel de digitalización en la competencia.
- Posibilidad de diferenciarse mediante mejor organización y control.
- Comercios sin horario corrido generan oportunidades de captar más clientes.

Debilidades

- Gestión manual similar a la de muchos competidores.
- Falta de control preciso del stock.
- Dependencia de procesos informales.

Amenazas

- Alta densidad de competidores (30 locales).
- Comercios con mayor variedad (almacén + kiosco).
- Diferencias en flujo de clientes entre negocios.
- Competidores con mejor ubicación puntual o mayor visibilidad.

3.1.5.6 Análisis de factibilidad

Factibilidad técnica

El maxikiosco MTK cuenta con los recursos tecnológicos básicos necesarios para implementar mejoras en su gestión, ya que dispone de al menos una computadora operativa dentro del local. La misma cuenta con sistema operativo Windows 10, 8 GB de memoria RAM para el correcto funcionamiento del Sistema Gestorix y el software Excel instalado, lo que permite la migración y administración de la información del negocio de manera adecuada.

Por otro lado, el equipo desarrollador dispone de los recursos necesarios para llevar adelante el proyecto, contando con cinco computadoras para el desarrollo, lo que garantiza la viabilidad técnica del mismo.

Factibilidad económica

Desde el punto de vista económico, la propuesta resulta viable debido a que el costo de implementación será accesible, adaptado a la realidad del comercio y experiencia del equipo.

El kiosco se encuentra en una situación estable, con actividad constante y proyección a futuro, lo que permite afrontar la incorporación de mejoras sin comprometer su funcionamiento.

Además, la inversión se justifica por los beneficios esperados, como la reducción de pérdidas, un mejor control del stock y una mayor eficiencia en la gestión.

Equipo:

- 5 personas
- 7 meses
- 25 usd por hora

Costo:

Por día 2hs × 25 usd = USD 50

Por semana 5 días x 50 usd= USD 250

Por mes 4 Semanas x 250 = USD 1000 En 7

meses USD 7000 (Total x 5 empleados)

En pesos (aprox):

- Si 1 USD ≈ 1300 ARS
  >> **9.100.000,00 ARS**

Factibilidad operativa

Las tareas del kiosco son repetitivas y estructuradas (ventas, reposición, control de productos), lo que facilita la incorporación de mejoras en la gestión.

El personal ya se encuentra habituado a la operatoria del negocio, por lo que podrá adaptarse progresivamente a nuevas herramientas sin generar un impacto negativo en la atención al cliente ni en el funcionamiento diario.

Asimismo, el equipo de trabajo está compuesto por empleados que pueden utilizar estas herramientas en sus tareas diarias, lo que facilita la incorporación de nuevas formas de gestión sin requerir conocimientos técnicos avanzados.

3.1.6 Descripción general de Stakeholders y Usuarios

Los principales stakeholders son el dueño del comercio, quien además cumple funciones operativas dentro del local, dos empleados encargados de las tareas diarias, el equipo desarrollador compuesto por cinco integrantes responsables del análisis, diseño e implementación del sistema y todos los clientes del kiosco a los que el uso del sistema les beneficiara con una mayor velocidad de atención.

3.1.6.1 Resumen de Stakeholders

Stakeholder	Rol en el proyecto	Interés principal	Nivel de Acceso
Dueño del kiosco	Cliente y usuario	Mejorar la gestión y control del negocio.	Completo

Empleados	Usuario operativo	Registrar ventas; facilitar el control de stock, reposición y verificación de vencimientos.	Operativo limitado
Equipo desarrollador	Desarrollo del sistema	Diseñar e implementar el sistema.	Especializado
Clientes finales	Consumidor del kiosco	Mejor experiencia de la atención recibida.	Nulo

Usuarios clave

Nombre	Descripción	Responsabilidad
Alberto Suarez	Dueño del Kiosco y trabajador en el mismo.	Define intereses, necesidades, alcances y aporta datos e información.

Resumen de Usuarios

Rol	Descripción	Nombre	StakeHolder
Dueño del kiosco	Responsable de la administración general del sistema.	Alberto Suarez	Dueño

	Gestiona ventas, compras, stock, reportes, configuración y permisos de usuarios.		
Empleado	Responsable de registrar ventas, cobros y movimientos de caja. Consulta stock y controla vencimientos de productos.	Matias	Ventas
Empleada	Responsable de registrar ventas, cobros y movimientos de caja. Consulta stock y controla vencimientos de productos.	Silvina	Ventas

3.1.6.2 Entorno de Usuario

El sistema será utilizado en el maxikiosco MTK, ubicado en Lomas de Zamora, mediante computadoras de escritorio conectadas a una red local interna.

Los usuarios accederán al sistema desde equipos con sistema operativo Windows 10, utilizando una interfaz gráfica intuitiva diseñada para facilitar la operación diaria del negocio. El acceso al sistema requerirá autenticación mediante nombre de usuario y contraseña, permitiendo identificar a cada operador y habilitando las funciones correspondientes según el perfil asignado.

La aplicación trabajará con una base de datos SQL Server conectada en forma local, garantizando rapidez de acceso y disponibilidad de la información durante la jornada comercial.

El entorno de uso se caracteriza por una dinámica de atención simultánea al cliente y una operatoria continua entre las 8:00 y las 22:00 hs, por lo que el sistema deberá responder de manera rápida, simple y confiable, minimizando tiempos de carga y errores operativos.
Los usuarios poseen conocimientos informáticos básicos, principalmente relacionados con el uso cotidiano de sistemas de facturación y herramientas de oficina. Debido a ello, la interfaz deberá priorizar:

- Simplicidad de uso.
- Facilidad de aprendizaje.
- Rapidez en las operaciones.
- Claridad visual.
- Seguridad en el manejo de la información.

No se prevé una capacitación técnica extensa, aunque será necesaria una breve inducción sobre el uso específico del sistema y la gestión de usuarios.

Los reportes generados por el sistema podrán exportarse en formatos digitales como Excel para su almacenamiento.

3.1.6.3 Perfil de los Stakeholders

3.1.6.3.1 Dueño

Representante	Administración general del kiosco MTK
Descripción	Responsable de administrar el sistema y supervisar el funcionamiento general del kiosco.
Tipo	Usuario Clave
Responsabilidad	Gestionar usuarios, controlar permisos, supervisar ventas, compras, stock y reportes generales del sistema.
Criterio de éxito	Que el sistema permita administrar correctamente todas las operaciones del kiosco de forma rápida y segura.

Grado de participación	Alta.
Comentarios	Cualquier modificación importante del sistema deberá ser aprobada por el administrador.

3.1.6.3.2 Empleado

Representante	Personal operativo del kiosco MTK
Descripción	Usuarios encargados de las operaciones diarias del kiosco, incluyendo ventas, atención al cliente, control de stock y compras.
Tipo	Usuario Operativo
Responsabilidad	Registrar ventas, controlar caja, consultar stock, registrar compras, gestionar reposición de productos y verificar vencimientos.
Criterio de éxito	Realizar las operaciones diarias de manera rápida, ordenada y sin errores, manteniendo actualizado el inventario y garantizando una correcta atención al cliente.
Grado de participación	Alta
Comentarios	Utilizan el sistema diariamente durante toda la jornada laboral y dependen directamente de su correcto funcionamiento.

3.1.6.3.3 Equipo de desarrollo

Representante	Grupo de desarrollo del sistema
Descripción	Equipo responsable del análisis, diseño, desarrollo, implementación y mantenimiento del sistema del kiosco.

Tipo	Stakeholder Técnico
Responsabilidad	Desarrollar el sistema según los requerimientos establecidos, corregir errores, implementar mejoras y garantizar el correcto funcionamiento del software.
Criterio de éxito	Entregar un sistema funcional, seguro, estable y adaptado a las necesidades del kiosco.
Grado de participación	Alta
Comentarios	El equipo participa durante todas las etapas del proyecto: análisis, desarrollo, pruebas e implementación.

3.1.6.3.4 Cliente del Kiosco

Representante	Consumidores finales
Descripción	Personas que realizan compras en el kiosco.
Tipo	Usuario Externo
Responsabilidad	Realizar compras y recibir atención rápida y eficiente.
Criterio de éxito	Ser atendido de manera rápida y obtener disponibilidad de productos.
Grado de participación	Baja
Comentarios	No interactúa directamente con el sistema, pero es beneficiado por su funcionamiento.

3.2 Descripción Global del Producto

3.2.1 Perspectiva del Producto

Gestorix es un sistema de escritorio orientado a la gestión operativa de maxikioscos y pequeños comercios minoristas.

Su objetivo principal es digitalizar y optimizar procesos que actualmente se realizan de manera manual, como el control de stock, registro de ventas, compras, usuarios y alertas de productos.

El sistema busca brindar una solución simple, accesible y segura para negocios con bajo nivel de digitalización.

Gestorix es un producto independiente desarrollado para ejecutarse en computadoras de escritorio con sistema operativo Windows.

El sistema interactúa con:

- Base de datos SQL Server para almacenamiento persistente
- Usuarios internos (administrador y empleados)
- Sistema local del kiosco

Se integra conceptualmente con la operatoria habitual del negocio:

- Ingreso de mercadería
- Ventas diarias
- Control de inventario
- Administración de usuarios

3.2.2 Resumen de Características

El sistema contará con los siguientes módulos funcionales:

Característica	Descripción
Gestión de Usuarios	Permite administrar a los empleados junto a sus permisos. ● Alta de usuarios. ● Modificación. ● Baja lógica. ● Asignación de roles.
Gestión de Productos	Permite administrar el stock, flujo, precios, fechas y categorías de la mercadería. ● Registrar productos. ● Editar información. ● Categorizar productos. ● Visualizar stock.
Gestión de Ventas	Permite realizar, registrar y consultar las ventas. ● Registrar ventas. ● Descontar stock automáticamente. ● Registrar medio de pago. ● Consultar historial.
Gestión de Compras	Permite visualizar las compras realizadas, información de proveedores y el ingreso del inventario. ● Registrar compras a proveedores. ● Actualizar el inventario. ● Historial de compras.
Gestión de Inventario	Permite visualizar el stock en tiempo real, junto a las respectivas alertas de vencimiento o bajo stock. ● Visualizar el stock. ● Visualizar alertas por stock crítico.

	● Visualizar alertas por productos próximos a vencer.
Reportes	Permite acceder a los informes de ventas. ● Ganancias diarias. ● Ganancias mensuales. ● Movimientos de stock.
Seguridad	Permite controlar el acceso a información sensible y el acceso a datos de cada empleado. ● Login de usuarios. ● Primer inicio obligatorio. ● Cambio de contraseña. ● Recuperación de contraseña. ● Preguntas de seguridad.

3.2.3 Suposiciones y Dependencias

Para el correcto funcionamiento del sistema se asume que el maxikiosco dispone de al menos una computadora operativa con sistema operativo Windows 10 y acceso a la red local interna.

Se supone además que los usuarios poseen conocimientos básicos de informática para operar el sistema, registrar ventas y consultar información de stock.

El sistema dependerá de una base de datos SQL Server correctamente instalada y configurada, utilizada para el almacenamiento y recuperación de la información comercial.

La aplicación requerirá conexión local entre el sistema y la base de datos para garantizar el acceso a los datos de ventas, productos y stock en tiempo real.

El sistema podrá interactuar con herramientas externas para la exportación de reportes e informes, especialmente mediante archivos compatibles con Microsoft Excel.

Para agilizar la operatoria diaria se considera el uso de un lector de código de barras compatible con el sistema.

La recuperación de credenciales por correo electrónico dependerá de la disponibilidad de conexión a internet y de un servicio de correo configurado correctamente.

El correcto funcionamiento del sistema dependerá de la disponibilidad del suministro eléctrico y del correcto estado del equipamiento informático utilizado en el kiosco.

3.3 Descripción Detallada del Producto

El sistema estará compuesto por diferentes módulos funcionales destinados a optimizar y digitalizar la administración integral del kiosco.

3.3.1 Módulo de Gestión de Usuarios

Este módulo permitirá administrar los usuarios que tendrán acceso al sistema. El administrador podrá crear, modificar, desactivar y controlar usuarios mediante una interfaz simple e intuitiva.

Cada usuario contará con:

- Nombre de usuario.
- Datos Personales.
- Contraseña cifrada mediante SHA256.
- Correo electrónico.
- Rol asignado dentro del sistema.

El sistema manejará distintos niveles de acceso según el rol y los permisos asignados de cada uno.

Funcionamiento del módulo

El administrador registrará un nuevo usuario indicando sus datos principales y el rol correspondiente. El sistema generará una contraseña temporal que deberá ser modificada obligatoriamente en el primer inicio de sesión.

Cada vez que un usuario intente acceder:

1. El sistema validará credenciales.
2. Verificará que el usuario se encuentre activo.
3. Comparará la contraseña cifrada almacenada en la base de datos.
4. Habilitará únicamente los módulos permitidos según el rol.

Controles implementados:

- Validación de campos obligatorios.
- Restricción de acceso según permisos.
- Contraseñas cifradas.

- Recuperación de contraseña mediante correo electrónico.
- Control de usuarios activos/inactivos.

3.3.2 Módulo de Productos

Este módulo permitirá administrar todos los productos comercializados por el kiosco.

Cada producto contará con:

- Código.
- Nombre.
- Categoría.
- Precio.
- Stock disponible.
- Fecha de vencimiento.
- Estado.
- Lote.

Funcionamiento del módulo:

El usuario autorizado podrá registrar nuevos productos, modificar información existente o desactivar productos que ya no se comercialicen.

El sistema permitirá:

- Buscar productos rápidamente.
- Filtrar por categoría.
- Consultar stock disponible.
- Detectar productos próximos a vencer.

Controles implementados:

- Validación de precios positivos.
- Control de stock mínimo.
- Restricción de eliminación física de productos.
- Verificación de fechas de vencimiento.

3.3.3 Módulo de Ventas

Este módulo estará orientado a agilizar el proceso de atención al cliente y registro de ventas.

Funcionamiento del módulo:

El empleado seleccionará productos mediante búsqueda o lectura de código de barras. El sistema calculará automáticamente:

- Subtotal.
- Cantidad de productos.
- Total final.

Al confirmar la venta:

1. Se registrará la operación.
2. Se actualizará automáticamente el stock.
3. Se almacenará el movimiento en la base de datos.
4. Se emitirá un comprobante simple.

Controles implementados:

- Validación de stock disponible.
- Restricción de ventas con stock insuficiente.
- Registro automático de movimientos.
- Cálculo automático de importes.

3.3.4 Módulo de Compras

Permitirá registrar compras realizadas a proveedores y actualizar el inventario del sistema.

Funcionamiento del módulo:

El empleado encargado podrá:

- Registrar proveedores.
- Generar órdenes de compra.

- Registrar ingreso de mercadería.

Cuando una compra sea confirmada:

1. El sistema actualizará el stock automáticamente.
2. Registrará costos de compra.
3. Asociará productos al proveedor correspondiente.

Controles implementados:

- Validación de proveedores existentes.
- Control de cantidades ingresadas.
- Registro histórico de compras.
- Actualización automática del inventario.

3.3.5 Módulo de Inventario

Este módulo permitirá controlar permanentemente el estado del stock del kiosco.

Funcionamiento del módulo:

El sistema realizará controles automáticos para:

- Detectar faltantes.
- Identificar productos críticos.
- Visualizar vencimientos próximos.
- Generar alertas visuales.

También permitirá:

- Consultar movimientos de stock.
- Visualizar historial de reposiciones.

Controles implementados:

- Alertas de stock crítico.
- Alertas de vencimiento.
- Registro automático de movimientos.
- Actualización en tiempo real del inventario.

3.3.6 Módulo de Reportes y Balances

Permitirá obtener información estadística y administrativa del kiosco.

Funcionamiento del módulo:

El usuario podrá generar reportes:

- Diarios.
- Semanales. ● Mensuales.

El sistema mostrará:

- Ventas realizadas.
- Ganancias.
- Productos más vendidos.
- Movimientos de caja.
- Compras registradas.

Controles implementados:

- Filtros por fecha.
- Validación de rangos de búsqueda.
- Generación automática de balances.
- Exportación de información.

3.3.7 Módulo de Autenticación y Seguridad

Este módulo garantizará la seguridad del acceso al sistema.

Funcionamiento del módulo:

El usuario deberá ingresar:

- Nombre de usuario.
- Contraseña.

El sistema:

1. Cifrará la contraseña ingresada.
2. Comparará el hash con la base de datos.
3. Validará permisos.
4. Permitirá o rechazará el acceso.

También incluirá:

- Recuperación de contraseña.
- Cambio obligatorio de contraseña temporal.
- Cierre seguro de sesión.

Controles implementados:

Contraseñas cifradas con SHA256.

- Validación de credenciales.
- Restricción según roles.
- Recuperación segura mediante correo electrónico.
- Control de sesión activa.

3.4 Restricciones

El sistema estará desarrollado únicamente para las funcionalidades y especificaciones acordadas con el cliente al inicio del proyecto, reemplazando los métodos manuales y herramientas actualmente utilizadas en la administración del kiosco.

La Empresa desarrolladora cumplirá con las acciones establecidas durante el desarrollo e implementación del sistema, respetando los requerimientos funcionales y técnicos definidos para el proyecto.

Tareas de la Empresa Desarrolladora

- Desarrollo del sistema de acuerdo con los requerimientos y especificaciones establecidas.
- Realización de reuniones periódicas con el cliente para seguimiento y validación del proyecto.
- Instalación y configuración del sistema en el entorno operativo del kiosco.

- Capacitación básica del personal para el uso correcto del software.
- Elaboración de documentación y material de ayuda para los usuarios.
- Soporte técnico frente a problemas o fallas imprevistas originadas durante el uso del sistema.

El sistema no contemplará la gestión completa de empleados, ya que únicamente registrará usuarios necesarios para operar dentro del software, almacenando solamente la información requerida para autenticación, roles y permisos.

Además, el sistema presentará las siguientes restricciones técnicas y operativas:

- Funcionamiento exclusivo en sistemas operativos Windows 10 o superiores.
- Sistema desarrollado únicamente para entorno de escritorio.
- Administración limitada a una única sucursal.
- No incluirá integración con facturación electrónica.
- No poseerá integración bancaria ni pasarelas de pago online.
- No contemplará gestión logística externa ni distribución de productos.
- Dependencia de SQL Server como sistema gestor de base de datos.

3.4.1 Otros Requisitos del Producto

La organización deberá garantizar un entorno de trabajo adecuado para asegurar el correcto funcionamiento del sistema y el cumplimiento de los requisitos del producto.

El sistema deberá ofrecer:

- Interfaz gráfica intuitiva y sencilla.
- Navegación rápida y organizada entre módulos.
- Seguridad básica mediante autenticación y control de permisos.
- Persistencia y almacenamiento seguro de la información.
- Integridad y consistencia de los datos registrados.
- Facilidad de uso para el personal operativo.
- Estabilidad durante jornadas prolongadas de trabajo.

3.4.2 Estándares Aplicables

Principales estándares considerados

Estándares de calidad

- ISO 9001.
- ISO/IEC 25000 .

Estándares de desarrollo

- ISO/IEC 12207 .
- IEEE 830 .

Estándares de seguridad

- ISO/IEC 27001 .
- Buenas prácticas OWASP para validación y seguridad de accesos.

Estándares de codificación

- Desarrollo realizado en lenguaje C#.
- Uso de arquitectura en capas:
    - Presentación.
    - Lógica.
    - Acceso a datos.
- Uso de convenciones de nombres propias de C#.
- Implementación de consultas SQL parametrizadas.
- Validación de datos en formularios.
- Manejo de excepciones para control de errores.
- Separación entre interfaz gráfica y lógica del sistema.
- Cifrado de contraseñas mediante SHA256.
- Compatibilidad con .NET 8, Windows Forms y SQL Server.

3.4.3 Requisitos de Sistema

Requisitos funcionales

El sistema deberá permitir:

- Autenticación de usuarios.
- Gestión de productos.

- Registro de ventas.
- Registro de compras. ● Control de inventario.
- Generación de reportes y balances.
- Administración de usuarios y permisos.

Requisitos no funcionales

El sistema deberá garantizar:

- Seguridad de acceso mediante roles y contraseñas cifradas.
- Integridad y consistencia de datos.
- Compatibilidad con Windows 10 o superior.
- Estabilidad durante jornadas prolongadas de uso.
- Rendimiento adecuado en operaciones diarias.

3.4.4 Requisitos de Desempeño

Velocidad de respuesta

- Inicio de sesión rápido, tiempo máximo 1.5 segundos.
- Registro de ventas en tiempo real, tiempo máximo 2 segundos.
- Consultas rápidas de productos y stock , máximo 3 segundos.
- Generación de reportes en menos de 5 segundos dependiendo del volumen de información.

Capacidad

- Soporte para múltiples operaciones diarias, un solo usuario.
- Manejo simultáneo de consultas y registros sin degradación notable.

Disponibilidad

- Funcionamiento continuo durante toda la jornada laboral del kiosco, 99.9% uptime.

Eficiencia de recursos

- Uso optimizado de memoria y procesamiento.
- Estabilidad en equipos de recursos moderados.

3.4.5 Requisitos de Entorno

Los requisitos de entorno especifican las condiciones necesarias para el correcto funcionamiento del sistema dentro del kiosco.

Hardware

- Computadora principal para operación del sistema.
- Procesador y memoria compatibles con .NET 8.
- Dispositivos básicos de entrada y salida.

Software

- Windows 10 o superior.
- SQL Server instalado y configurado.
- .NET Runtime compatible.

Red y conectividad

- Conexión local entre aplicación y base de datos.
- Acceso a internet para recuperación de contraseñas por correo electrónico.

Entorno operativo

- Kiosco comercial de atención al público.
- Uso continuo diario.
- Atención simultánea a clientes.
- Operación local desde computadora principal de caja.

3.5 Casos de Uso

Casos de Uso de Negocio

CU01 - Registrar Producto

ID y Nombre	**CU01 Registrar Producto**	
Descripción	Permite al dueño o empleado autorizado cargar un nuevo producto al inventario del kiosco, indicando su nombre, marca, unidad de venta y rubro, y opcionalmente el código de barras y la descripción. De forma opcional se cargan los precios (costo, IVA y margen de ganancia, con los que el sistema calcula el precio de venta), el control de stock, las alertas de vencimiento, el lote y los proveedores que lo venden con su sucursal.	
Actores	Administrador - Empleado autorizado (permiso 1.1.1 Alta de Artículos).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 1.1.1 Alta de Artículos. Deben existir marcas, rubros y unidades de venta registrados.	
Post-Condición	El producto queda registrado en la base de datos y disponible en el inventario. Si se cargaron, quedan registrados el stock, el lote y los proveedores del producto. Se calcula el dígito verificador del producto y la operación queda registrada en la bitácora.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Artículos > Cargar artículo nuevo.	
	2. El sistema muestra el formulario con el SKU generado automáticamente.	
	3. El usuario ingresa los datos obligatorios: nombre, marca, unidad de venta y rubro. El código y la descripción son opcionales.	
	4. El usuario completa, si corresponde, los datos opcionales: precio de costo, IVA, margen de ganancia, stock actual, stock mínimo y máximo, vencimiento, lote y proveedores con su sucursal.	

	5. El usuario presiona "Guardar".	
	6. El sistema valida los datos ingresados.	6.1 Datos incompletos o inválidos: El sistema informa "Complete los campos obligatorios (nombre, marca, unidad de venta y rubro) y revise los valores ingresados". 6.2 Lote sin proveedor: El sistema informa que para cargar lote o vencimiento debe agregar al menos un proveedor. 6.3 Proveedor repetido: El sistema informa que ese proveedor y sucursal ya están en la lista.
	7. El sistema guarda el producto, su stock, su lote y sus proveedores en la base de datos.	7.1 Error al guardar: El sistema informa que no se pudo registrar el artículo, o que se registró pero no se pudieron guardar el lote o los proveedores.
	8. El sistema confirma el registro exitoso.	

Diagrama de caso de uso

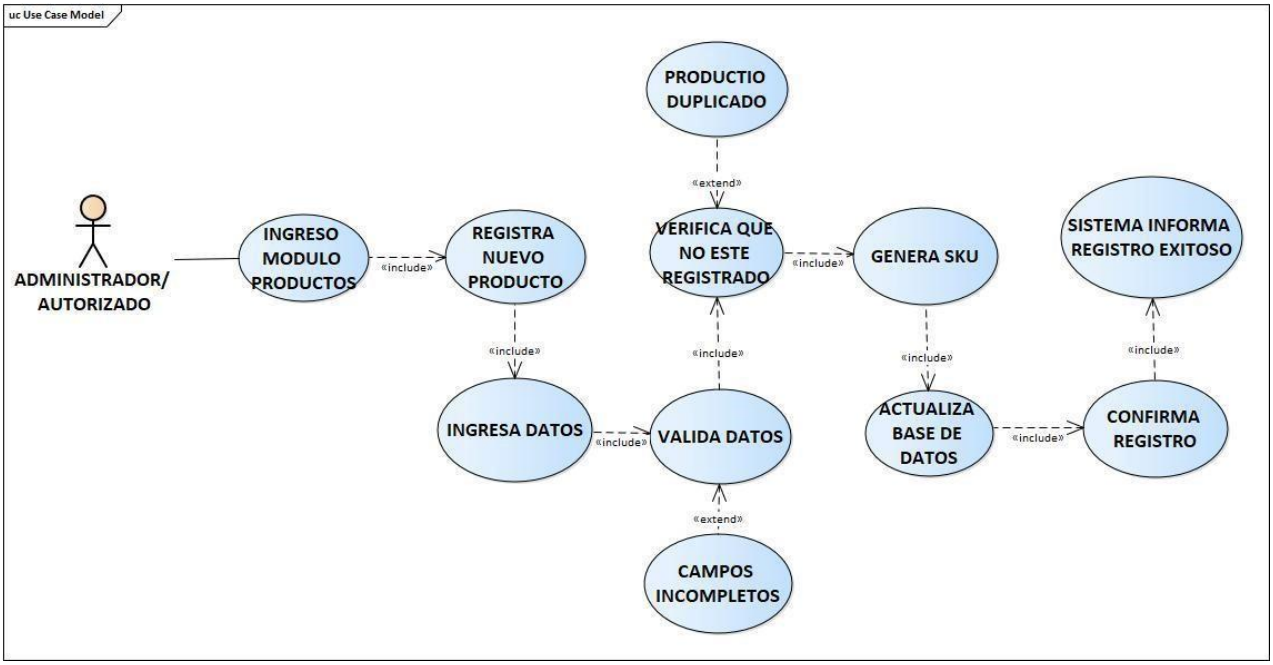


Diagrama de secuencia

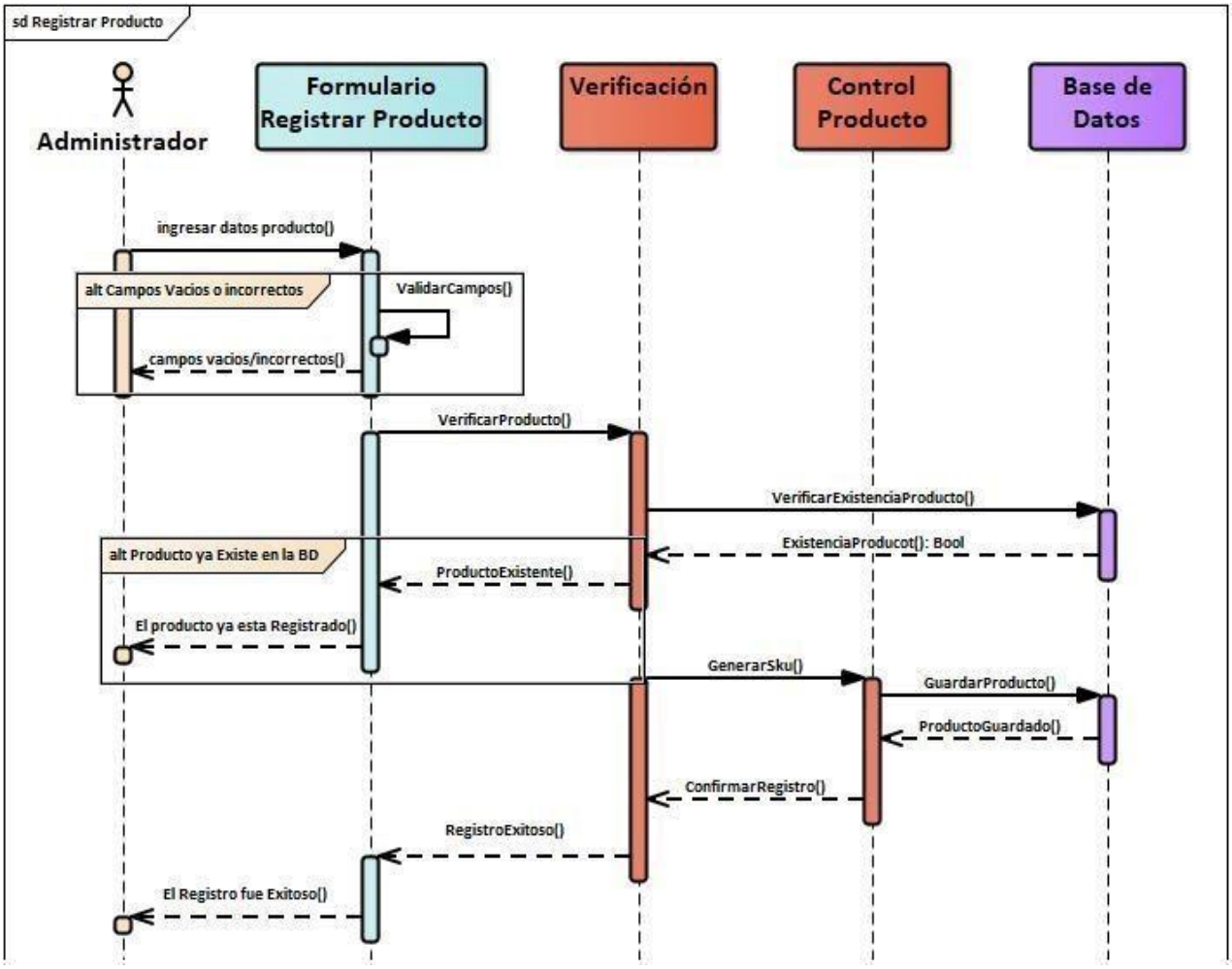
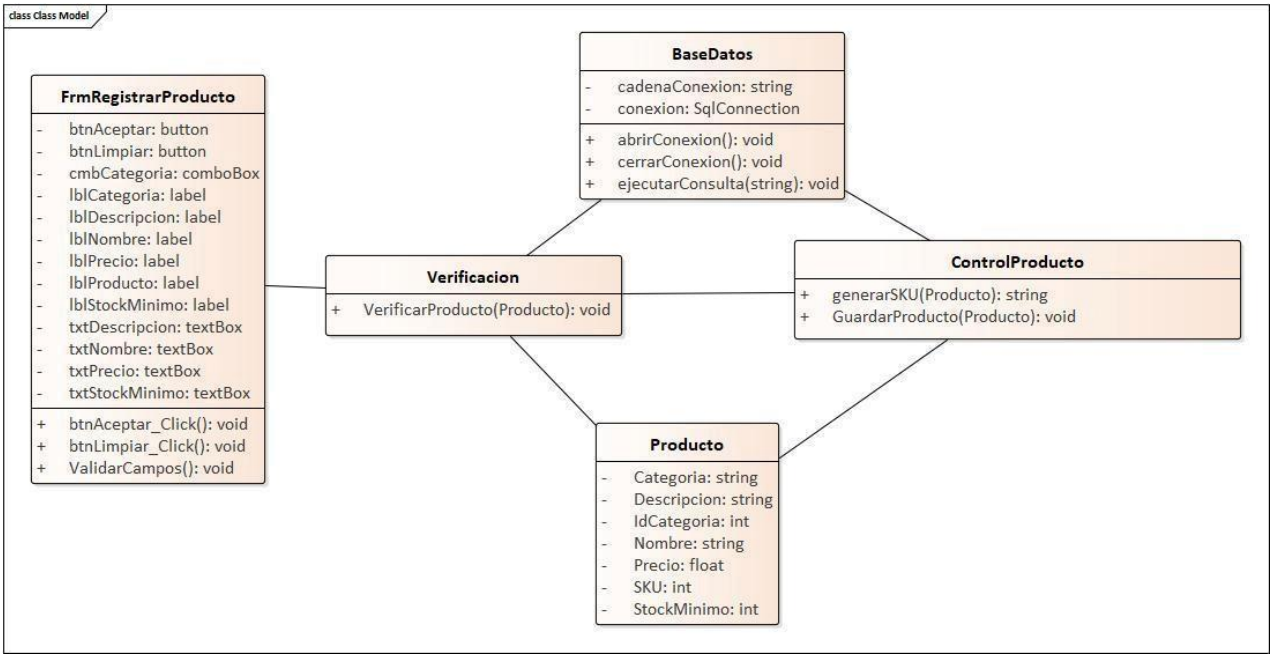


Diagrama de clases



CU02 - Registrar Venta

ID y Nombre	CU02 Registrar Venta	
Descripción	Permite al usuario autorizado registrar una venta desde la pantalla principal. Los productos se cargan con el lector de código de barras o con la búsqueda de artículos; el sistema verifica el stock, aplica las promociones vigentes y calcula subtotal, descuento y total. Al cobrar se pueden usar hasta dos medios de pago (efectivo, transferencia o débito) y el sistema calcula el vuelto. Una vez confirmada, la venta queda registrada, se descuenta el stock y se genera el movimiento de caja.	
Actores	Administrador - Empleado autorizado (permiso 2.1.0 Realizar Ventas).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 2.1.0 Realizar Ventas. La caja debe estar abierta (CU31). Los productos deben estar registrados, activos y con precio de venta.	
Post-Condición	La venta queda registrada con sus productos y medios de pago. El stock de los productos vendidos se actualiza automáticamente. Se registra el movimiento de caja para el arqueo, los balances y los informes.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario se encuentra en la pantalla principal.	1.1 Caja cerrada: El sistema pregunta si desea abrir la caja (CU31); si el usuario no tiene permiso, le pide que avise a un encargado.
	2. El usuario escanea el código de barras o busca el producto y lo agrega a la venta.	2.1 Producto no encontrado o sin precio: El sistema informa que no existe un producto con ese código o que no tiene precio de venta.
	3. El sistema verifica la disponibilidad de stock.	3.1 Stock insuficiente: El sistema informa el stock disponible y el usuario puede modificar la cantidad.
	4. El sistema aplica las promociones vigentes y calcula subtotal, descuento y total.	
	5. El usuario puede cambiar la cantidad o quitar un producto.	

	6. El usuario presiona "F2-Cobrar".	6.1 Venta vacía: El sistema informa que debe agregar al menos un producto para realizar el cobro.
	7. El usuario elige uno o dos medios de pago e ingresa los montos.	7.1 Más de dos medios de pago: El sistema informa que solo se permiten dos métodos de pago diferentes por venta.
	8. El sistema calcula lo pagado, lo restante y el vuelto.	
	9. El usuario presiona "Confirmar venta".	9.1 Pago incompleto: El sistema informa el saldo pendiente y no confirma la venta. 9.2 Cancelación de la venta: El usuario presiona "Cancelar" y el sistema no registra cambios.
	10. El sistema registra la venta, actualiza el stock y registra el movimiento de caja.	
	11. El sistema informa que la venta fue registrada correctamente.	

Diagrama de caso de uso

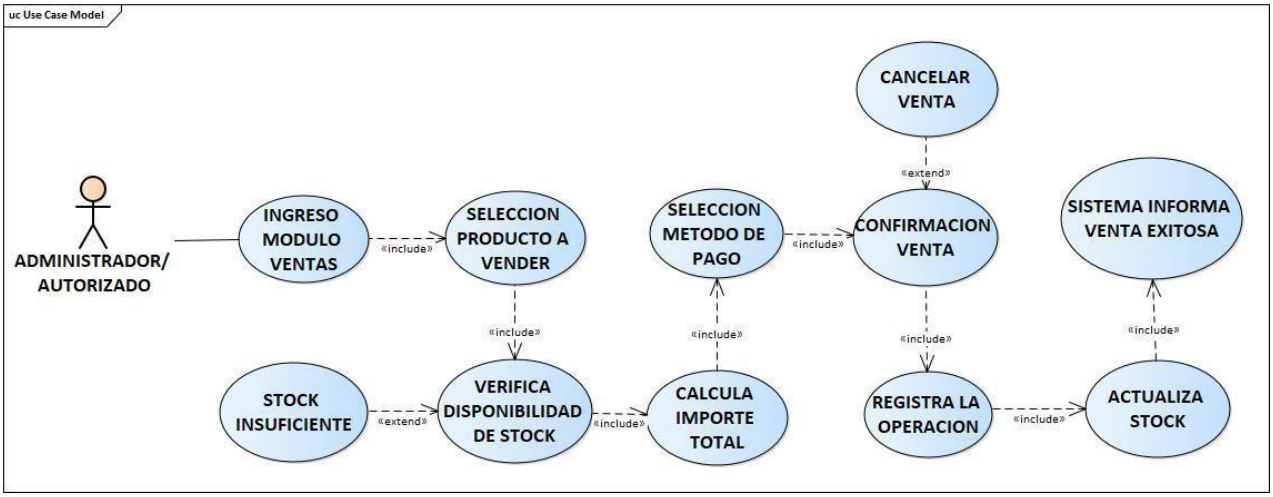


Diagrama de secuencia

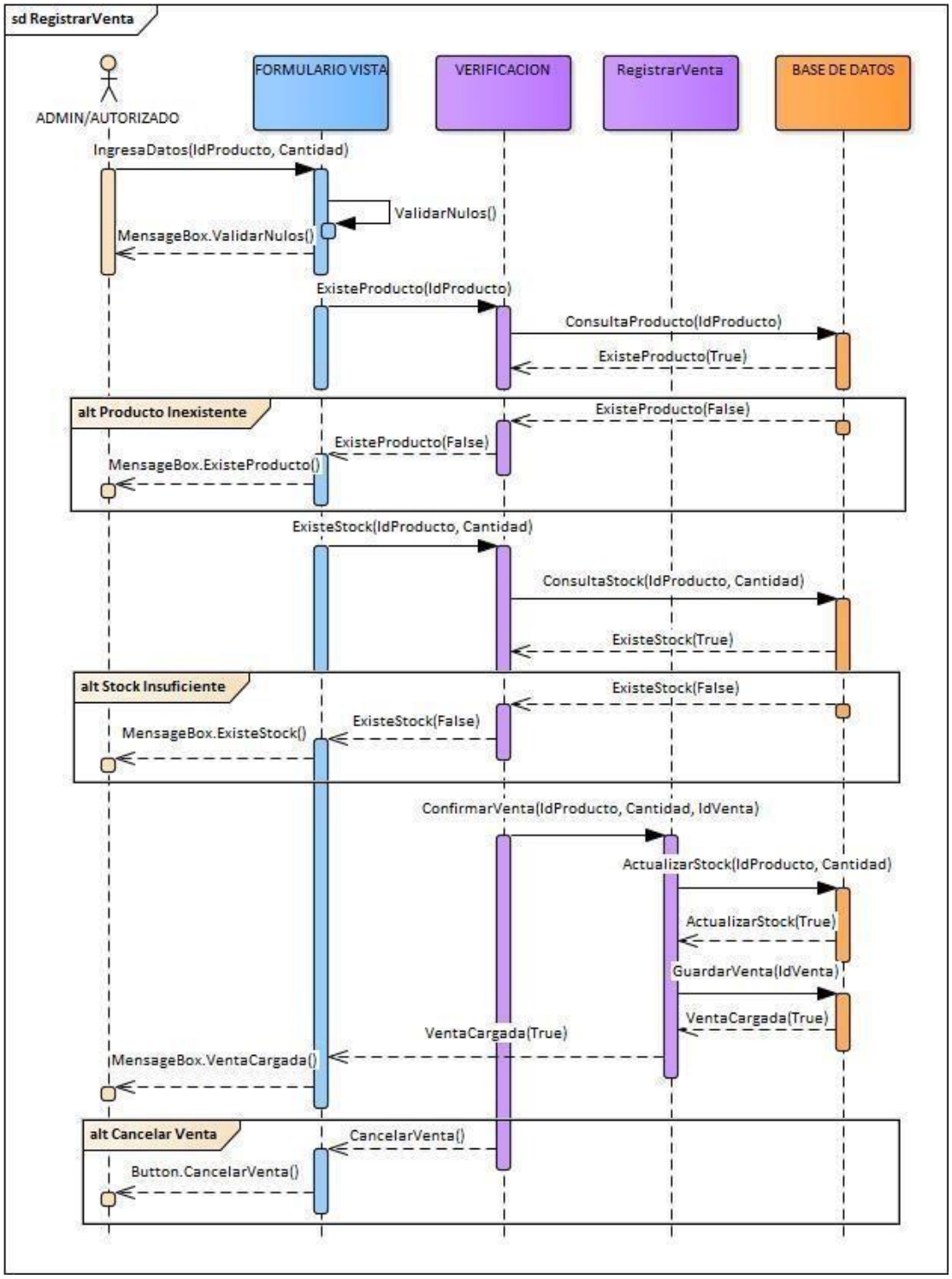
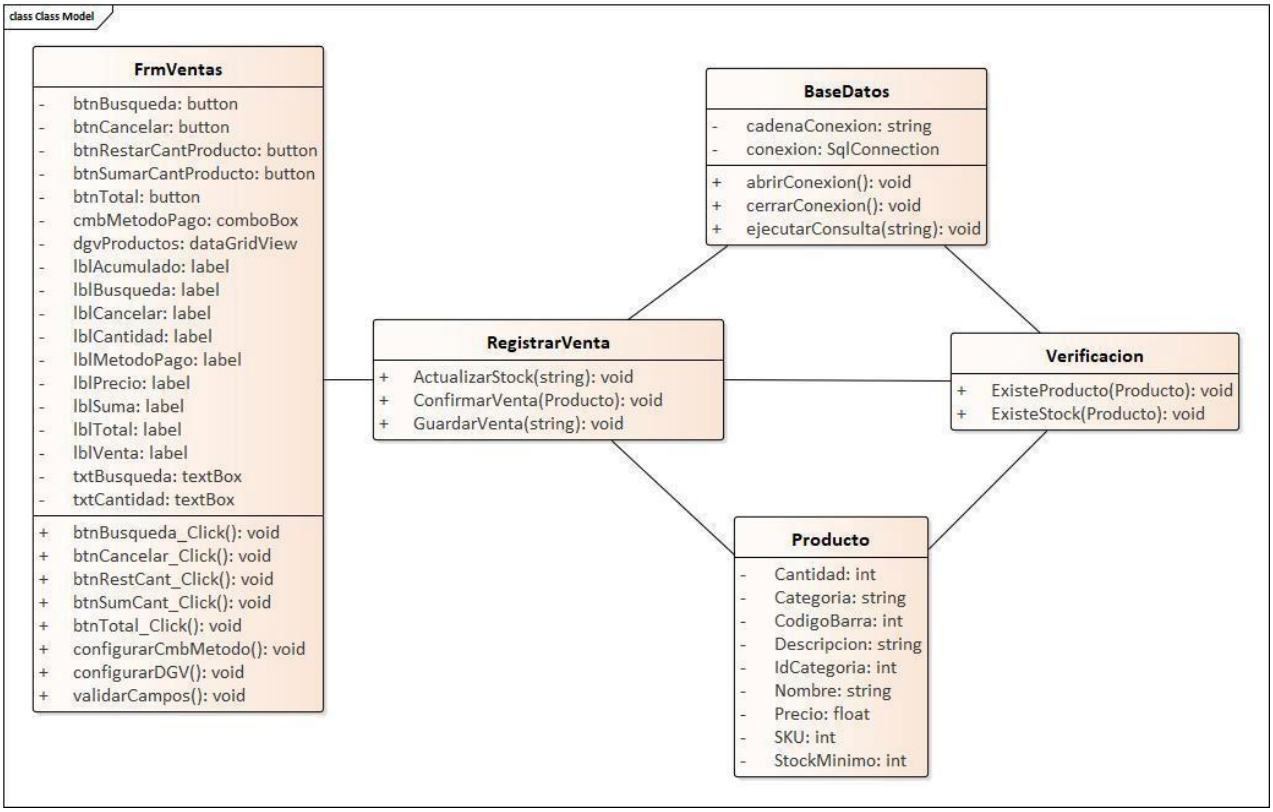


Diagrama de clases



CU03 - Carga Empleado

ID y Nombre	CU03 Carga Empleado	
Descripción	Permite al Administrador registrar un nuevo empleado como usuario del sistema. Durante el proceso se ingresan los datos personales (nombre, apellido, DNI, CUIL, fecha de nacimiento y género), la dirección, el teléfono, el correo electrónico y el rol que desempeñará. El sistema valida que el CUIL corresponda al DNI, genera automáticamente un nombre de usuario y una contraseña temporal y los envía por correo electrónico. Por último registra la información en la base de datos.	
Actores	Administrador (permiso 10.1.0 Alta Empleado).	
Pre-Condición	El administrador debe haber iniciado sesión y tener el permiso 10.1.0 Alta Empleado. Deben existir roles registrados en el sistema.	
Post-Condición	El empleado queda registrado con su usuario y rol. Se genera un nombre de usuario único y una contraseña temporal, que el empleado deberá cambiar en su primer ingreso. La operación queda registrada en la bitácora.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El administrador accede al menú Administración > Empleado.	
	2. El administrador presiona "Nuevo" y el sistema habilita los campos de carga.	
	3. El administrador ingresa los datos personales, la dirección, el teléfono, el correo y selecciona el rol.	
	4. El administrador presiona "Crear empleado".	
	5. El sistema valida los datos ingresados y que el DNI corresponda al CUIL.	5.1 Datos incompletos o incorrectos: El sistema informa el campo a corregir (por ejemplo, correo inválido, rol sin seleccionar o DNI que no corresponde al CUIL).

	6. El sistema verifica que el empleado no esté registrado.	6.1 Datos duplicados: El sistema informa que el empleado ya está registrado.
	7. El sistema genera el nombre de usuario en base al nombre y apellido y una contraseña temporal aleatoria.	7.1 Usuario existente: El sistema genera una variante del nombre de usuario y continúa.
	8. El sistema almacena la información del empleado en la base de datos.	8.1 Error al registrar: El sistema informa que no se pudo crear el empleado y no almacena cambios.
	9. El sistema envía el nombre de usuario y la contraseña temporal al correo electrónico ingresado.	9.1 Error de envío: El empleado queda creado y el sistema informa que no se pudo enviar el correo con la contraseña temporal.
	10. El sistema informa que el empleado se creó correctamente.	

Diagrama de caso de uso

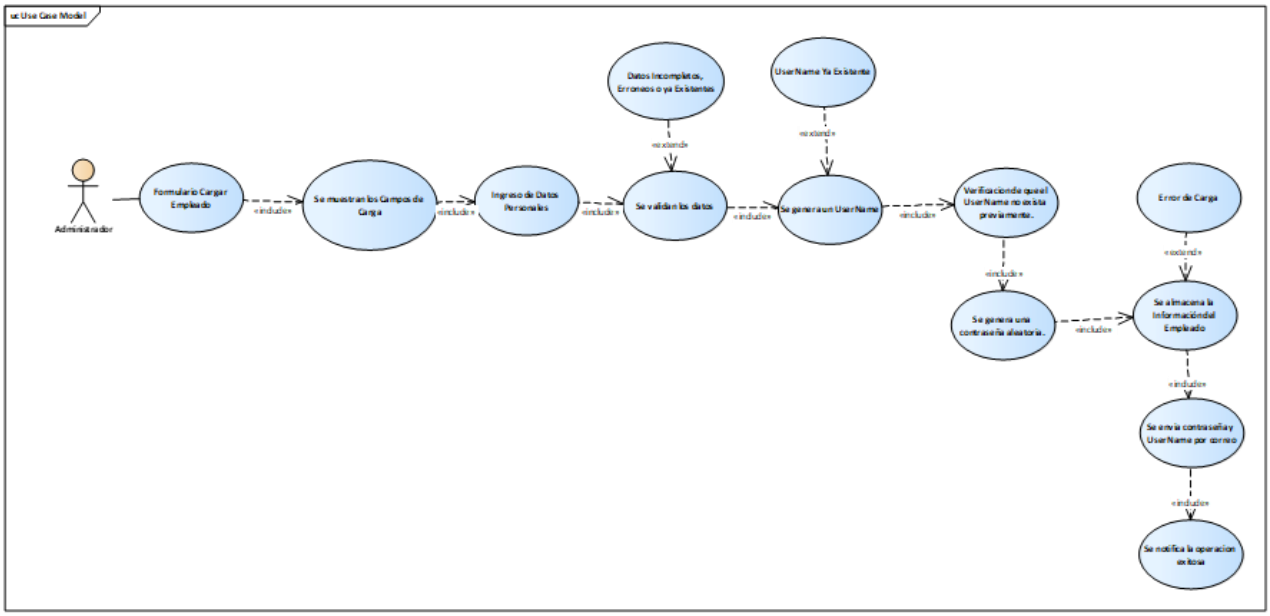


Diagrama de secuencia

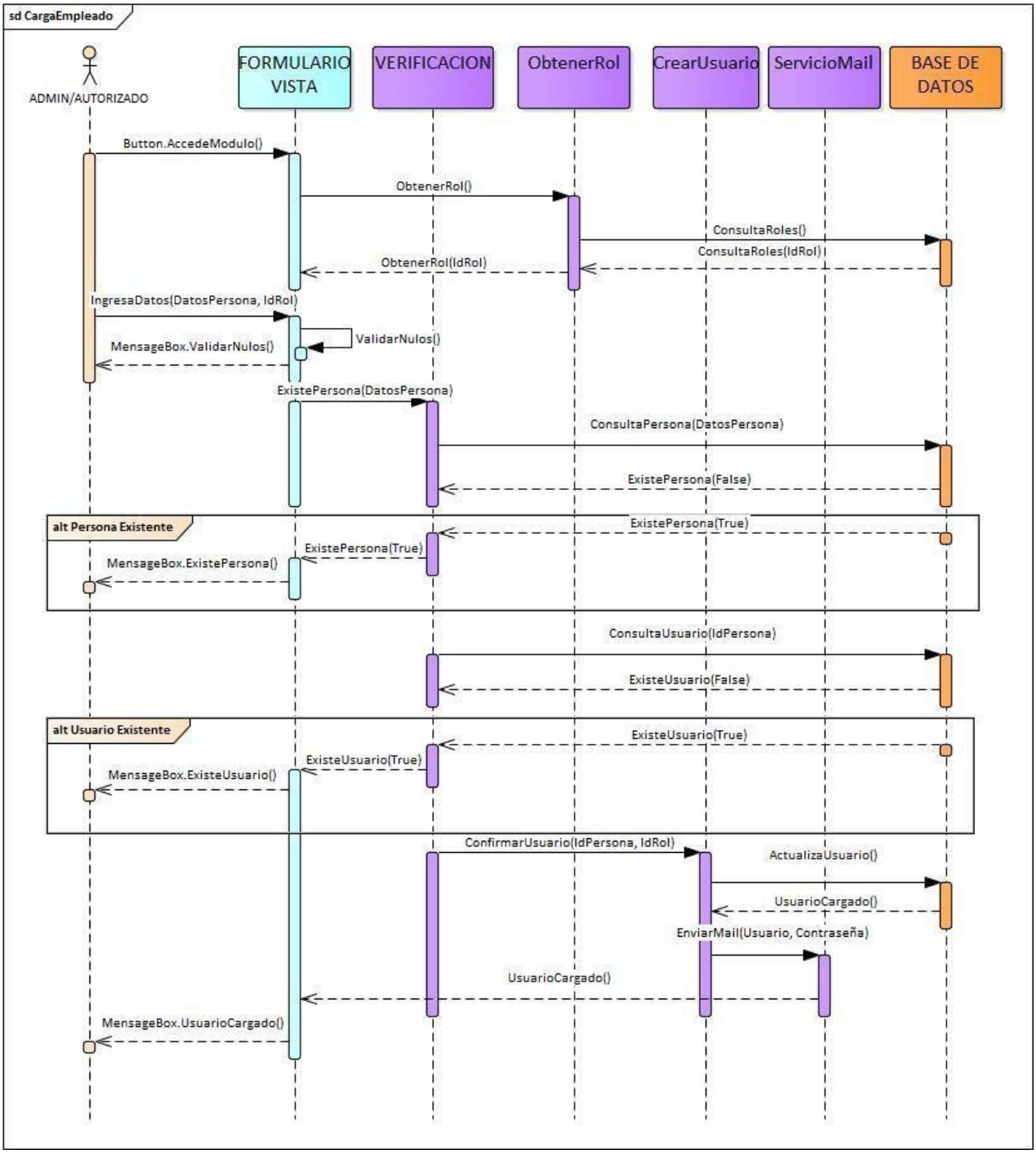
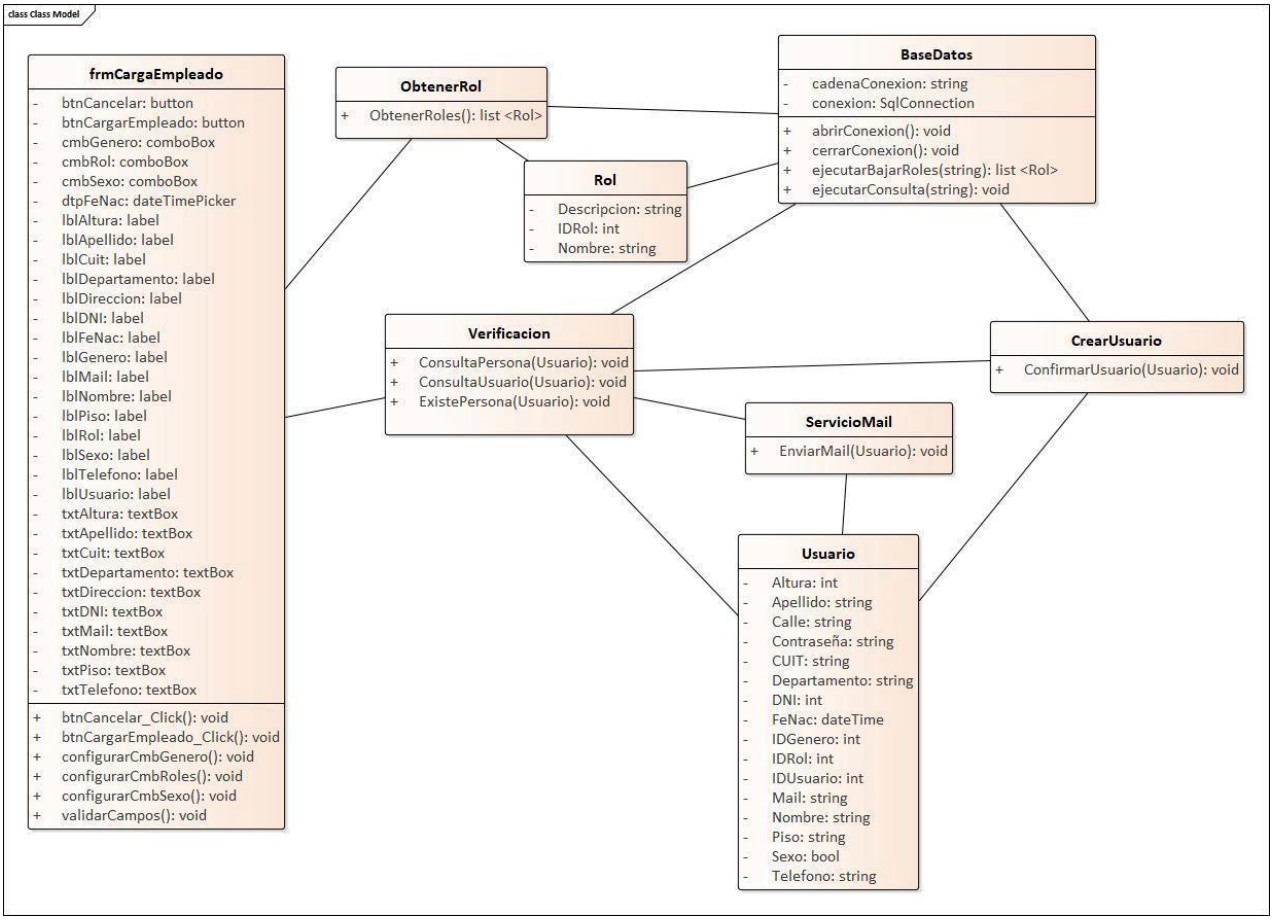


Diagrama de clases



CU04 - Crear Rol

ID y Nombre	**CU04 Crear Rol**	
Descripción	Permite al Administrador crear nuevos roles dentro del sistema y definir los permisos que tendrán asociados. Desde la pantalla de permisos, el administrador crea el rol ingresando un nombre identificatorio y luego marca, en el árbol de permisos, las funcionalidades a las que podrán acceder los usuarios con ese rol. El sistema valida que el nombre del rol no exista previamente y guarda las asociaciones entre el rol y sus permisos. Los permisos dependen del rol: cambiarlos afecta a todos los empleados que lo tengan asignado. De esta manera, el administrador puede personalizar los niveles de acceso sin realizar cambios en el código fuente del sistema.	
Actores	Administrador (permiso 12.7.0 Roles y Permisos).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 12.7.0 Roles y Permisos. Deben existir permisos registrados en el sistema.	
Post-Condición	El rol queda registrado en la base de datos. Los permisos seleccionados quedan asociados al rol. El rol se encuentra disponible para asignar a los empleados.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El administrador accede al menú Administración > Empleado y presiona "Permisos del rol".	
	2. El sistema muestra los roles y el árbol de permisos.	
	3. El administrador presiona "Nuevo Rol".	
	4. El administrador ingresa el nombre del rol y presiona "Guardar".	4.1 Nombre vacío: El sistema informa "Ingrese un nombre para el rol".
	5. El sistema verifica que el nombre del rol no exista.	5.1 Nombre de rol existente: El sistema informa que ya existe un rol con ese nombre.

	6. El sistema registra el nuevo rol y lo muestra en la lista.	
	7. El administrador selecciona el rol y marca los permisos que tendrá.	
	8. El administrador presiona "Guardar".	8.1 Cancelar: El administrador presiona "Cancelar" y el sistema descarta los cambios. 8.2 Eliminar rol: El administrador presiona "Eliminar Rol"; los roles principales del sistema están protegidos y no se pueden eliminar.
	9. El sistema registra las asociaciones entre el rol y los permisos seleccionados.	
	10. El sistema confirma que se guardaron los permisos del rol.	

Diagrama de caso de uso

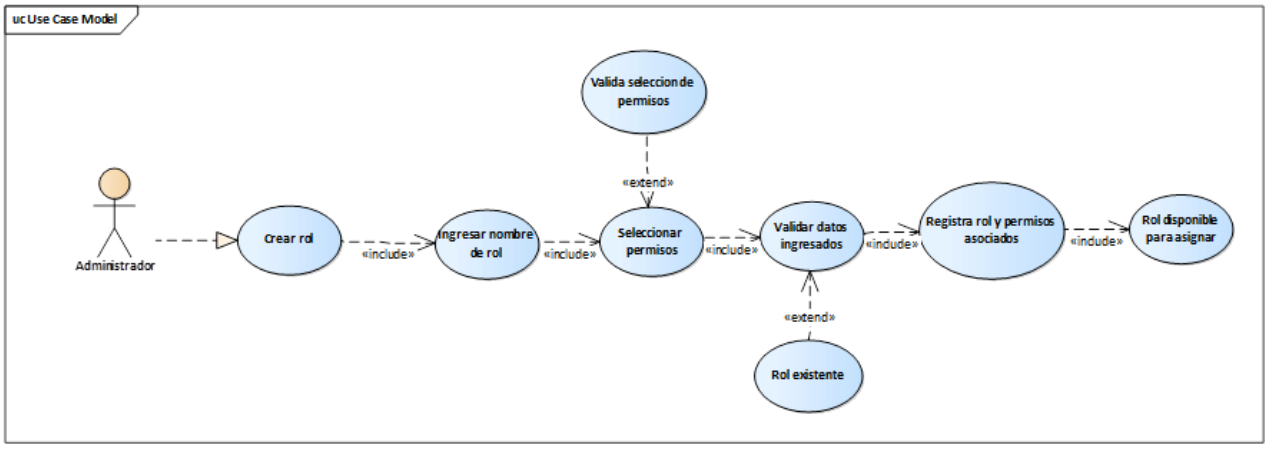


Diagrama de secuencia

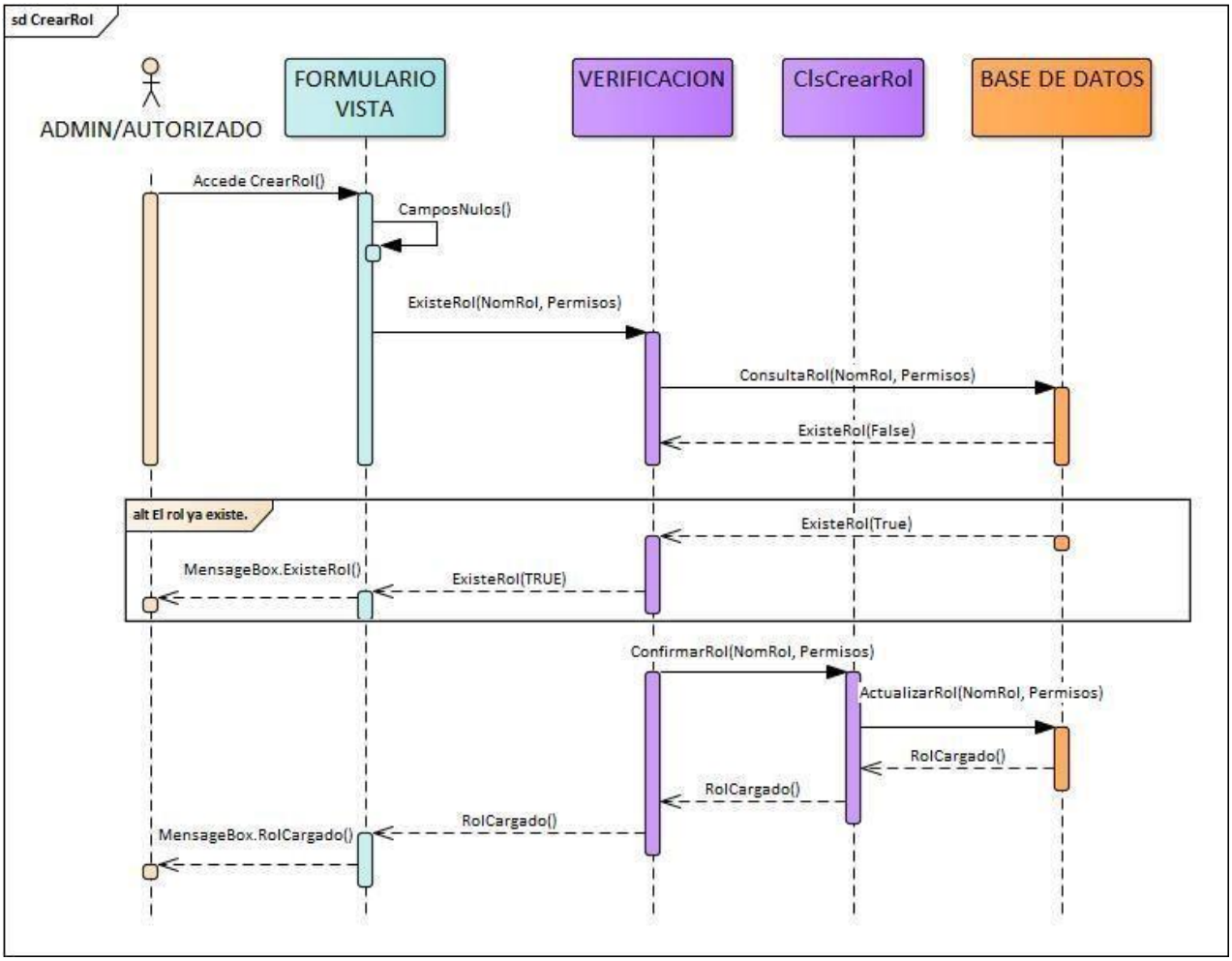
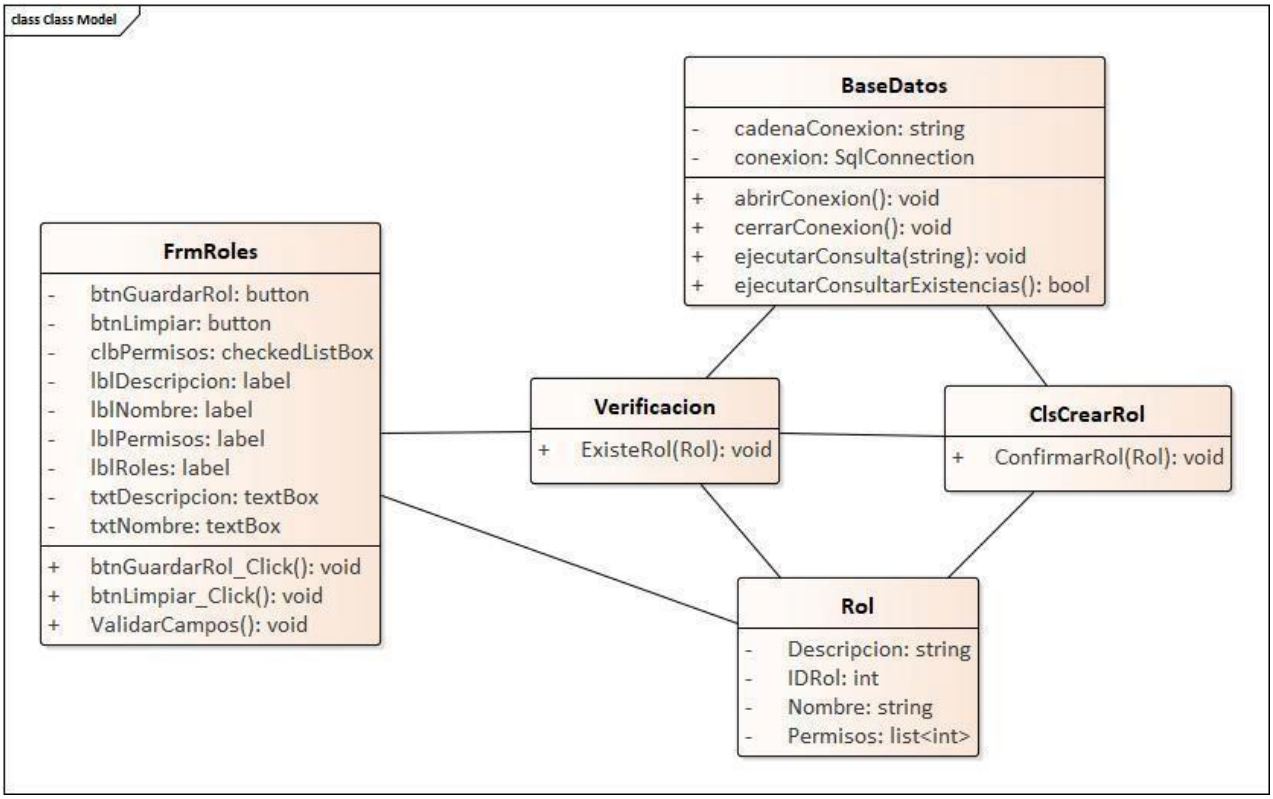


Diagrama de clases



CU05 - Control de Caja Diaria

ID y Nombre	**CU05 Control de Caja Diaria**	
Descripción	Permite al dueño o empleado autorizado realizar el cierre de caja (arqueo) al finalizar la jornada o el turno. El sistema muestra el fondo inicial, los ingresos y egresos en efectivo y el efectivo esperado; el usuario ingresa el efectivo contado y el sistema calcula la diferencia.	
Actores	Administrador - Empleado autorizado (permiso 7.4.0 Cerrar Caja).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 7.4.0 Cerrar Caja. Debe haber una caja abierta (CU31).	
Post-Condición	La caja queda cerrada con el efectivo contado, la diferencia y la observación. El cierre queda disponible en el historial de cajas y en el balance.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al menú Caja > Cerrar Caja.	1.1 Caja no abierta: El sistema informa que la caja ya no está abierta.
	2. El sistema muestra el arqueo: fondo inicial, ingresos en efectivo, egresos en efectivo y efectivo esperado.	
	3. El usuario ingresa el efectivo contado.	
	4. El sistema calcula la diferencia.	
	5. El usuario presiona "Cerrar caja".	5.1 Diferencia sin observación: Si hay diferencia, el sistema pide ingresar una observación que la explique.

	6. El sistema solicita confirmación.	6.1 Cancelar: El usuario no confirma y la caja sigue abierta.
	7. El sistema registra el cierre de la caja.	
	8. El sistema informa que la caja se cerró correctamente.	

Diagrama de caso de uso

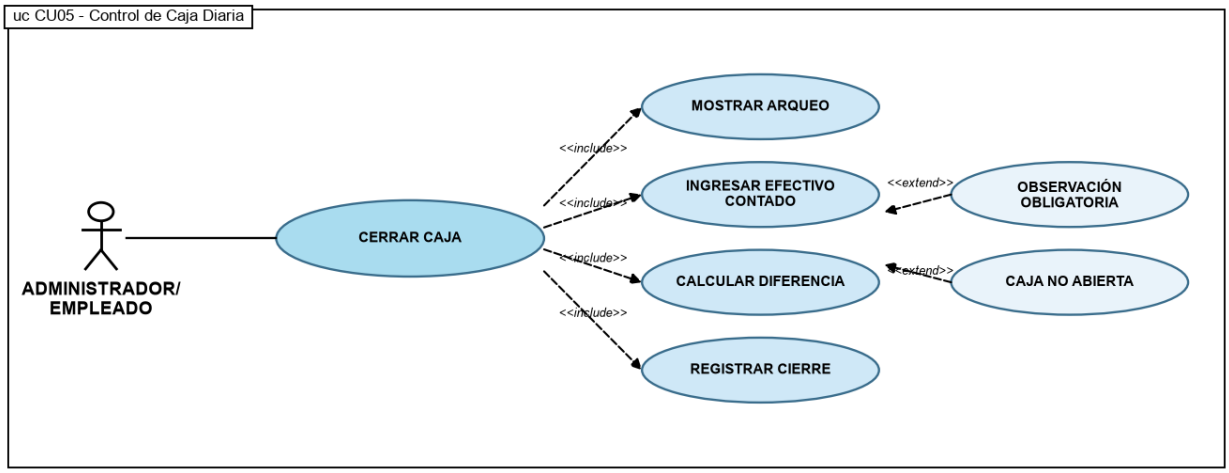


Diagrama de secuencia

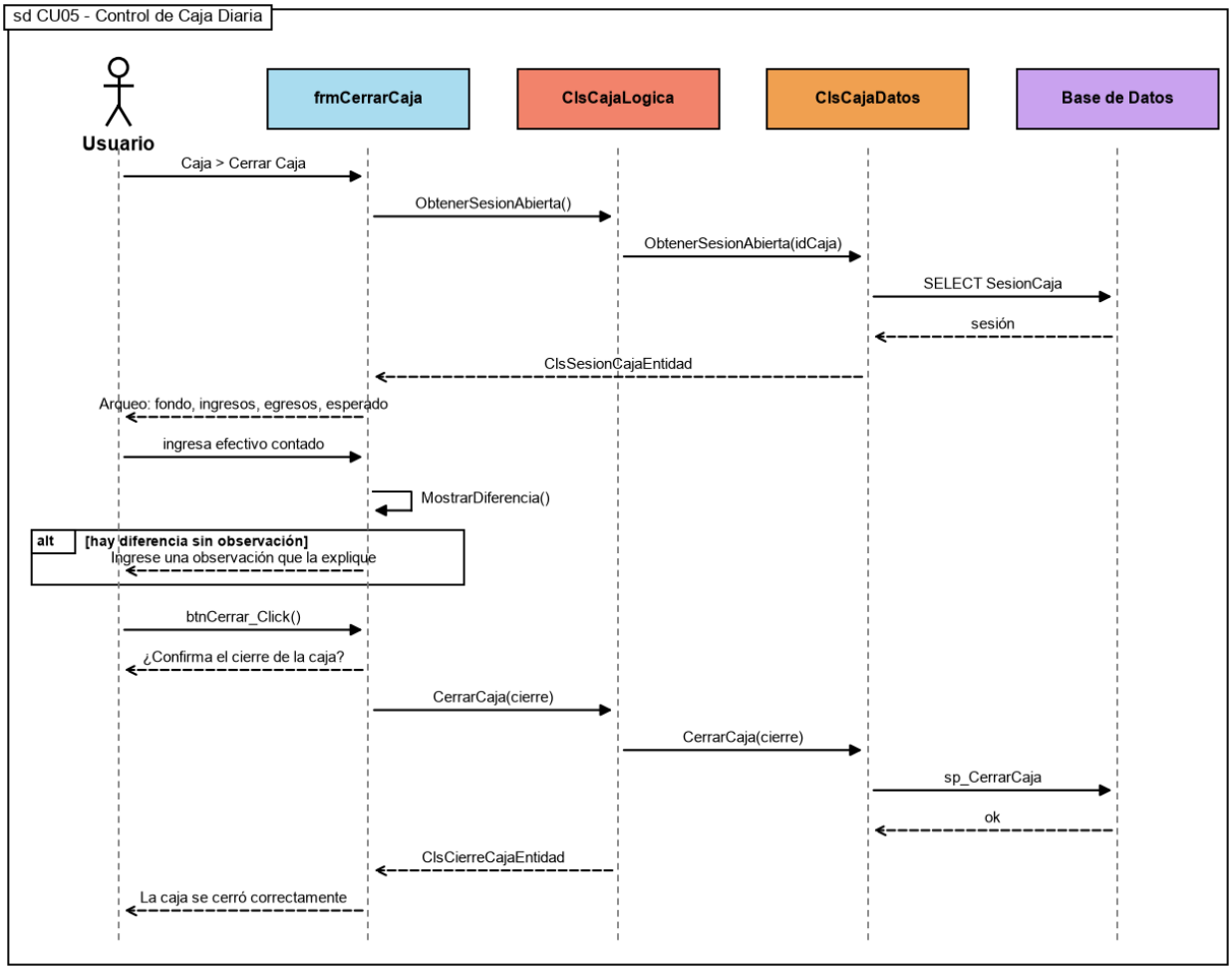
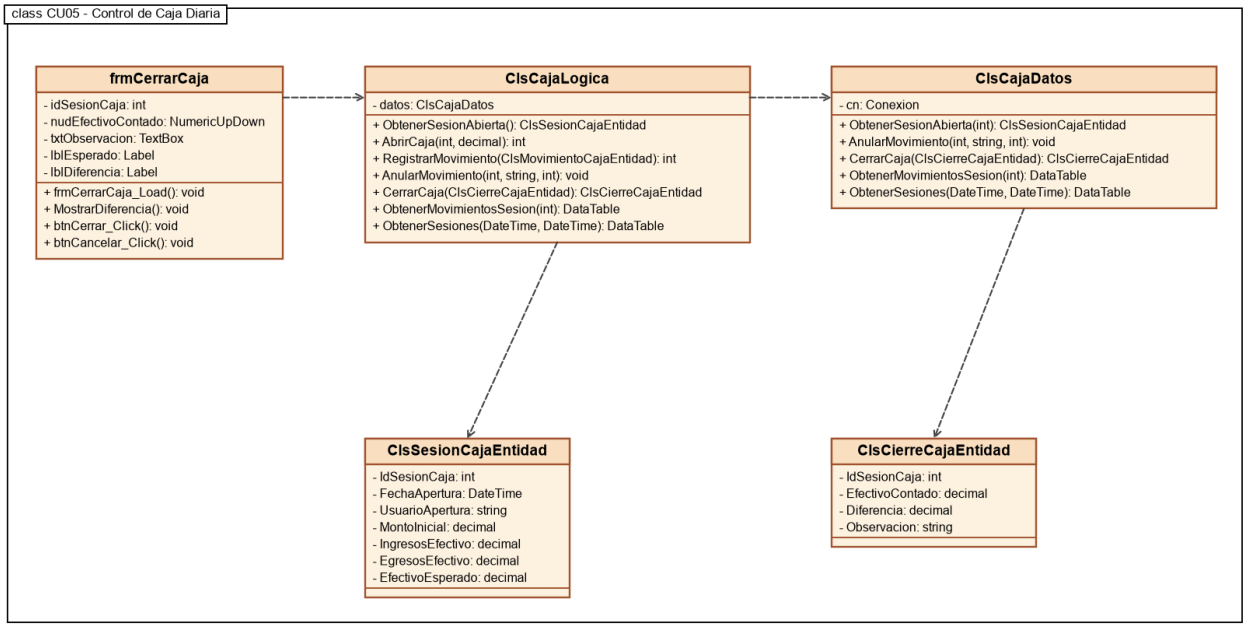


Diagrama de clases



CU06 - Registrar Proveedor

ID y Nombre	**CU06 Registrar Proveedor**	
Descripción	Permite al dueño o empleado autorizado registrar un proveedor nuevo en el sistema con su primera sucursal: razón social, CUIT, condición de IVA, teléfono, correo electrónico y dirección. Un mismo proveedor puede tener varias sucursales.	
Actores	Administrador (permiso 4.1.1 Alta de Proveedores).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 4.1.1 Alta de Proveedores.	
Post-Condición	El proveedor queda registrado en el sistema junto con su sucursal.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Proveedores > Administrador de Proveedores.	
	2. El usuario presiona "Nuevo proveedor".	
	3. El usuario ingresa los datos obligatorios (marcados con *): razón social, sucursal, condición de IVA, teléfono, correo, provincia, partido, localidad, calle y número.	
	4. El sistema valida los datos ingresados.	4.1 Campos vacíos: El sistema detecta campos obligatorios incompletos y solicita completarlos. 4.2 Campos con datos incorrectos: El sistema detecta datos inválidos (por ejemplo, CUIT o correo) y solicita corregirlos.

	5. El sistema verifica que el proveedor no se encuentre registrado previamente.	5.1 Proveedor ya registrado: El sistema informa que ya existe un proveedor con ese nombre.
	6. El sistema guarda el proveedor y su sucursal en la base de datos.	
	7. El sistema informa que el registro fue exitoso y pregunta si desea cargar otra sucursal.	7.1 Otra sucursal: El usuario acepta y el sistema abre el formulario para cargar otra sucursal del mismo proveedor.

Diagrama de caso de uso

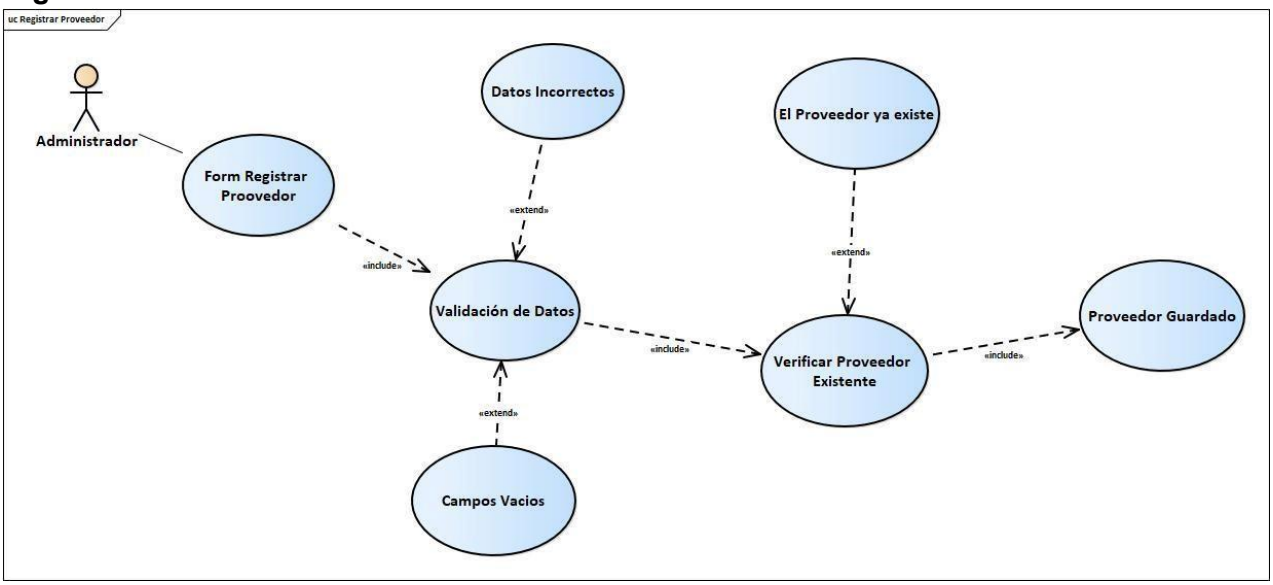


Diagrama de secuencia

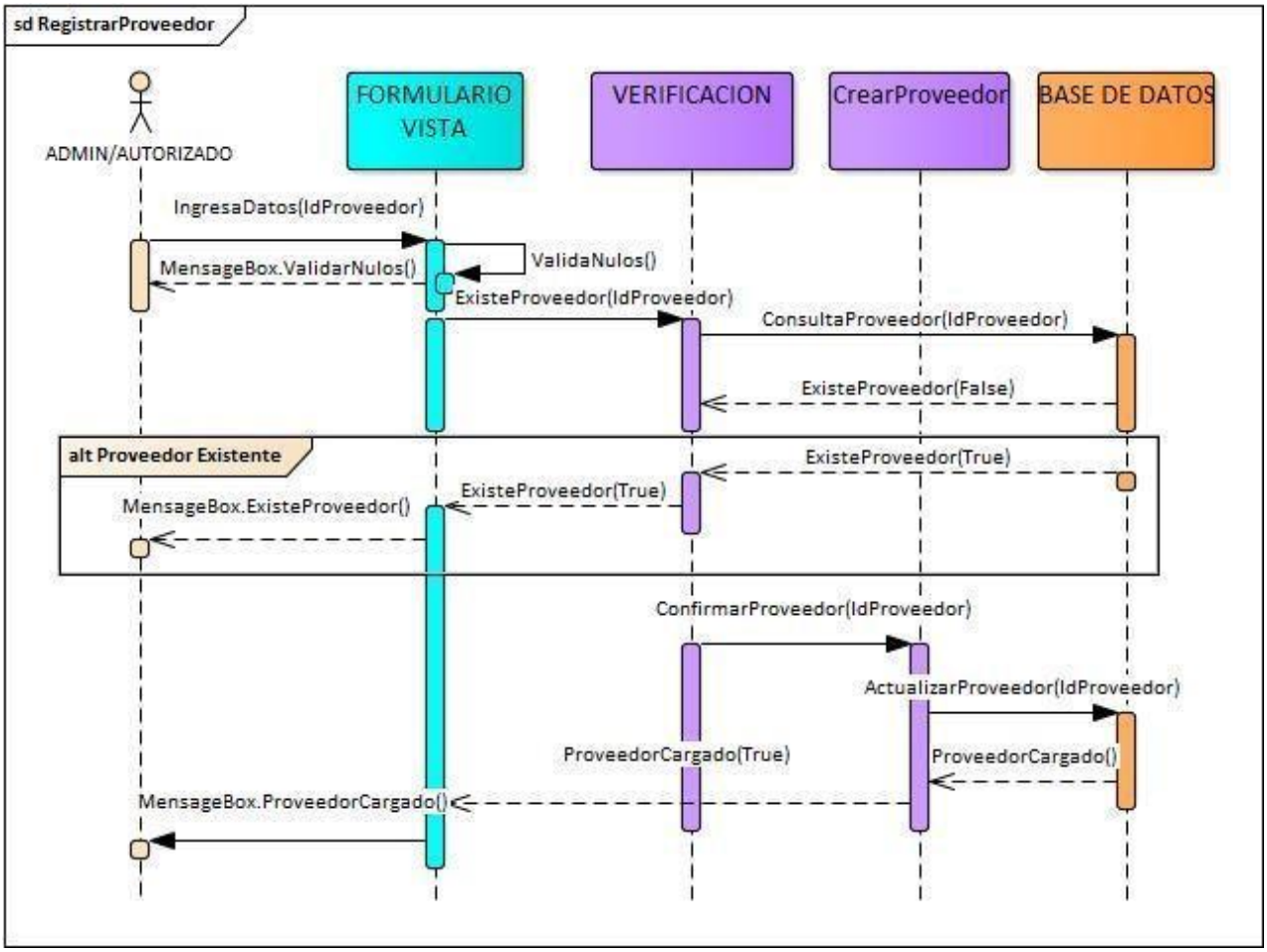
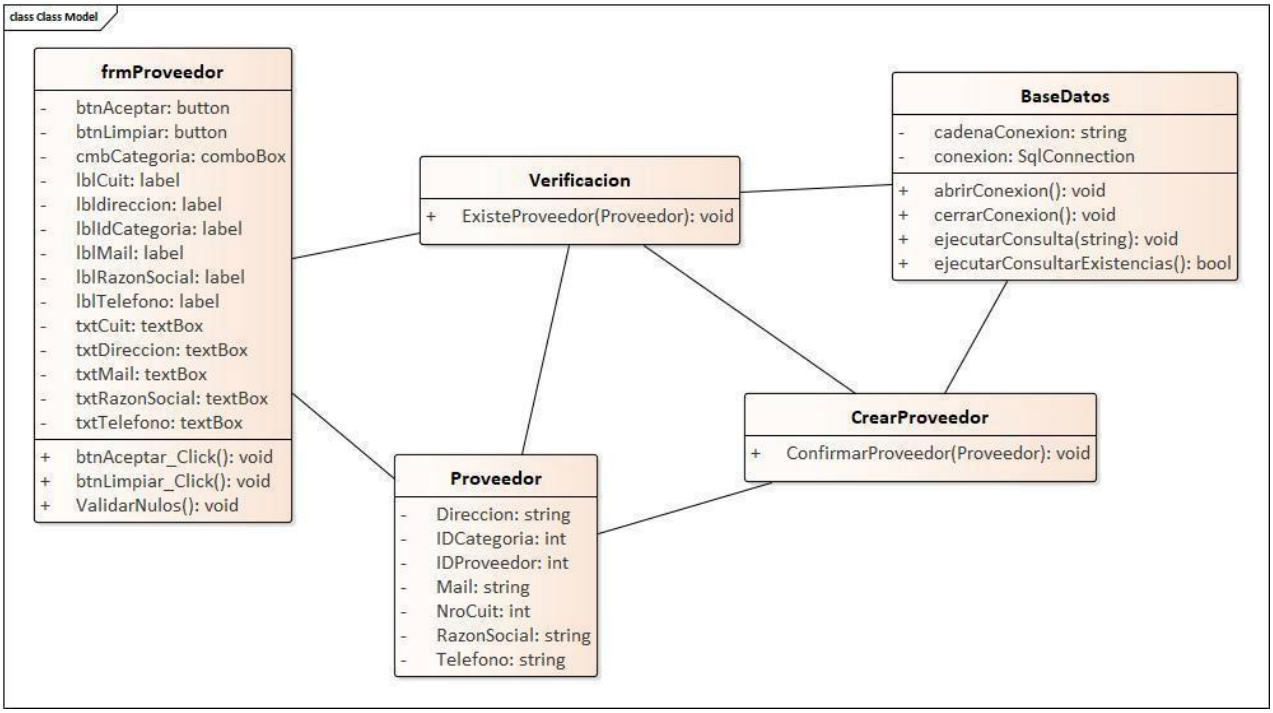


Diagrama de clases



CU07 - Devolución

ID y Nombre	**CU07 Devolución**	
Descripción	Permite al dueño o empleado autorizado registrar la devolución de un producto vendido. Se busca la venta de origen, se elige el producto, la cantidad y el motivo, y se indica si se devuelve el dinero en efectivo o se entrega otra unidad del mismo producto, y si la unidad devuelta vuelve al stock.	
Actores	Administrador - Empleado autorizado (permiso 2.2.1 Realizar Devoluciones).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 2.2.1 Realizar Devoluciones. La venta de origen debe estar registrada. Para devolver dinero, la caja debe estar abierta.	
Post-Condición	La devolución queda registrada con su motivo. Si se devuelve dinero, se registra el egreso en la caja; si la unidad está en condiciones, vuelve al stock.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Ventas > Devolución.	
	2. El usuario presiona "Nueva devolución".	
	3. El usuario busca la venta por rango de fechas o por número de venta.	3.1 Venta no encontrada: El sistema informa que no se encontró la venta con ese número. 3.2 Fechas inválidas: El sistema informa que la fecha desde no puede ser mayor a la fecha hasta.
	4. El usuario selecciona la venta y el producto a devolver.	
	5. El usuario ingresa la cantidad, el motivo y el tipo de reintegro, y marca si la unidad vuelve al stock.	

	6. El sistema valida los datos ingresados.	6.1 Datos incompletos: El sistema informa que debe seleccionar la venta, el producto y el motivo, y que la cantidad debe ser mayor a cero. 6.2 Caja cerrada o sin efectivo: Si se devuelve dinero, el sistema informa que no hay una caja abierta o que no hay efectivo suficiente.
	7. El usuario confirma la devolución.	
	8. El sistema guarda la devolución en la base de datos.	
	9. El sistema informa que la devolución se registró correctamente.	

Diagrama de caso de uso

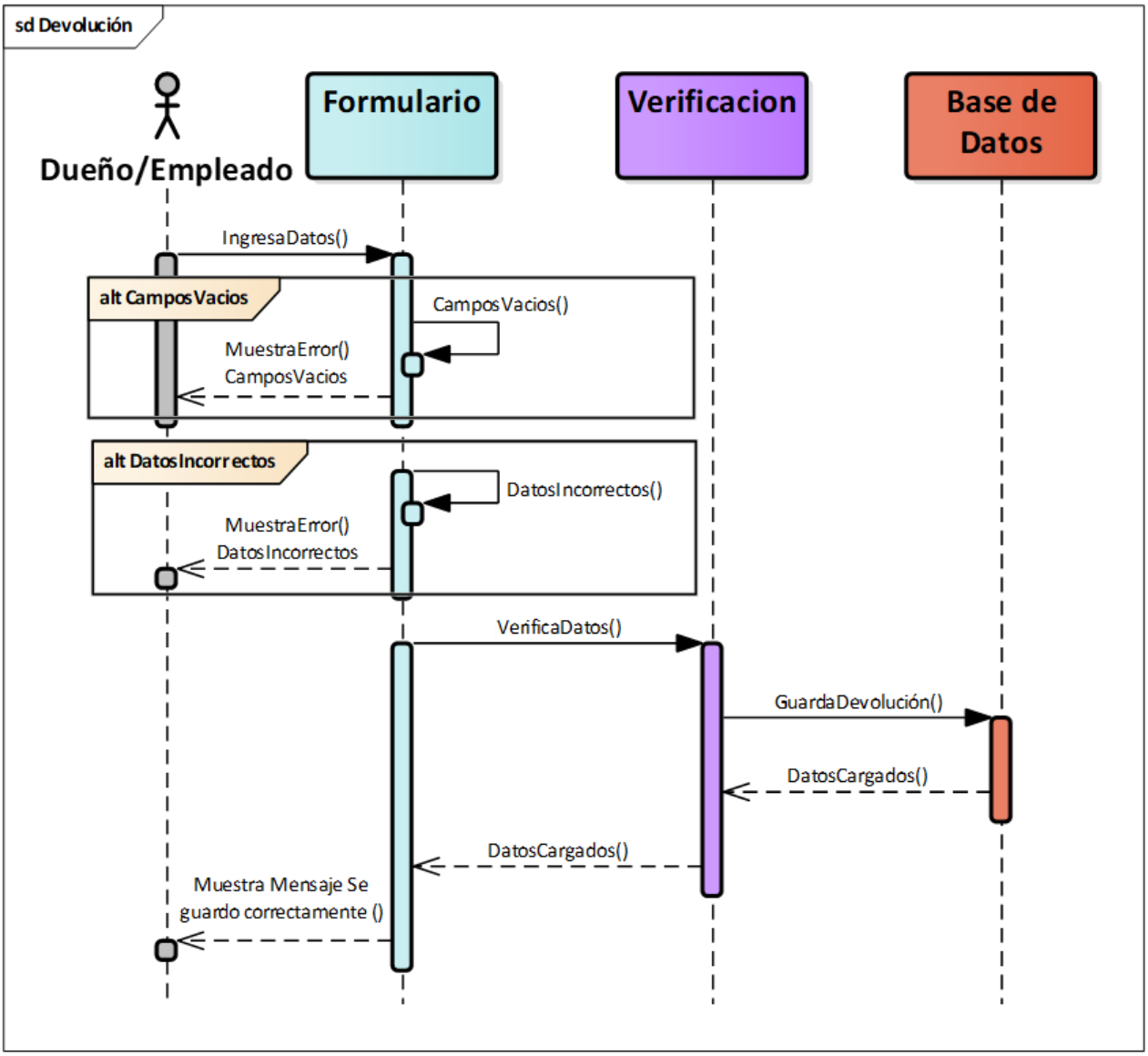


Diagrama de secuencia

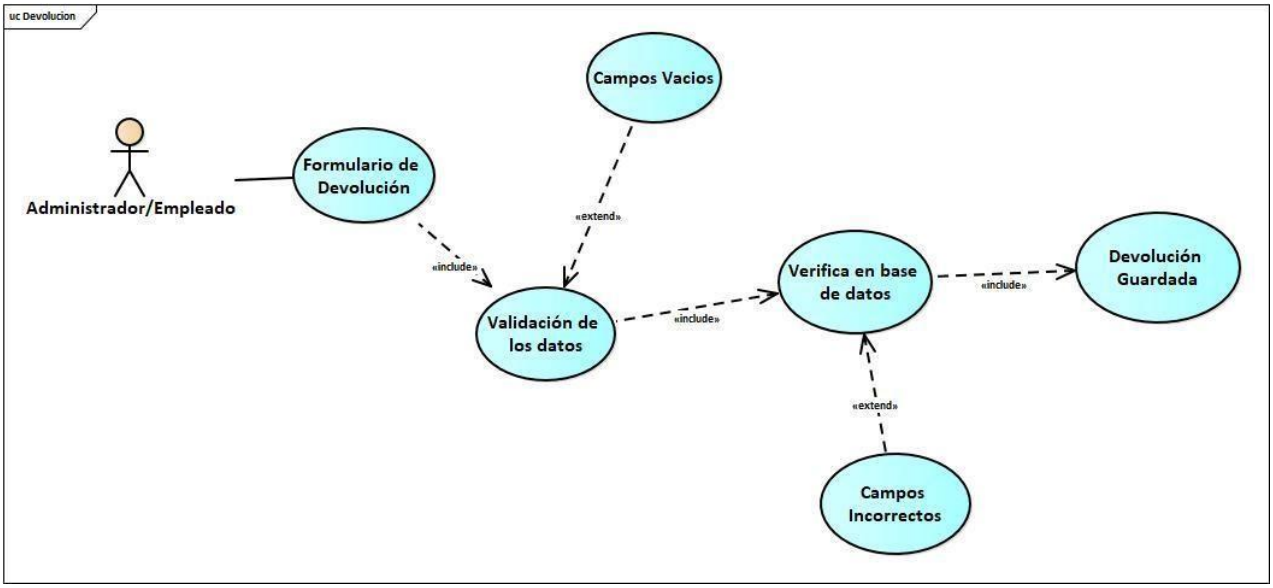
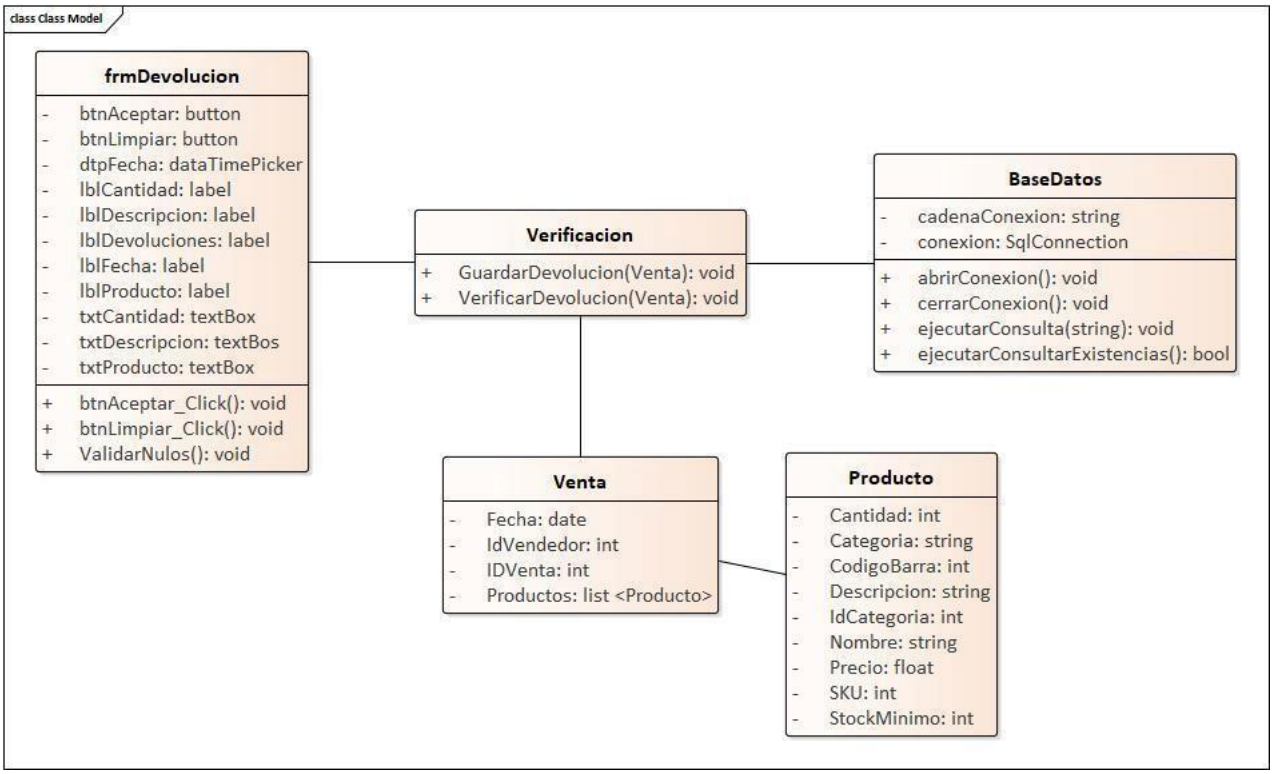


Diagrama de clases



CU08 - Buscar Artículo

ID y Nombre	**CU08 Buscar Artículo**	
Descripción	Permite al dueño o empleado autorizado buscar los artículos por nombre, código de barras o SKU. La búsqueda se usa desde la venta, las compras y la gestión de artículos.	
Actores	Dueño, empleado autorizado.	
Pre-Condición	El dueño o empleado debe estar logueado con los permisos correspondientes.	
Post-Condición	El sistema muestra en pantalla los datos del producto seleccionado.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario abre la búsqueda de artículos.	
	2. El usuario ingresa el nombre, el código de barras o el SKU y presiona "Buscar".	
	3. El sistema muestra los artículos que coinciden, ocultando los discontinuados.	3.1 Sin resultados: El sistema avisa que el artículo no se encuentra en la base de datos. 3.2 Ver discontinuados: El usuario destilda "Ocultar discontinuo" y el sistema muestra también los artículos discontinuados.
	4. El usuario selecciona el artículo y presiona "Aceptar".	4.1 No se seleccionó artículo: El sistema informa "Seleccione un artículo".

	5. El sistema muestra los datos del artículo seleccionado.	

Diagrama de caso de uso

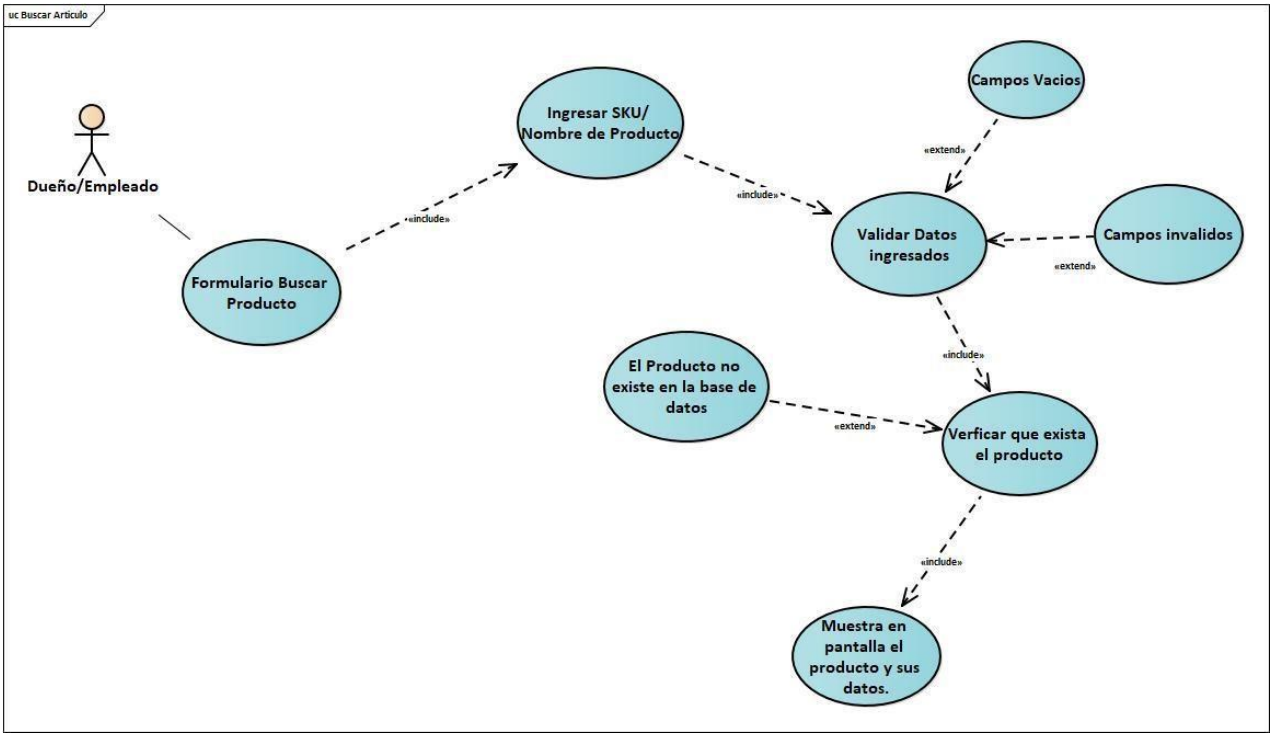


Diagrama de secuencia

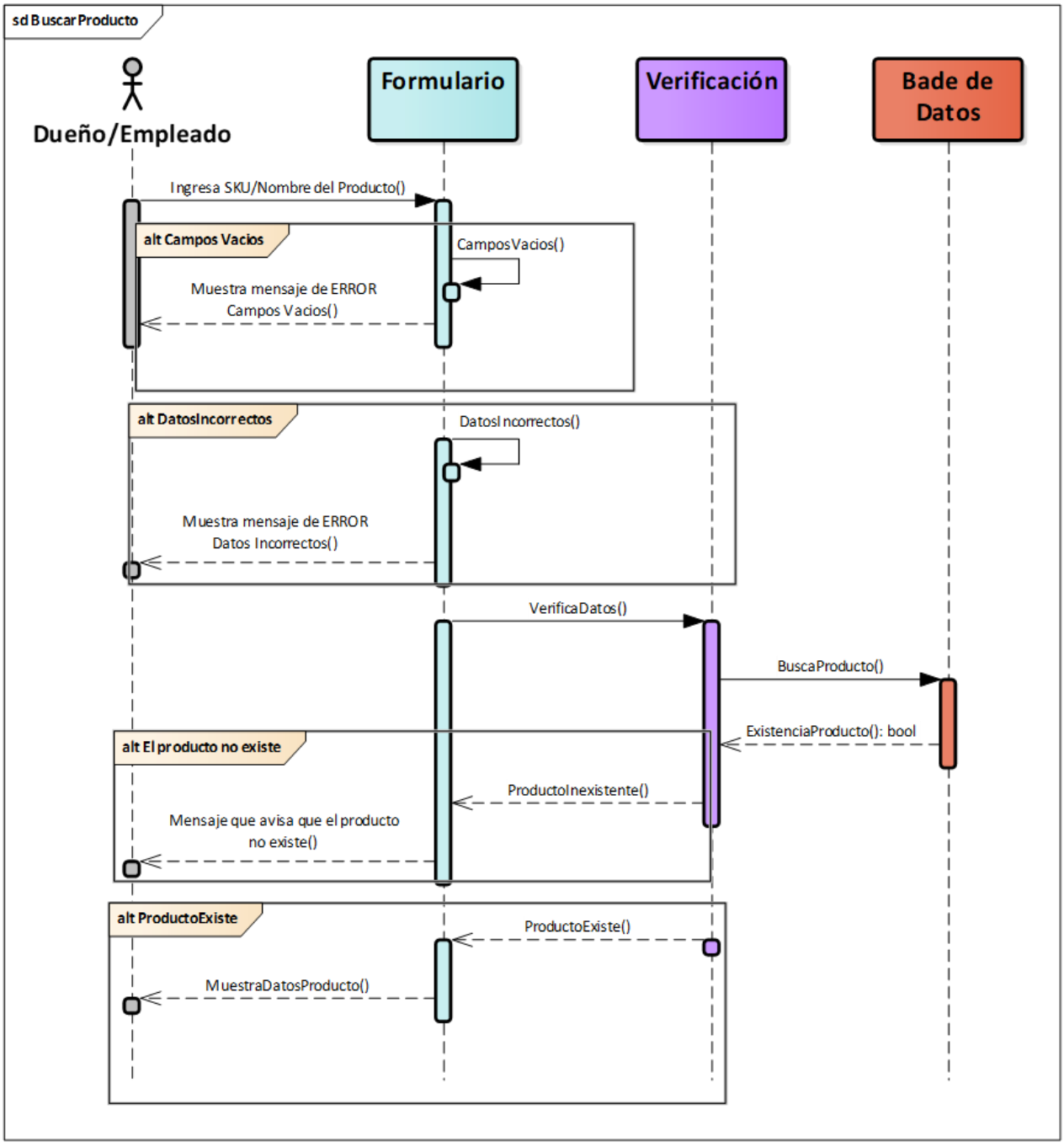
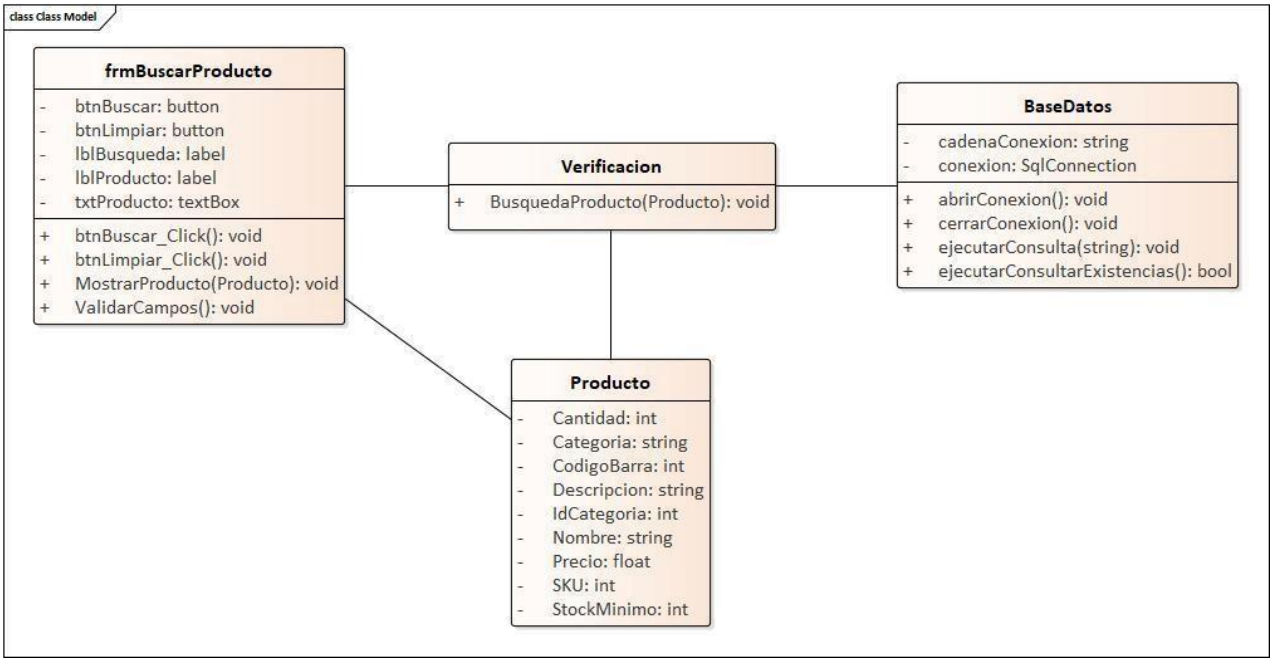


Diagrama de clases



CU09 - Productos Próximos a Vencer

ID y Nombre	CU09 Productos Próximos a Vencer	
Descripción	Permite al dueño o empleado autorizado ver los productos que estén por vencer. Cada lote tiene su fecha de vencimiento y los días de anticipación con los que se quiere el aviso; cuando se llega a esa fecha, el sistema muestra la alerta de tipo vencimiento en el módulo de Alertas.	
Actores	Dueño, empleado autorizado (permiso 11.1.0 Visualizar Alarma).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 11.1.0 Visualizar Alarma. El producto debe tener activadas las alertas de vencimiento y la fecha de vencimiento cargada.	
Post-Condición	El sistema muestra en pantalla la lista de productos próximos a vencer o vencidos.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario presiona la campana de alertas de la pantalla principal.	
	2. El sistema obtiene de la base de datos los lotes con stock cuya fecha de aviso ya llegó.	2.1 No hay productos: En caso de no existir productos próximos a vencer, la lista queda vacía y el contador de alertas no los incluye.
	3. El usuario filtra por el tipo de alerta de vencimiento.	
	4. El sistema muestra cada producto con su lote y la fecha en que vence o venció, marcando como urgentes los vencidos.	

Diagrama de caso de uso

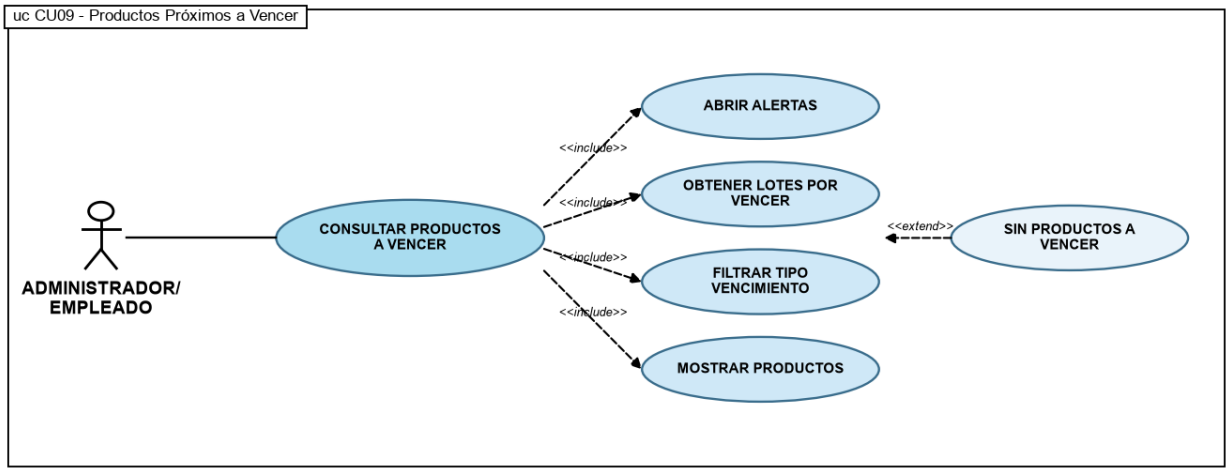


Diagrama de secuencia

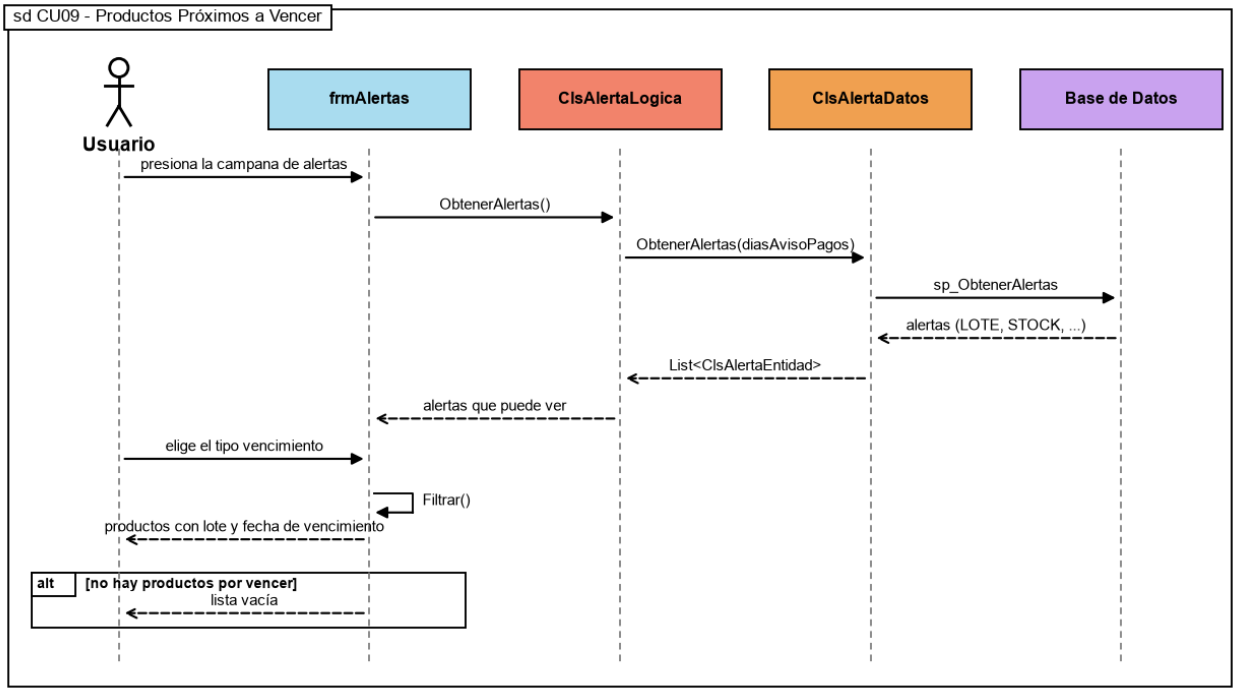
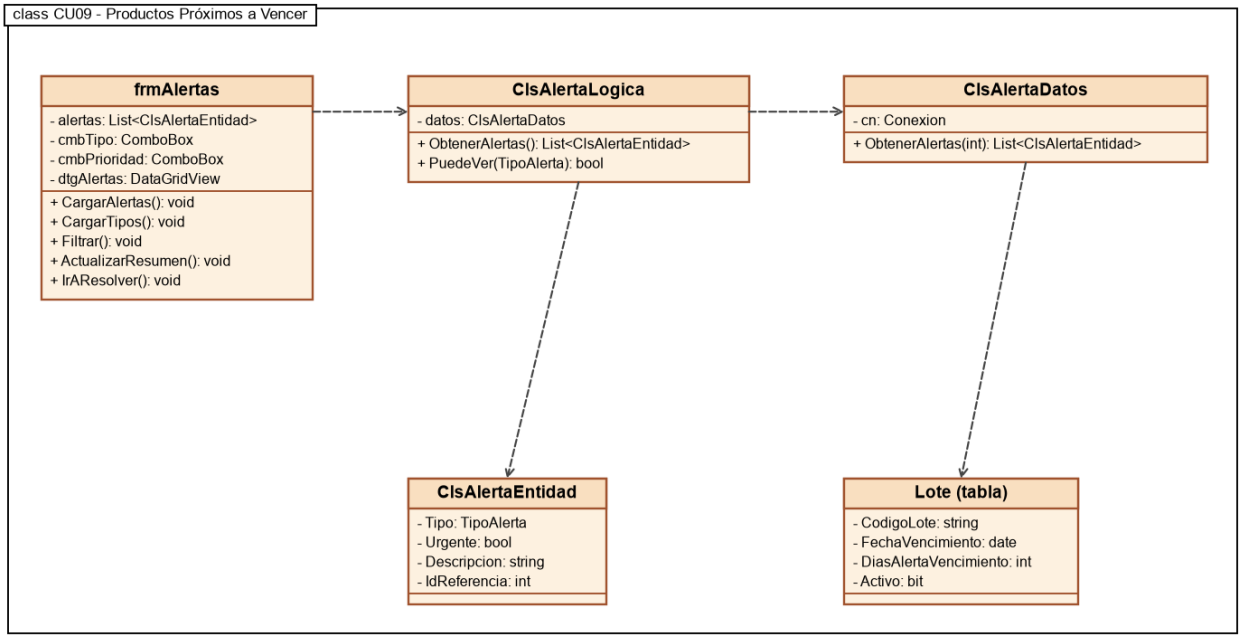


Diagrama de clases



CU10 - Modificar Alerta Stock Mínimo

ID y Nombre	**CU10 Modificar Alerta Stock Mínimo**	
Descripción	Permite al dueño o empleado autorizado modificar el stock mínimo y el stock máximo de cada producto. Cuando el stock actual llega al mínimo, el sistema genera la alerta de stock.	
Actores	Administrador - Empleado autorizado (permiso 1.1.2 Baja/Modificación de Artículos).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 1.1.2. El producto debe tener activado el control de stock.	
Post-Condición	Queda actualizado el stock mínimo y la alerta se dispara según el valor configurado.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Artículos > Modificar artículo.	
	2. El usuario busca el producto a modificar.	2.1 El sistema no encuentra el producto e informa que no se encuentra en la base de datos.
	3. El sistema muestra los datos del producto, incluyendo el stock mínimo para alertas y el stock máximo.	
	4. El usuario modifica el stock mínimo.	
	5. El sistema valida los datos ingresados.	5.1 Campos con datos incorrectos: El sistema informa que revise los valores ingresados.
	6. El usuario confirma la modificación presionando "Guardar".	6.1 Cancelar los cambios del stock mínimo: El usuario presiona "Salir" y no se guardan cambios.

	7. El sistema actualiza el valor en la base de datos.	
	8. El sistema muestra que la modificación fue exitosa.	

Diagrama de caso de uso

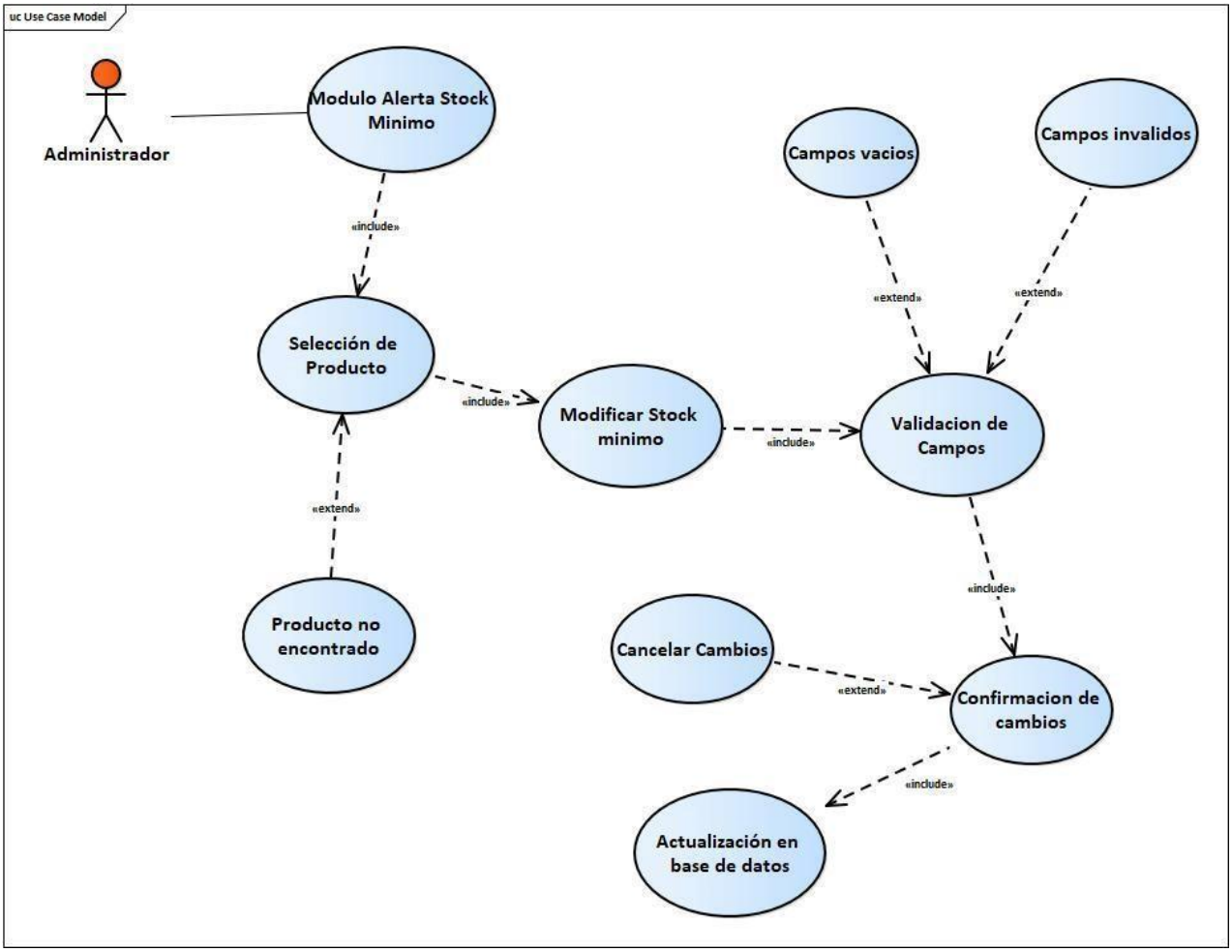


Diagrama de secuencia

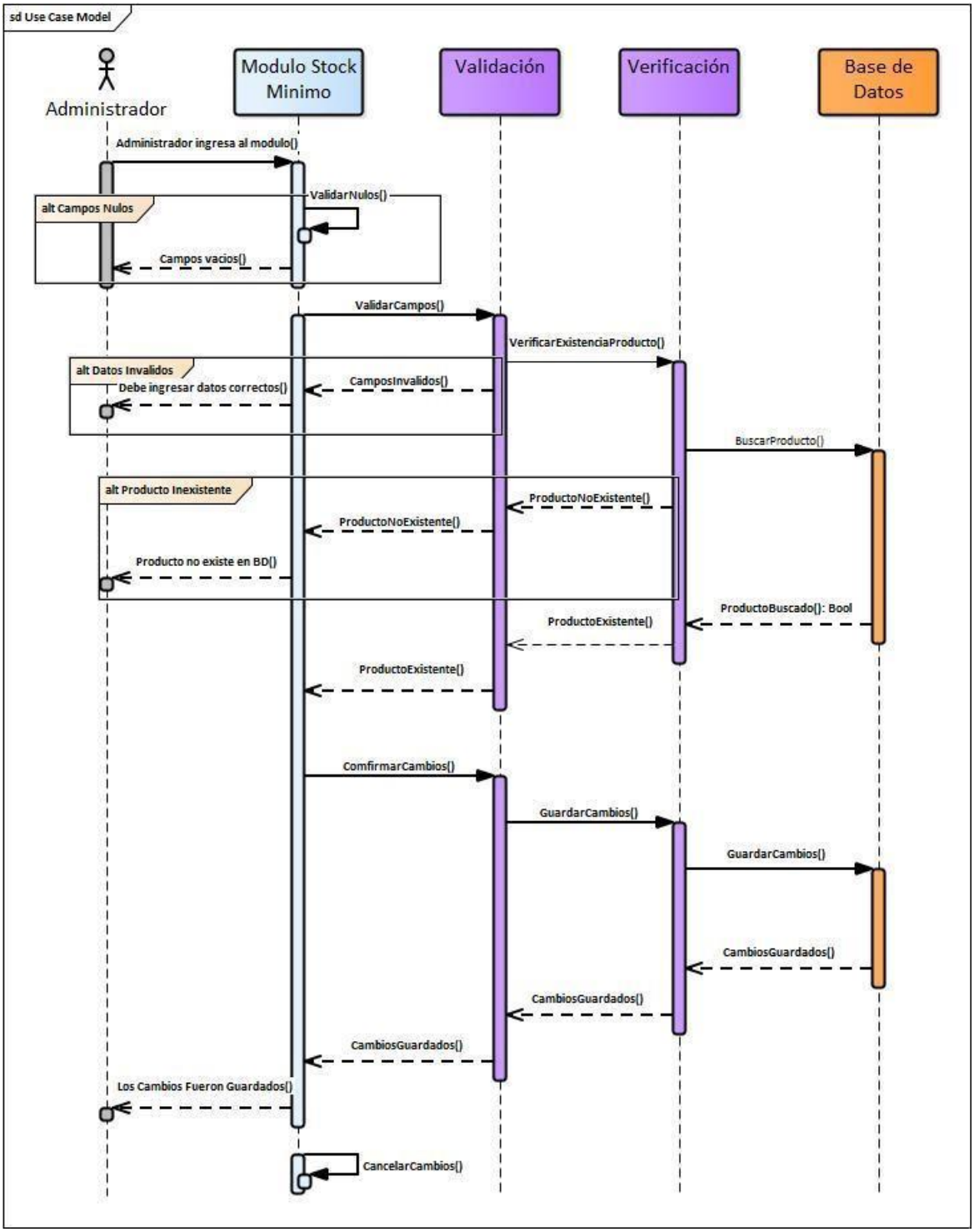
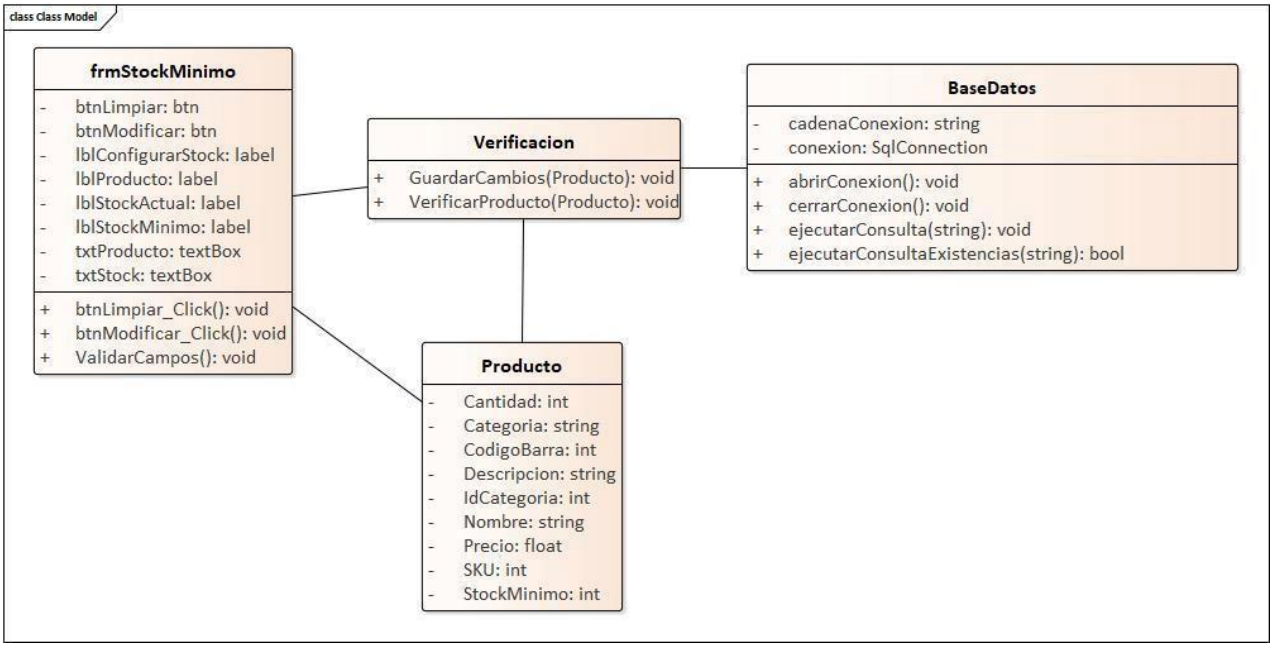


Diagrama de clases



CU11 - Modificar / dar de baja producto

ID y Nombre	**CU11 Modificar / dar de baja producto**	
Descripción	Permite al dueño o empleado autorizado modificar los datos de un producto ya registrado (código, nombre, descripción, marca, rubro, unidad, precios, stock, lote y proveedores) o darlo de baja lógica (discontinuarlo). Un producto discontinuado deja de mostrarse para la venta, pero no se elimina de la base de datos, por lo que se conserva su historial; se puede volver a activar.	
Actores	Administrador - Empleado autorizado (permiso 1.1.2 Baja/Modificación de Artículos).	
Pre-Condición	El usuario debe haber iniciado sesión y poseer el permiso 1.1.2 Baja/Modificación de Artículos. El producto debe estar registrado en el sistema.	
Post-Condición	Los datos del producto quedan actualizados, o el producto queda en estado Discontinuado. Se recalcula el dígito verificador del producto y la operación queda registrada en la bitácora.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Artículos > Modificar artículo.	
	2. El usuario busca el producto por código o nombre, o lo filtra por rubro o marca (CU08 Buscar Artículo).	2.1 Producto no encontrado: El sistema informa que el producto no se encuentra registrado.
	3. El usuario selecciona el producto.	
	4. El usuario presiona "Modificar articulo" y el sistema muestra sus datos.	4.1 Discontinuar producto: El usuario presiona "Discontinuar", confirma y el sistema marca el producto como inactivo. Con "Mostrar artículos discontinuados" puede volver a activarlo. 4.2 Eliminar producto: El usuario presiona "Eliminar" y el sistema solicita confirmación antes de eliminarlo.

	5. El usuario modifica los datos necesarios.	
	6. El sistema valida los datos ingresados.	6.1 Campos vacíos o datos incorrectos: El sistema informa que complete los campos obligatorios y revise los valores ingresados. 6.2 Código de barras duplicado: El sistema detecta que el código de barras pertenece a otro producto e informa al usuario que ingrese uno distinto.
	7. El usuario confirma la modificación presionando "Guardar".	7.1 Cancelar los cambios: El usuario presiona "Salir" y el sistema descarta los cambios.
	8. El sistema actualiza el producto en la base de datos y recalcula el dígito verificador.	
	9. El sistema registra la operación en la bitácora.	
	10. El sistema informa que la operación se realizó con éxito.	

Diagrama de caso de uso

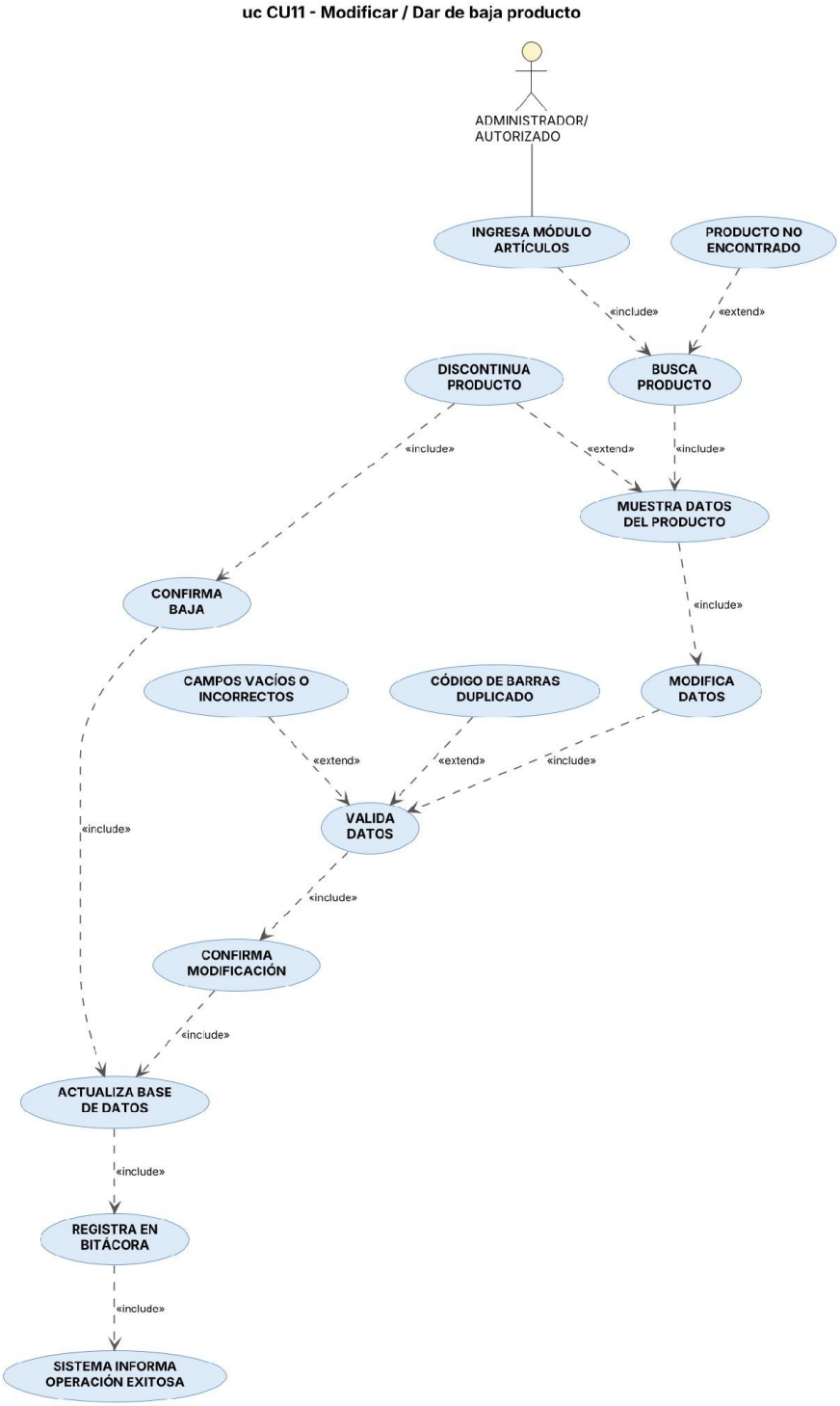


Diagrama de secuencia

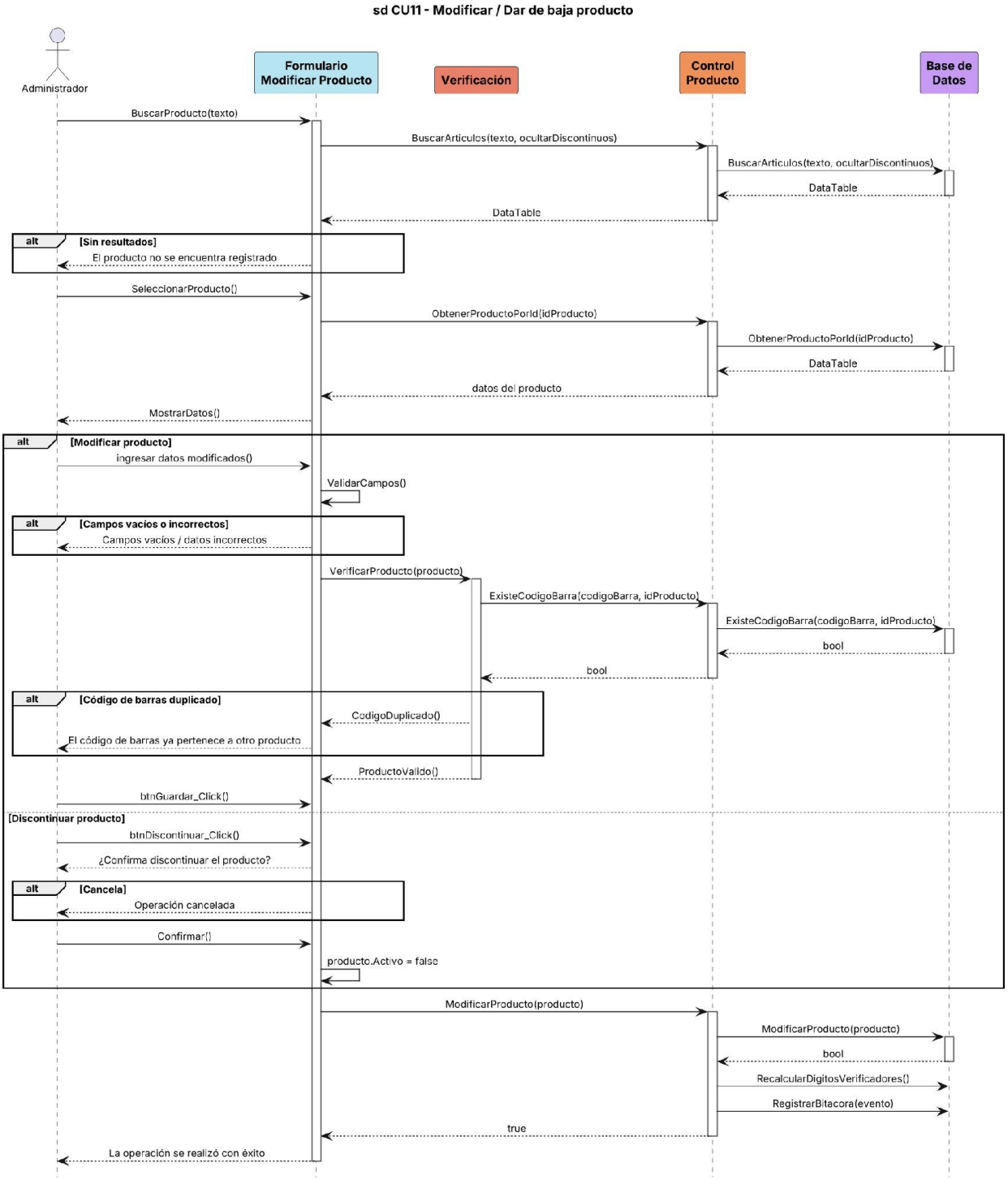
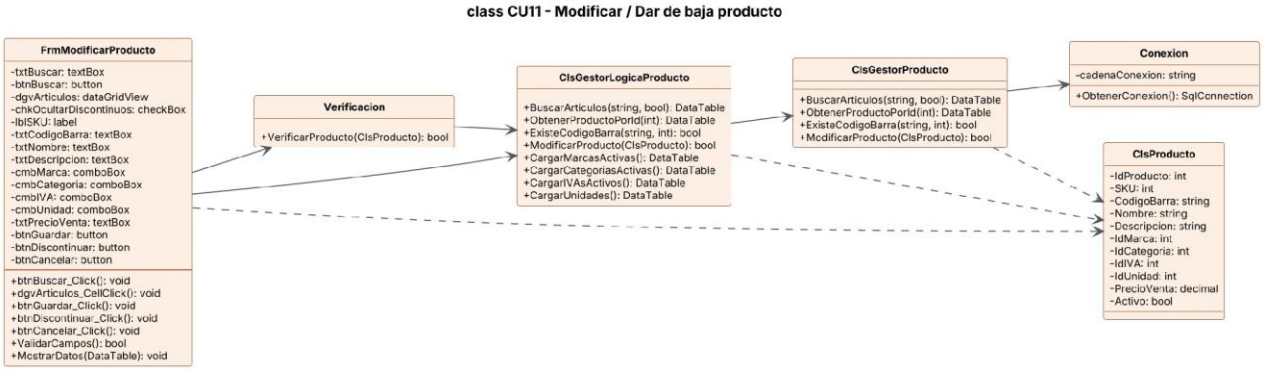


Diagrama de clases



CU12 - Cargar stock a artículo existente

ID y Nombre	CU12 Cargar stock a artículo existente	
Descripción	Permite al dueño o empleado autorizado sumar unidades al stock de productos que ya están registrados desde la pantalla de Gestión rápida de productos. Los productos se filtran por proveedor, sucursal, categoría o marca, o se buscan por código, SKU o nombre, y en la grilla se carga la cantidad a reponer de cada uno. Desde la misma pantalla se pueden actualizar los precios, uno por uno o aplicando un aumento por porcentaje.	
Actores	Administrador - Empleado autorizado (permiso 1.1.3 Carga de Artículos).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 1.1.3 Carga de Artículos. Los productos deben estar registrados y activos.	
Post-Condición	El stock de los productos queda actualizado.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Artículos > Reposición rápida de stock.	
	2. El usuario elige "Reponer stock".	2.1 Actualizar precios: El usuario elige "Actualizar precios", edita el precio en la grilla o aplica un aumento por porcentaje y guarda.
	3. El usuario filtra por proveedor, sucursal, categoría o marca, o busca por código, SKU o nombre.	
	4. El sistema muestra los productos con su stock actual.	
	5. El usuario ingresa la cantidad a reponer de cada producto.	

	6. El usuario presiona "Guardar cambios".	6.1 Cancelar: Si hay cambios sin guardar, el sistema pregunta si desea salir igual.
	7. El sistema valida las cantidades ingresadas.	7.1 Cantidad inválida: El sistema informa que ingrese una cantidad válida para reponer. 7.2 Sin cantidades: El sistema informa que no hay cantidades para reponer.
	8. El sistema actualiza el stock de cada producto.	
	9. El sistema informa que el stock se actualizó correctamente.	

Diagrama de caso de uso

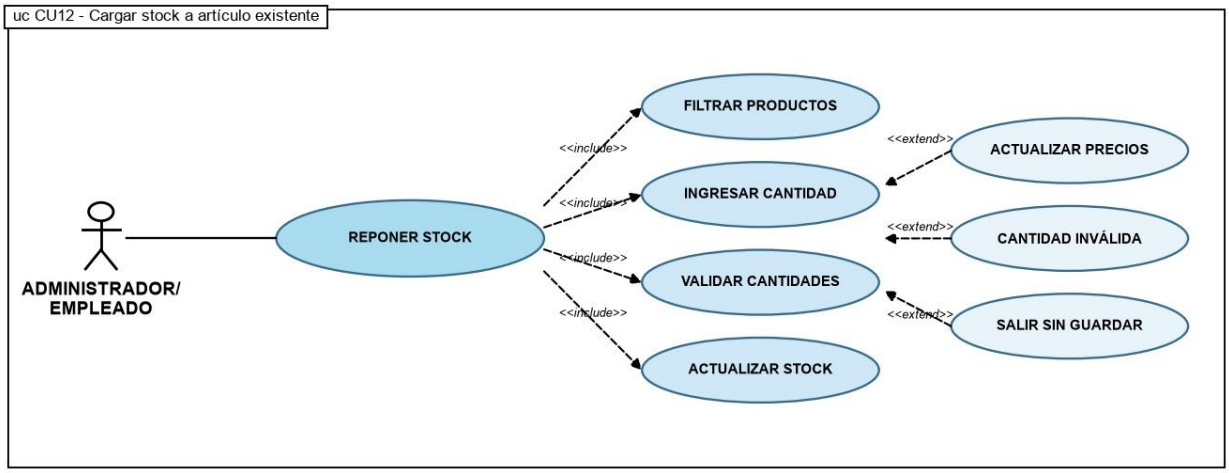


Diagrama de secuencia

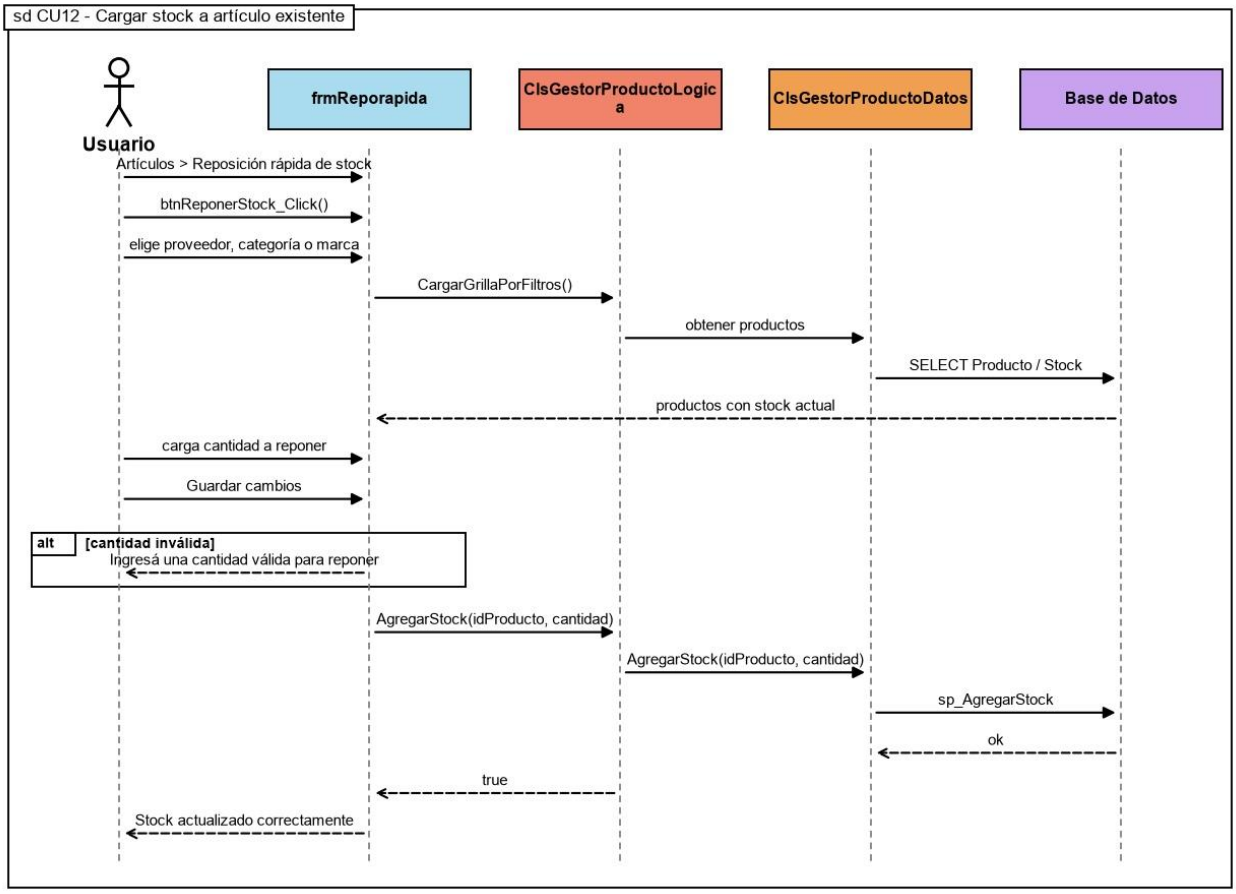
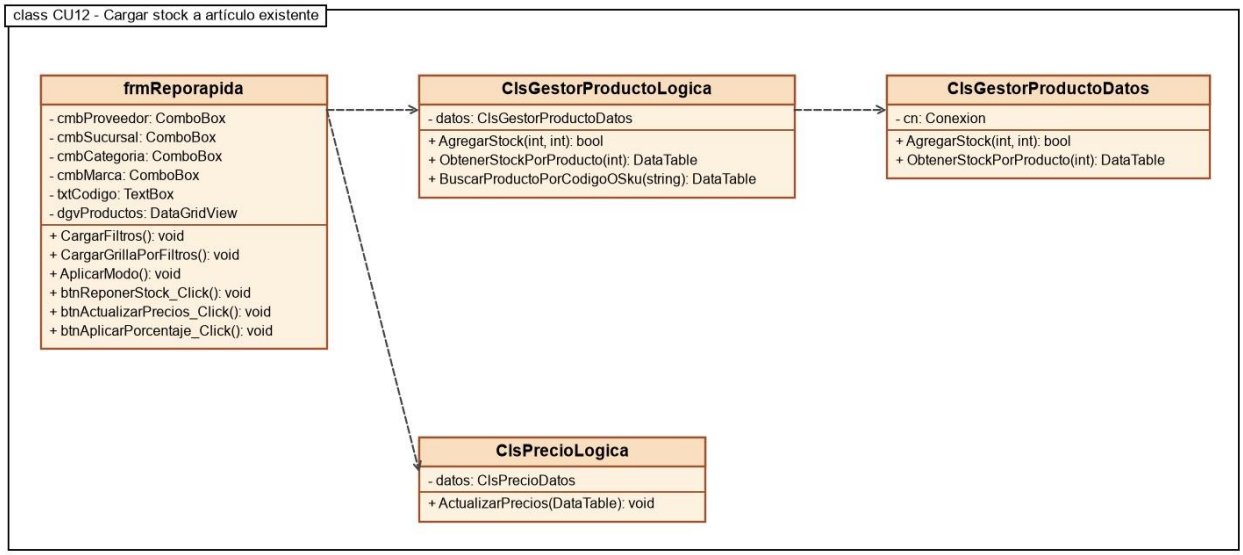


Diagrama de clases



CU13 - Gestionar rubros

ID y Nombre	**CU13 Gestionar rubros**	
Descripción	Permite al dueño o empleado autorizado administrar los rubros (categorías) con los que se clasifican los productos: dar de alta un rubro nuevo, modificar su nombre, discontinuarlo, volver a habilitarlo o eliminarlo. Los rubros y las marcas se administran desde la misma pantalla. Los rubros discontinuados no se eliminan, solo dejan de ofrecerse al cargar productos.	
Actores	Administrador - Empleado autorizado (permiso 1.2.0 Gestión de Rubros).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 1.2.0 Gestión de Rubros (o 1.2.1 / 1.2.2 según la operación).	
Post-Condición	El rubro queda registrado, modificado, discontinuado, habilitado o eliminado según la operación elegida.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Artículos > Categorías y marcas.	
	2. El usuario elige "Categorías".	
	3. El sistema muestra el listado de rubros con su estado (Activo / Discontinuado).	
	4. El usuario presiona "Nuevo".	4.1 Modificar rubro: El usuario selecciona un rubro, presiona "Modificar" y edita el nombre; continúa en el paso 6. 4.2 Discontinuar / Habilitar rubro: El usuario selecciona el rubro y presiona "Discontinuar" o "Habilitar"; el sistema pide confirmación y continúa en el paso 7. 4.3 Eliminar rubro:

		El usuario presiona "Eliminar" y confirma; las categorías protegidas no se pueden eliminar.
	5. El usuario ingresa el nombre del rubro y presiona "Aceptar".	5.1 Cancelar: El usuario presiona "Volver" y el sistema descarta los cambios.
	6. El sistema valida los datos ingresados.	6.1 Campo vacío: El sistema informa "Ingrese el nombre del rubro".
	7. El sistema guarda los cambios en la base de datos.	
	9. El sistema informa que la operación se realizó con éxito y actualiza el listado.	

Diagrama de caso de uso

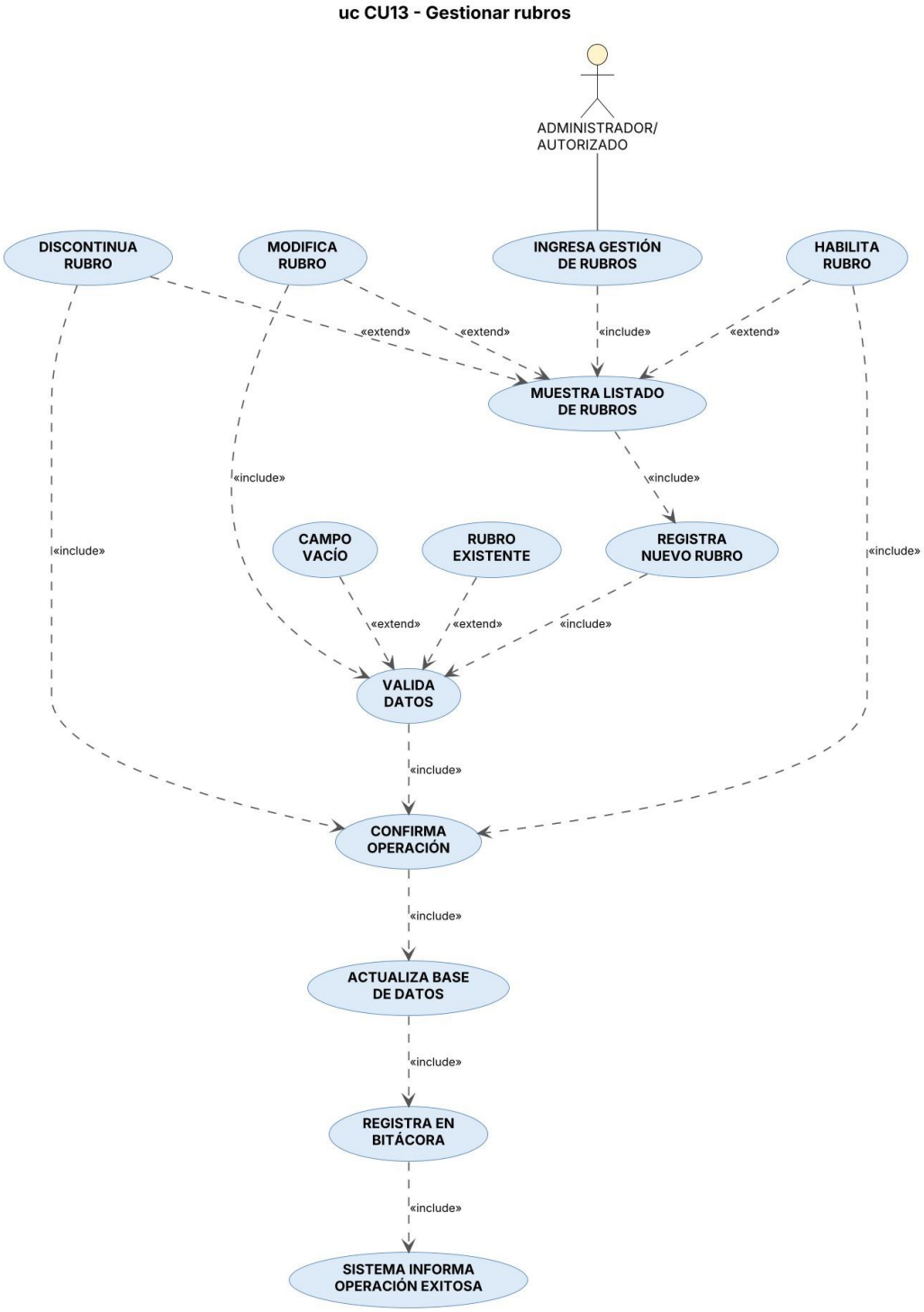


Diagrama de secuencia

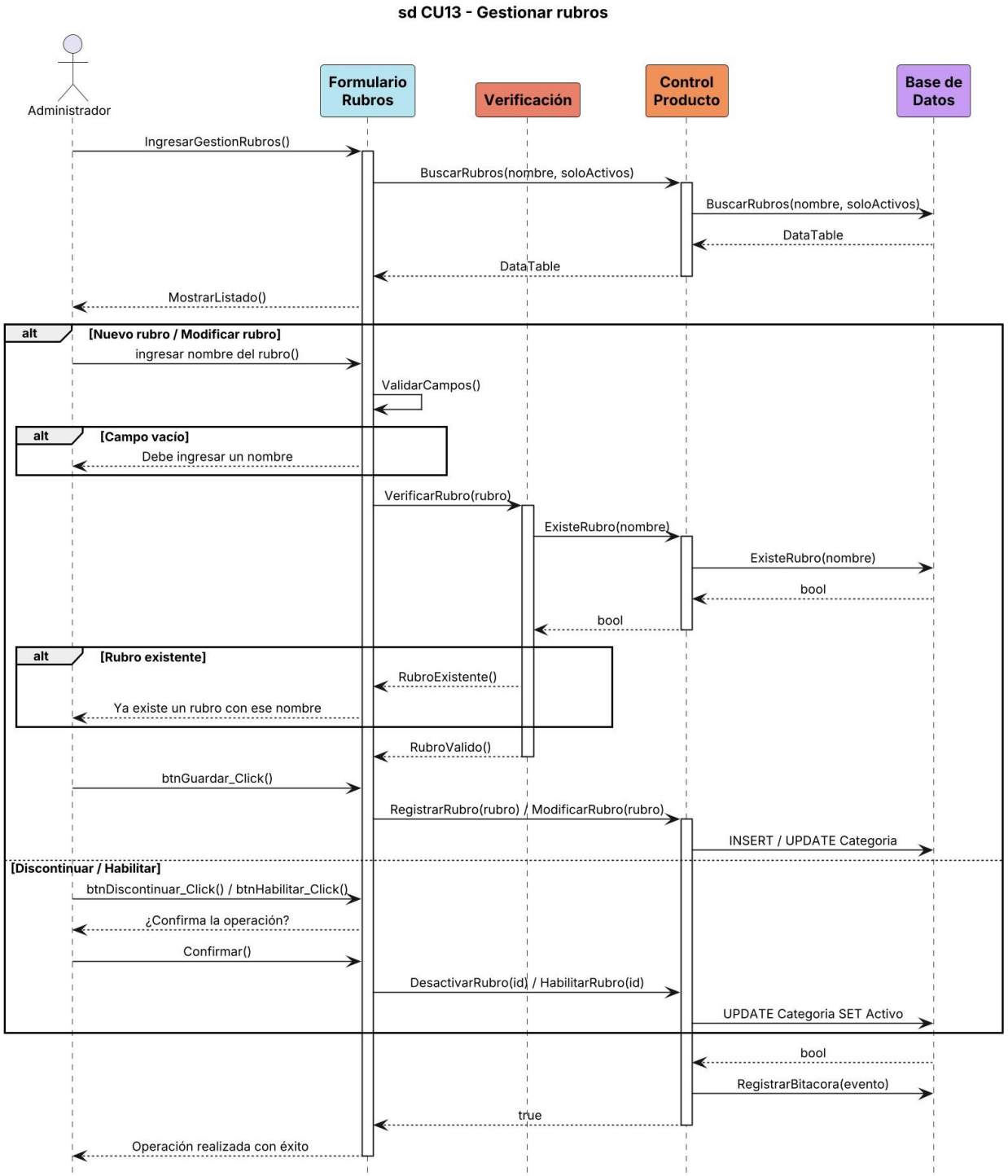
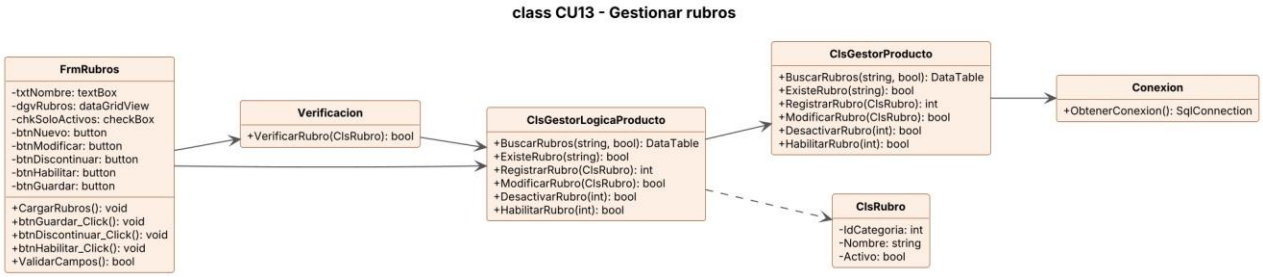


Diagrama de clases



CU14 - Gestionar marcas

ID y Nombre	**CU14 Gestionar marcas**	
Descripción	Permite al dueño o empleado autorizado administrar las marcas de los productos: registrar una marca nueva, modificarla, discontinuarla, volver a habilitarla o eliminarla. Se administran desde la misma pantalla que los rubros. Las marcas discontinuadas no se eliminan; se pueden ver destildando "Mostrar solo activos".	
Actores	Administrador - Empleado autorizado (permiso 1.3.0 Gestión de Marcas).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 1.3.0 Gestión de Marcas (o 1.3.1 / 1.3.2 según la operación).	
Post-Condición	La marca queda registrada, modificada, discontinuada, habilitada o eliminada según la operación elegida.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Artículos > Categorías y marcas.	
	2. El usuario elige "Marcas".	
	3. El sistema muestra el listado de marcas con su estado (Activo / Discontinuado).	
	4. El usuario busca la marca por nombre, pudiendo filtrar solo las activas.	
	5. El usuario presiona "Nuevo".	5.1 Modificar marca: El usuario selecciona una marca y presiona "Modificar"; continúa en el paso 6. 5.2 Discontinuar marca:

		El usuario selecciona una marca activa y presiona "Discontinuar"; el sistema pregunta "¿Está seguro de discontinuar la marca?" y continúa en el paso 8. 5.3 Habilitar marca: El usuario selecciona una marca discontinuada y presiona "Habilitar"; continúa en el paso 8.
	6. El usuario ingresa el nombre de la marca y presiona "Aceptar".	6.1 Cancelar: El usuario presiona "Volver" y el sistema descarta los cambios.
	7. El sistema valida los datos ingresados.	7.1 Campo vacío: El sistema informa "Ingrese el nombre de la marca".
	8. El sistema guarda los cambios en la base de datos.	
	10. El sistema informa que la operación se realizó con éxito y actualiza el listado.	

Diagrama de caso de uso

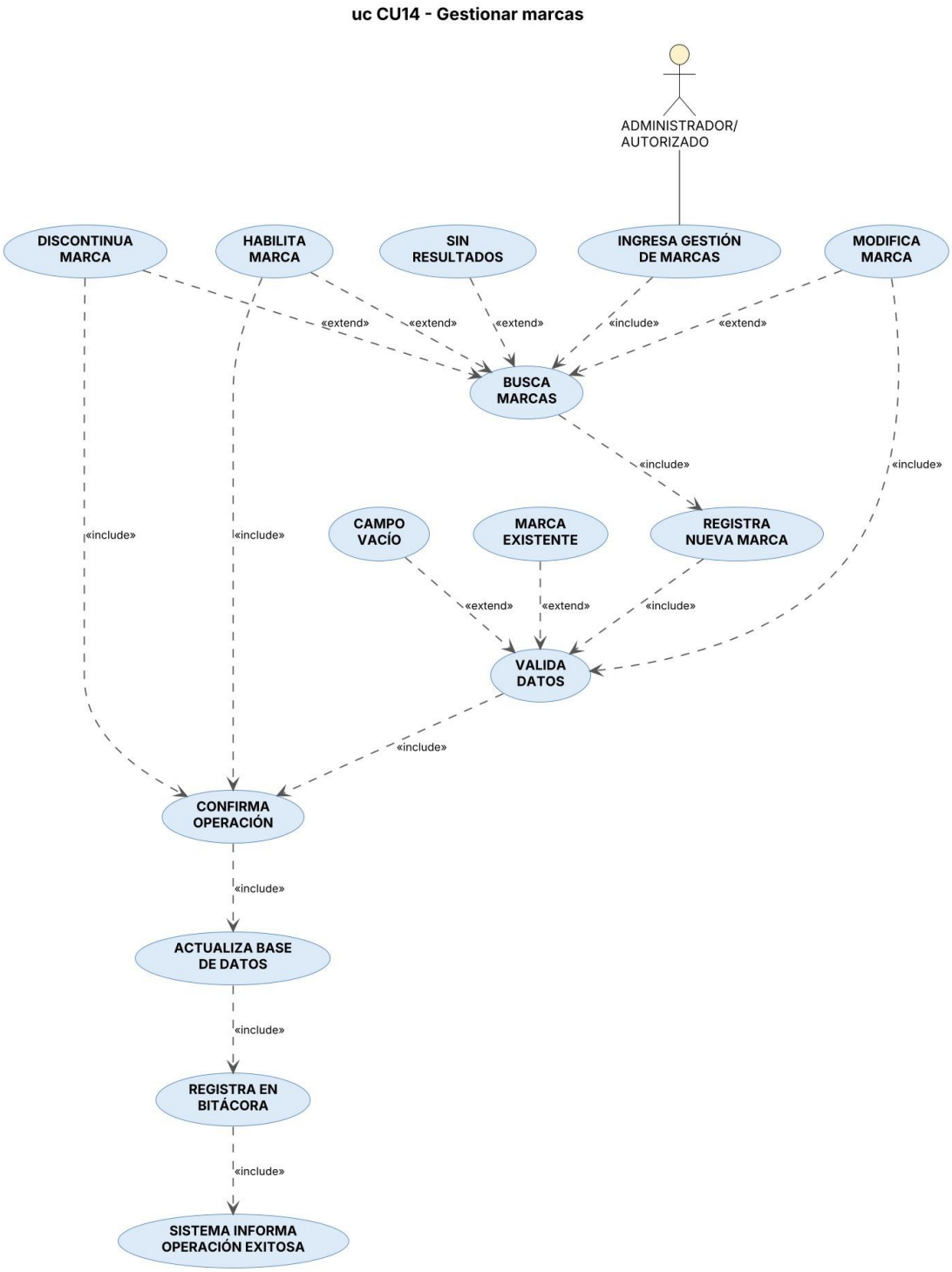


Diagrama de secuencia

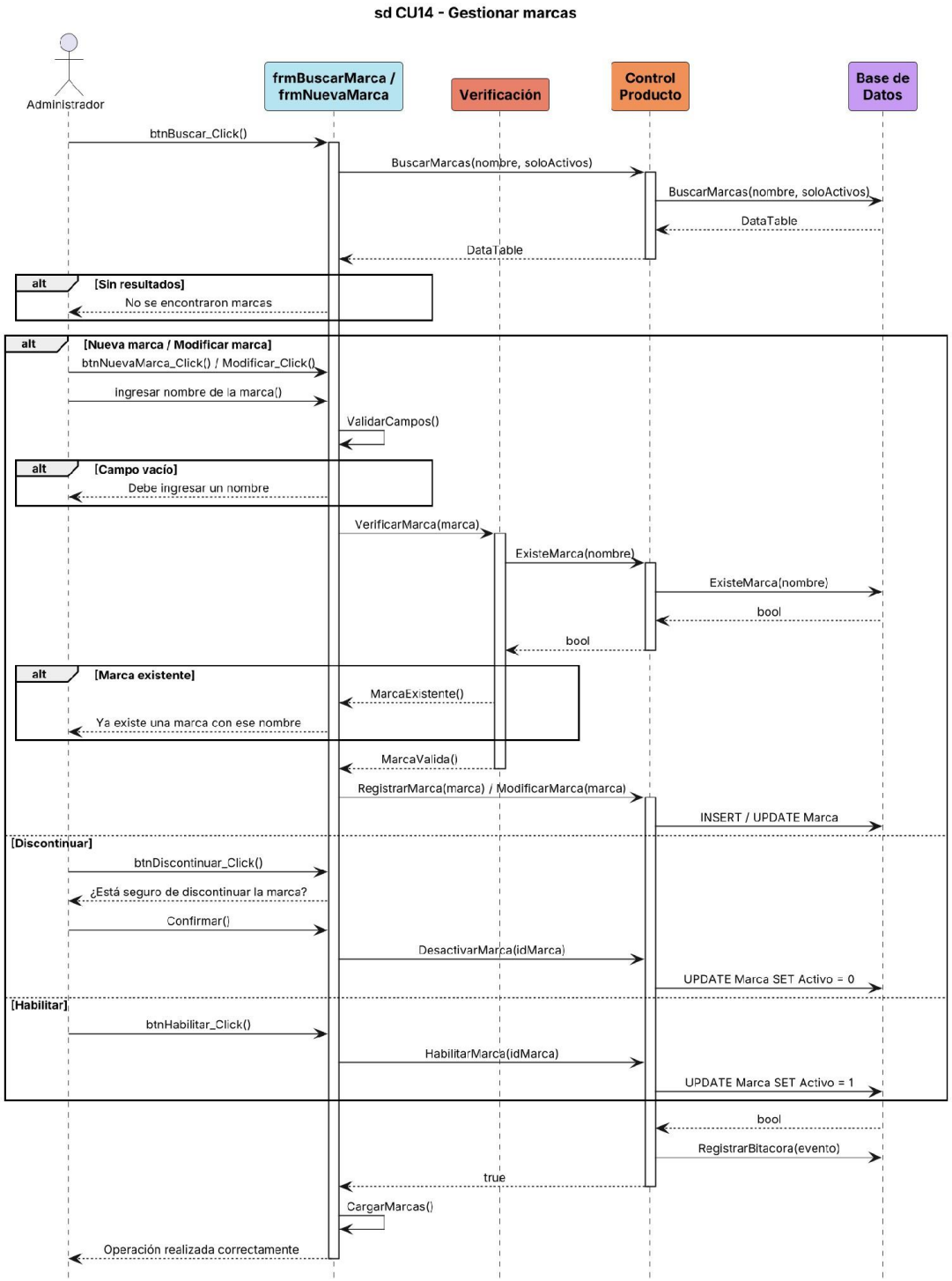
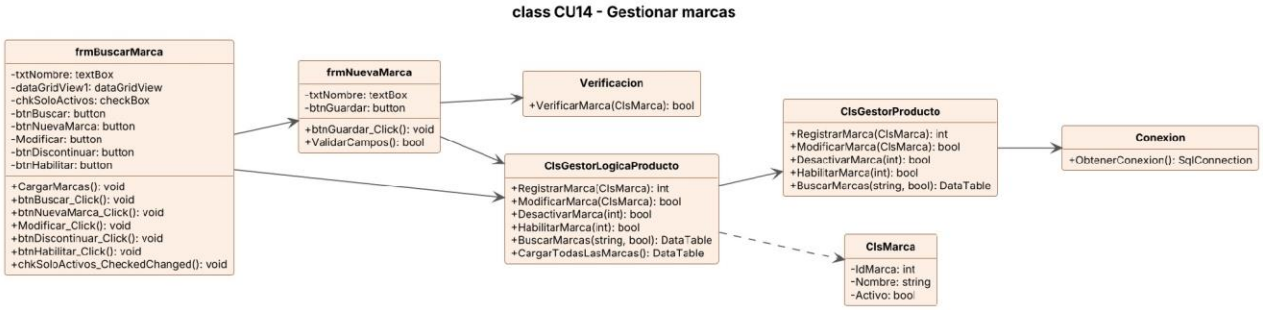


Diagrama de clases



CU15 - Importar artículos desde Excel

ID y Nombre	**CU15 Importar artículos desde Excel**	
Descripción	Permite al dueño o empleado autorizado cargar varios artículos de una sola vez desde un archivo Excel. El sistema ofrece una planilla modelo con las hojas ARTICULOS y PROVEEDORES_COSTOS, lee cada fila del archivo, valida los datos y registra los productos válidos junto con su stock, lote y proveedores. Al finalizar informa cuántos artículos se importaron y cuáles filas tuvieron errores.	
Actores	Administrador - Empleado autorizado (permiso 1.4.0 Importar Artículos desde Excel).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 1.4.0. El archivo debe estar en formato Excel (.xlsx) y respetar la estructura de la planilla modelo.	
Post-Condición	Los artículos válidos quedan registrados con su stock, lote y relación con el proveedor. Las filas con errores no se importan y se informan al usuario.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Artículos > Importar artículos desde Excel.	
	2. El usuario presiona "Seleccionar" y elige el archivo Excel.	2.1 Descargar planilla modelo: El usuario presiona "Descargar Planilla"; el sistema genera la plantilla y la guarda en la ruta elegida. 2.2 No se seleccionó archivo: El sistema informa "Debe seleccionar un archivo de Excel".
	3. El usuario presiona "Aceptar" para iniciar la importación.	
	4. El sistema abre el archivo y lee las hojas ARTICULOS y PROVEEDORES_COSTOS.	4.1 Archivo inválido: El sistema informa que el archivo no existe, que no contiene la hoja ARTICULOS o PROVEEDORES_COSTOS, o que no pudo leerlo.

	5. El sistema valida los datos de cada fila.	5.1 Fila con errores: El sistema omite la fila y guarda el motivo para el informe final. 5.2 Sin artículos: El sistema informa que no se encontraron artículos para importar.
	6. El sistema registra cada producto válido con su stock, lote y proveedores.	
	7. El sistema registra la importación en la bitácora.	
	8. El sistema informa la cantidad de artículos importados y el detalle de las filas con errores.	

Diagrama de caso de uso

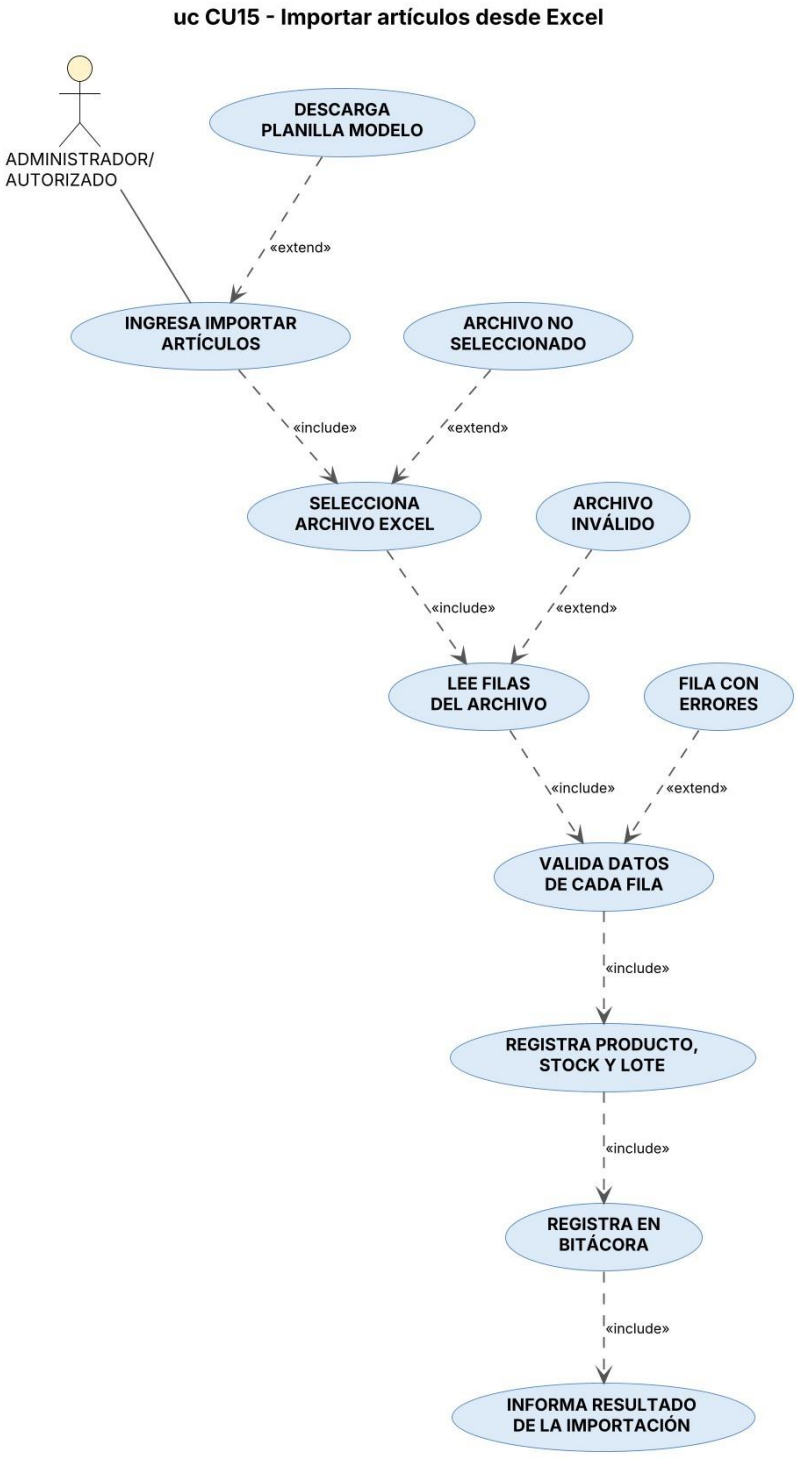


Diagrama de secuencia

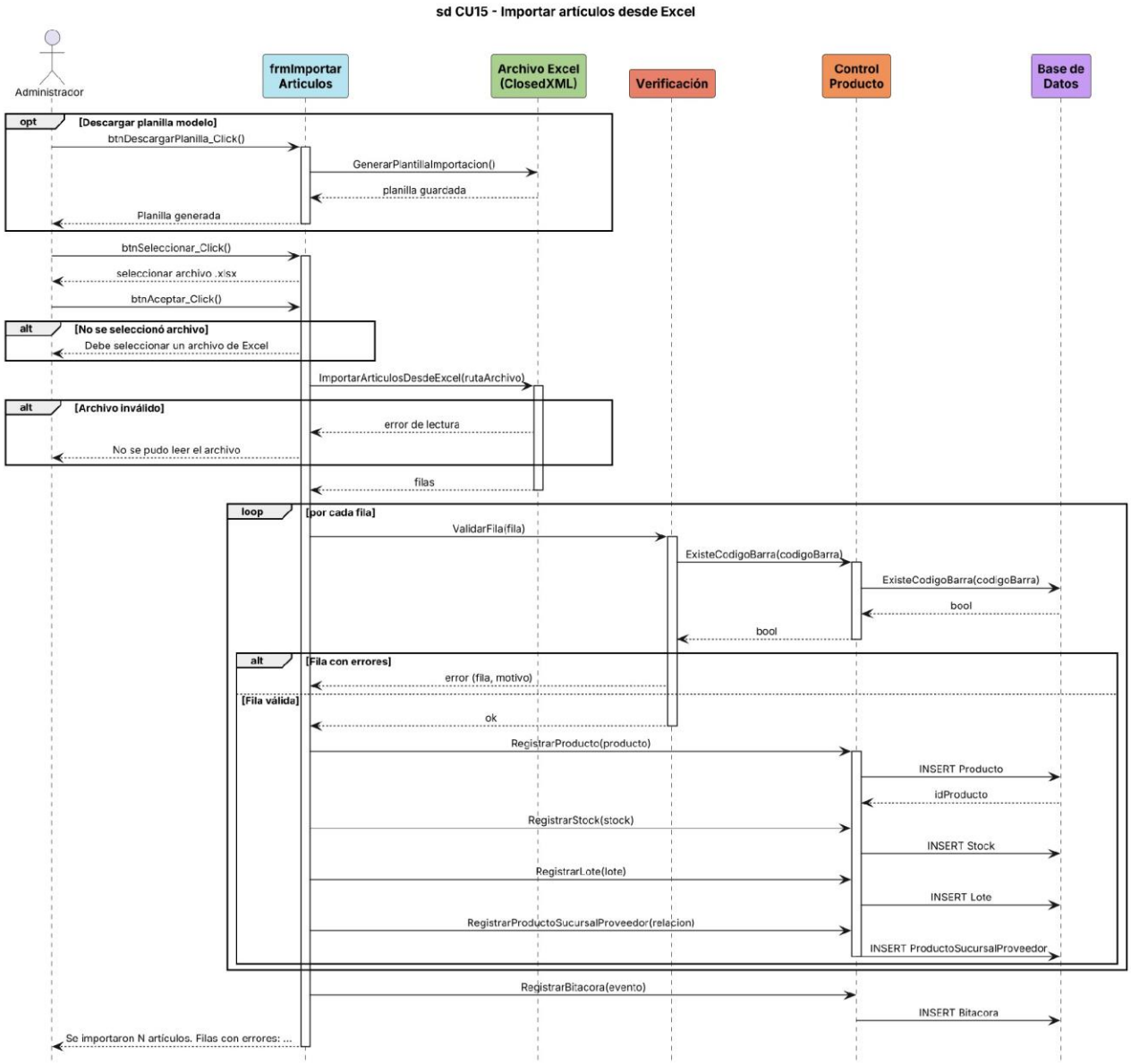
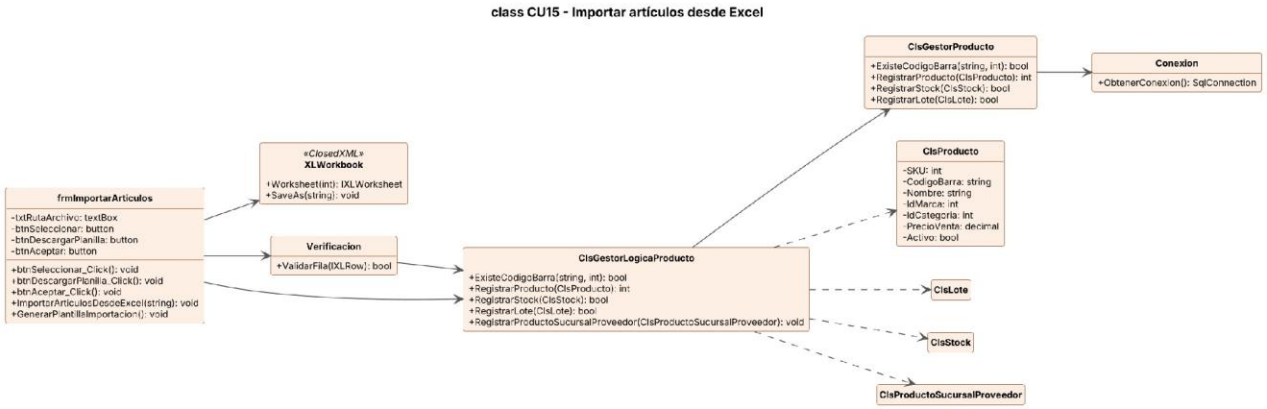


Diagrama de clases



CU16 - Exportar artículos a Excel

ID y Nombre	**CU16 Exportar artículos a Excel**	
Descripción	Permite al dueño o empleado autorizado generar un archivo Excel con el listado de artículos, filtrado por proveedor, sucursal del proveedor, rubro, marca y estado, y opcionalmente solo los artículos con stock, con vencimiento o con control de lote. Sirve para controlar precios y stock fuera del sistema o para enviar listados a proveedores.	
Actores	Administrador (permiso 1.5.0 Exportar Artículos a Excel).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 1.5.0. Debe haber artículos registrados.	
Post-Condición	Se genera un archivo Excel con los artículos que cumplen los filtros, en la ubicación elegida por el usuario.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Artículos > Exportar artículos a Excel.	
	2. El sistema carga los filtros disponibles: proveedores, rubros, marcas y estados.	
	3. El usuario selecciona los filtros deseados (al elegir un proveedor, el sistema carga sus sucursales).	
	4. El usuario presiona "Exportar".	
	5. El sistema obtiene los artículos que cumplen los filtros.	5.1 Sin resultados: El sistema informa que no se encontraron artículos con los filtros seleccionados.

	6. El usuario elige la carpeta y el nombre del archivo.	6.1 Cancelar: El sistema no genera el archivo y vuelve al paso 3.
	7. El sistema genera el archivo Excel.	7.1 Error al guardar: El sistema informa que no pudo guardar el archivo (por ejemplo, porque está abierto).
	8. El sistema informa que el archivo se exportó correctamente y la cantidad de artículos exportados.	

Diagrama de caso de uso

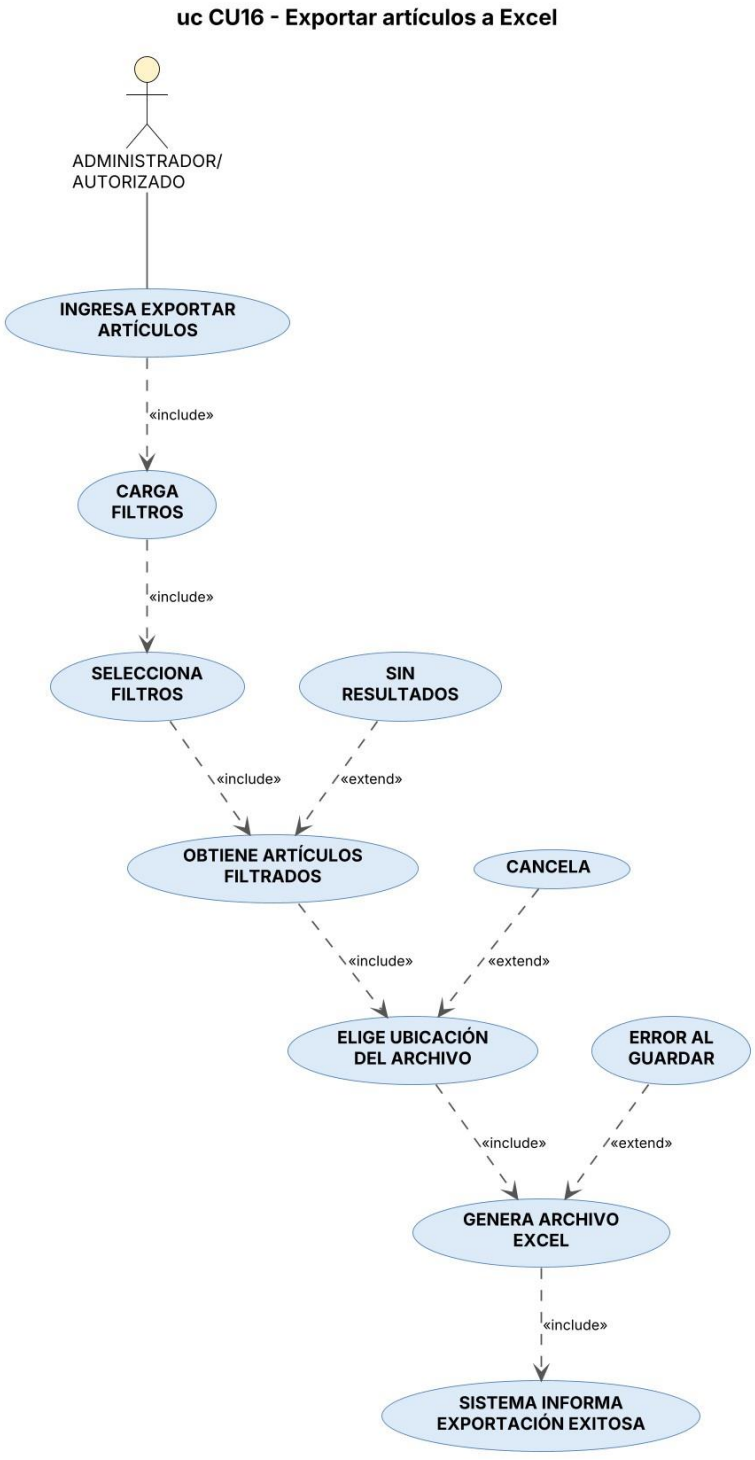


Diagrama de secuencia

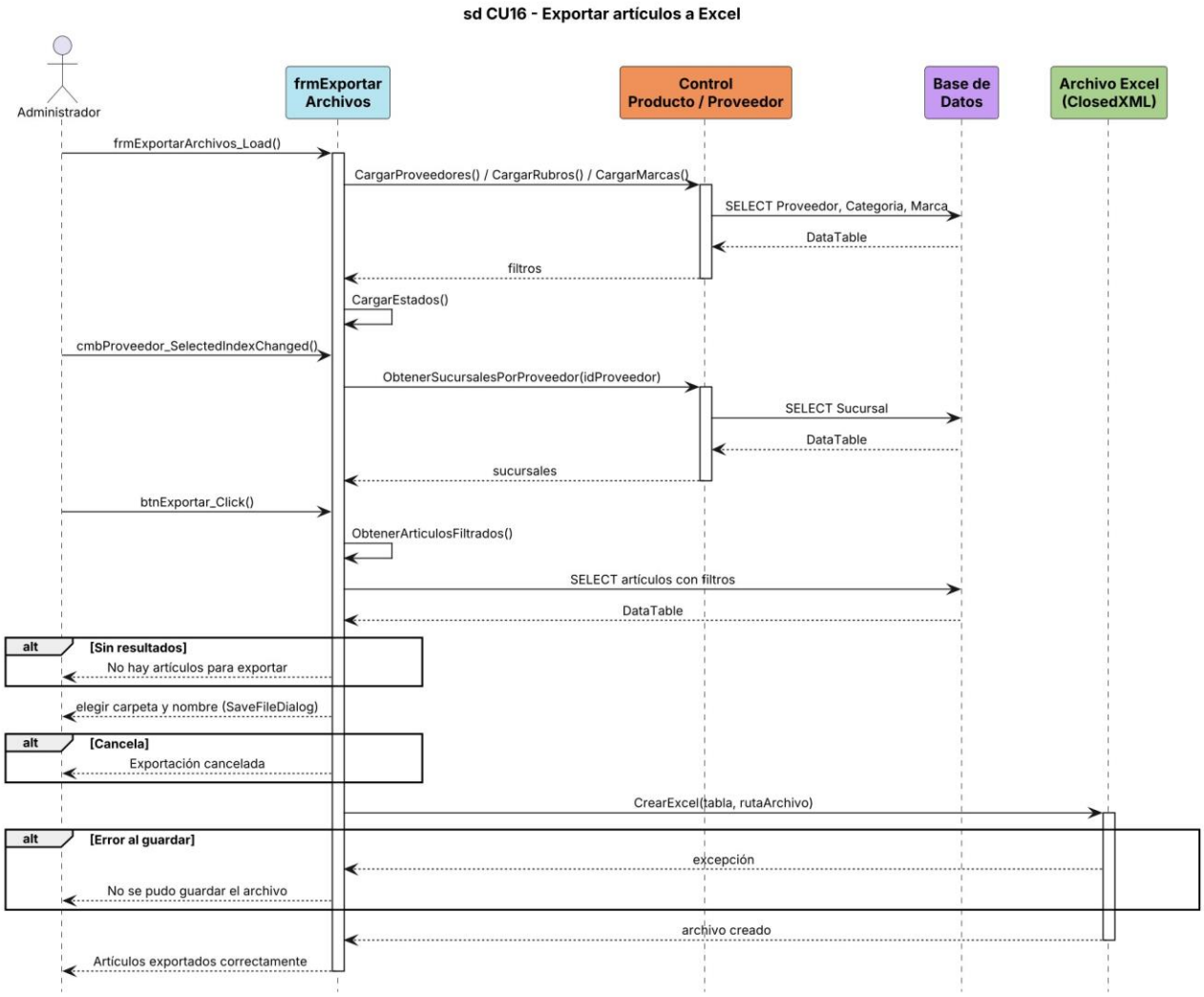
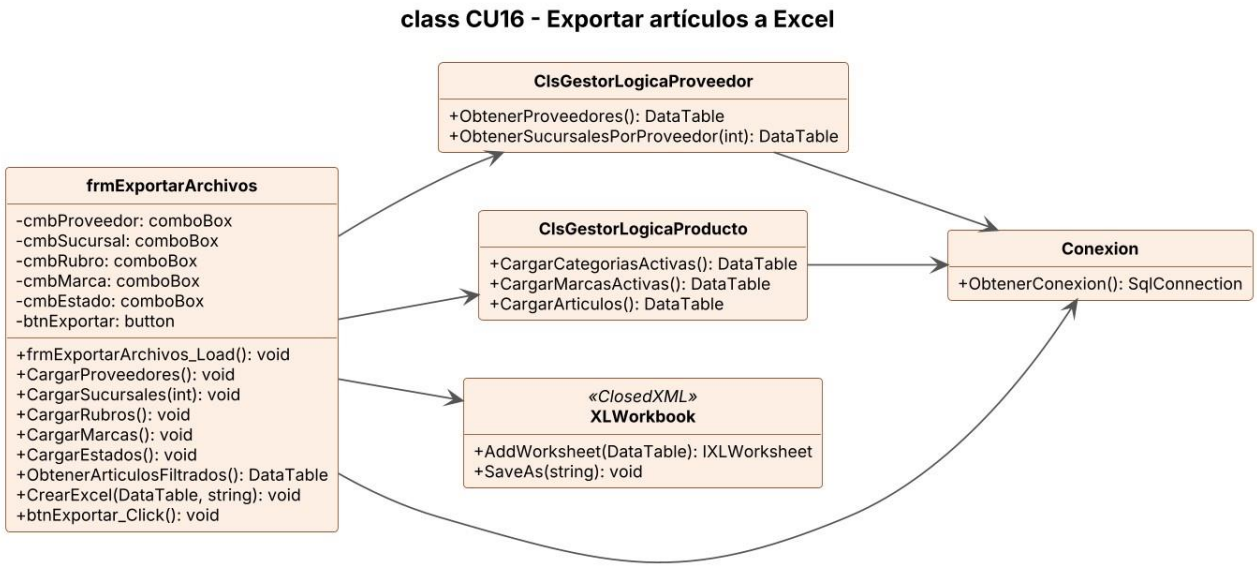


Diagrama de clases



CU17 - Consultar historial de ventas

ID y Nombre	**CU17 Consultar historial de ventas**	
Descripción	Permite al dueño o empleado autorizado consultar las ventas registradas, filtrarlas por estado o por texto y ver el detalle de cada venta: productos, cantidades, precios, medios de pago, cajero y caja. El Administrador además puede anular una venta indicando el motivo (el stock vuelve a sumarse) o volver a activarla.	
Actores	Administrador - Empleado autorizado (permiso 2.0.0 Ventas; 2.5.0 Anular Ventas para anular).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso correspondiente.	
Post-Condición	Se muestran las ventas que cumplen los filtros y, si se elige una, su detalle. Si se anula, la venta queda anulada con su motivo y el stock se repone.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Ventas > Administrador de Ventas.	
	2. El sistema muestra las ventas registradas.	
	3. El usuario filtra por estado o escribe en "Buscar".	
	4. El sistema muestra las ventas que cumplen los filtros.	
	5. El usuario selecciona una venta y presiona "Ver detalle" (o hace doble clic).	5.1 No se seleccionó venta: El sistema informa "Seleccione una venta para ver el detalle". 5.2 Anular venta: El Administrador presiona "Anular", ingresa el motivo y confirma; el sistema anula la venta y repone el stock. Sin el

		permiso 2.5.0, el sistema informa que no tiene permiso para anular ventas. 5.3 Activar venta: Si la venta está anulada, el sistema pregunta si desea activarla nuevamente.
	6. El sistema muestra el detalle de la venta: productos, pagos, cajero, caja, total y observaciones.	

Diagrama de caso de uso

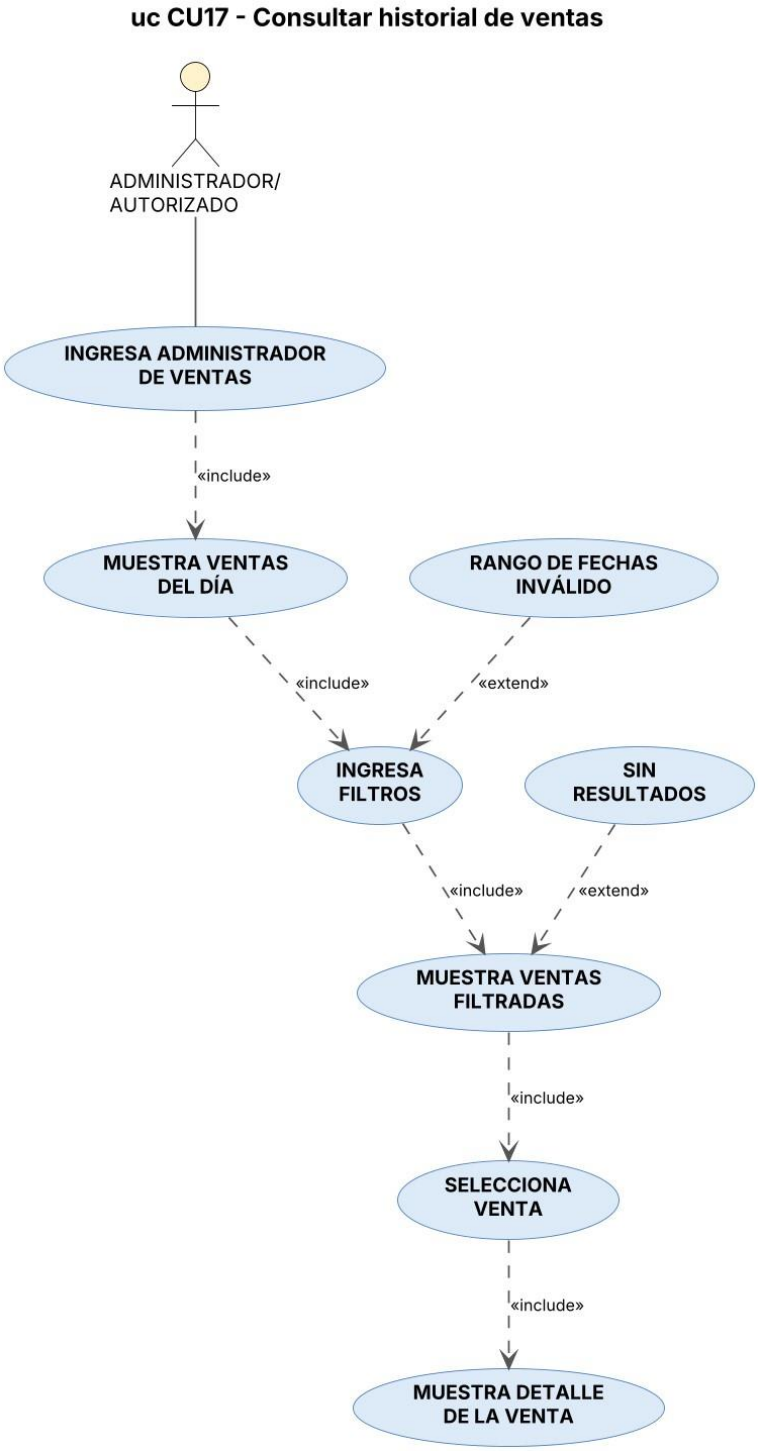


Diagrama de secuencia

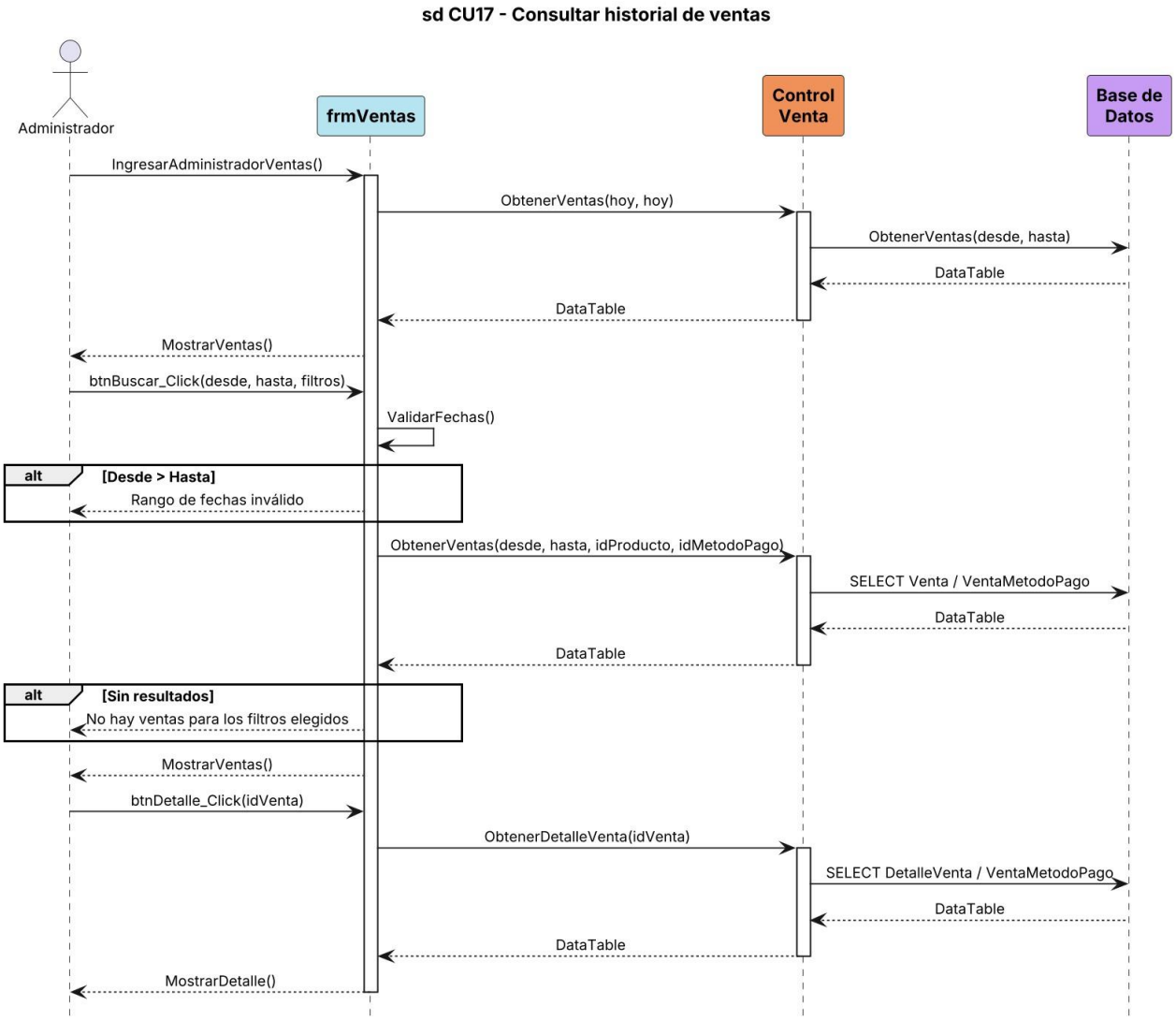
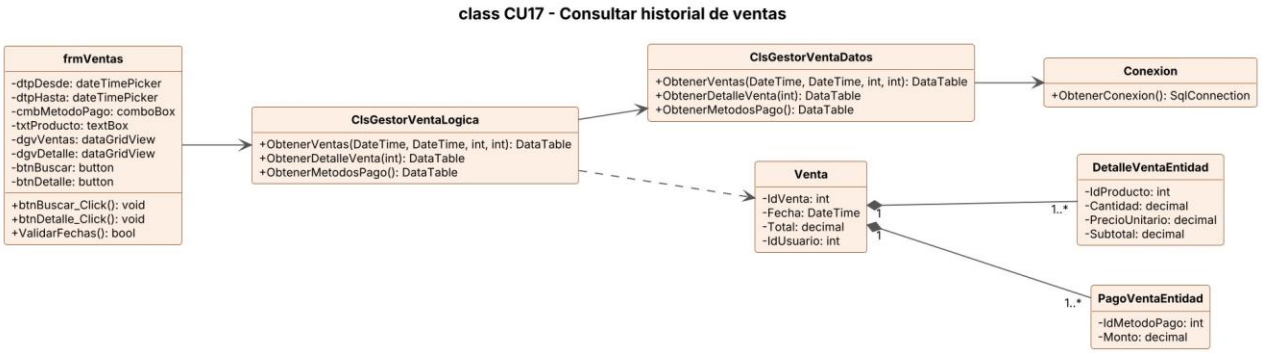


Diagrama de clases



CU18 - Consultar historial de devoluciones

ID y Nombre	**CU18 Consultar historial de devoluciones**	
Descripción	Permite al dueño o empleado autorizado consultar las devoluciones registradas, filtrarlas por tipo o por texto y ver el motivo de cada una. Desde la misma pantalla se accede a cargar una nueva devolución (CU07).	
Actores	Administrador - Empleado autorizado (permiso 2.2.2 Historial Devoluciones).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 2.2.2 Historial Devoluciones.	
Post-Condición	Se muestran las devoluciones que cumplen los filtros y el motivo de la elegida. No se modifican datos.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Ventas > Devolución.	
	2. El sistema muestra las devoluciones registradas con la venta de origen, el producto, la cantidad, el importe y el usuario.	2.1 Error al obtener datos: El sistema informa que no se pudieron obtener las devoluciones.
	3. El usuario filtra por tipo o escribe en "Buscar".	3.1 Limpiar filtros: El usuario presiona "Limpiar filtros" y el sistema vuelve a mostrar todas las devoluciones.
	4. El sistema actualiza el listado.	
	5. El usuario selecciona una devolución.	5.1 Nueva devolución: El usuario presiona "Nueva devolución" y continúa en el CU07.
	6. El sistema muestra el motivo de la devolución elegida.	

Diagrama de caso de uso

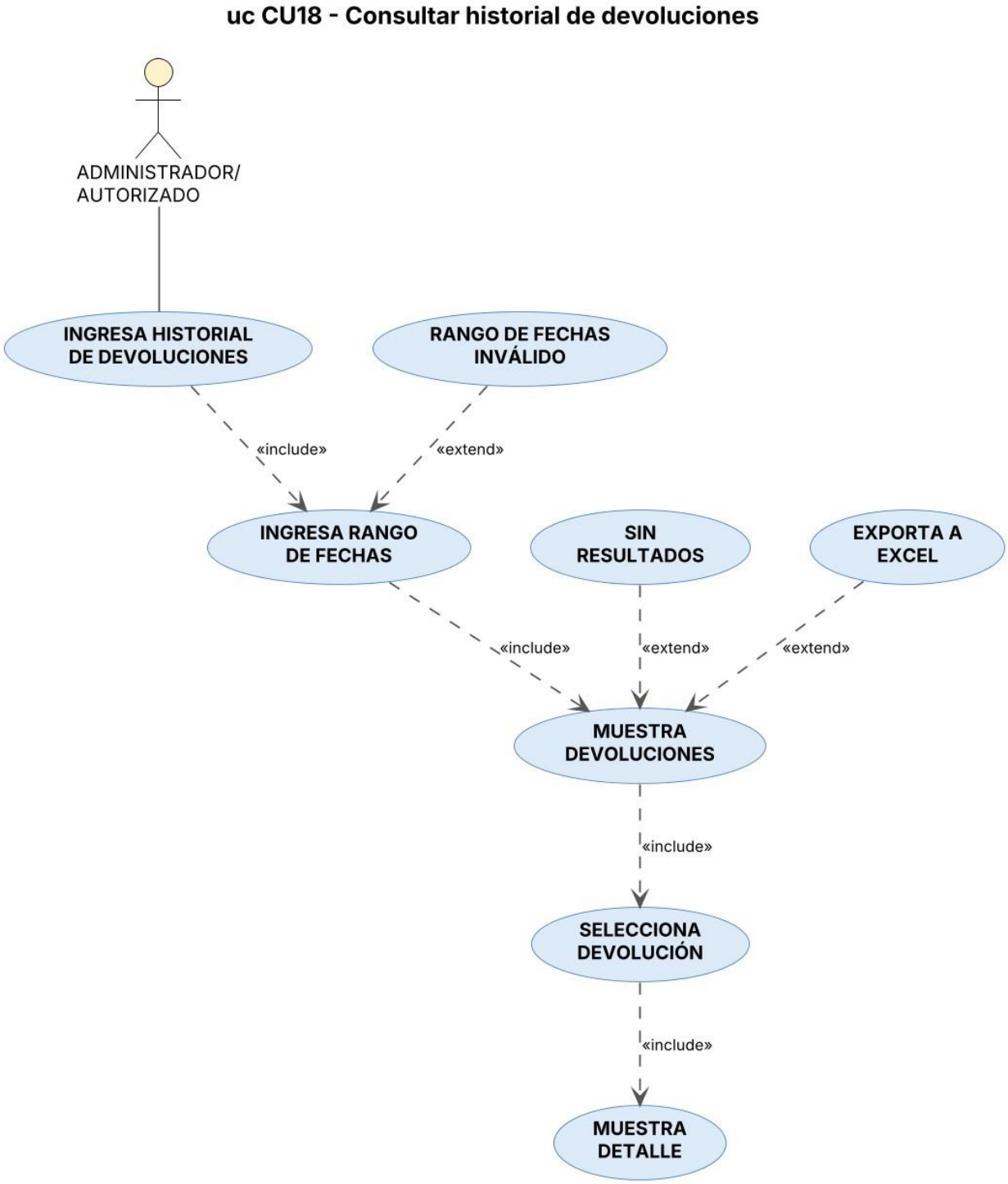


Diagrama de secuencia

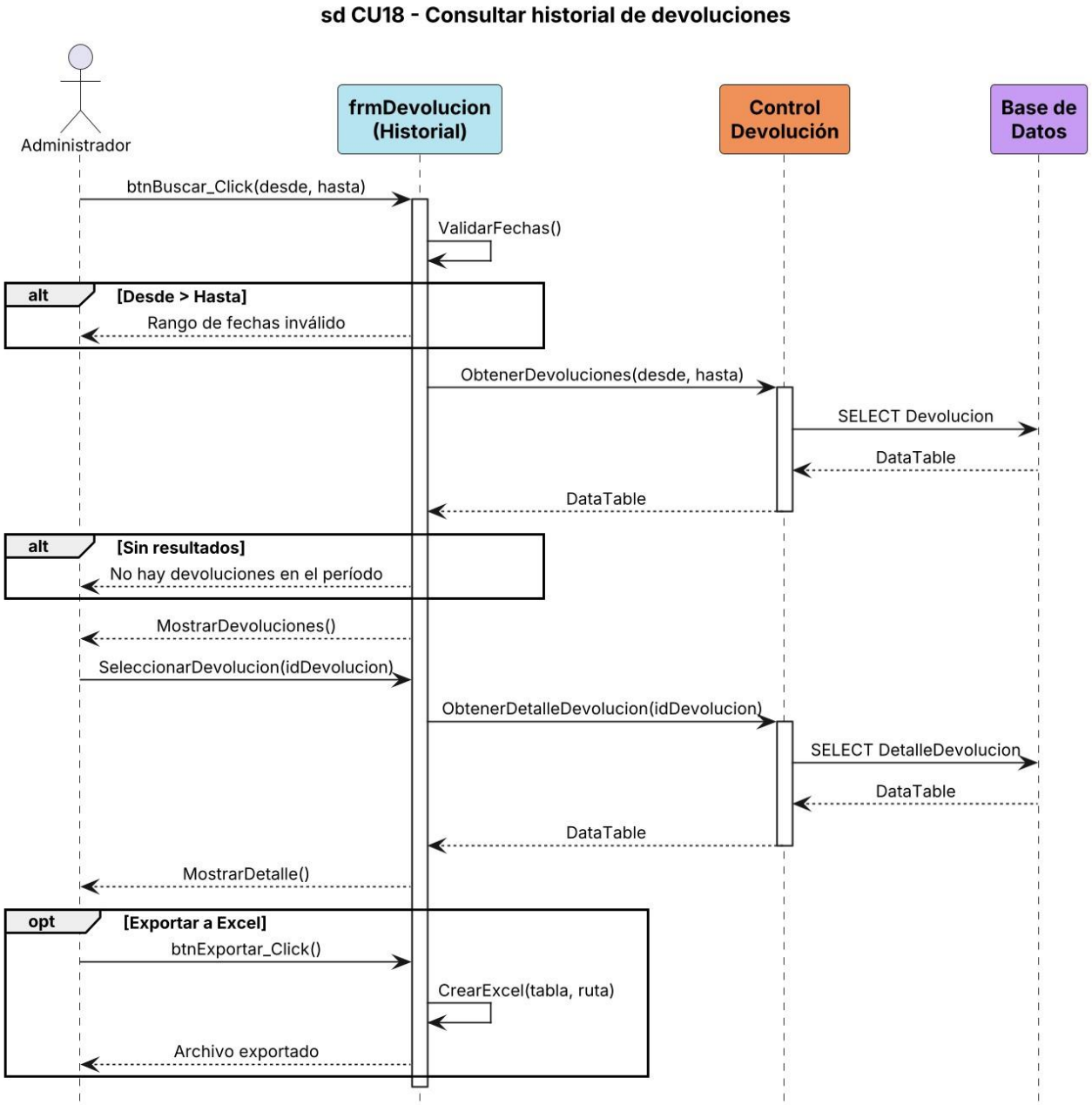
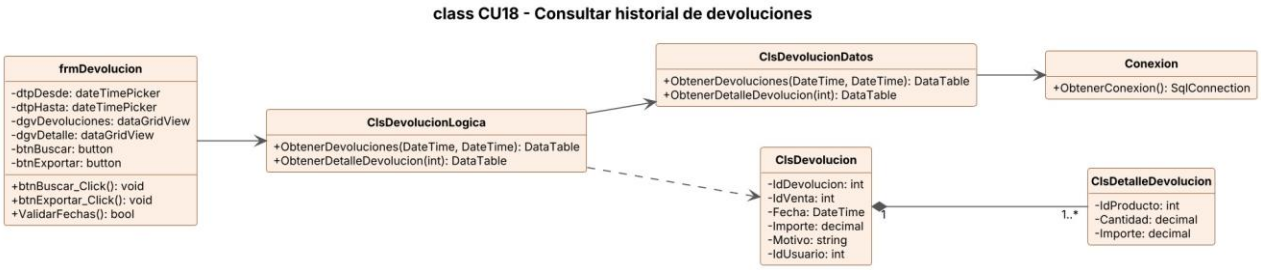


Diagrama de clases



CU19 - Crear promoción

ID y Nombre	**CU19 Crear promoción**	
Descripción	Permite al Administrador crear una promoción de venta indicando su nombre, los productos incluidos, la mecánica (precio promocional, descuento porcentual o descuento por volumen, por ejemplo 2x1 o 3x2) y la vigencia: fechas, días de la semana y franja horaria. Durante la vigencia, el sistema aplica la promoción al registrar una venta. Al guardarla, el sistema le asigna un código.	
Actores	Administrador - Empleado autorizado (permiso 2.3.0 Crear Promociones).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 2.3.0 Crear Promociones. Los productos de la promoción deben estar registrados y activos.	
Post-Condición	La promoción queda registrada y activa para la vigencia indicada.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al menú Promociones > Administrar Promociones.	
	2. El sistema muestra las promociones (las desactivadas en rojo).	
	3. El usuario ingresa el nombre de la promoción.	
	4. El usuario presiona "Configurar fechas y horarios" e indica fecha de inicio y fin, días de la semana y franja horaria.	4.1 Vigencia inválida: El sistema informa que la fecha de fin no puede ser anterior a la de inicio, que la hora de fin no puede ser igual a la de inicio o que debe seleccionar al menos un día.
	5. El usuario presiona "+Agregar producto" y busca cada producto por nombre, código de barras o SKU (CU08).	5.1 Producto repetido: El sistema informa que el artículo ya fue agregado a la promoción.

	6. El usuario elige la mecánica de la promoción y su valor.	6.1 Mecánica por volumen: El usuario presiona "+ Agregar mecánica" e indica la cantidad que lleva y la que paga; solo puede aplicarse a un mismo producto.
	7. El usuario presiona "Guardar Promoción".	
	8. El sistema valida los datos ingresados.	8.1 Datos incompletos: El sistema informa que debe agregar al menos un producto, configurar la vigencia o seleccionar una mecánica.
	9. El sistema registra la promoción y sus productos en la base de datos.	
	10. El sistema informa que la promoción se guardó correctamente con su código.	

Diagrama de caso de uso

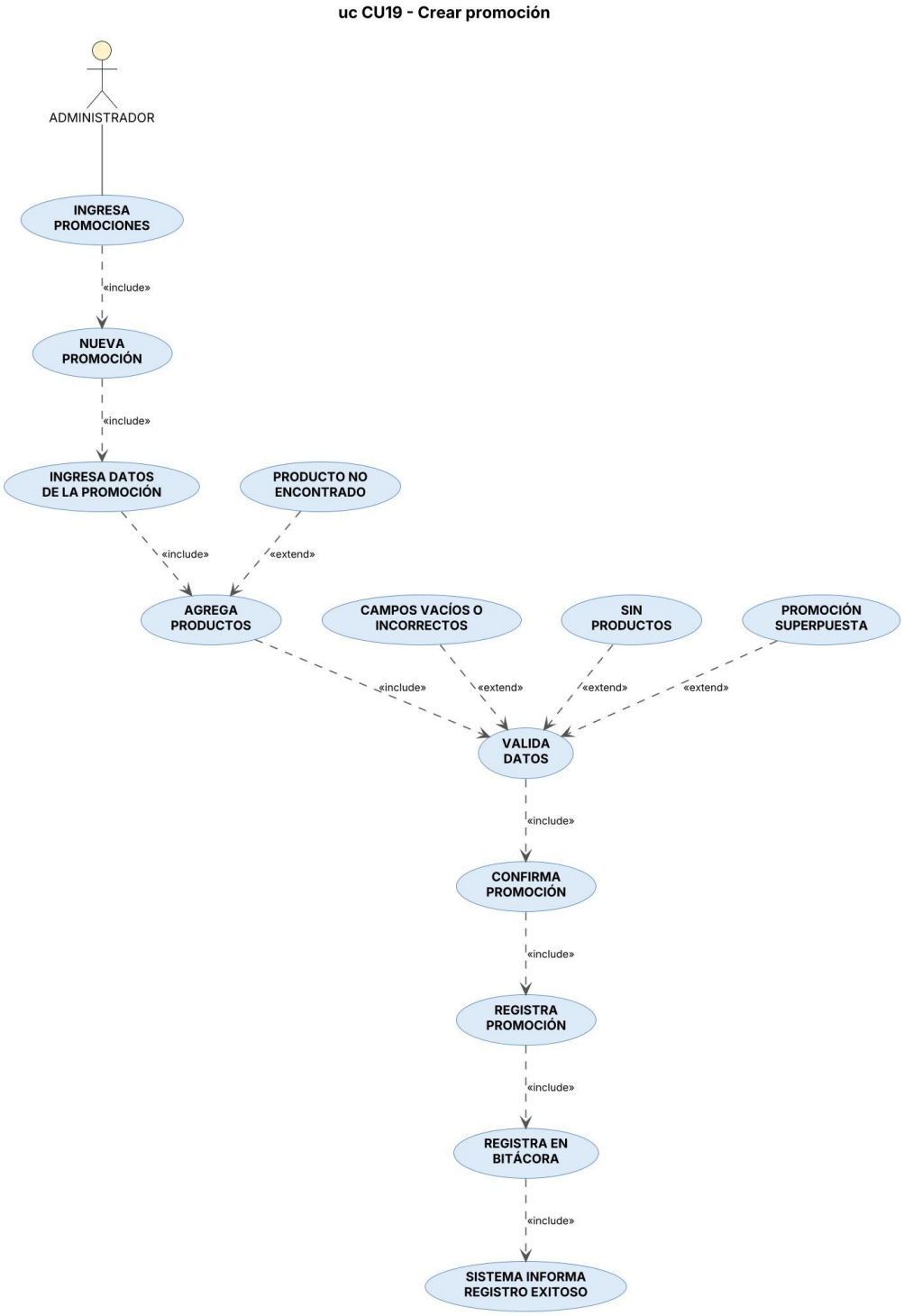


Diagrama de secuencia

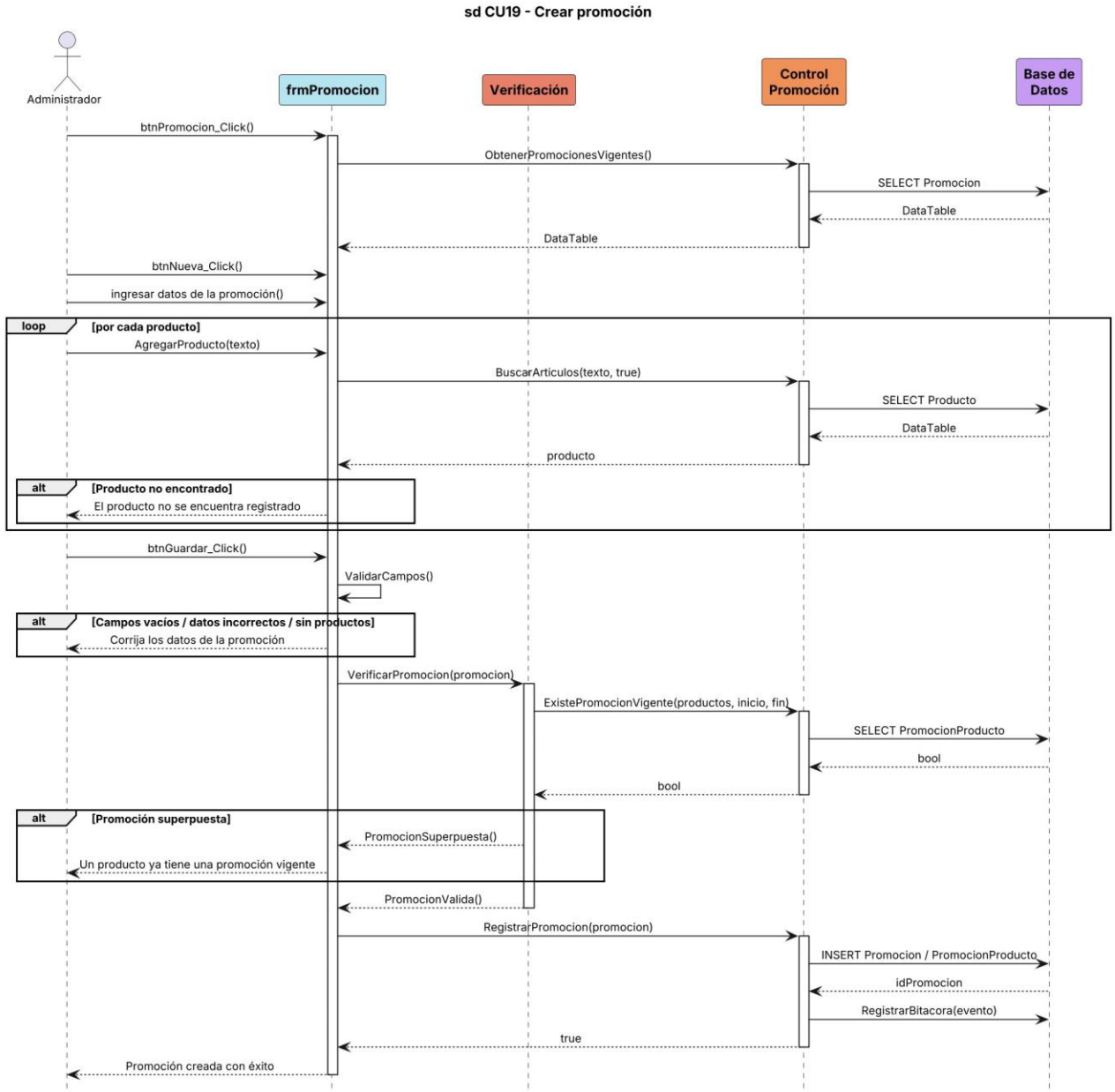
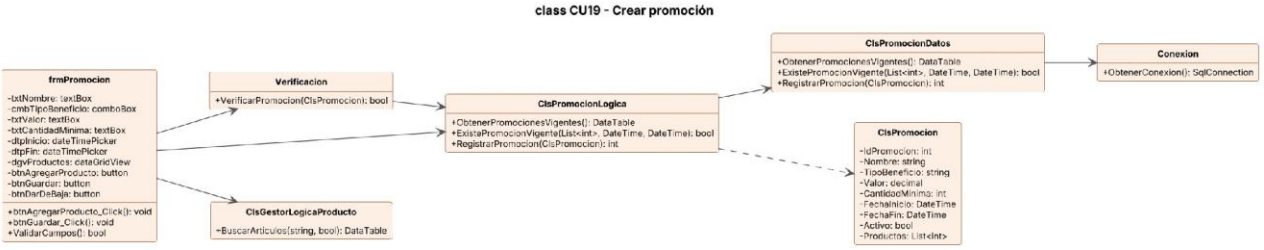


Diagrama de clases



CU20 - Dar de baja promoción

ID y Nombre	**CU20 Dar de baja promoción**	
Descripción	Permite al Administrador desactivar una promoción antes de su fecha de fin. La promoción deja de aplicarse a las ventas nuevas, pero se conserva en la base de datos junto con las ventas en las que ya se aplicó, y se puede volver a activar. Si la promoción no se usó, también se puede eliminar.	
Actores	Administrador - Empleado autorizado (permiso 2.4.0 Baja Promociones).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 2.4.0 Baja Promociones. Debe existir al menos una promoción registrada.	
Post-Condición	La promoción queda inactiva y deja de aplicarse en las ventas.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al menú Promociones > Administrar Promociones.	
	2. El sistema muestra las promociones.	
	3. El usuario busca y selecciona la promoción.	
	4. El usuario marca "Desactivar Promoción".	4.1 Activar promoción: El usuario destilda "Desactivar Promoción" y el sistema la vuelve a activar. 4.2 Eliminar promoción: El usuario presiona "Eliminar"; el sistema advierte que la acción no se puede deshacer y, si confirma, la elimina.
	5. El sistema marca la promoción como inactiva.	

	7. El sistema informa que la promoción se desactivó correctamente y la muestra en rojo.	

Diagrama de caso de uso

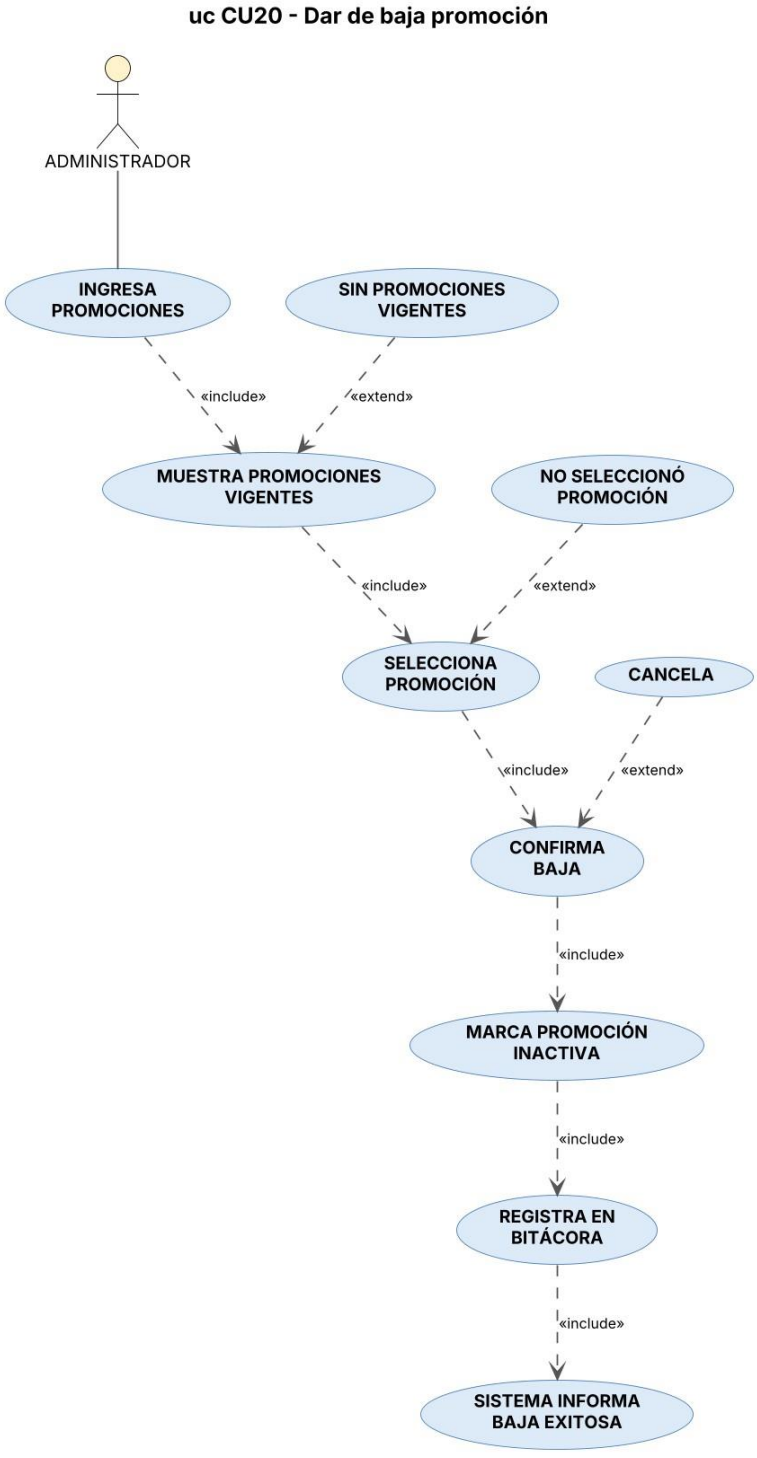


Diagrama de secuencia

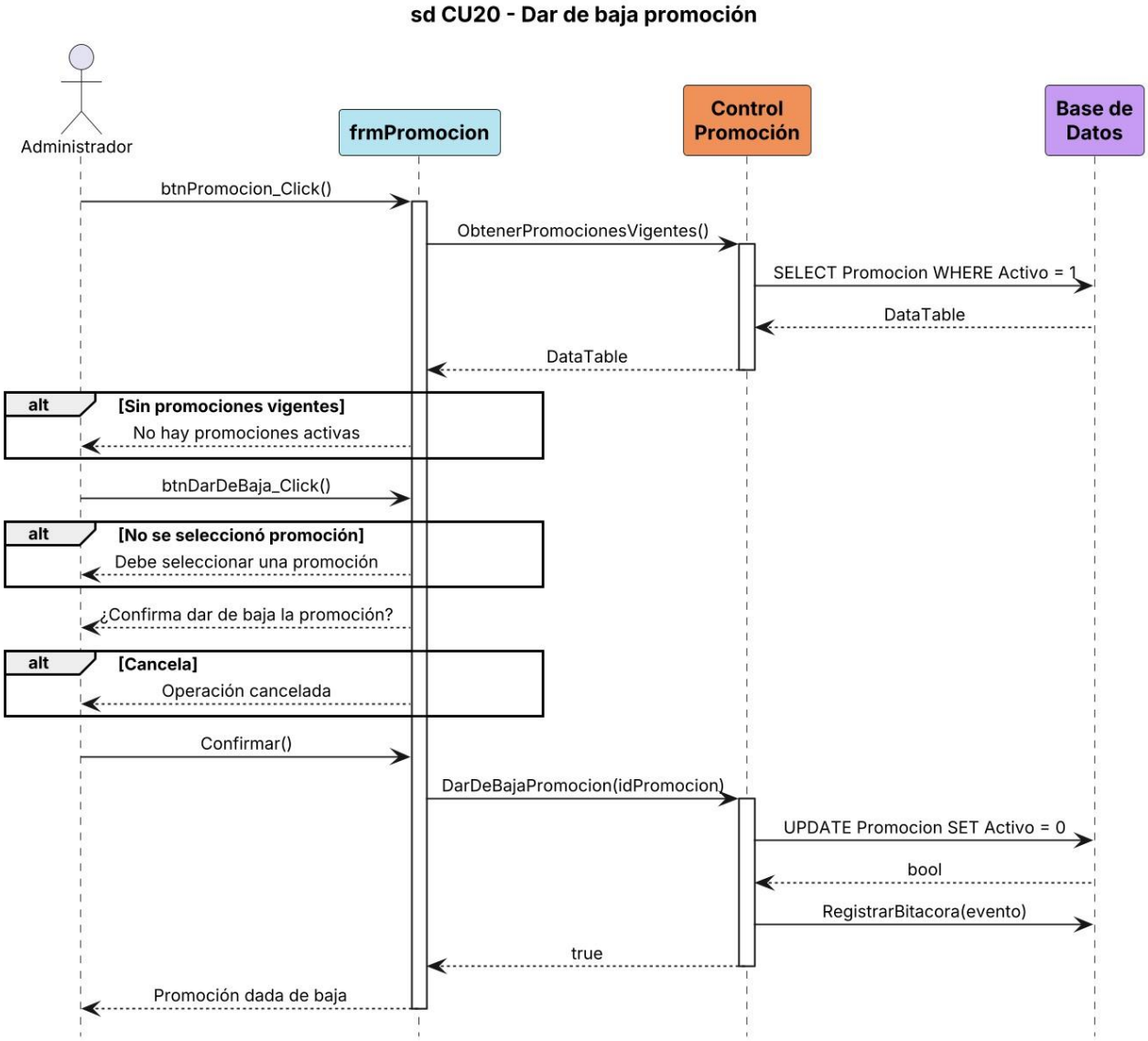
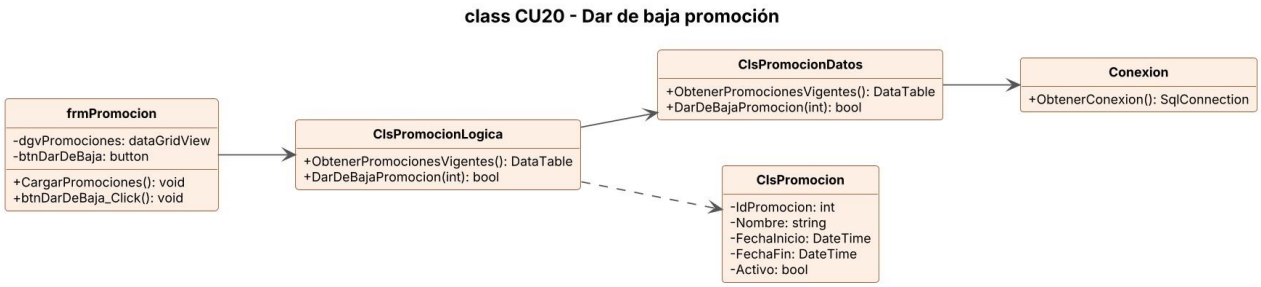


Diagrama de clases



CU21 - Registrar compra a proveedor

ID y Nombre	**CU21 Registrar compra a proveedor**	
Descripción	Permite al dueño o empleado de compras registrar una compra a un proveedor, indicando la sucursal, el comprobante (tipo y número), la condición de pago (contado o cuenta corriente), el medio de pago, la fecha de vencimiento y la entrega estimada, y cargando los productos con su cantidad, precio de costo sin IVA e IVA. El sistema calcula subtotal, IVA y total, actualiza el stock y deja la compra pendiente de recepción (CU23).	
Actores	Administrador - Empleado Compras (permiso 3.1.0 Nueva Compra).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 3.1.0 Nueva Compra. El proveedor debe estar activo y tener al menos una sucursal activa. Los productos deben estar registrados.	
Post-Condición	La compra queda registrada con su detalle y pendiente de recepción. El stock de cada producto se incrementa y se registran los movimientos de stock de tipo Ingreso. Si la compra es en cuenta corriente, queda pendiente de pago con el proveedor; si se paga desde la caja, se registra el egreso.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Compras > Administrador de Compras y presiona "Nueva compra" (o presiona F4 en la pantalla principal).	
	2. El usuario busca el proveedor por nombre o CUIT y elige la sucursal.	2.1 Proveedor no encontrado o sin sucursales: El sistema informa que no se encontró ningún proveedor activo o que no tiene sucursales activas.
	3. El usuario selecciona el tipo de comprobante, carga el número, la condición, el medio de pago y, si corresponde, el vencimiento y la entrega estimada.	3.1 Comprobante repetido: El sistema informa que ya existe ese comprobante para el proveedor.
	4. El usuario busca cada producto y lo agrega a la grilla.	
	5. El usuario ingresa la cantidad, el precio sin IVA y el IVA de cada producto.	5.1 Datos incorrectos: El sistema informa si la cantidad es menor o igual a cero, el precio es negativo o el IVA no está entre 0 y 100.

	6. El sistema calcula el subtotal sin IVA, el IVA y el total de la compra.	
	7. El usuario presiona "Guardar compra".	7.1 Cancelar: Si hay productos cargados, el sistema pregunta si desea salir sin guardar.
	8. El sistema valida los datos.	8.1 Sin productos: El sistema informa "Debe agregar al menos un producto a la compra". 8.2 Fechas inválidas: El sistema informa que la entrega estimada no puede ser anterior a hoy o que el vencimiento no puede ser anterior a la fecha de la compra.
	9. El sistema registra la compra y su detalle, actualiza el stock y registra los movimientos de stock.	
	10. El sistema informa que la compra se guardó correctamente con su número.	

Diagrama de caso de uso

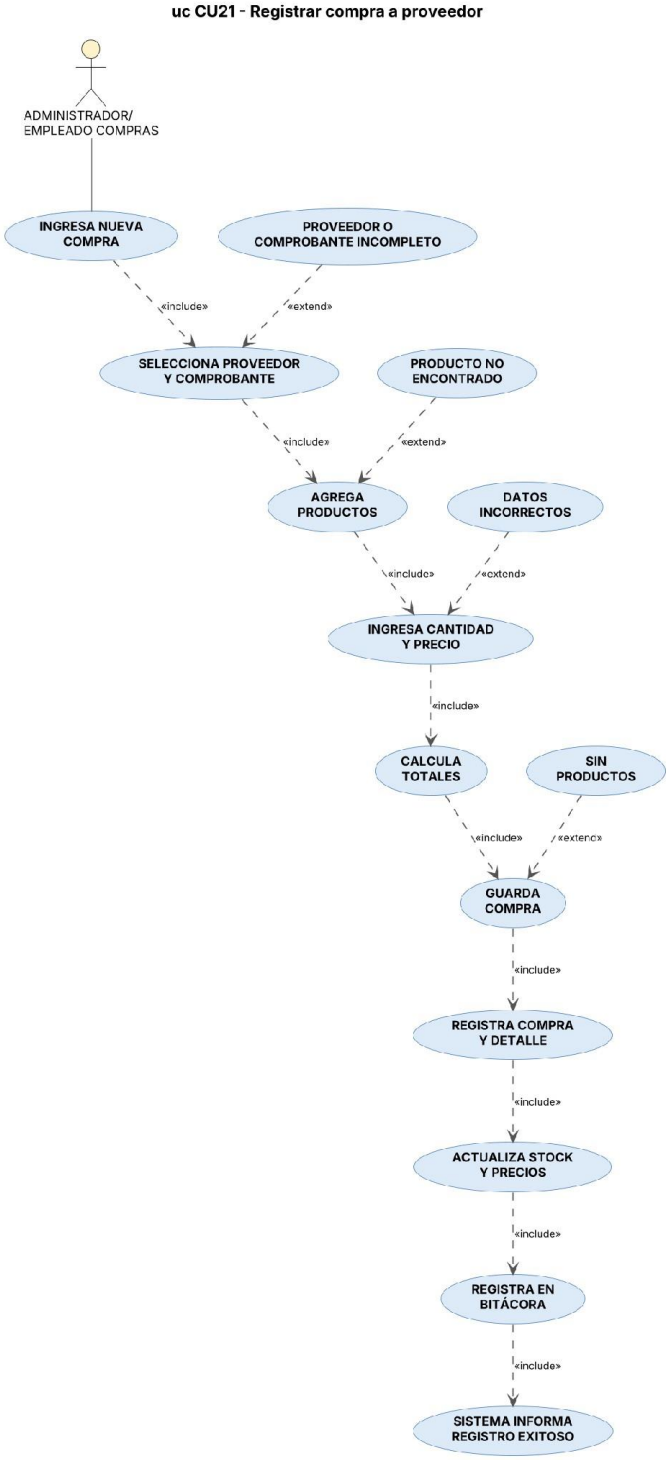


Diagrama de secuencia

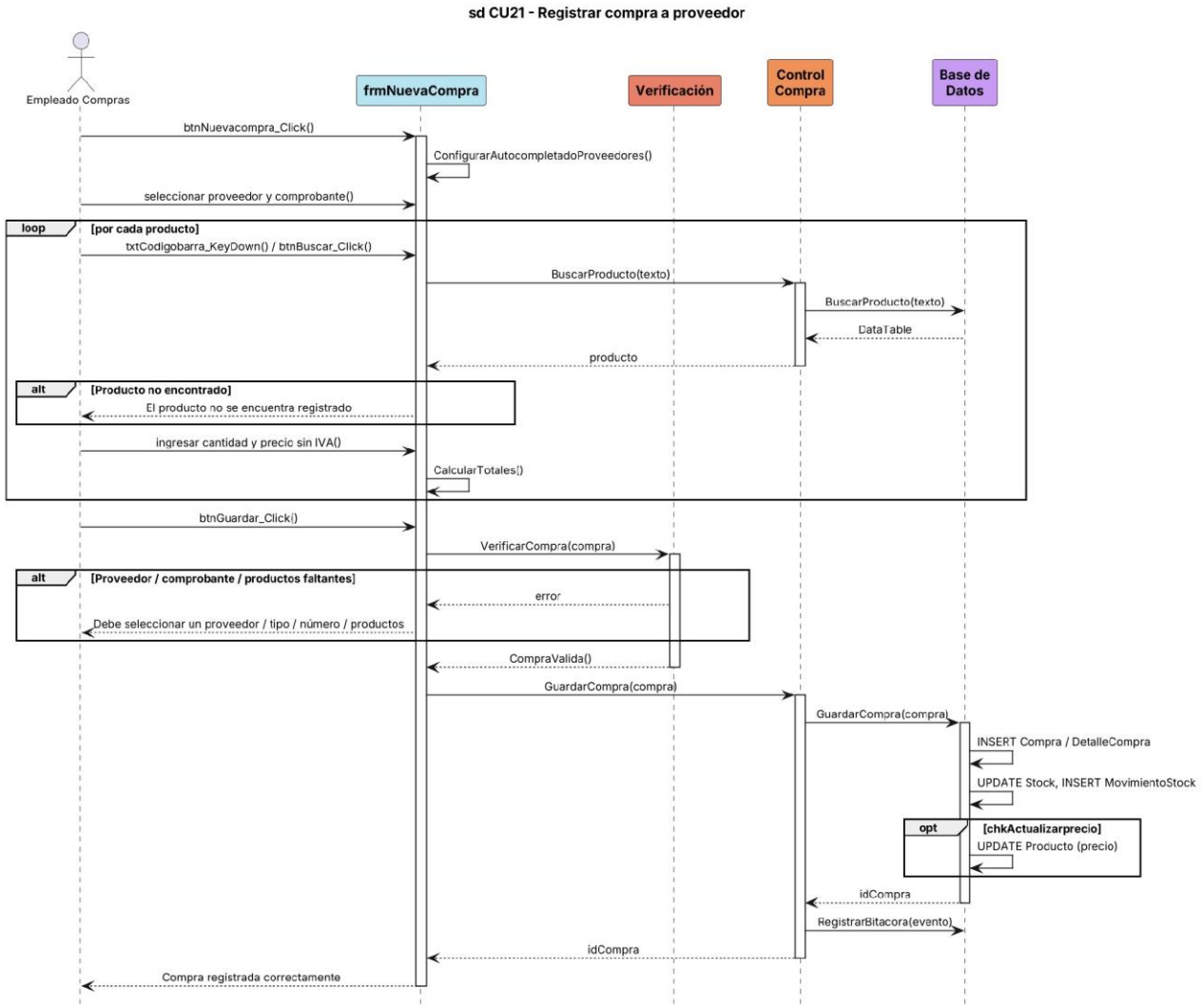
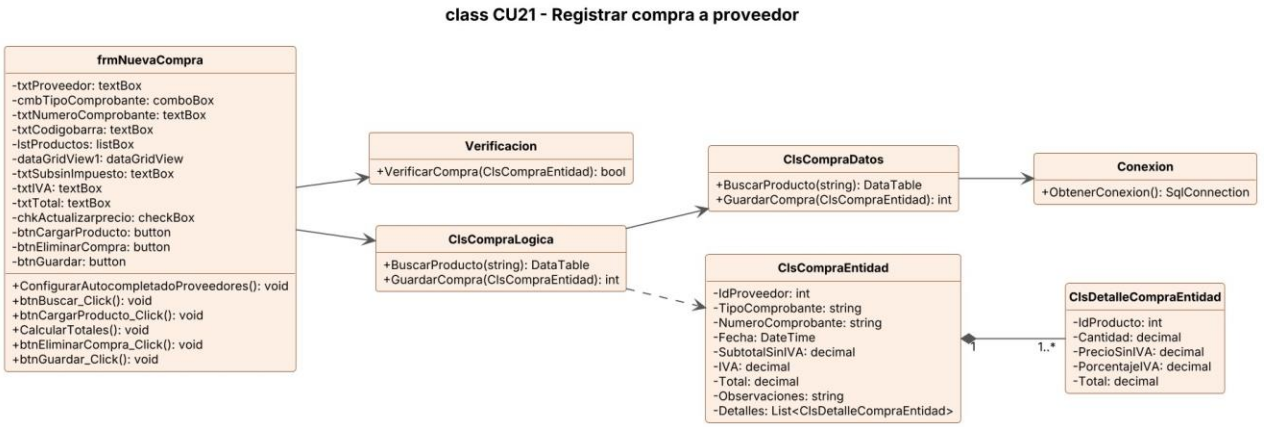


Diagrama de clases



CU22 - Consultar / anular compra

ID y Nombre	**CU22 Consultar / anular compra**	
Descripción	Permite al dueño o empleado de compras consultar las compras registradas filtrando por proveedor y fechas, ver el detalle de cada una y anular una compra cargada por error indicando obligatoriamente el motivo. Al anular, el sistema descuenta del stock la mercadería que había ingresado con esa compra.	
Actores	Administrador - Empleado Compras (permiso 3.0.0 Compras; 3.3.0 Anular Compras para anular).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso correspondiente.	
Post-Condición	Consulta: se muestran las compras y su detalle, sin modificar datos. Anulación: la compra queda anulada con su motivo, se revierte el stock y la operación queda en la bitácora.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Compras > Administrador de Compras.	
	2. El sistema muestra las compras registradas.	
	3. El usuario filtra por proveedor y rango de fechas.	3.1 Sin resultados: El sistema informa que el proveedor no tiene compras en el período.
	4. El usuario selecciona una compra y presiona "Ver detalle".	4.1 No se seleccionó compra: El sistema informa que debe seleccionar una compra.
	5. El sistema muestra el detalle: proveedor, fecha, comprobante, productos, subtotal sin IVA, IVA y total.	
	6. El usuario cierra el detalle y finaliza la consulta.	6.1 Anular compra: El usuario selecciona la compra, presiona "Anular" e ingresa el motivo; si el motivo está vacío, el sistema informa "Debe

		ingresar un motivo para anular la compra". El sistema solicita confirmación, anula la compra, descuenta el stock y lo registra en la bitácora. 6.2 Compra ya anulada: El sistema informa que la compra ya se encuentra anulada.

Diagrama de caso de uso

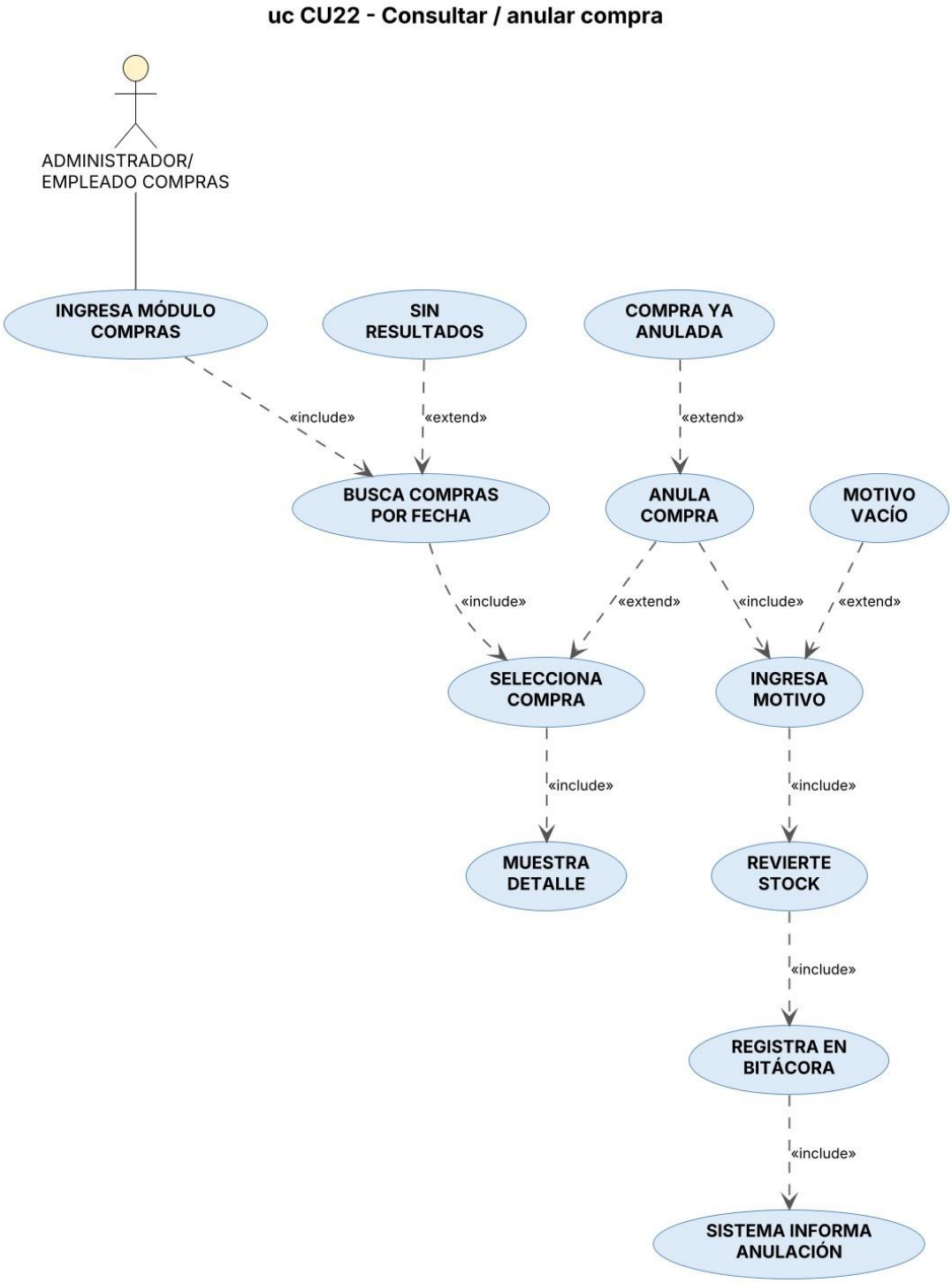


Diagrama de secuencia

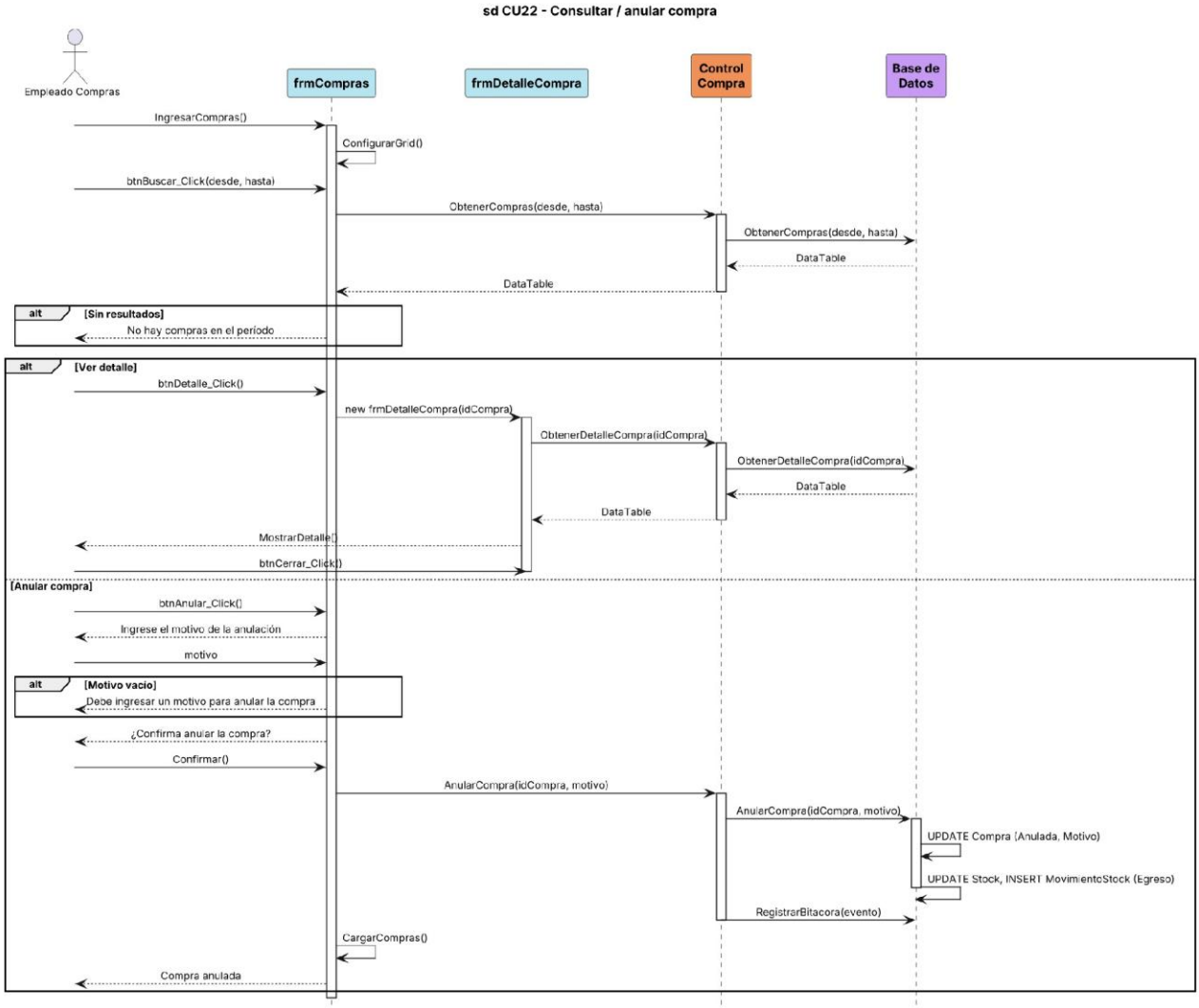
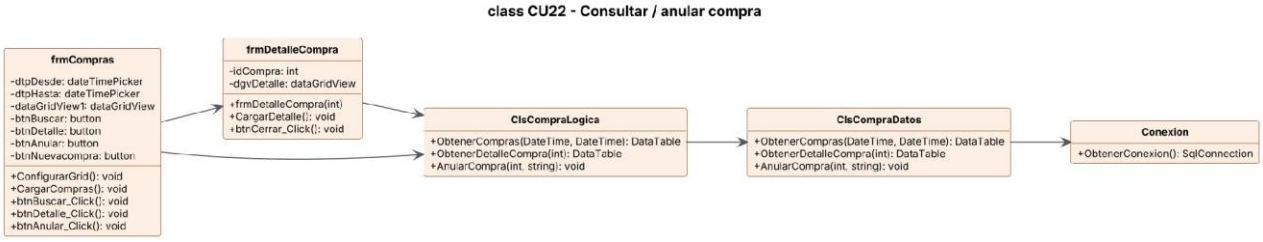


Diagrama de clases



CU23 - Recepción de mercadería

ID y Nombre	**CU23 Recepción de mercadería**	
Descripción	Permite al dueño o empleado de compras confirmar la llegada de la mercadería de una compra pendiente. El usuario controla la cantidad recibida de cada producto y el total de la boleta; si hay diferencias con lo comprado, indica el motivo y el sistema ajusta el stock (sobrante o faltante).	
Actores	Administrador - Empleado Compras (permiso 3.2.0 Recepción de Mercadería).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 3.2.0 Recepción de Mercadería. Debe existir al menos una compra pendiente de recepción.	
Post-Condición	La compra queda recibida con el detalle de lo que llegó. Si hubo diferencias, el stock se ajusta y se registran los movimientos de stock.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Compras > Recepción de mercadería.	
	2. El sistema muestra las compras pendientes de recepción, que se pueden filtrar por proveedor y fecha.	
	3. El usuario selecciona una compra.	
	4. El sistema muestra el proveedor, la fecha, el comprobante, el estado y los productos comprados.	
	5. El usuario ingresa la cantidad recibida de cada producto y el total de la boleta.	5.1 Producto que no llegó: El usuario indica 0 en la cantidad recibida; los productos de la compra no se pueden quitar. 5.2 Producto de más: El usuario presiona "+ Agregar producto" y lo agrega a la recepción.

		5.3 Datos inválidos: El sistema informa que ingrese una cantidad recibida válida (0 o mayor) o un total de boleta válido.
	6. El sistema calcula la diferencia con el total de la compra.	6.1 Diferencia: El usuario indica el motivo de la diferencia y el detalle u observaciones.
	7. El usuario presiona "Confirmar recepción".	7.1 Cancelar: El usuario presiona "Cancelar" y no se registran cambios.
	8. El sistema registra la recepción y ajusta el stock según las diferencias.	
	9. El sistema informa que la recepción se confirmó correctamente.	

Diagrama de caso de uso

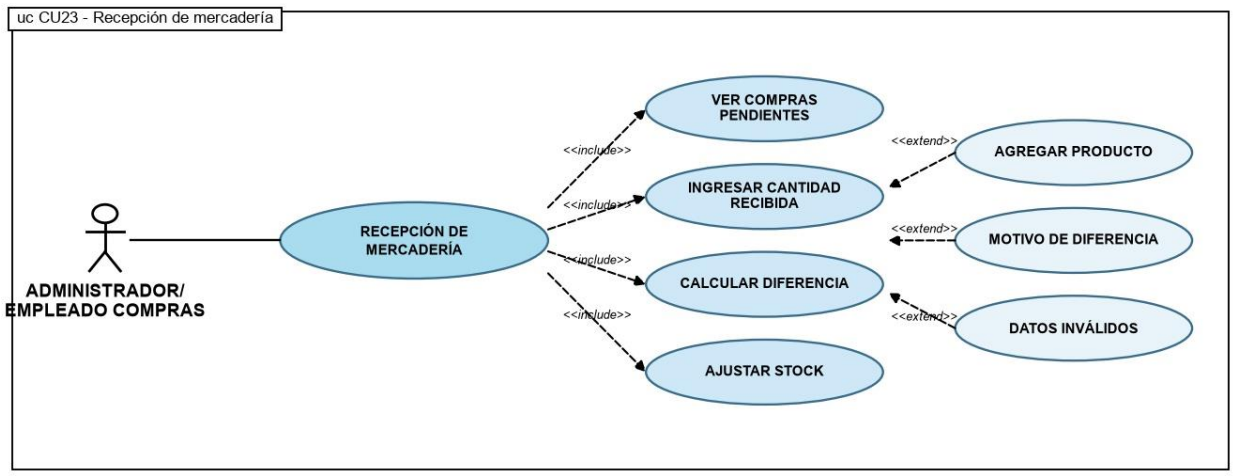


Diagrama de secuencia

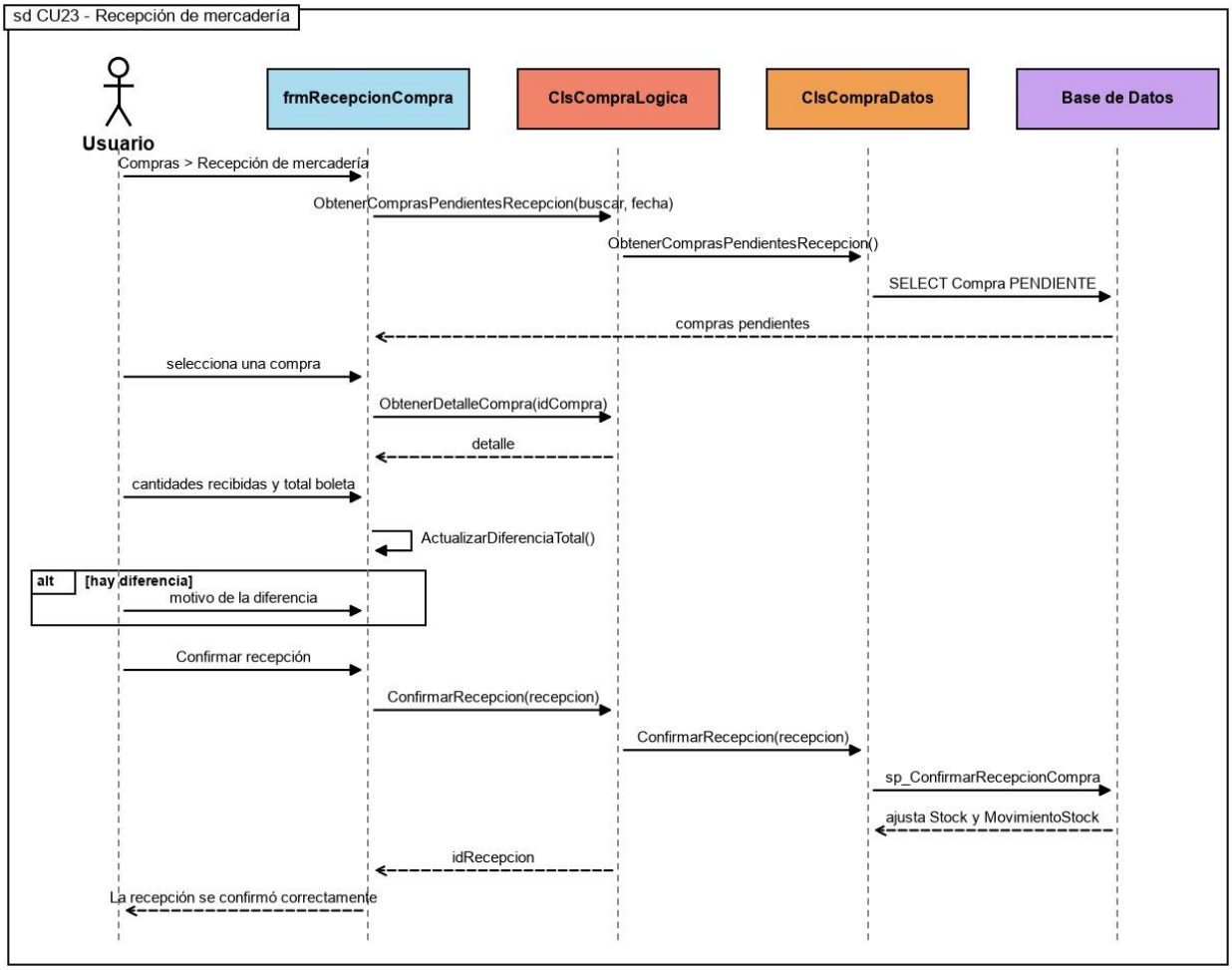
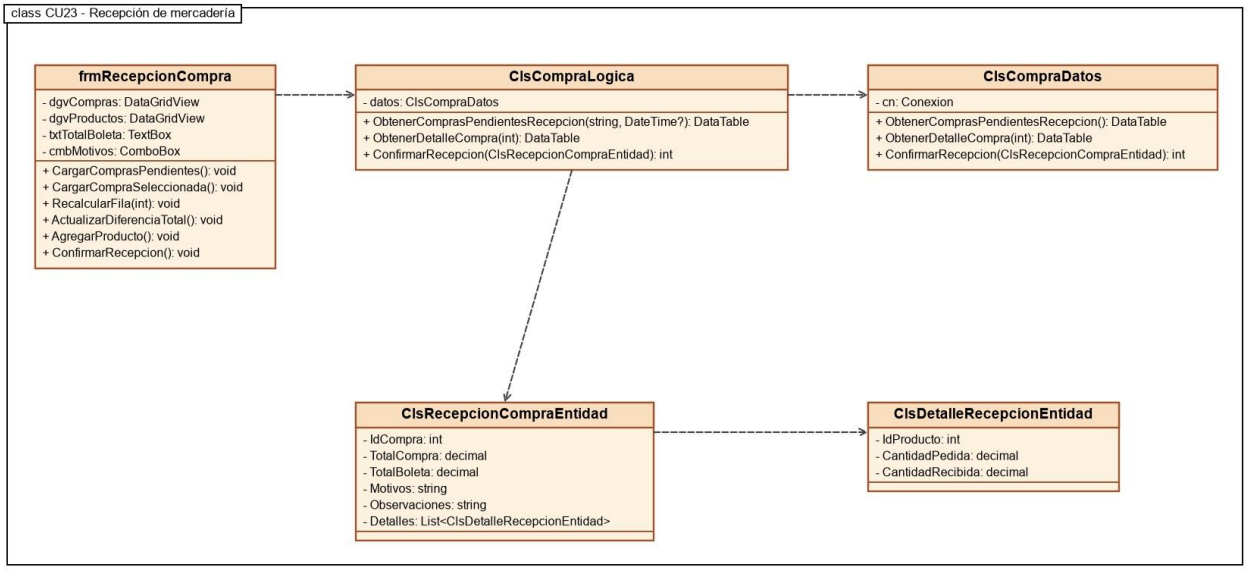


Diagrama de clases



CU24 - Modificar / dar de baja proveedor

ID y Nombre	**CU24 Modificar / dar de baja proveedor**	
Descripción	Permite al dueño o empleado autorizado modificar los datos de un proveedor ya registrado (razón social, CUIT, sucursales, teléfono, correo electrónico y dirección), agregarle sucursales o darlo de baja lógica (descontinuarlo). Un proveedor descontinuado no se puede elegir en compras nuevas, pero se conserva su historial de compras y su cuenta corriente, y se puede volver a activar.	
Actores	Administrador (permiso 4.1.2 Baja/Modificación de Proveedores).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 4.1.2. El proveedor debe estar registrado.	
Post-Condición	Los datos del proveedor quedan actualizados, o el proveedor queda inactivo.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Proveedores > Administrador de Proveedores.	
	2. El sistema muestra todos los proveedores con sus sucursales.	
	3. El usuario busca el proveedor por nombre o CUIT.	
	4. El usuario selecciona la sucursal y presiona "Modificar" (o hace doble clic en la fila).	4.1 Descontinuar proveedor: El usuario presiona "Descontinuar", el sistema solicita confirmación y marca el proveedor como inactivo. Con "Ocultar proveedores inactivos" destildado se lo puede volver a activar. 4.2 Nueva sucursal: El usuario presiona "Nueva sucursal" y carga otra sucursal del mismo proveedor.

	5. El usuario modifica los datos necesarios.	
	6. El sistema valida los datos ingresados.	6.1 Campos vacíos o datos incorrectos: El sistema informa los campos obligatorios sin completar o con formato inválido.
	7. El usuario confirma y el sistema actualiza el proveedor en la base de datos.	7.1 Cancelar: El sistema descarta los cambios y vuelve al paso 2.
	9. El sistema informa que el proveedor se modificó correctamente.	

Diagrama de caso de uso

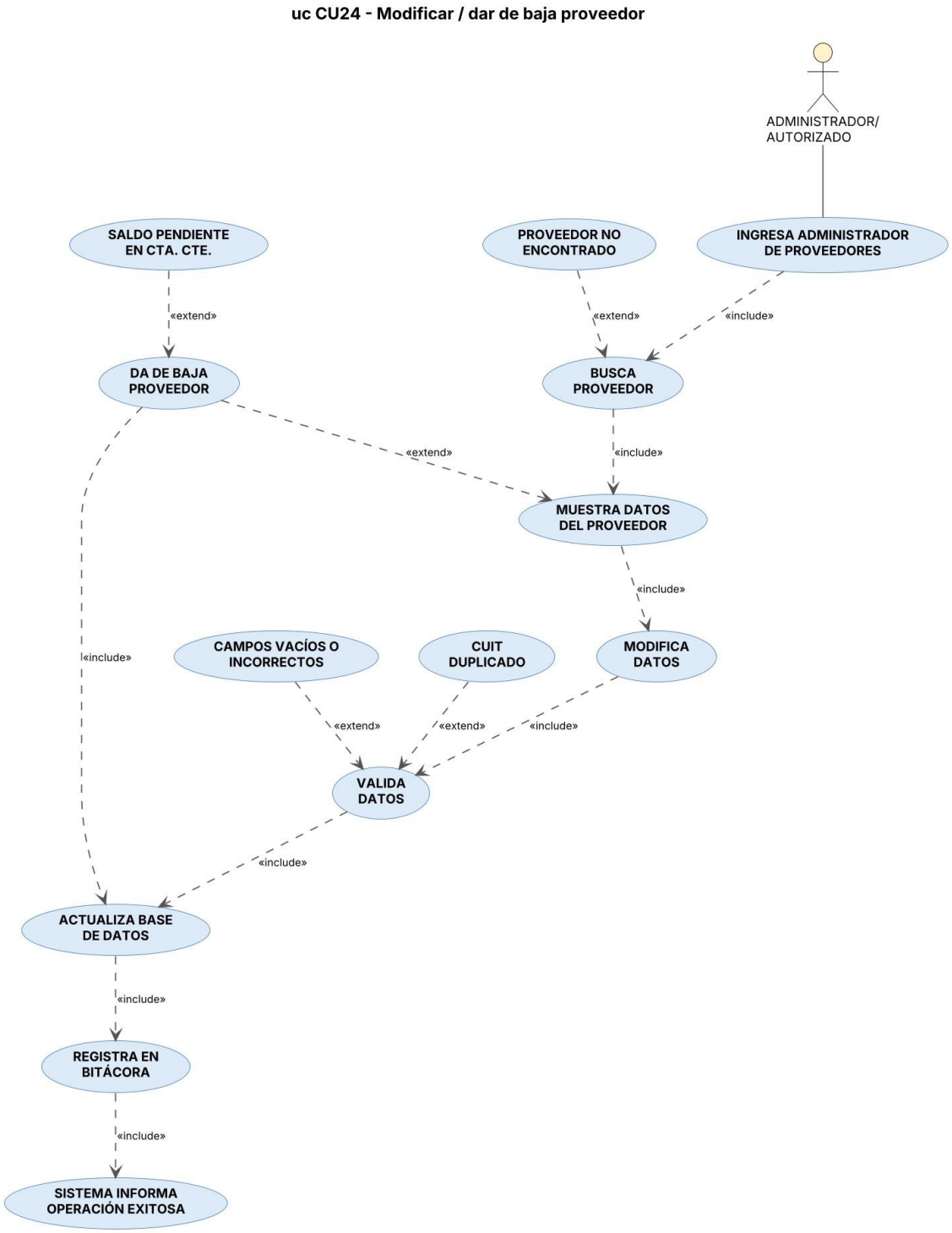


Diagrama de secuencia

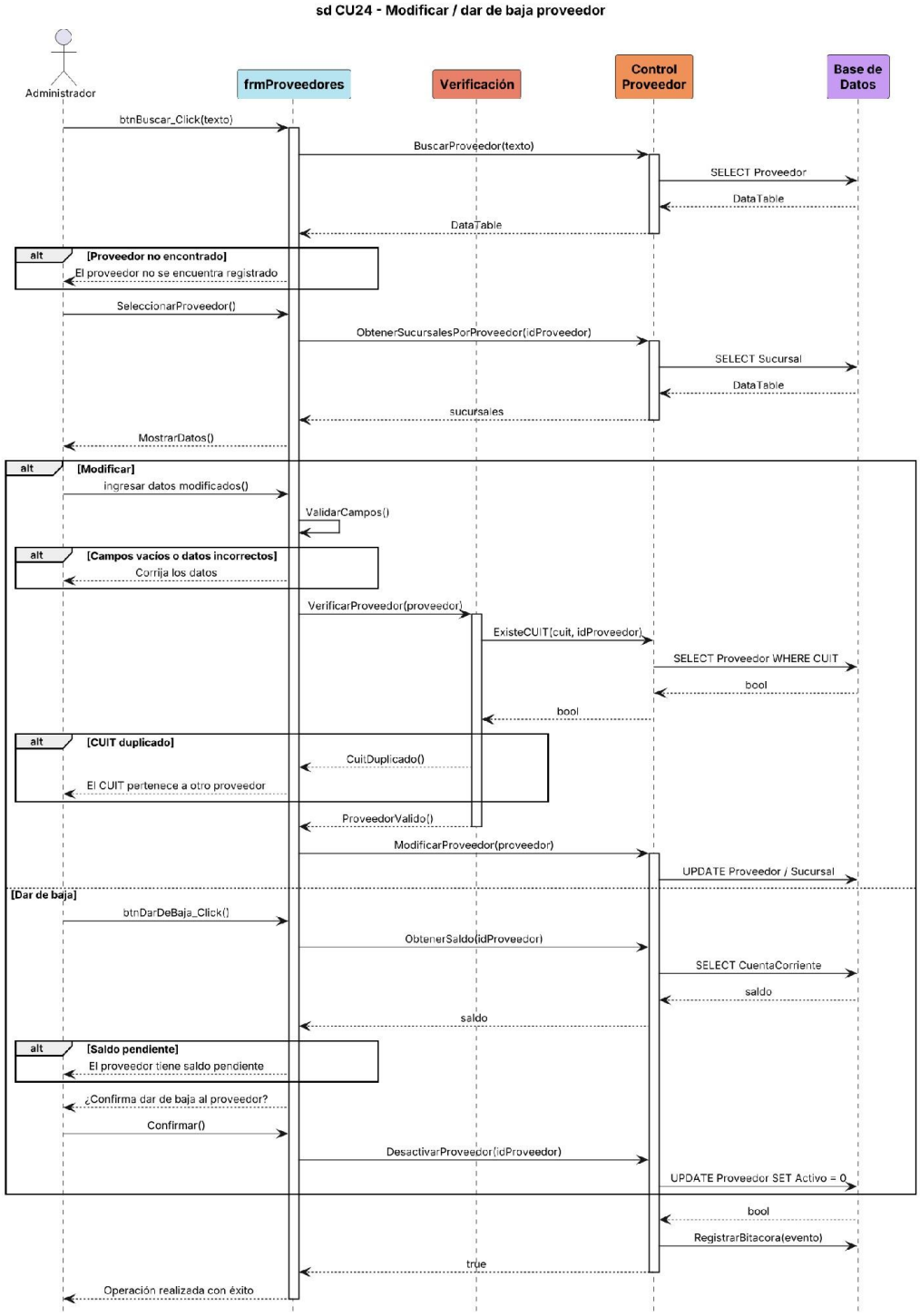
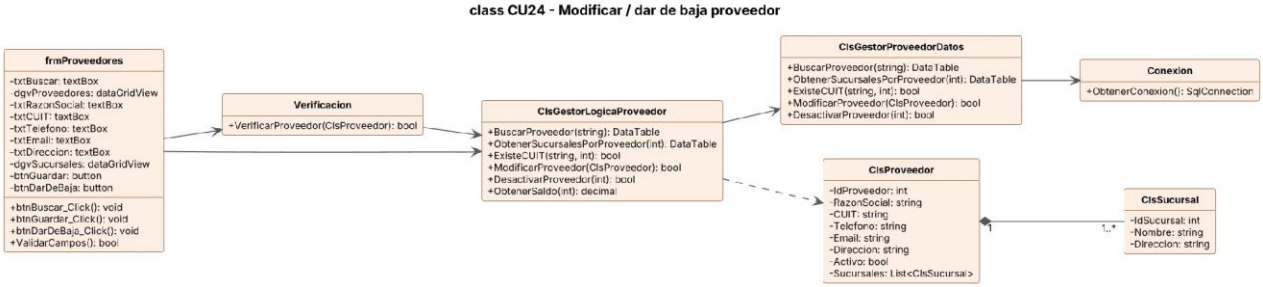


Diagrama de clases



CU25 - Registrar pago a proveedor (cuenta corriente)

ID y Nombre	**CU25 Registrar pago a proveedor (cuenta corriente)**	
Descripción	Permite al dueño o empleado de compras consultar la cuenta corriente de los proveedores (compras pendientes de pago, pagos realizados, deuda total y vencido) y registrar un pago total o parcial de una compra, indicando el medio de pago y el monto. Si el dinero sale de la caja abierta, el sistema registra también el egreso de caja. Un pago cargado por error se puede anular indicando el motivo.	
Actores	Administrador - Empleado Compras (permiso 4.2.0 Cuenta Corriente de Proveedores).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 4.2.0. El proveedor debe tener compras con saldo pendiente. Para pagar desde la caja, la caja debe estar abierta.	
Post-Condición	El pago queda registrado y el saldo de la compra se actualiza. Si el pago sale de la caja, se registra el egreso de caja.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Proveedores > Cuenta corriente de Proveedores.	
	2. El usuario selecciona el proveedor (o marca "Todos los proveedores") y presiona "Buscar".	
	3. El sistema muestra las compras con saldo, la deuda total y lo vencido.	3.1 Incluir pagadas: El usuario marca "Incluir compras pagadas" para ver también las compras ya canceladas.
	4. El usuario selecciona una compra y presiona "Registrar pago".	4.1 Compra ya pagada: El sistema informa que la compra seleccionada ya está pagada.
	5. El usuario elige el medio de pago, ingresa el monto (o presiona "Pagar todo") y, si corresponde, marca que el dinero sale de la caja.	

	6. El sistema valida los datos ingresados.	6.1 Monto inválido: El sistema informa que el monto debe ser mayor a cero o que supera el saldo pendiente. 6.2 Efectivo insuficiente: Si sale de la caja, el sistema informa que el monto supera el efectivo disponible.
	7. El usuario presiona "Guardar".	
	8. El sistema registra el pago, actualiza el saldo y, si corresponde, registra el egreso en la caja.	
	9. El sistema informa que el pago se registró correctamente y muestra los pagos de la compra.	9.1 Anular pago: El usuario selecciona un pago, presiona "Anular pago" e ingresa el motivo; el sistema anula el pago y actualiza el saldo.

Diagrama de caso de uso

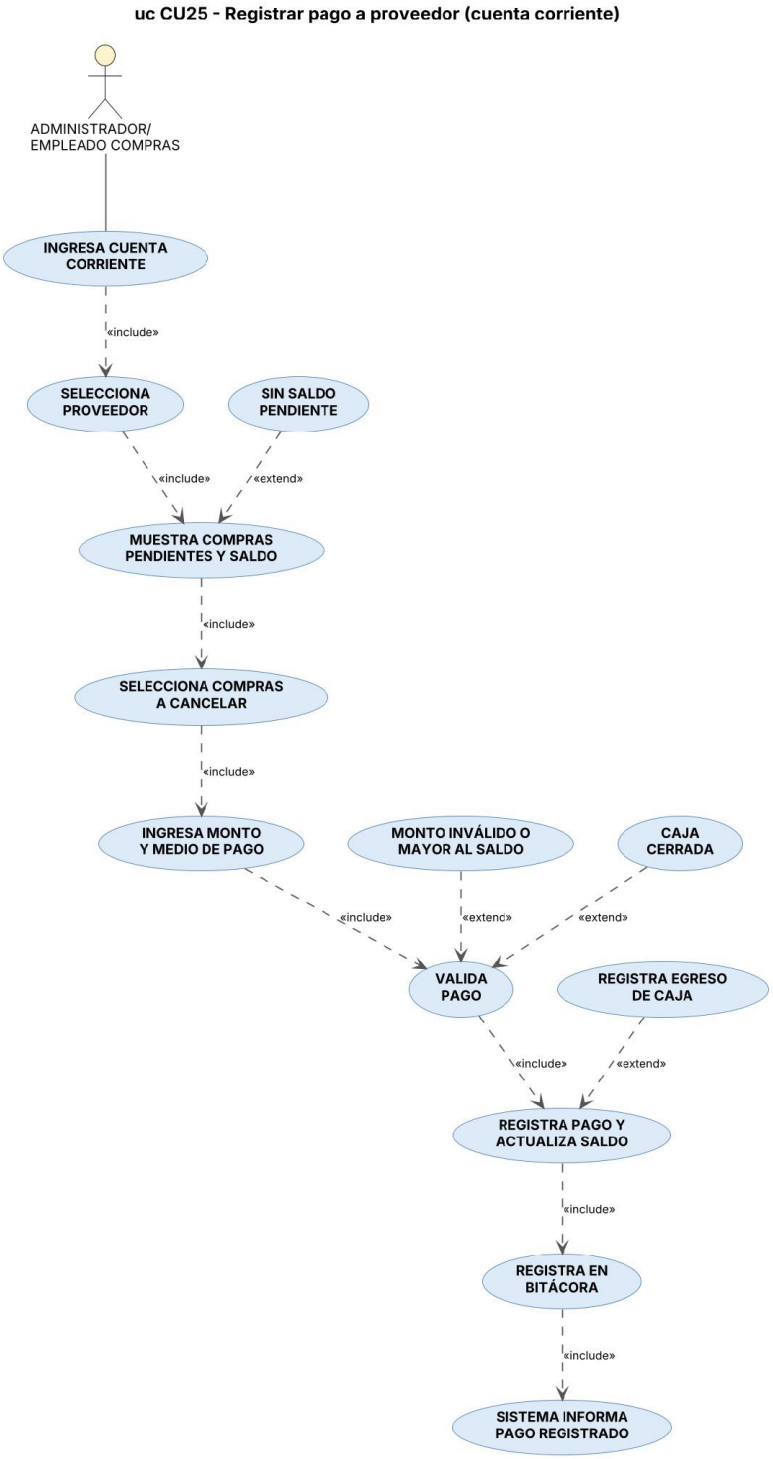


Diagrama de secuencia

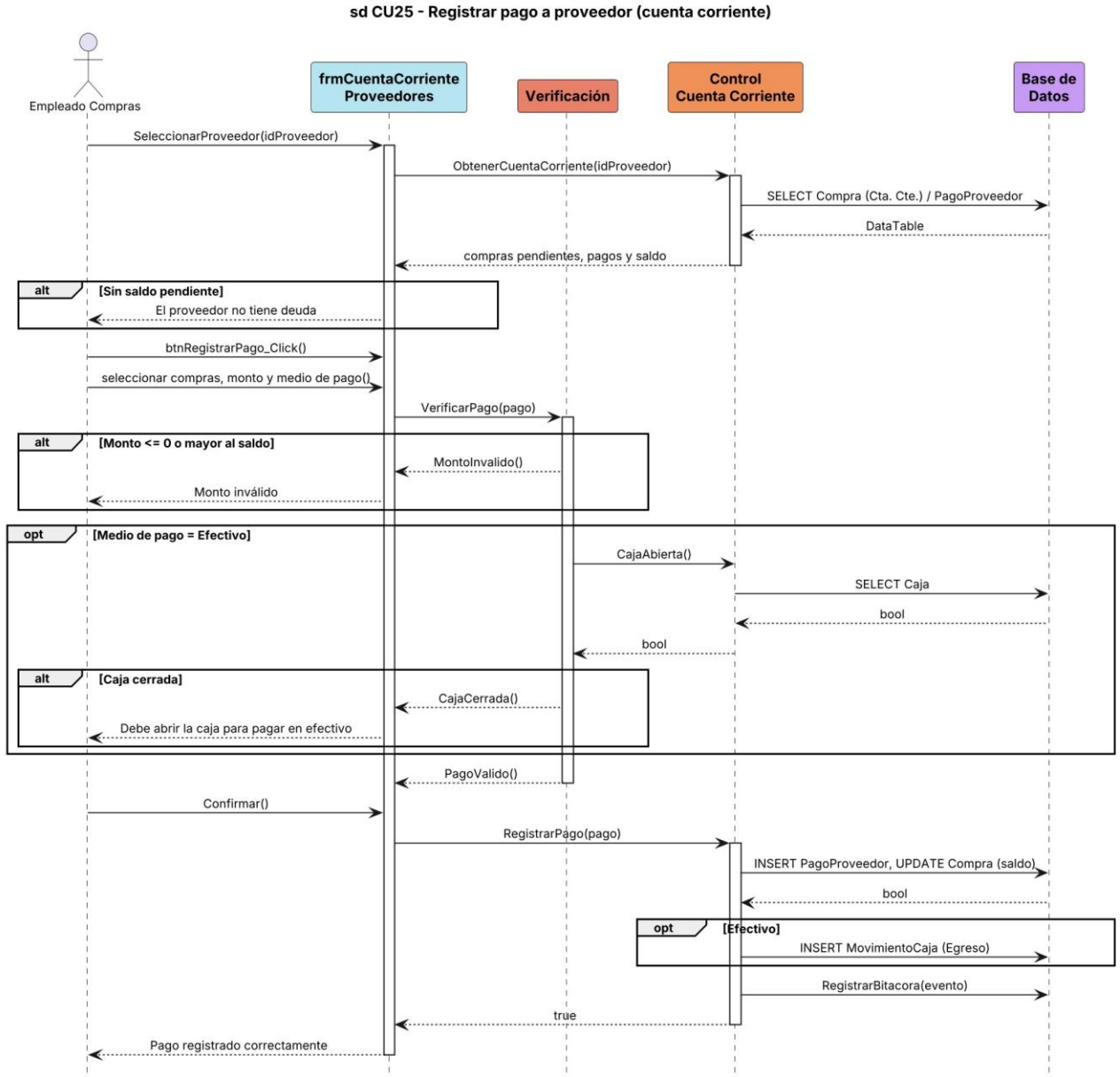


Diagrama de clases



CU26 - Consultar listado de precios

ID y Nombre	**CU26 Consultar listado de precios**	
Descripción	Permite a cualquier empleado con permiso consultar el listado de precios de venta de los productos activos, filtrarlo por proveedor, sucursal, categoría, marca, artículo o solo los que tienen oferta, y exportarlo a Excel para tenerlo en el mostrador.	
Actores	Administrador - Empleado General - Empleado Ventas (permiso 5.1.0 Listado de Precios).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 5.1.0 Listado de Precios.	
Post-Condición	Se muestra el listado de precios filtrado. Opcionalmente se genera el archivo Excel. No se modifican datos.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Listados > Precios y reposición.	
	2. El usuario elige "Lista de precios".	
	3. El usuario selecciona los filtros y presiona "Buscar".	3.1 Limpiar: El usuario presiona "Limpiar" y el sistema quita los filtros.
	4. El sistema muestra los productos con su precio de venta.	
	5. El usuario presiona "Exportar a Excel".	5.1 Sin datos: El sistema informa que no hay artículos para exportar.

	6. El sistema genera el archivo e informa que se exportó correctamente.	6.1 Error al exportar: El sistema informa que no pudo exportar el archivo.

Diagrama de caso de uso

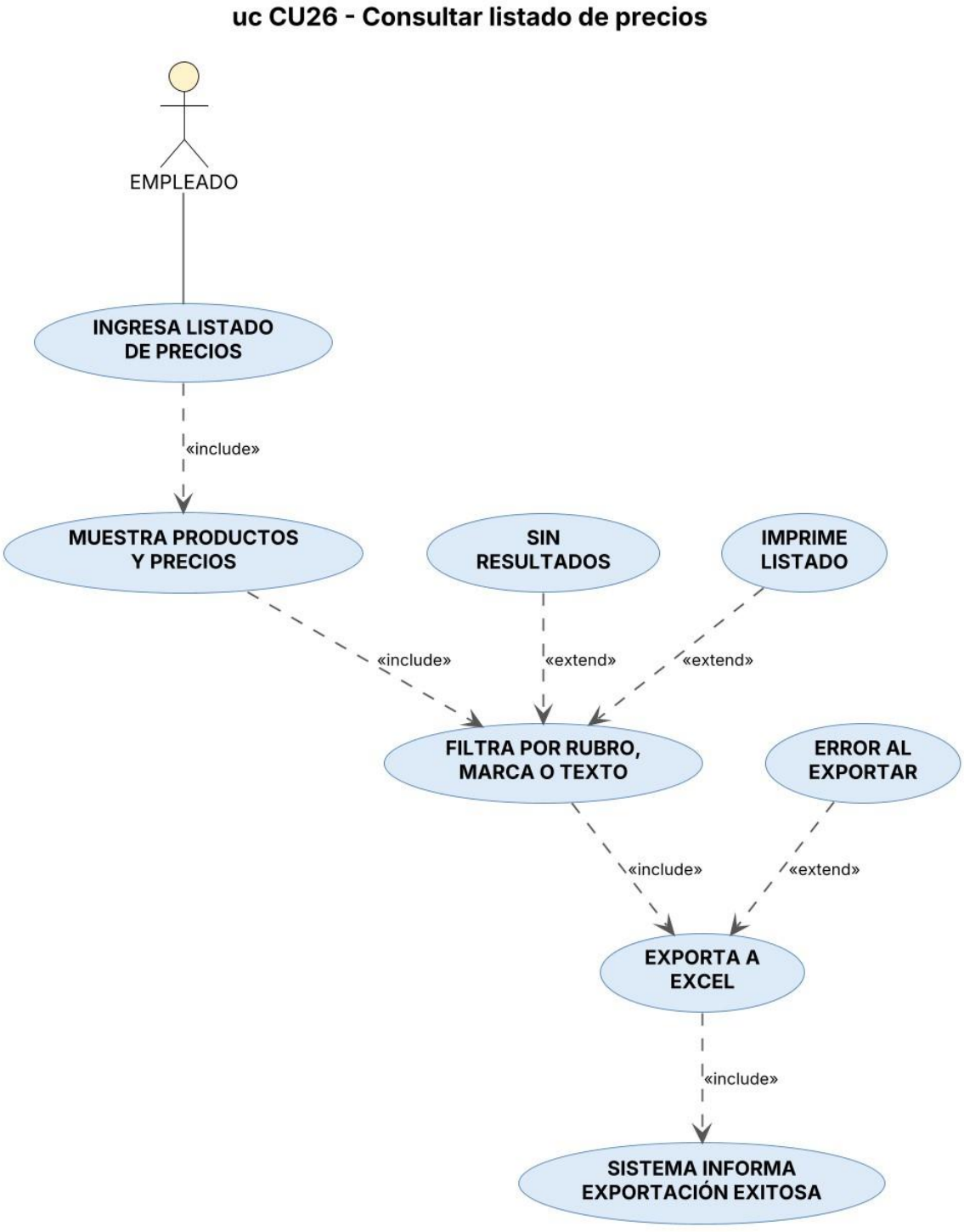


Diagrama de secuencia

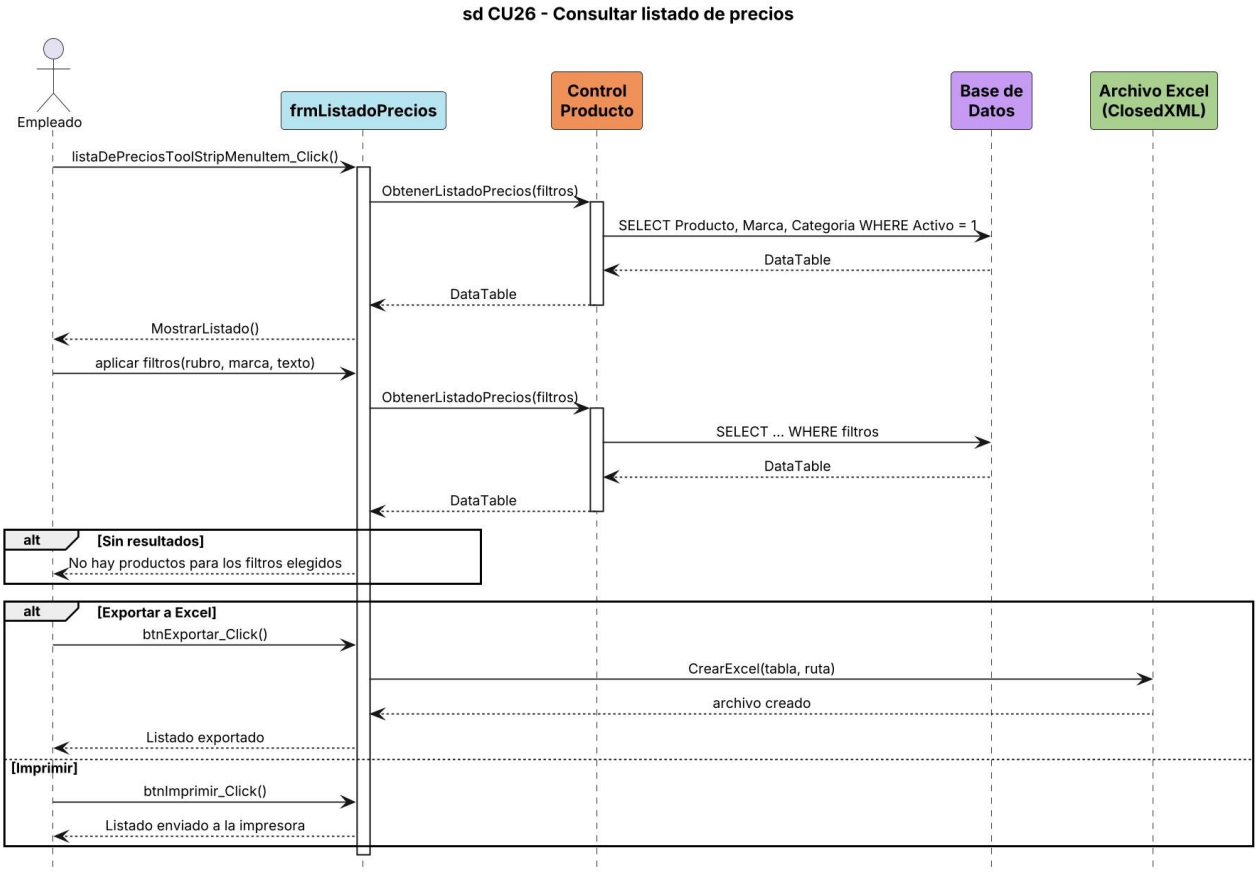
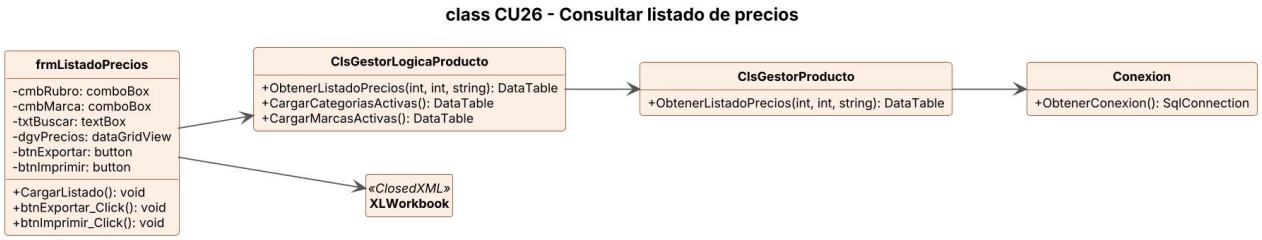


Diagrama de clases



CU27 - Consultar listado de reposición

ID y Nombre	CU27 Consultar listado de reposición	
Descripción	Permite al dueño o empleado de compras ver los productos cuyo stock actual está en o por debajo del stock mínimo, con la cantidad sugerida para reponer, filtrando por proveedor, sucursal, categoría, marca o artículo. El listado puede exportarse a Excel para armar el pedido al proveedor.	
Actores	Administrador - Empleado Compras - Empleado General (permiso 5.2.0 Listado de Reposición).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 5.2.0 Listado de Reposición. Los productos deben tener configurado el stock mínimo.	
Post-Condición	Se muestra el listado de productos a reponer. Opcionalmente se exporta. No se modifican datos del stock.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al módulo Listados > Precios y reposición.	
	2. El usuario elige "Reposición".	
	3. El usuario selecciona los filtros y presiona "Buscar".	
	4. El sistema muestra los productos a reponer con su stock actual y la cantidad sugerida.	4.1 Sin productos a reponer: El listado queda vacío porque todos los productos tienen stock suficiente.
	5. El usuario presiona "Exportar a Excel".	5.1 Sin datos: El sistema informa que no hay artículos para exportar.

	6. El sistema genera el archivo e informa que se exportó correctamente.	6.1 Error al exportar: El sistema informa que no pudo exportar el archivo.

Diagrama de caso de uso

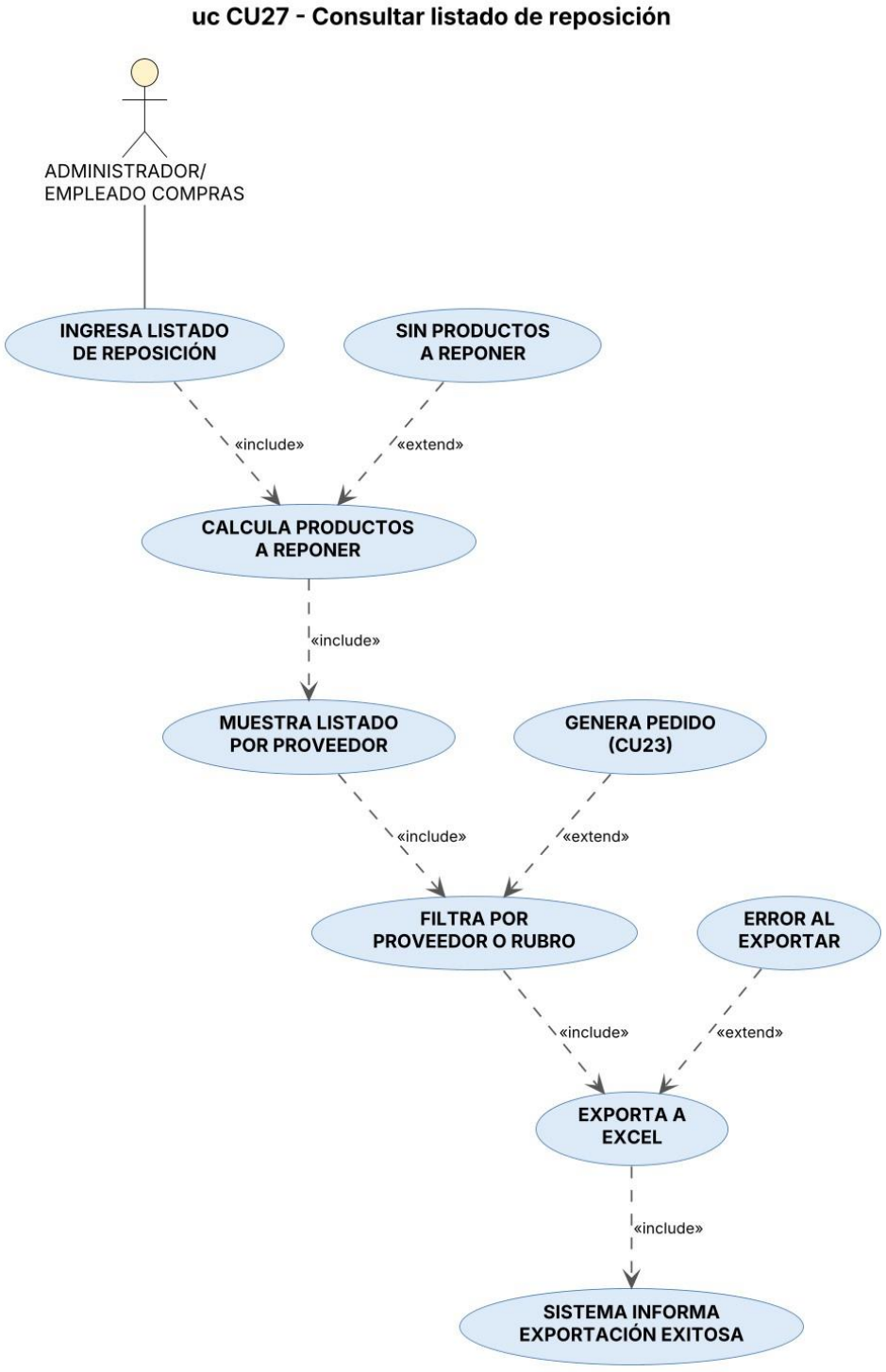


Diagrama de secuencia

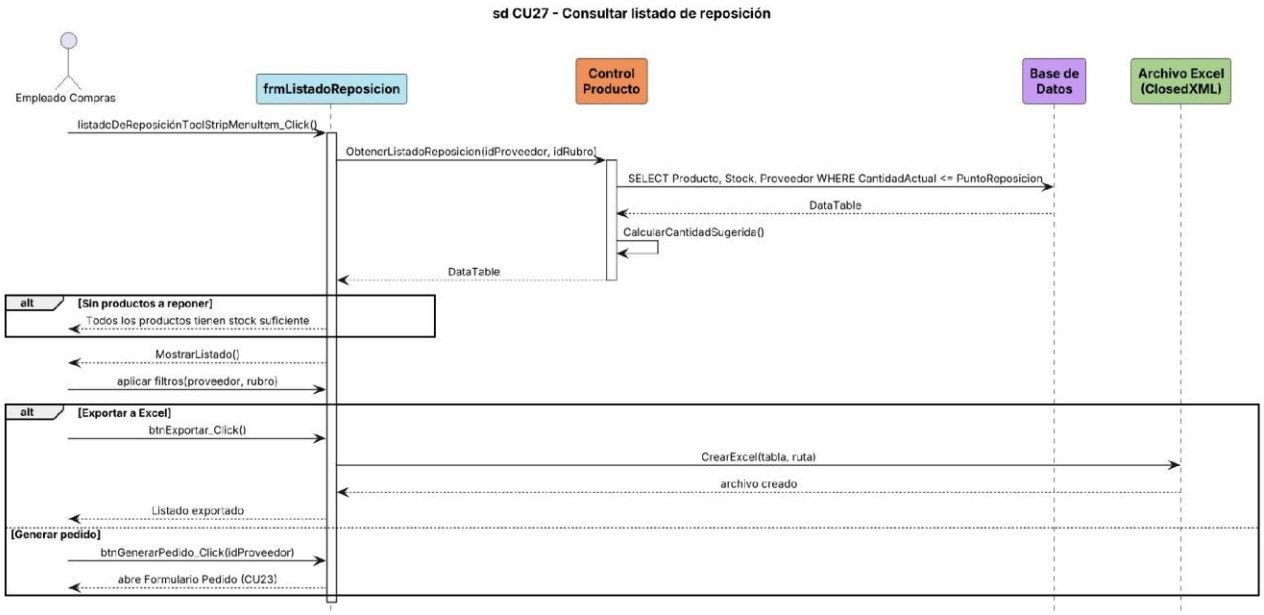
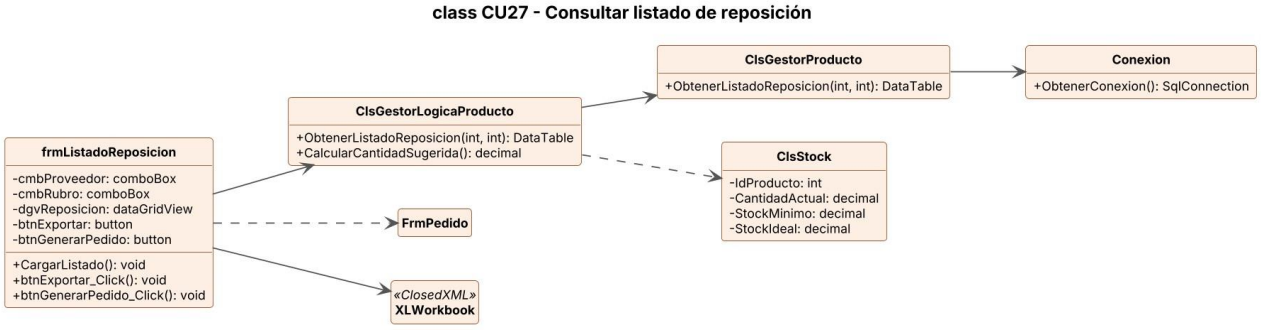


Diagrama de clases



CU28 - Generar informe de ventas

ID y Nombre	**CU28 Generar informe de ventas**	
Descripción	Permite al Administrador generar informes de ventas para un rango de fechas en cinco variantes: por fecha, detalle, por rubro, por artículo y por medio de pago. Además del rango de fechas se puede filtrar por vendedor, medio de pago, categoría o artículo. El informe se muestra en pantalla y puede exportarse a Excel.	
Actores	Administrador (permisos 6.1.0 a 6.5.0 Informes).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso del informe elegido.	
Post-Condición	Se muestra el informe con sus totales. Opcionalmente se genera el archivo Excel. No se modifican datos.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El Administrador accede al módulo Informes > Ventas.	
	2. El Administrador elige el tipo de informe: por fecha, detalle, por rubro, por artículo o por medio de pago.	
	3. El Administrador ingresa el rango de fechas y los filtros, y presiona "Buscar".	3.1 Rango de fechas inválido: El sistema informa que la fecha desde no puede ser posterior a la fecha hasta.
	4. El sistema obtiene las ventas del período agrupadas según el tipo de informe.	
	5. El sistema muestra el informe con cantidades, importes y totales.	

	6. El Administrador presiona "Exportar a Excel".	6.1 Sin datos: El sistema informa que no hay datos para exportar.
	7. El sistema genera el archivo e informa que el informe se exportó correctamente.	7.1 Error al exportar: El sistema informa que no pudo exportar el archivo.

Diagrama de caso de uso

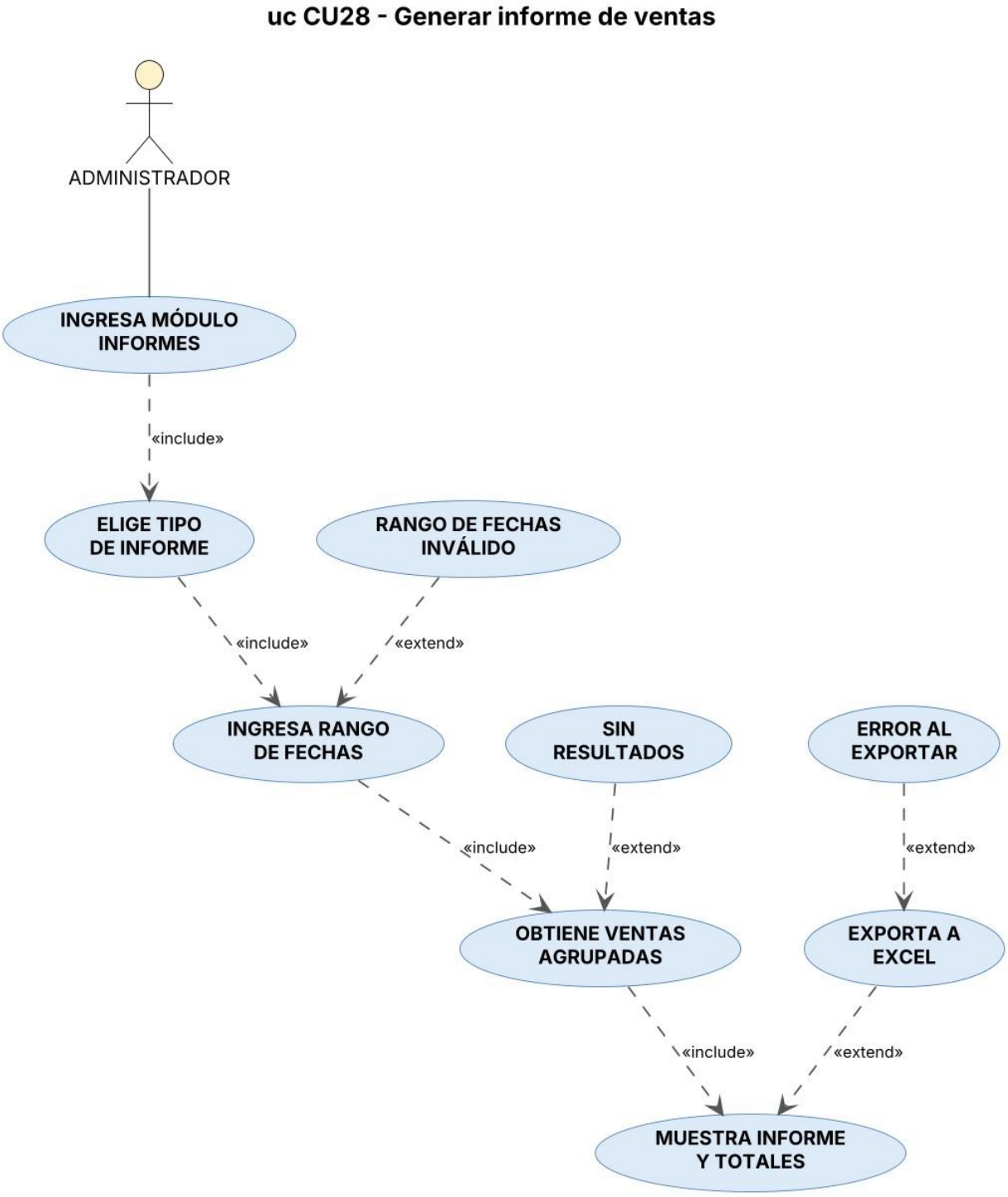


Diagrama de secuencia

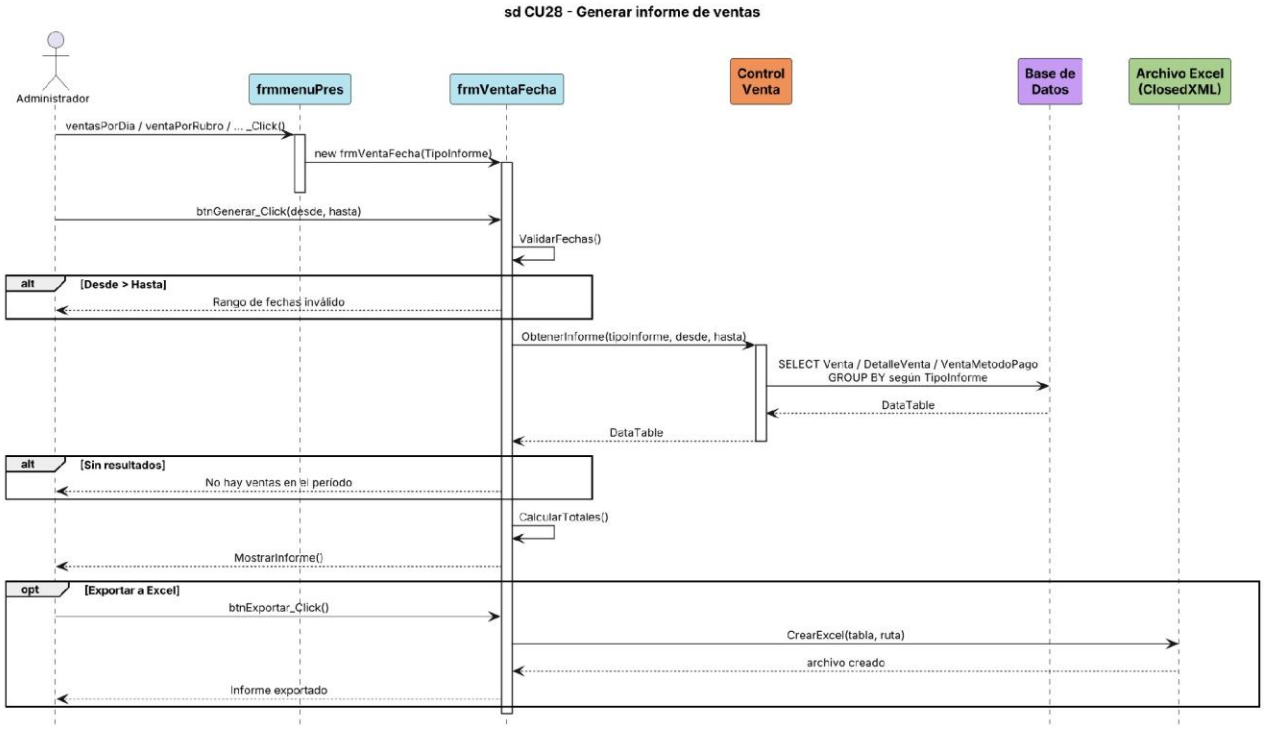
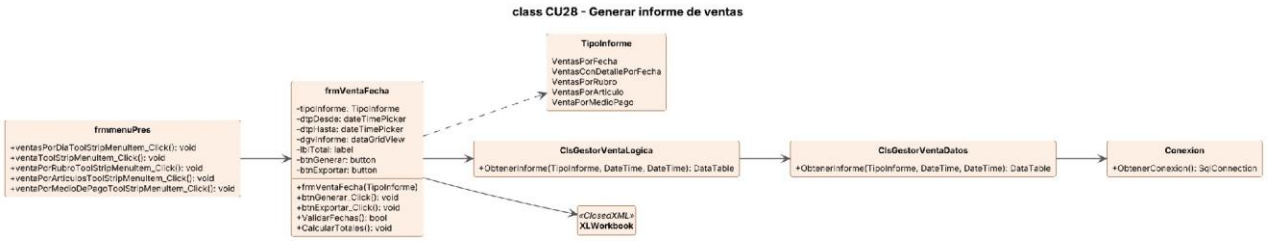


Diagrama de clases



CU29 - Generar balance de ingresos y egresos

ID y Nombre	**CU29 Generar balance de ingresos y egresos**	
Descripción	Permite al Administrador obtener el balance del kiosco para un período: indicadores de ventas, ticket promedio, ventas anuladas, devoluciones, cajas cerradas y cajas con faltante, junto con el detalle de los movimientos de ingresos y egresos, que se puede filtrar por tipo, medio de pago y usuario que lo registró. El balance puede exportarse a Excel.	
Actores	Administrador (permiso 6.6.0 Balance).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 6.6.0 Balance.	
Post-Condición	Se muestra el balance del período con sus indicadores y el detalle de movimientos. Opcionalmente se genera el archivo Excel. No se modifican datos.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El Administrador accede al módulo Informes > Balance.	
	2. El Administrador elige el período con "Hoy", "Este mes" o un rango de fechas.	2.1 Rango de fechas inválido: El sistema informa que la fecha desde no puede ser mayor a la fecha hasta.
	3. El Administrador presiona "Consultar".	
	4. El sistema obtiene los ingresos y egresos del período.	4.1 Error al obtener datos: El sistema informa que no se pudo obtener el balance.
	5. El sistema muestra los indicadores y el detalle de movimientos.	

	6. El Administrador filtra el detalle por tipo, medio de pago o usuario y presiona "Filtrar".	
	7. El Administrador presiona "Exportar a Excel".	
	8. El sistema genera el archivo e informa que el balance se exportó correctamente.	8.1 Error al exportar: El sistema informa que no se pudo exportar el balance.

Diagrama de caso de uso

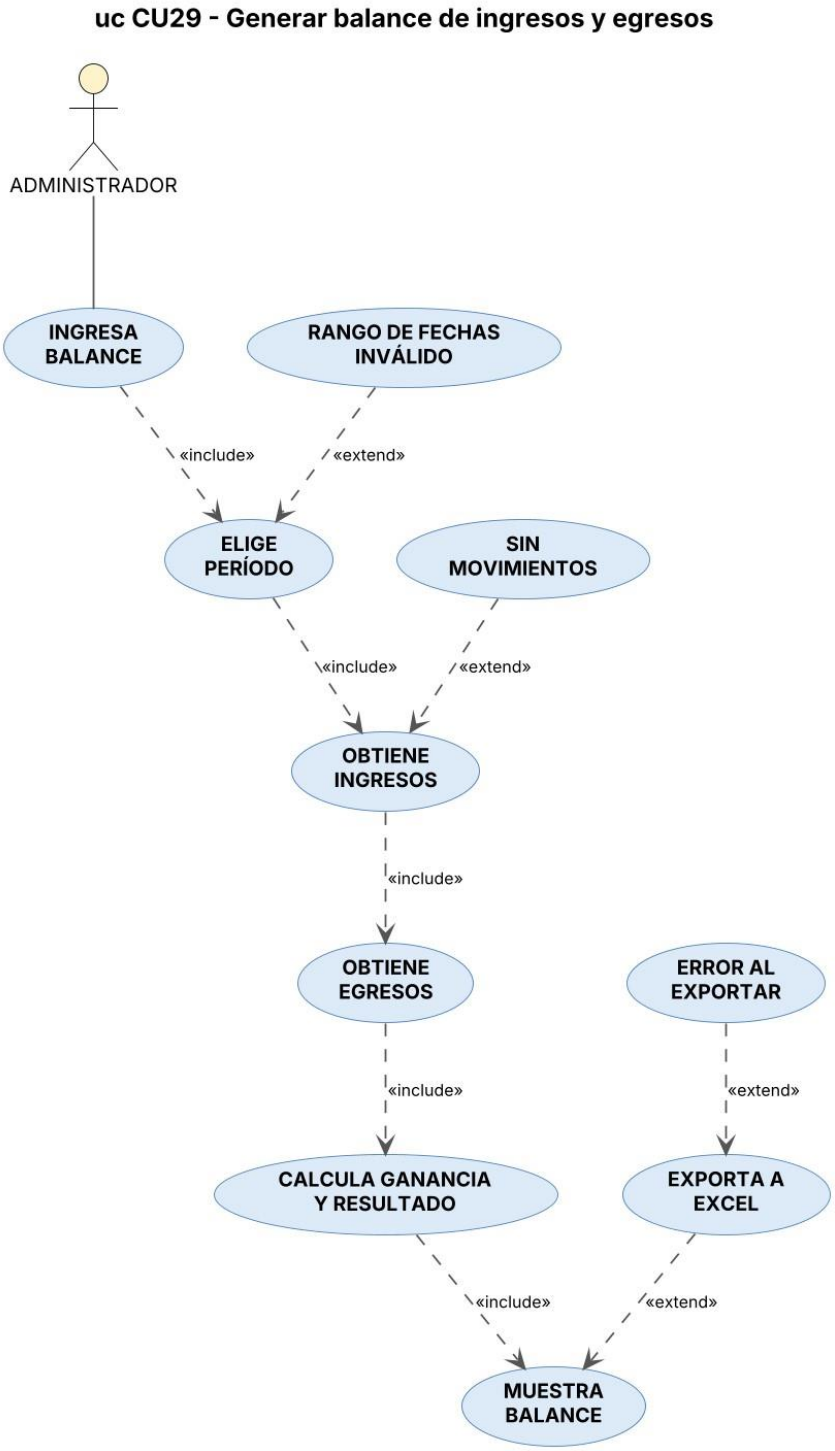


Diagrama de secuencia

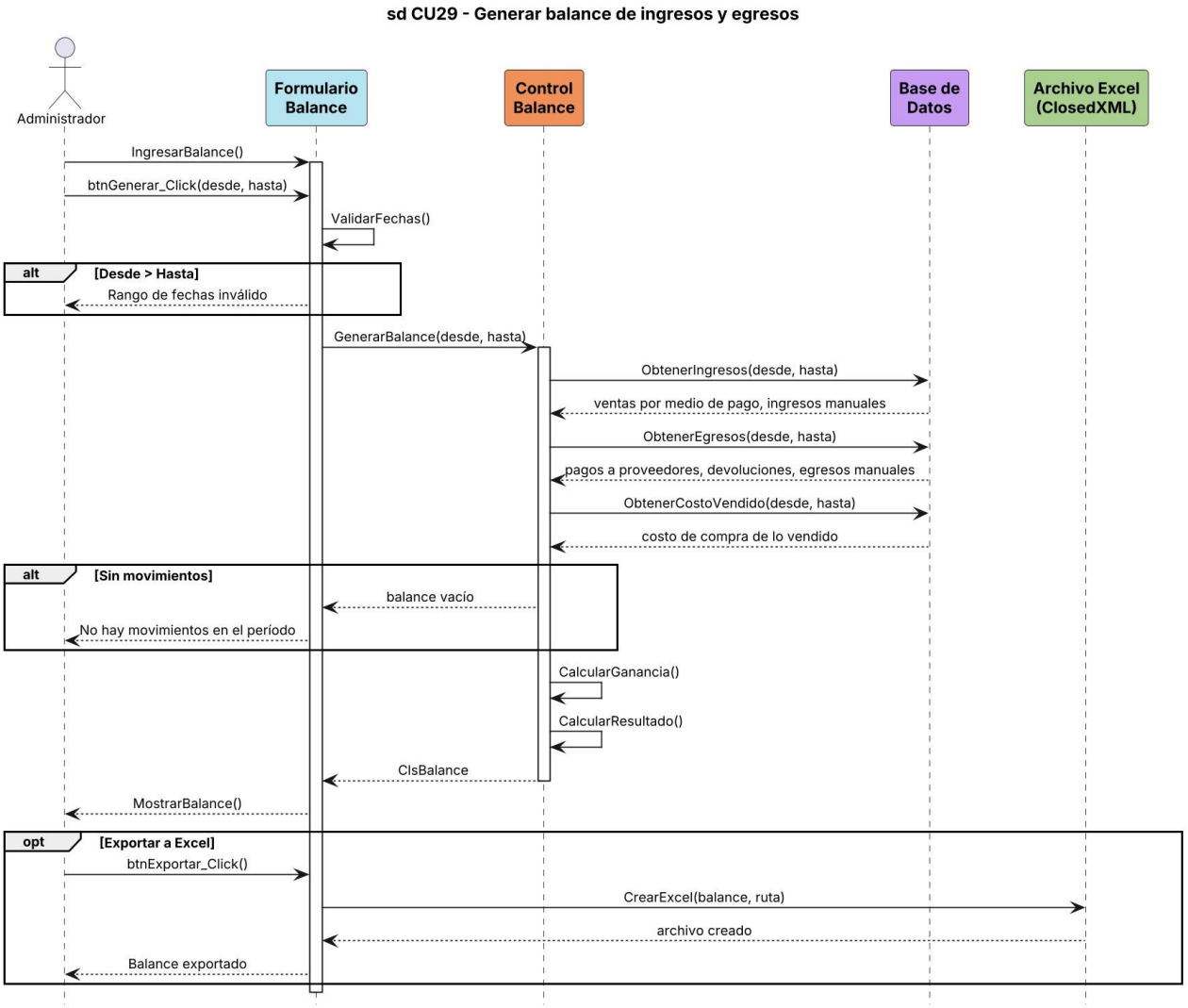
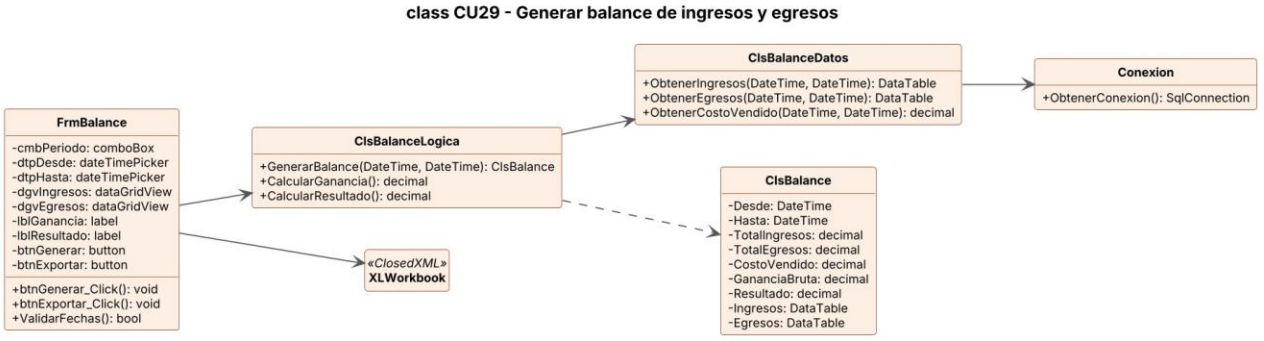


Diagrama de clases



CU30 - Consultar estado e historial de caja

ID y Nombre	**CU30 Consultar estado e historial de caja**	
Descripción	Permite al dueño o empleado autorizado ver el estado de la caja abierta (quién la abrió, fondo inicial, ingresos y egresos en efectivo, efectivo esperado y otros medios) con los movimientos de la sesión, y consultar el historial de cajas anteriores por rango de fechas.	
Actores	Administrador - Empleado autorizado (permiso 7.5.0 Estado e Historial de Caja).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 7.5.0.	
Post-Condición	Se muestra el estado de la caja y, si se elige, el historial. No se modifican datos.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al menú Caja > Estado de Caja.	
	2. El sistema muestra si la caja está abierta o cerrada, quién la abrió y cuándo.	2.1 Caja cerrada: El sistema muestra "CAJA CERRADA" y permite abrirla (CU31).
	3. El sistema muestra el fondo inicial, los ingresos y egresos en efectivo, el efectivo esperado y los otros medios.	
	4. El sistema muestra los movimientos de la sesión.	4.1 Acciones desde el estado: El usuario puede registrar un ingreso o egreso (CU32 / CU33), anular un movimiento (CU35) o cerrar la caja (CU05).
	5. El usuario presiona "Historial".	

	6. El usuario ingresa el rango de fechas y presiona "Buscar".	6.1 Rango de fechas inválido: El sistema informa que la fecha desde no puede ser mayor a la fecha hasta.
	7. El sistema muestra las cajas del período con su apertura, cierre y diferencia.	

Diagrama de caso de uso

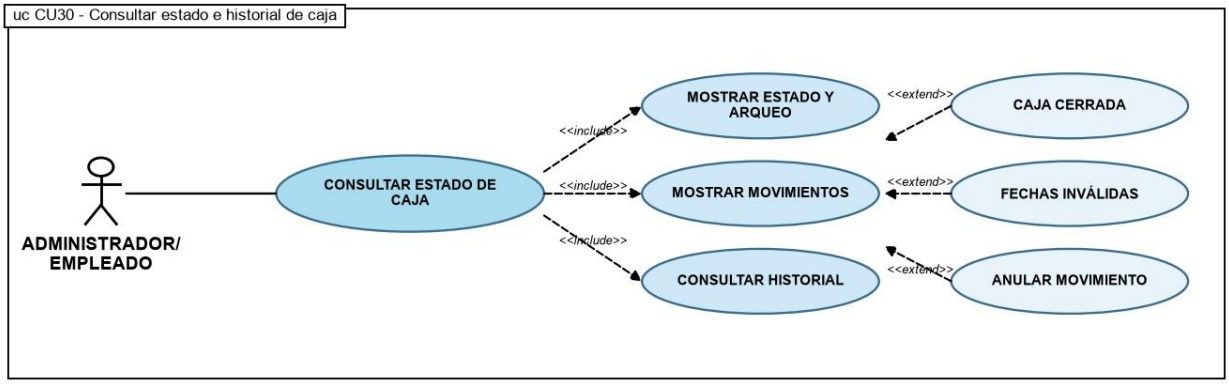


Diagrama de secuencia

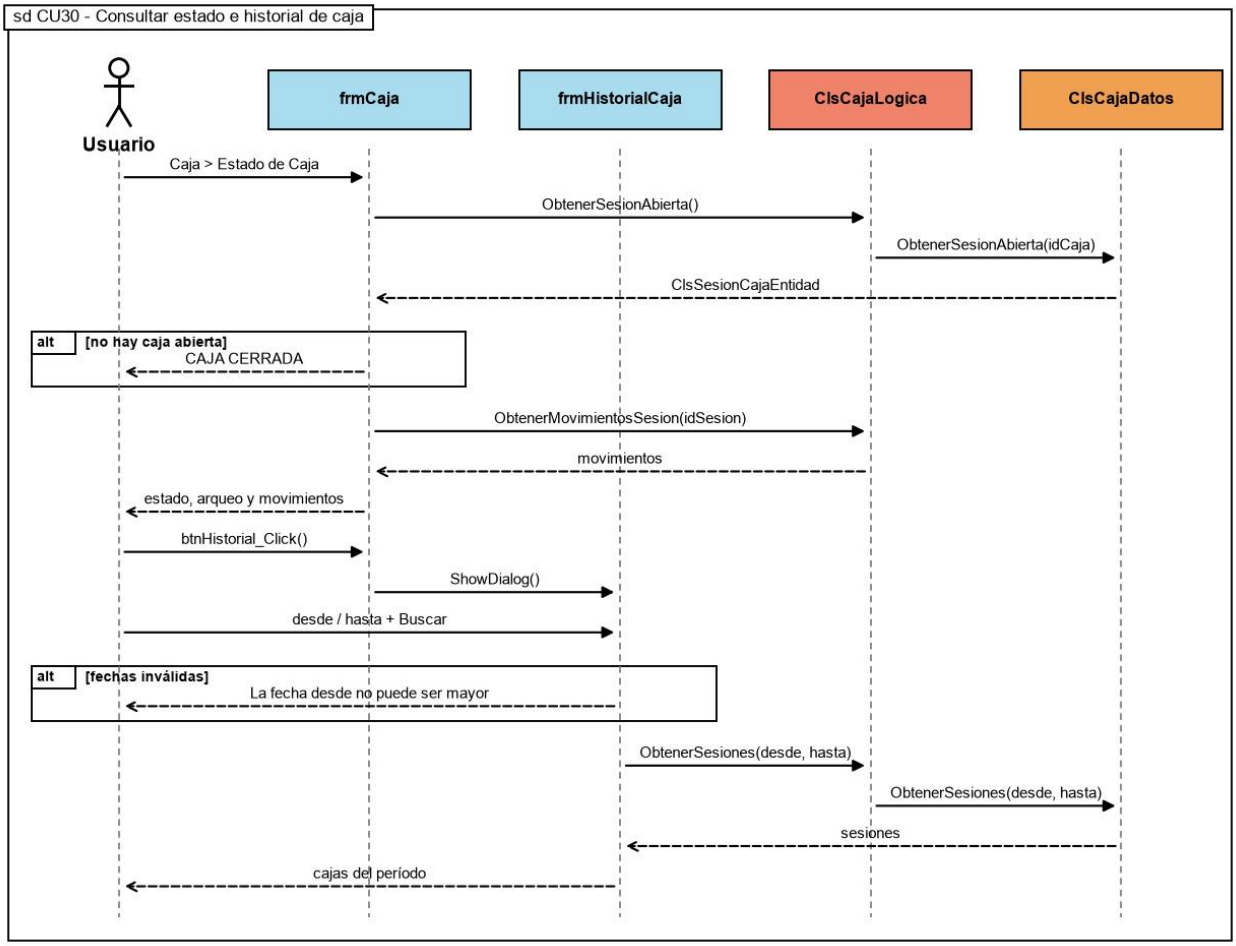
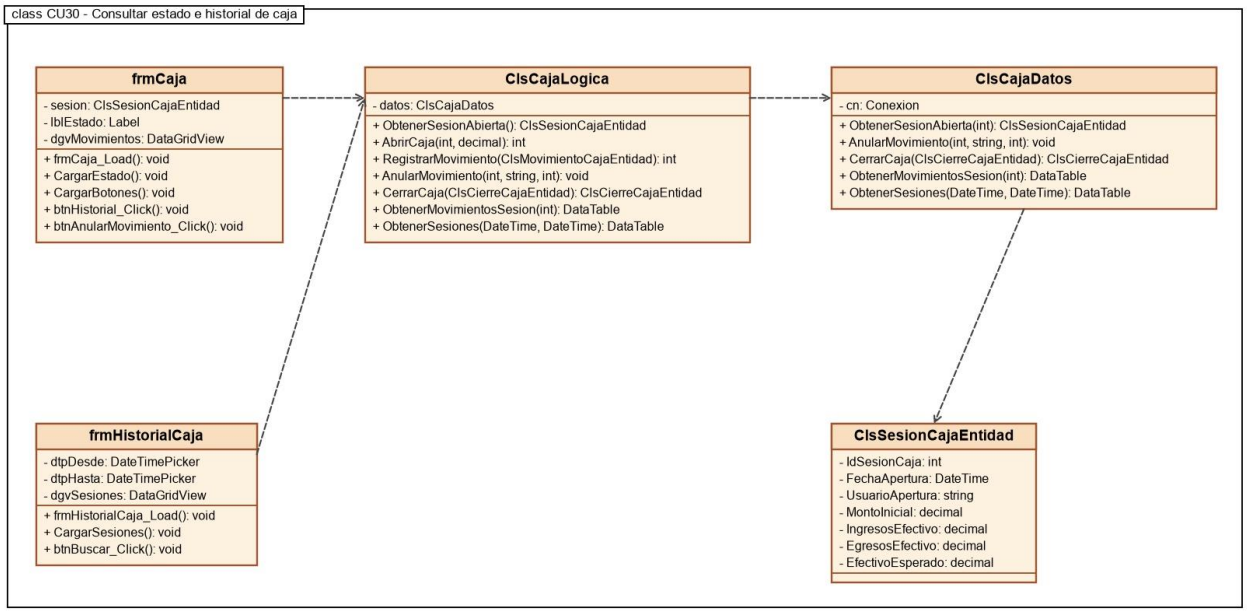


Diagrama de clases



CU31 - Abrir caja

ID y Nombre	**CU31 Abrir caja**	
Descripción	Permite al dueño o empleado autorizado abrir la caja al comenzar la jornada o el turno, ingresando el fondo inicial en efectivo (el cambio con el que arranca la caja). A partir de la apertura se habilitan las ventas, los cobros en efectivo y los movimientos manuales de caja, que luego se controlan en el cierre (CU05).	
Actores	Administrador - Empleado autorizado (permiso 7.3.0 Abrir Caja).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 7.3.0 Abrir Caja. No debe haber otra caja abierta.	
Post-Condición	La caja queda abierta con el fondo inicial, la fecha, la hora y el usuario. Se habilitan las ventas y los movimientos de caja.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al menú Caja > Abrir Caja.	1.1 Apertura desde la venta: Si el usuario intenta vender con la caja cerrada, el sistema pregunta si desea abrirla ahora.
	2. El sistema muestra el usuario y la fecha de apertura.	
	3. El usuario ingresa el fondo inicial en efectivo.	
	4. El usuario presiona "Abrir caja".	4.1 Sin fondo inicial: Si el monto es cero, el sistema pregunta si desea abrir la caja sin fondo inicial. 4.2 Cancelar: El sistema no abre la caja y no permite vender hasta que se abra.
	5. El sistema registra la apertura de caja.	5.1 Error al abrir: El sistema informa que no se pudo abrir la caja.

	6. El sistema informa que la caja se abrió correctamente con su número de sesión.	

Diagrama de caso de uso

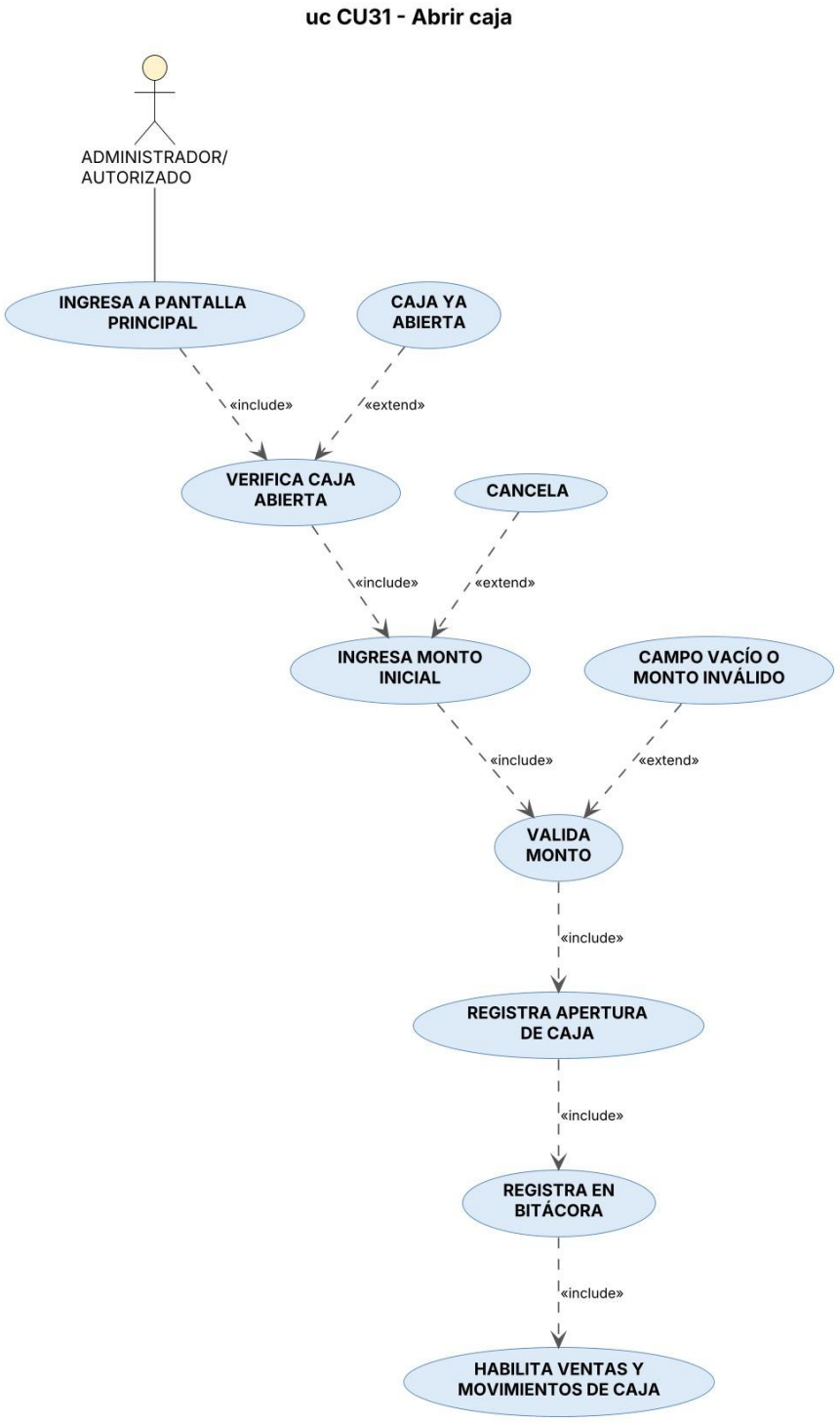


Diagrama de secuencia

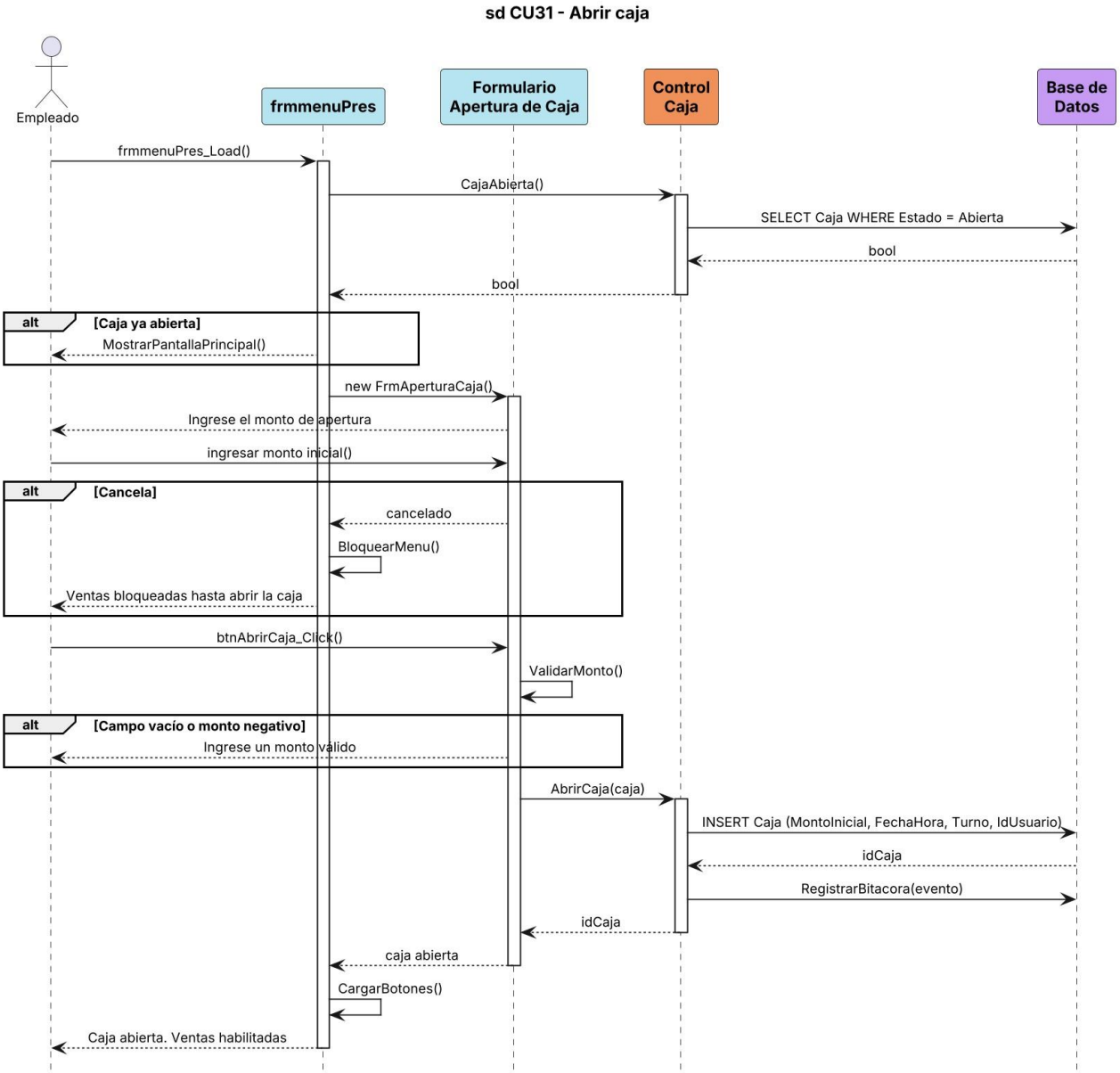
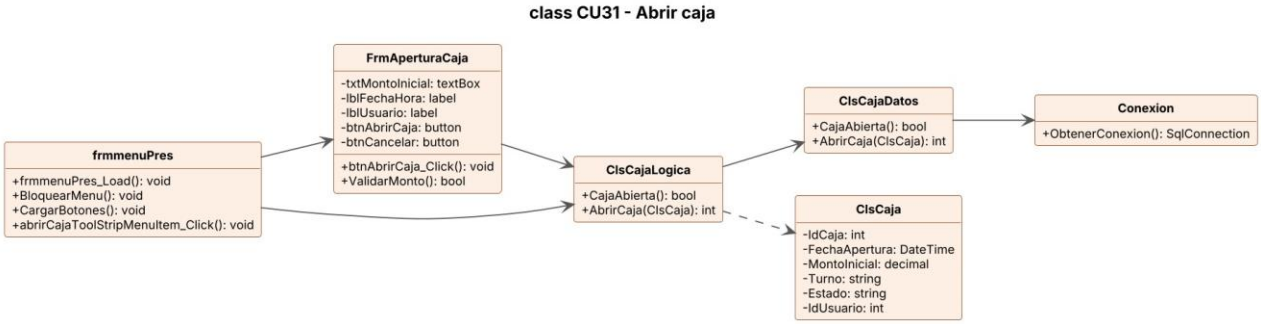


Diagrama de clases



CU32 - Registrar ingreso manual de caja

ID y Nombre	**CU32 Registrar ingreso manual de caja**	
Descripción	Permite al dueño o empleado autorizado registrar un ingreso de dinero en la caja que no surge de una venta (por ejemplo, un aporte de cambio), indicando el concepto, el medio de pago, el importe y la observación con el motivo, que es obligatoria.	
Actores	Administrador - Empleado autorizado (permiso 7.1.0 Nuevo Ingreso).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 7.1.0 Nuevo Ingreso. La caja debe estar abierta (CU31).	
Post-Condición	El ingreso queda registrado en la caja con concepto, importe, observación, fecha, hora y usuario, y el saldo de caja se actualiza.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al menú Caja > Nuevo Ingreso.	1.1 Caja cerrada: El sistema informa que debe abrir la caja antes de registrar movimientos.
	2. El sistema abre el formulario de ingreso de caja.	
	3. El usuario selecciona el concepto y el medio de pago.	
	4. El usuario ingresa el importe.	
	5. El usuario ingresa la observación (motivo) del ingreso.	
	6. El usuario presiona "Guardar".	6.1 Cancelar: El sistema cierra el formulario sin registrar el movimiento.

	7. El sistema valida los datos.	7.1 Campos vacíos: El sistema informa que debe seleccionar el concepto y el medio de pago e ingresar la observación. 7.2 Importe inválido: El sistema informa que el importe debe ser mayor a cero.
	8. El sistema registra el ingreso en la caja y actualiza el saldo.	
	9. El sistema informa que el movimiento se registró correctamente.	

Diagrama de caso de uso

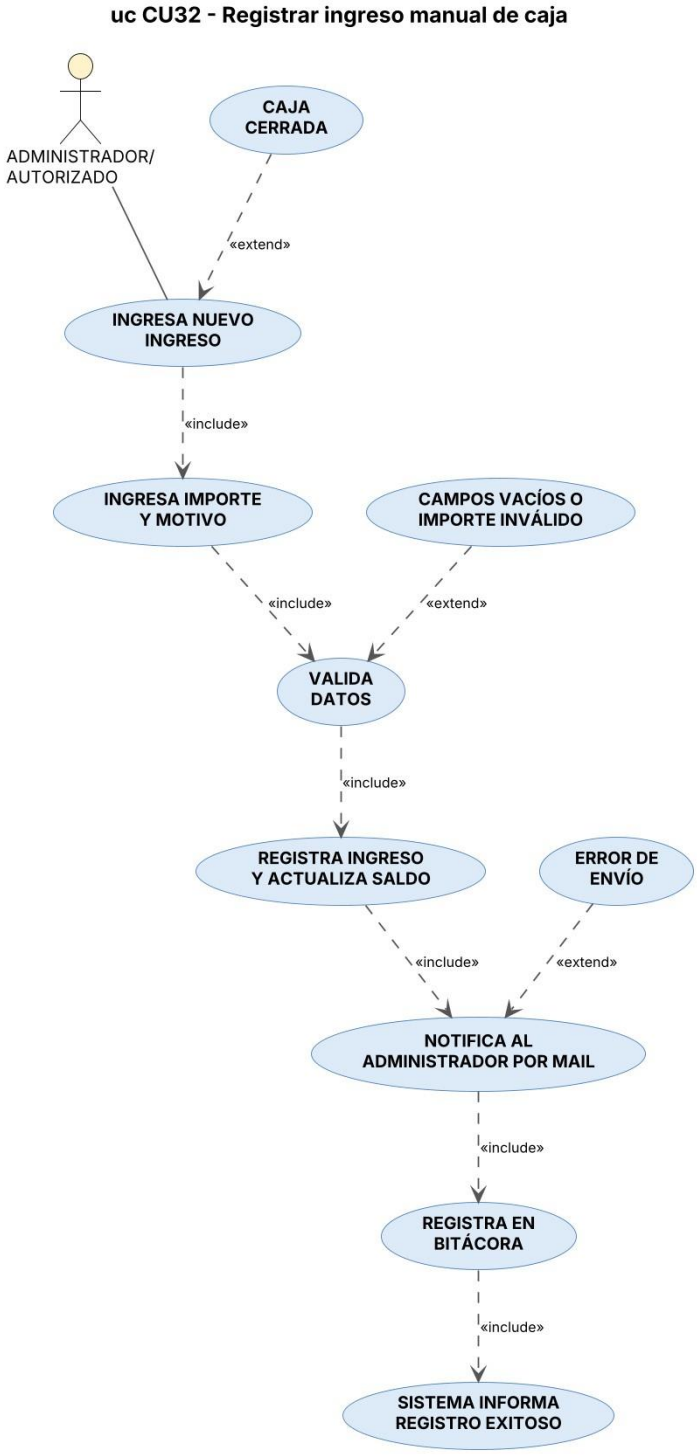


Diagrama de secuencia

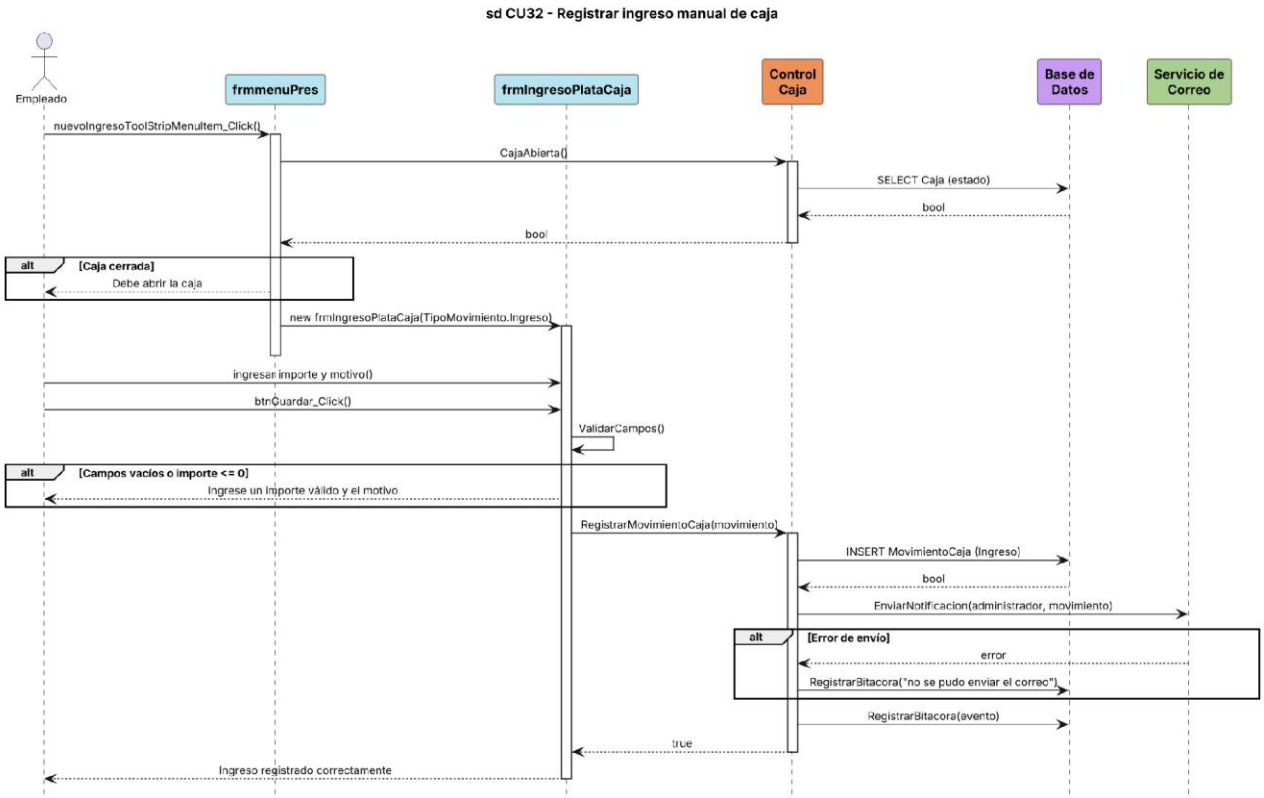
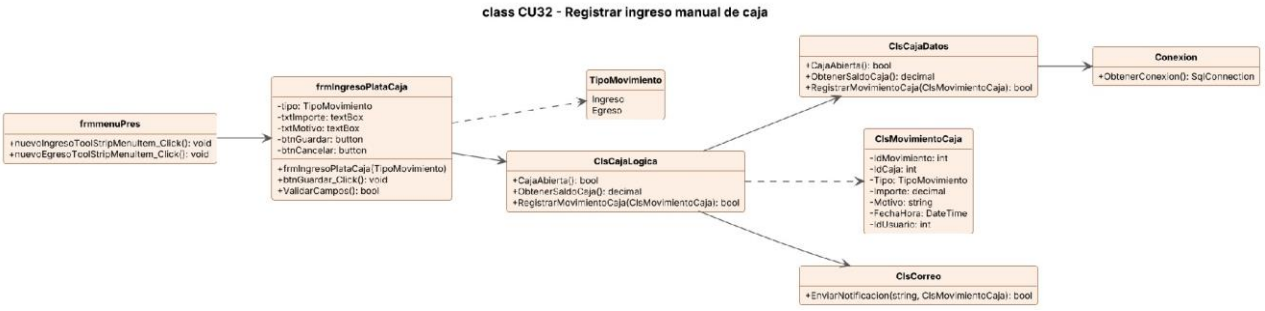


Diagrama de clases



CU33 - Registrar egreso manual de caja

ID y Nombre	**CU33 Registrar egreso manual de caja**	
Descripción	Permite al dueño o empleado autorizado registrar un egreso de dinero de la caja que no surge de una venta (retiro de caja, gasto, vale o adelanto a un empleado o pago de sueldo), indicando el concepto, el medio de pago, el importe y la observación con el motivo. Para los vales y los sueldos se indica además el empleado que recibe el dinero.	
Actores	Administrador - Empleado autorizado (permiso 7.2.0 Nuevo Egreso).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 7.2.0 Nuevo Egreso. La caja debe estar abierta (CU31).	
Post-Condición	El egreso queda registrado en la caja con concepto, importe, observación, fecha, hora y usuario, y el saldo de caja se actualiza.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al menú Caja > Nuevo Egreso.	1.1 Caja cerrada: El sistema informa que debe abrir la caja antes de registrar movimientos.
	2. El sistema abre el formulario de egreso de caja.	
	3. El usuario selecciona el concepto y el medio de pago.	3.1 Concepto con empleado: Si el concepto es un vale o un pago de sueldo, el usuario selecciona el empleado que recibe el dinero.
	4. El usuario ingresa el importe.	
	5. El usuario ingresa la observación (motivo) del egreso.	
	6. El usuario presiona "Guardar".	6.1 Cancelar: El sistema cierra el formulario sin registrar el movimiento.

	7. El sistema valida los datos.	7.1 Campos vacíos: El sistema informa los datos que faltan completar. 7.2 Importe inválido: El sistema informa que el importe debe ser mayor a cero. 7.3 Saldo insuficiente: El sistema informa que el importe supera el efectivo disponible en la caja.
	8. El sistema registra el egreso en la caja y actualiza el saldo.	
	9. El sistema informa que el movimiento se registró correctamente.	

Diagrama de caso de uso

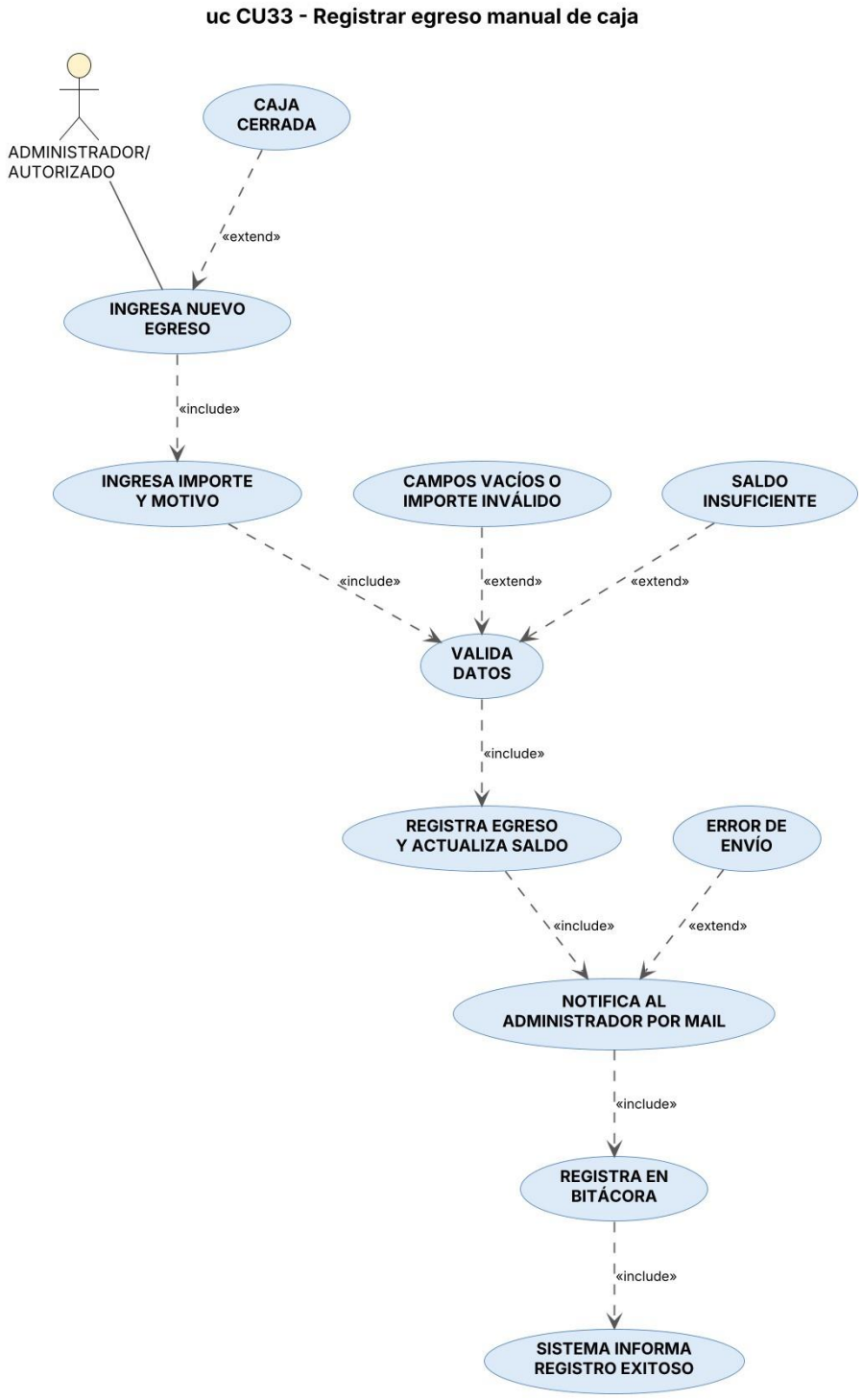


Diagrama de secuencia

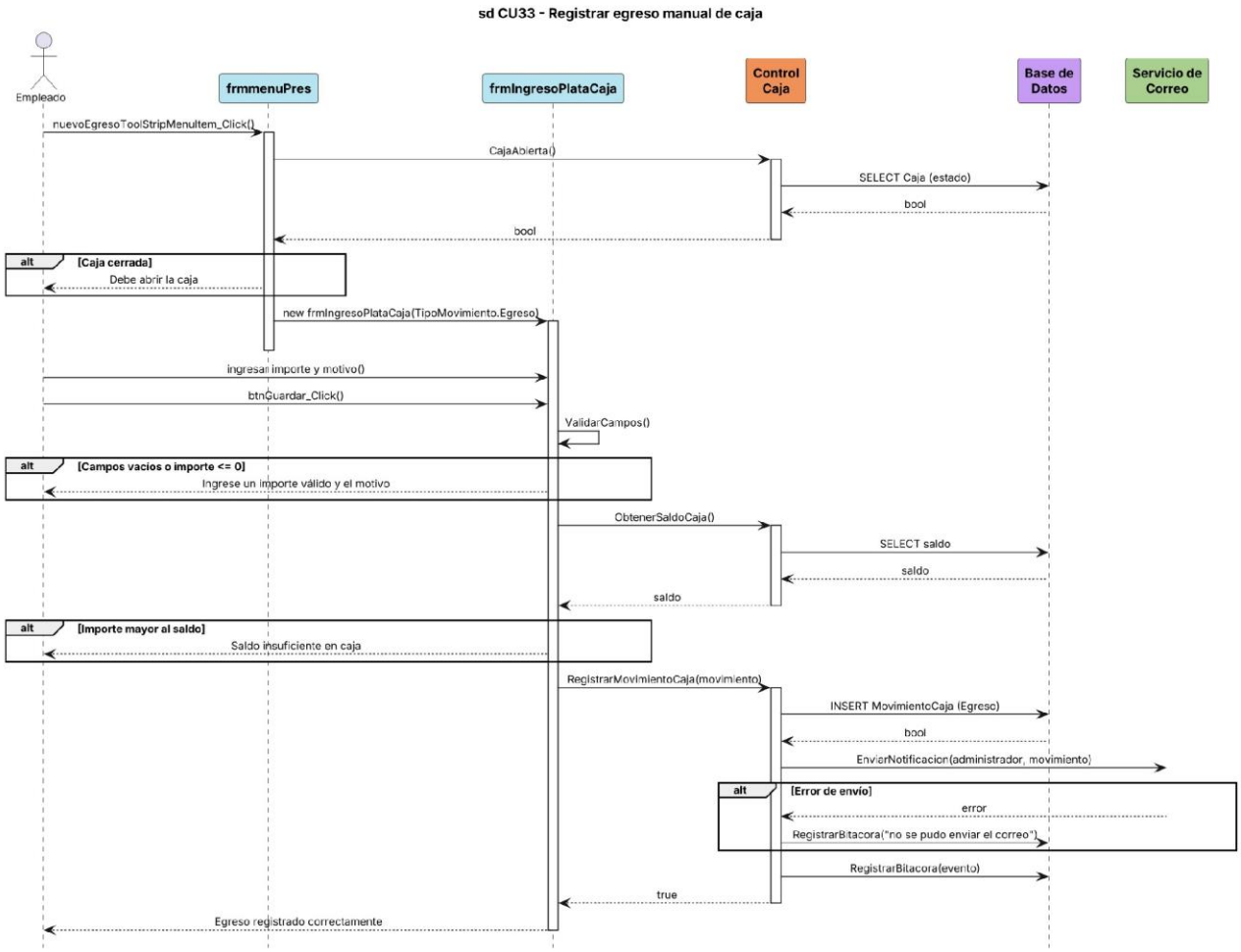
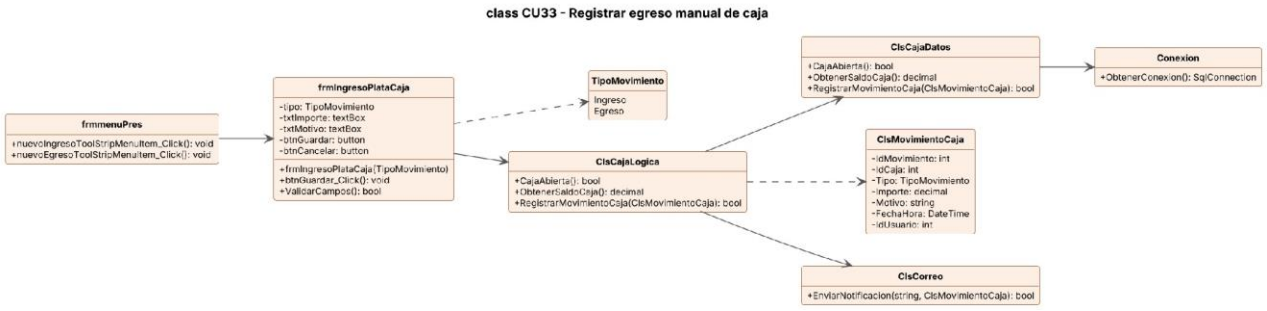


Diagrama de clases



CU34 - Consultar alertas

ID y Nombre	**CU34 Consultar alertas**	
Descripción	Permite a los empleados con permiso consultar las alertas del sistema: productos con stock bajo o sin stock, lotes por vencer o vencidos, compras con entrega atrasada, caja abierta de un día anterior, pagos a proveedores por vencer, promociones vencidas que siguen activas y productos que se venden por debajo del costo. La pantalla principal muestra una campana con la cantidad de alertas, y desde la lista se puede ir a resolver cada una.	
Actores	Administrador - Empleado General - Empleado Compras - Empleado Ventas (permiso 11.1.0 Visualizar Alarma).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 11.1.0 Visualizar Alarma.	
Post-Condición	Se muestran las alertas pendientes. No se modifican datos.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario presiona la campana de alertas de la pantalla principal.	
	2. El sistema calcula las alertas pendientes.	2.1 Sin alertas: La lista queda vacía y la campana no muestra pendientes.
	3. El sistema muestra las alertas con su tipo, prioridad y descripción, y los contadores de total, urgentes y avisos.	
	4. El usuario filtra por tipo, prioridad o texto.	
	5. El usuario selecciona una alerta y presiona "Ir a resolver" (o hace doble clic).	5.1 Sin permiso para resolver: El sistema informa que no tiene permiso para resolver esa alerta y que avise al administrador. 5.2 Sin pantalla asociada:

		El sistema informa que la alerta no tiene una pantalla asociada para resolverla.
	6. El sistema abre la pantalla asociada para resolverla (por ejemplo, la reposición de stock o la cuenta corriente).	

Diagrama de caso de uso

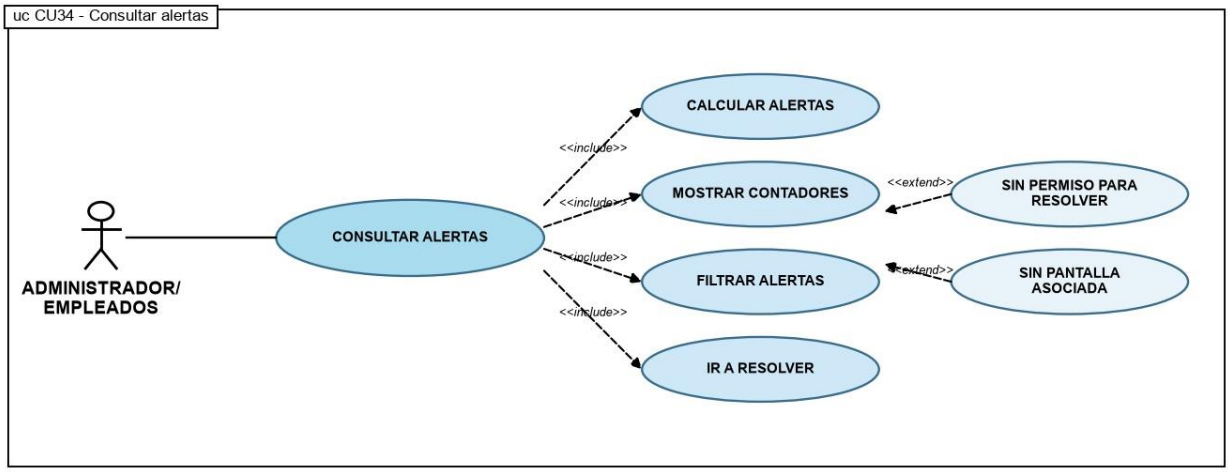


Diagrama de secuencia

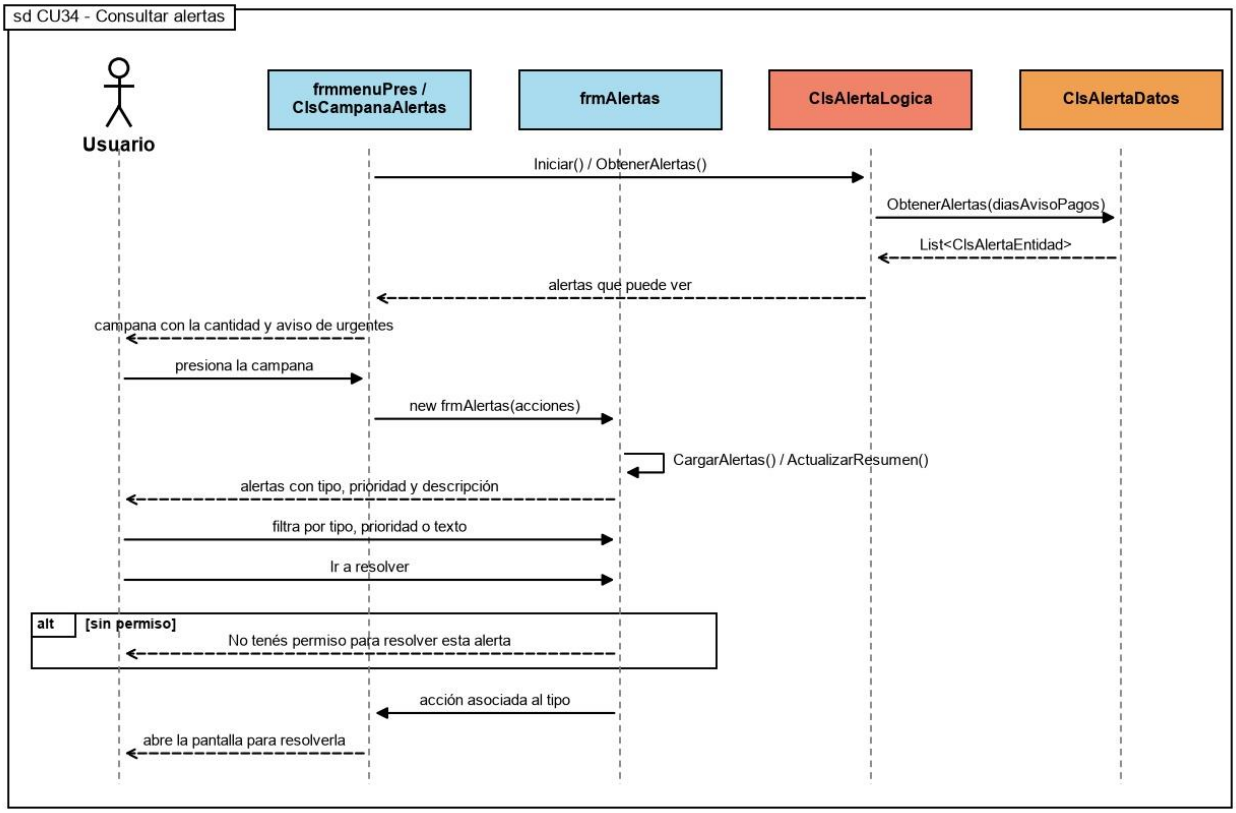
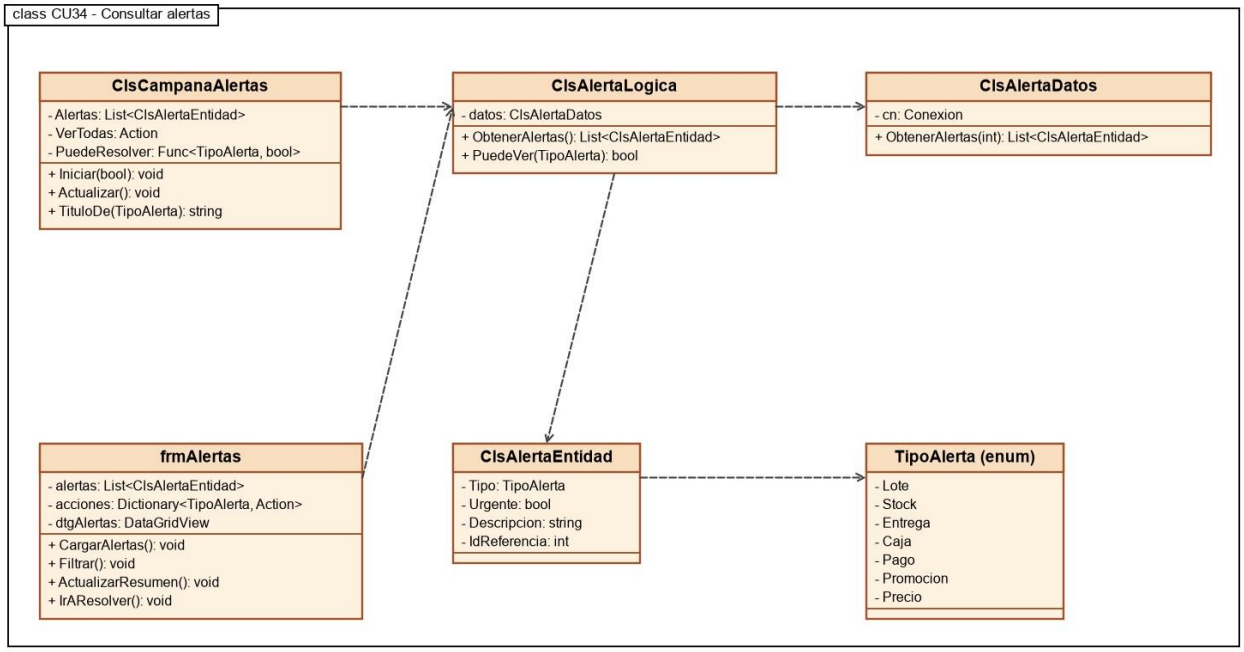


Diagrama de clases



CU35 - Anular movimiento de caja

ID y Nombre	**CU35 Anular movimiento de caja**	
Descripción	Permite al Administrador anular un ingreso o egreso manual de la caja cargado por error, indicando obligatoriamente el motivo. El movimiento no se elimina: queda anulado y deja de sumarse en el saldo de la caja.	
Actores	Administrador (permiso 7.6.0 Anular Movimiento de Caja).	
Pre-Condición	El usuario debe haber iniciado sesión y tener el permiso 7.6.0. Debe haber una caja abierta con movimientos manuales.	
Post-Condición	El movimiento queda anulado con el motivo, y el saldo de la caja se actualiza.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al menú Caja > Estado de Caja (CU30).	
	2. El sistema muestra los movimientos de la sesión.	
	3. El usuario selecciona el movimiento y presiona "Anular movimiento".	3.1 No se seleccionó movimiento: El sistema informa "Seleccione un movimiento". 3.2 Movimiento que no es manual: El sistema informa que desde acá solo se pueden anular los ingresos y egresos manuales.
	4. El sistema solicita el motivo de la anulación.	

	5. El usuario ingresa el motivo y confirma.	5.1 Motivo vacío: El sistema informa que debe ingresar un motivo para anular el movimiento. 5.2 Cancelar: El sistema no realiza cambios y vuelve al paso 2.
	6. El sistema anula el movimiento y actualiza el saldo.	
	7. El sistema informa que el movimiento fue anulado correctamente.	

Diagrama de caso de uso

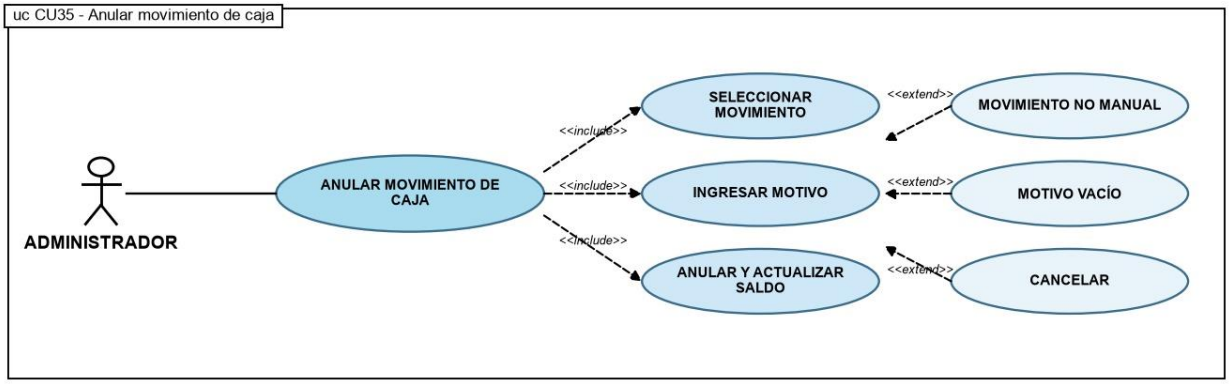


Diagrama de secuencia

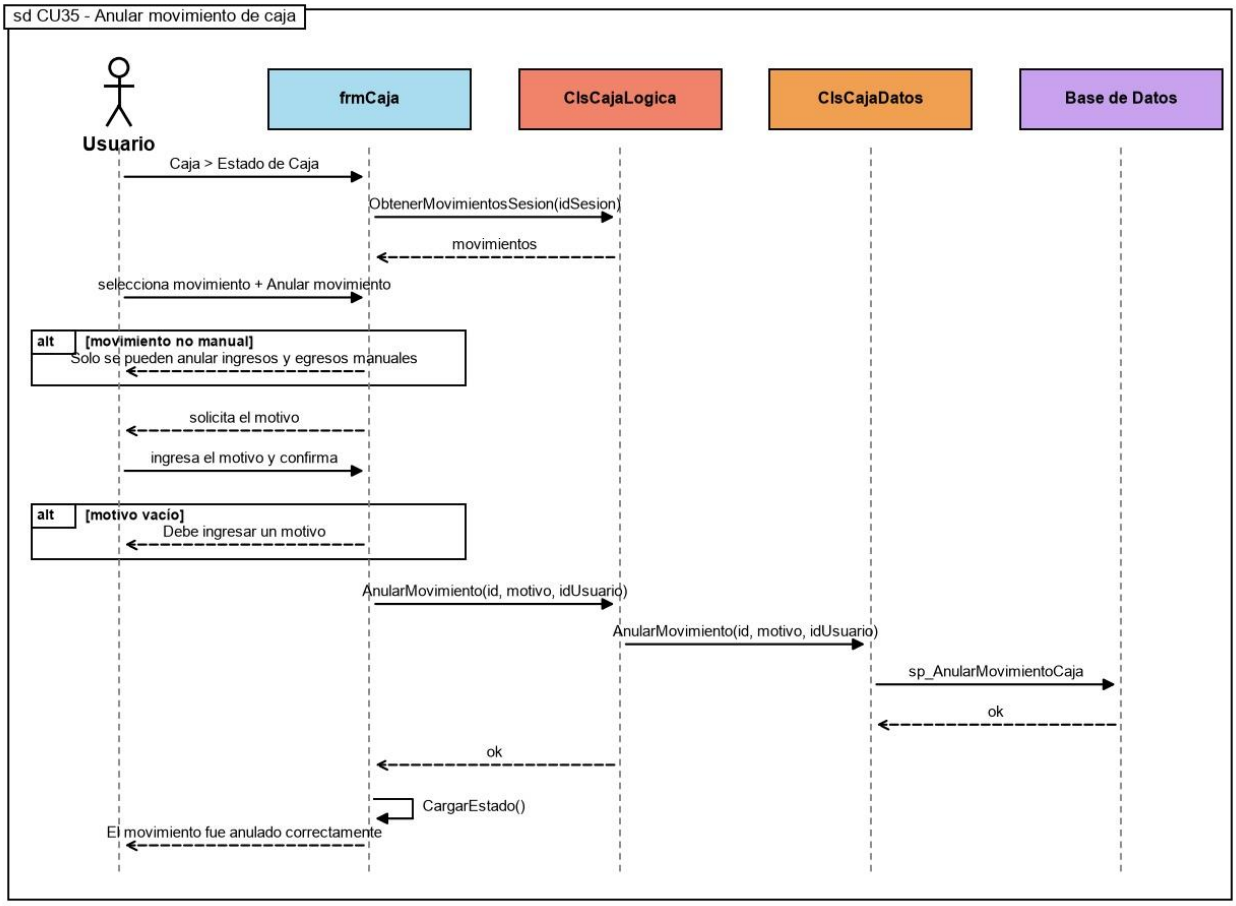
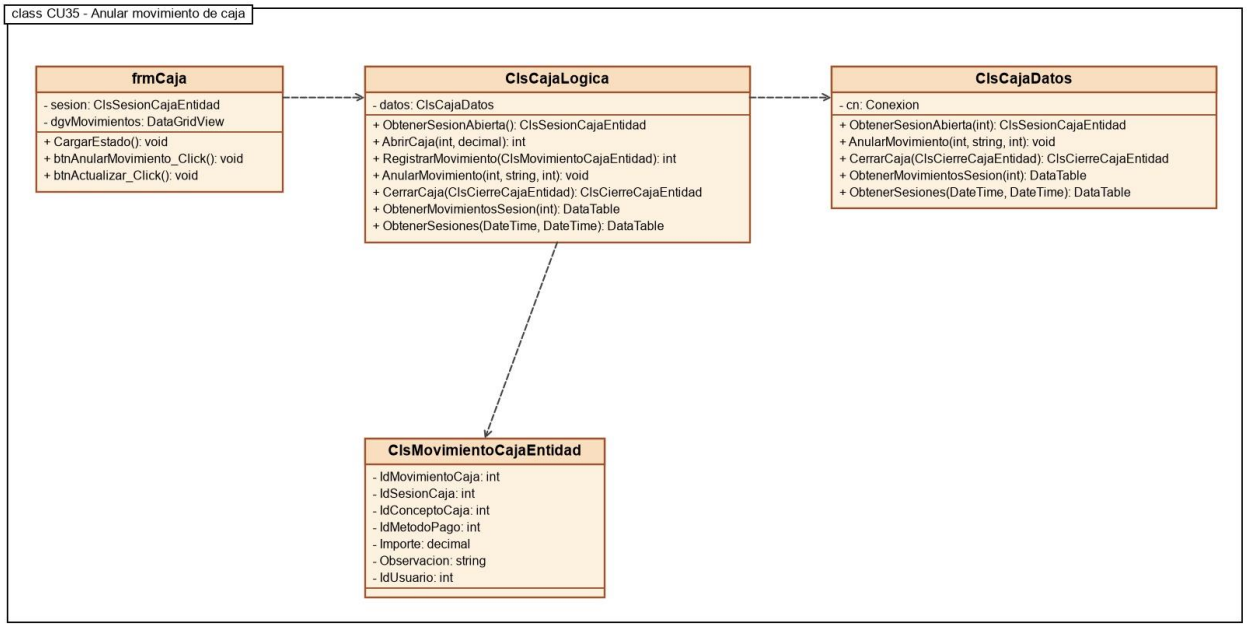


Diagrama de clases



Casos de Uso de Seguridad

CUS01 - Cambiar Idioma

ID y Nombre	**CUS01 Cambiar Idioma**	
Descripción	Permite a cualquier usuario del sistema seleccionar el idioma en el que se muestra la interfaz (Español o English). El sistema traduce los textos de todos los formularios abiertos (patrón Observer) sin reiniciar la aplicación, adapta el formato de los importes (ARS en español, USD en inglés) y guarda la elección para aplicarla en el próximo inicio.	
Actores	Administrador - Empleado.	
Pre-Condición	El usuario debe haber iniciado sesión.	
Post-Condición	La interfaz se muestra en el idioma seleccionado. El idioma queda guardado para el próximo inicio de la aplicación.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario accede al menú Idioma.	
	2. El sistema muestra los idiomas disponibles: English y Español.	
	3. El usuario selecciona el idioma deseado.	
	4. El sistema cambia el idioma y el formato de los importes.	4.1 Texto sin traducir: Si una etiqueta no tiene traducción, el sistema la muestra en Español.
	5. El sistema notifica a todos los formularios abiertos, que actualizan sus textos al nuevo idioma.	

	6. El sistema guarda el idioma elegido.	6.1 Error al guardar: El idioma se aplica solamente para la sesión actual.

Diagrama de caso de uso

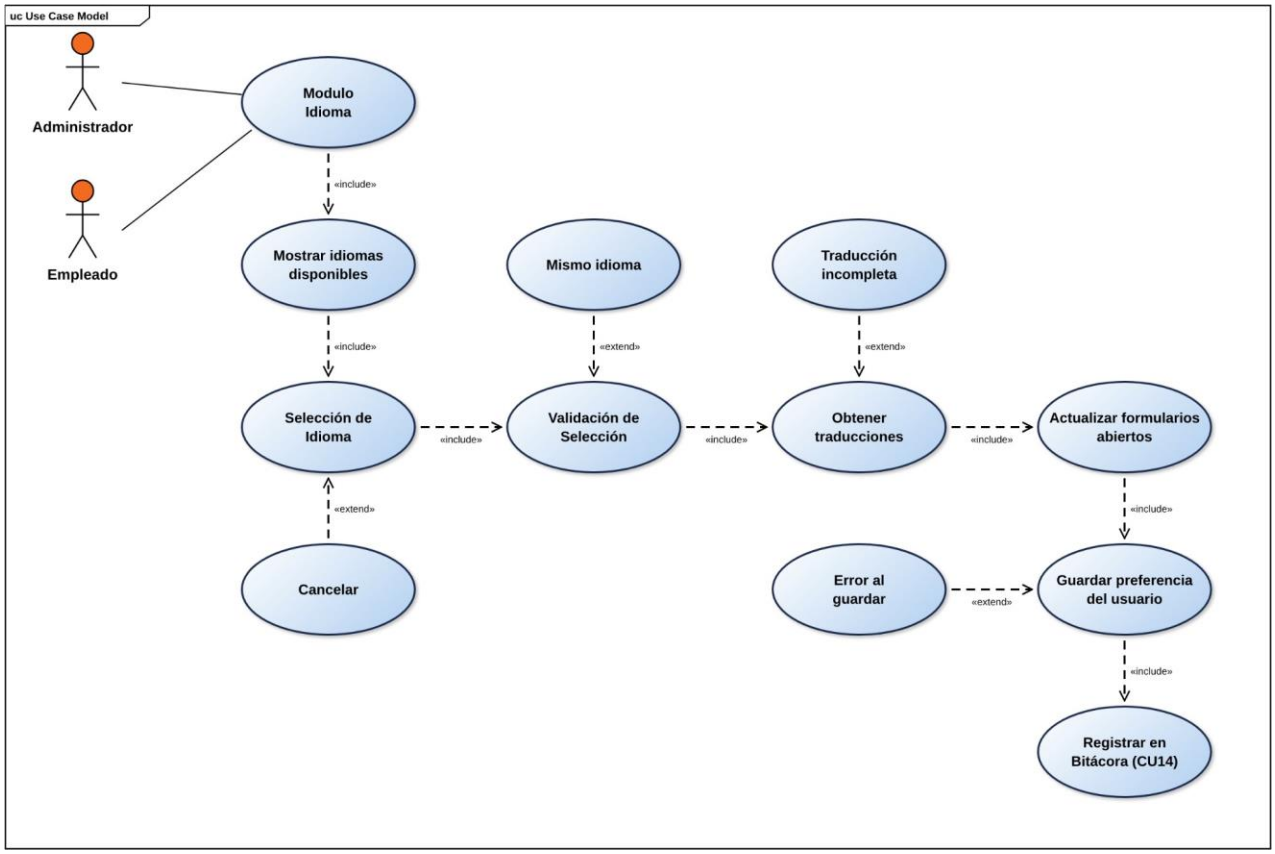


Diagrama de secuencia

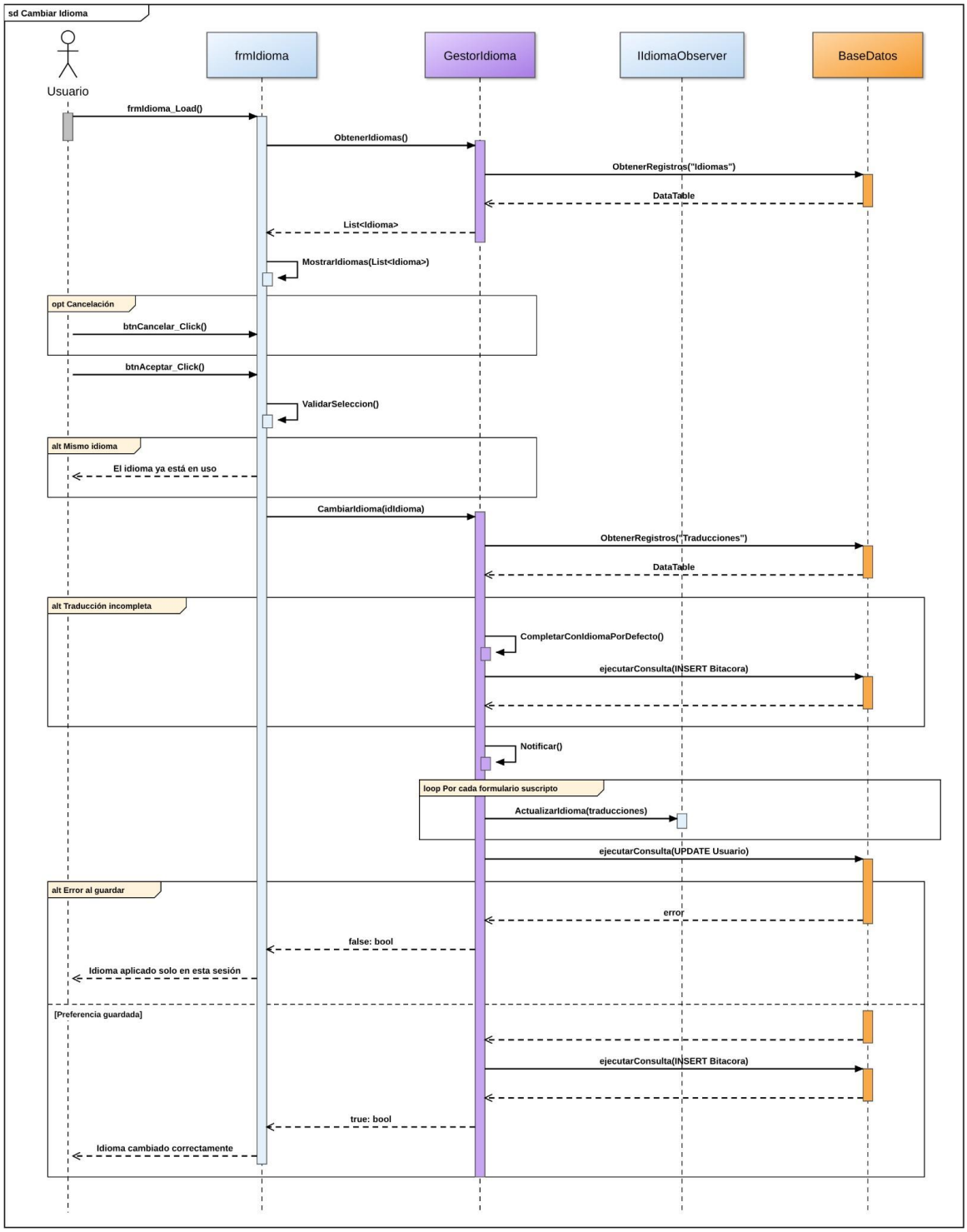
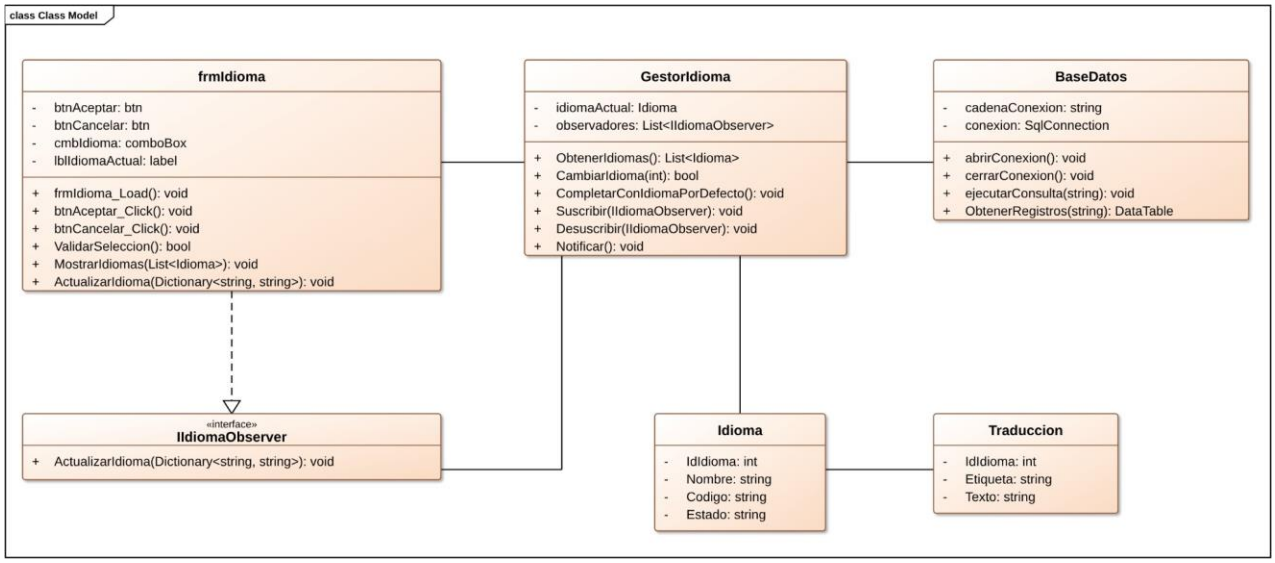


Diagrama de clases



CUS02 - Consultar Bitácora

ID y Nombre	**CUS02 Consultar Bitácora**	
Descripción	Permite al Administrador consultar los eventos registrados en la bitácora del sistema, separados por categoría: Usuarios, Seguridad, Productos, Compras, Promociones y Errores técnicos. Dentro de cada categoría puede buscar por texto y filtrar por rango de fechas, y ver el detalle completo de un registro. Los registros de la bitácora solo pueden consultarse: no pueden modificarse ni eliminarse.	
Actores	Administrador.	
Pre-Condición	El administrador debe haber iniciado sesión. Debe poseer el permiso de Bitácora (12.9.0).	
Post-Condición	El sistema muestra en pantalla los eventos que cumplen con los filtros seleccionados y, opcionalmente, el detalle de un evento.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El administrador accede al menú Administración > Bitácora.	
	2. El sistema muestra los registros de la bitácora.	
	3. El administrador elige la categoría.	
	4. El administrador ingresa el texto a buscar y/o el rango de fechas y presiona "Filtrar".	4.1 Limpiar: El administrador presiona "Limpiar" y el sistema quita los filtros.
	5. El sistema busca en la base de datos los eventos que cumplen con los filtros.	5.1 Sin resultados: La grilla queda vacía. 5.2 Error: El sistema informa que no se pudieron cargar los registros.

	6. El sistema muestra los eventos encontrados en la grilla.	
	7. El administrador selecciona un evento.	
	8. El sistema muestra el detalle del registro: fecha y hora, usuario, acción y descripción.	

Diagrama de caso de uso

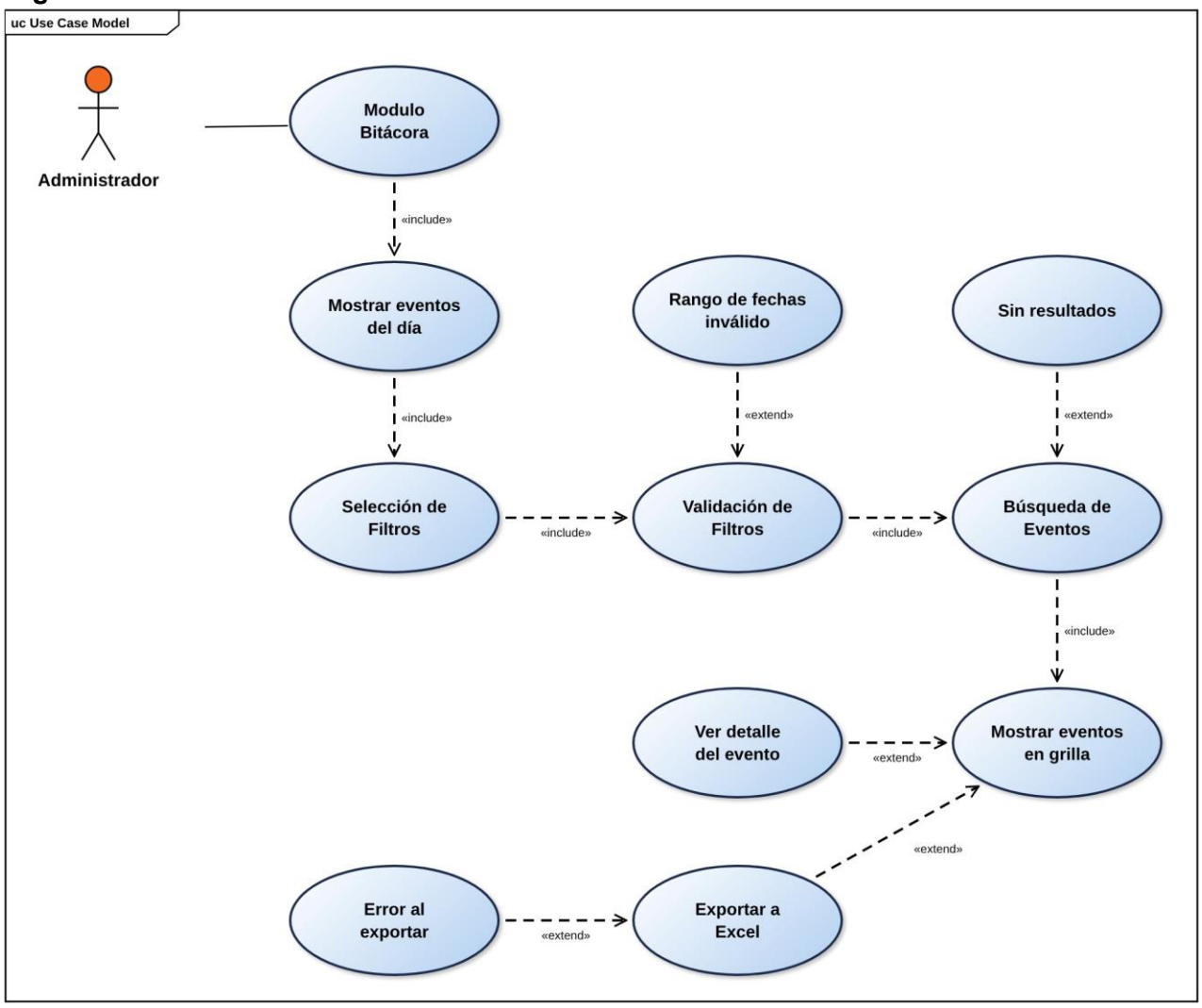


Diagrama de secuencia

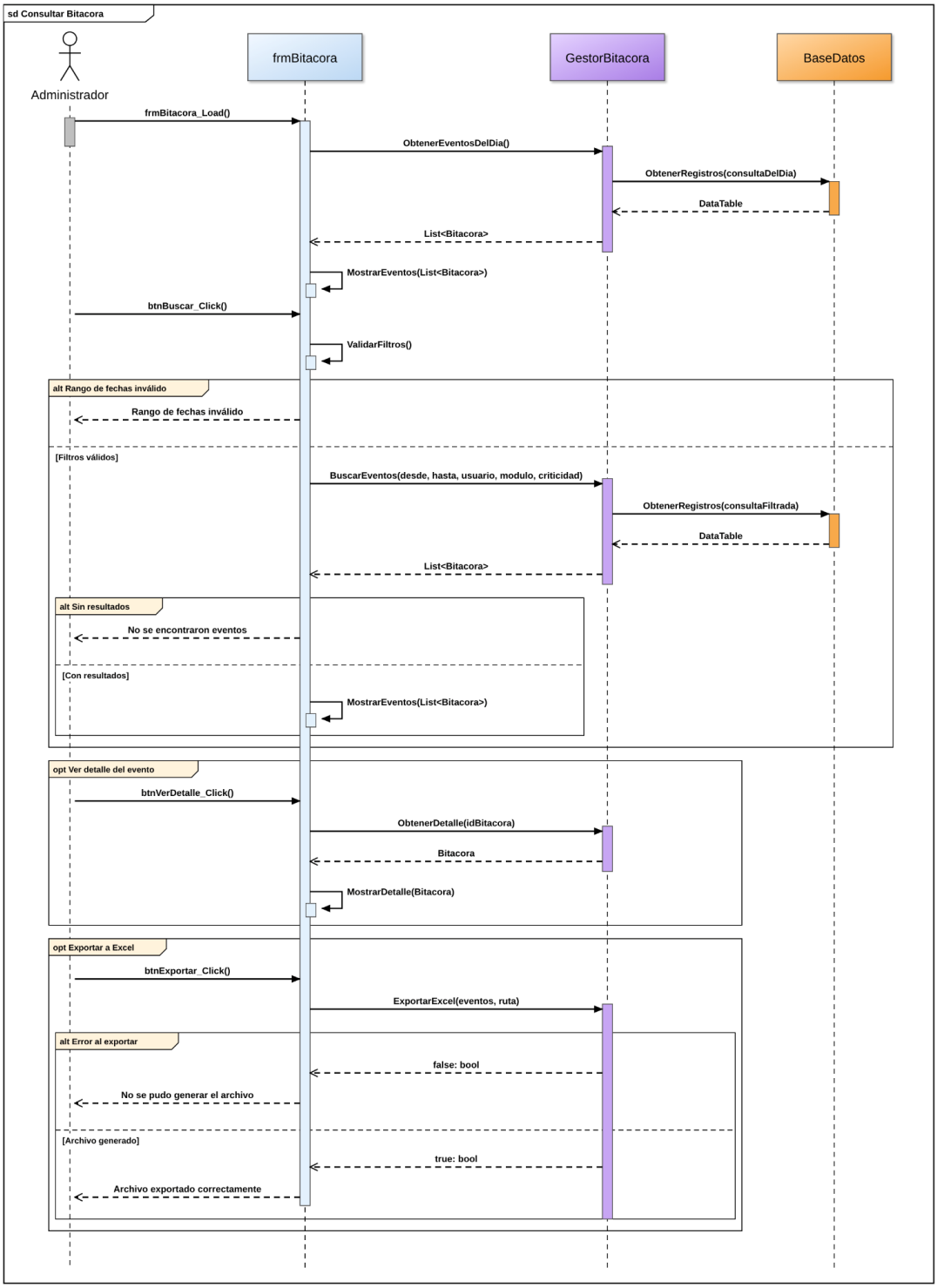
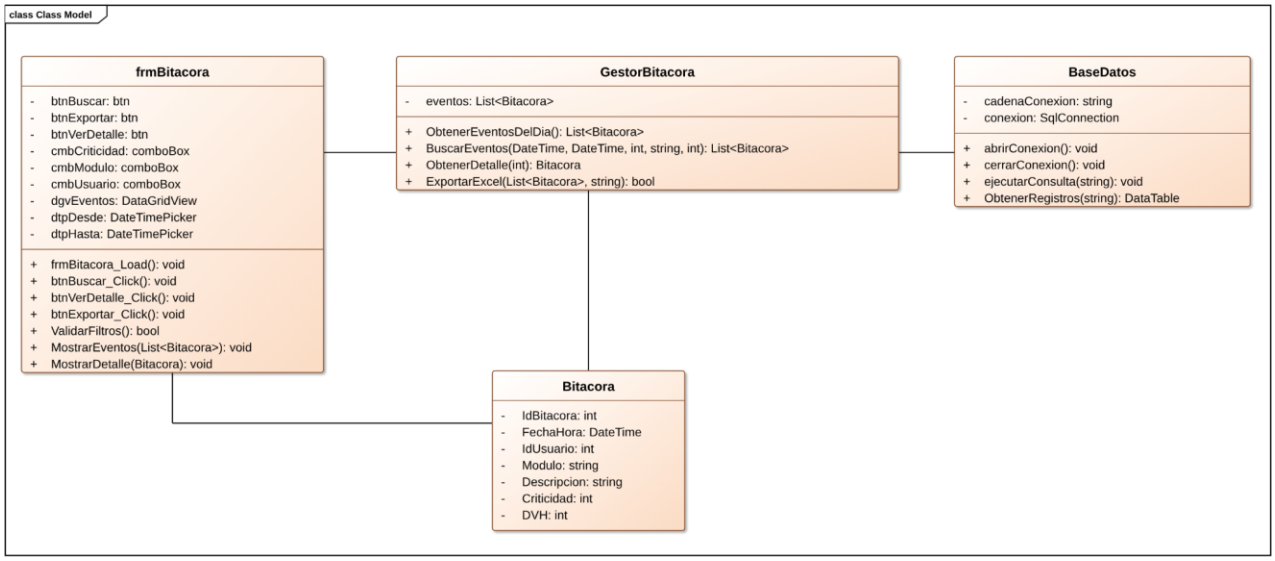


Diagrama de clases



CUS03 - Realizar Backup

ID y Nombre	**CUS03 Realizar Backup**	
Descripción	Permite al Administrador generar una copia de seguridad completa de la base de datos de Gestorix en la carpeta que elija (disco local, pendrive o disco externo) y restaurar un backup existente. El nombre del archivo se genera automáticamente con la fecha y hora (por ejemplo: Gestorix_20-10-2026_220000.bak) y junto al backup se guarda un archivo .hash con su firma. Al restaurar, el sistema verifica la firma y avisa si el archivo fue modificado o no fue generado por Gestorix. El avance de la copia y de la restauración se muestra con barras de progreso.	
Actores	Administrador.	
Pre-Condición	El administrador debe haber iniciado sesión y poseer el permiso de Gestión de Backup (12.8.0). La ubicación de destino debe estar disponible y contar con espacio suficiente.	
Post-Condición	Se genera el archivo de backup y su firma en la ubicación seleccionada. Si se restaura, la base de datos queda igual que en el backup y el sistema se reinicia.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El administrador accede al menú Administración > Gestión de Backup.	
	2. El administrador presiona "Examinar" y selecciona la carpeta de destino.	
	3. El sistema muestra los backups que hay en esa carpeta.	
	4. El administrador presiona "Generar Backup".	4.1 Restaurar backup: El administrador selecciona un backup de la lista y presiona "Restaurar"; el sistema advierte que se perderán los cambios posteriores, verifica la firma, restaura mostrando el progreso y reinicia el sistema. 4.2 Firma inválida:

		Si el backup no tiene una firma válida, el sistema lo advierte y pregunta si desea continuar.
	5. El sistema genera el nombre del archivo y ejecuta el backup mostrando el progreso.	5.1 Error durante la copia: El sistema informa el error y no se genera el backup. 5.2 Cerrar durante el proceso: El sistema pide esperar a que termine el backup o la restauración.
	6. El sistema guarda la firma del archivo.	
	7. El sistema informa que el backup se generó correctamente en la ubicación elegida y actualiza la lista.	

Diagrama de caso de uso

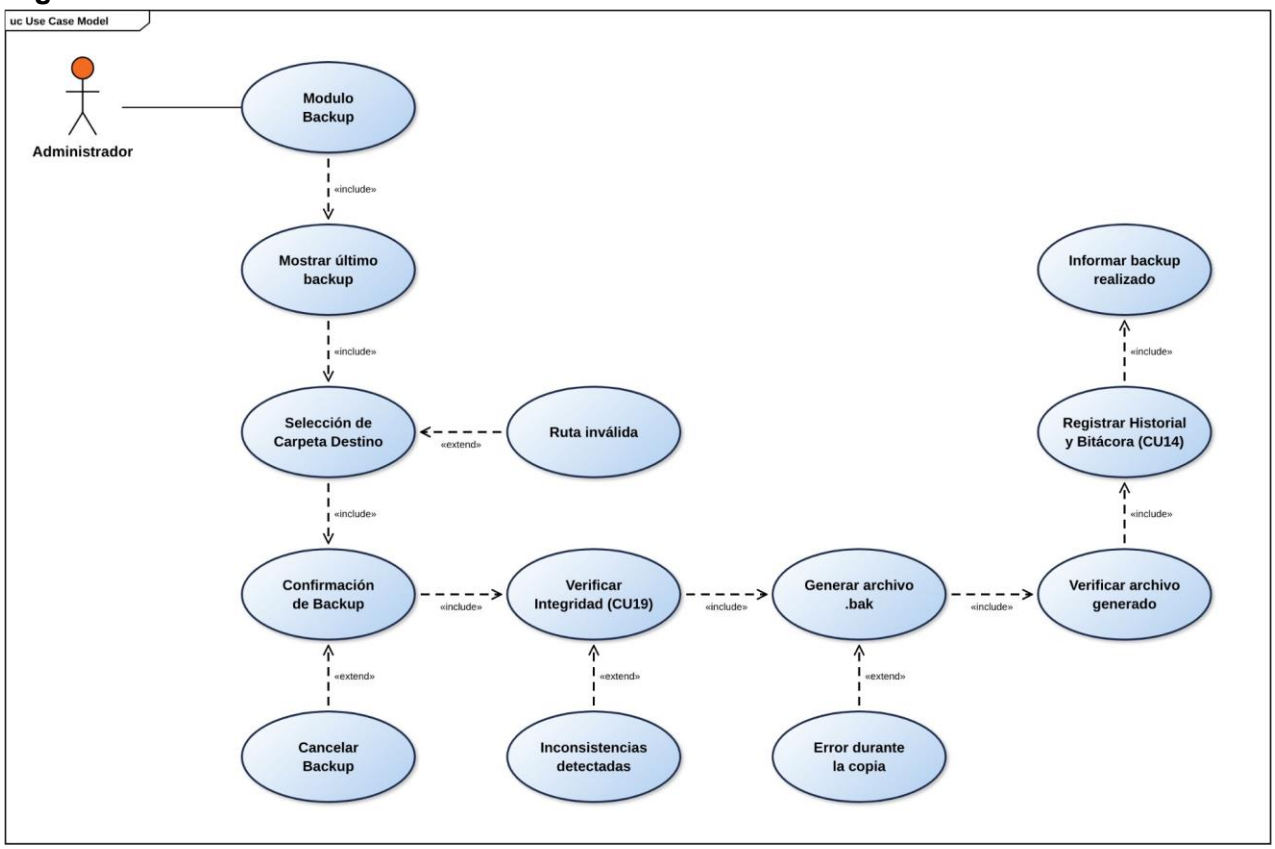


Diagrama de secuencia

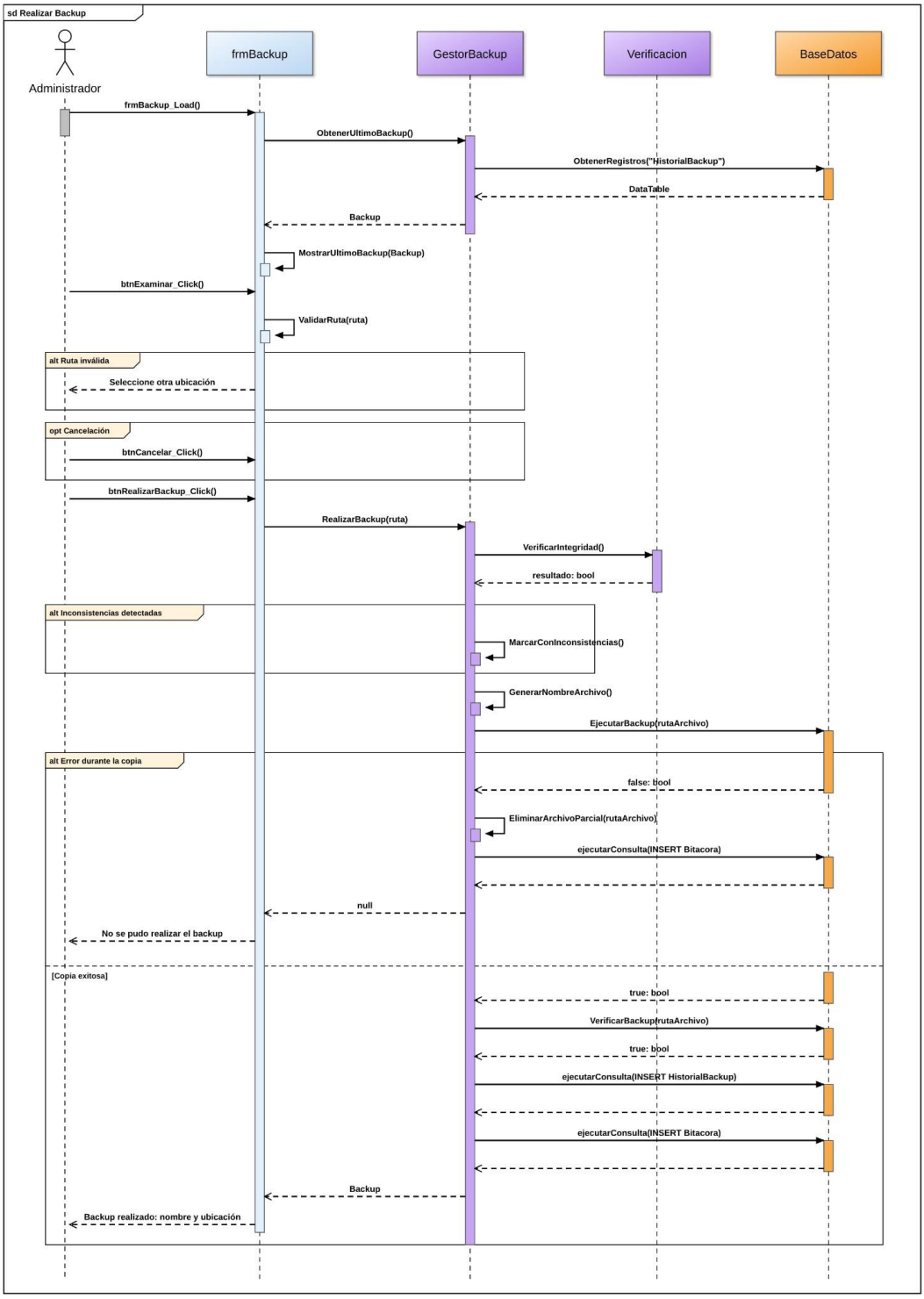
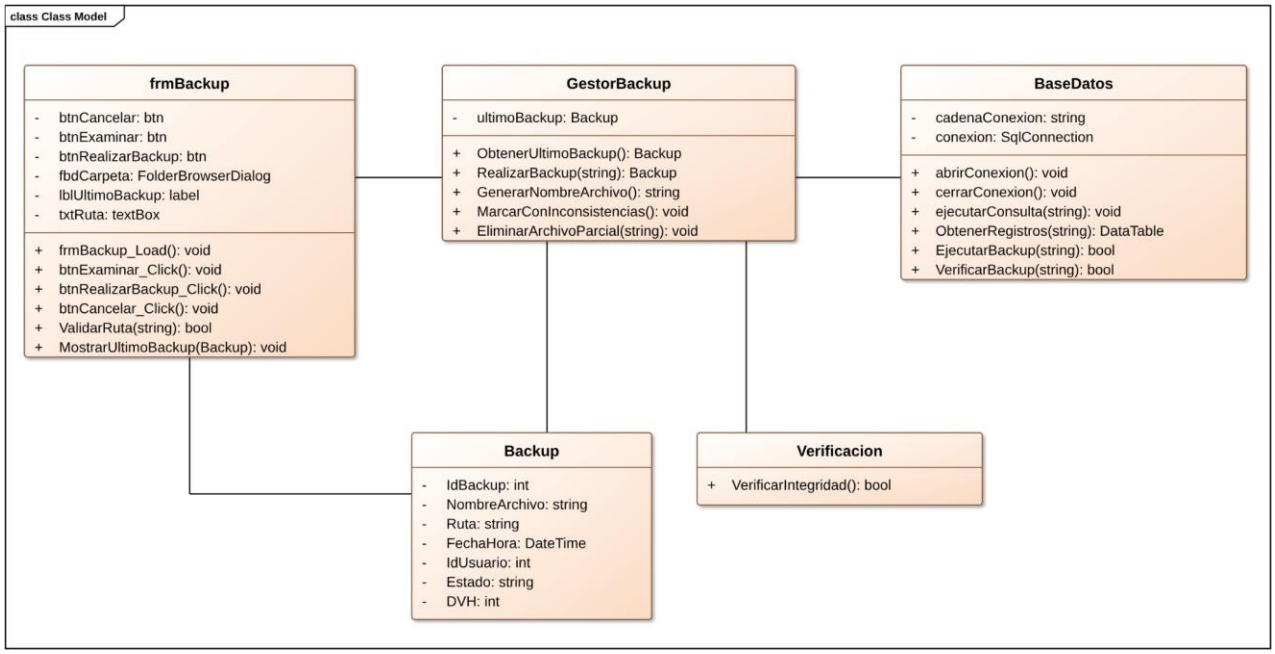


Diagrama de clases



CUS04 - Gestión de Dígitos Verificadores

ID y Nombre	**CUS04 Gestión de Dígitos Verificadores**	
Descripción	Permite controlar que la información de la base de datos no haya sido alterada por fuera del sistema. En cada inicio de sesión, luego de validar las credenciales, el sistema recalcula el Dígito Verificador Horizontal (DVH) de cada registro de las tablas protegidas (Producto, Usuario y Venta) y lo compara con el almacenado. Si la base de datos es íntegra, el usuario ingresa normalmente. Si hay inconsistencias, se bloquea el acceso a los usuarios sin permiso de backup y se muestra al Administrador la pantalla de Verificar integridad, desde donde puede recalcular los DVH (aceptando los datos actuales como válidos) o restaurar un backup. El DVH se calcula además automáticamente cada vez que se da de alta o se modifica un registro de una tabla protegida.	
Actores	Administrador - Empleado.	
Pre-Condición	El usuario debe estar registrado en el sistema. La base de datos debe estar accesible. Las tablas protegidas deben tener su DVH calculado.	
Post-Condición	Si los datos son íntegros, el usuario ingresa al menú principal. Si existen inconsistencias, quedan registradas en la bitácora y el acceso queda restringido al administrador. Si el administrador recalcula, todos los DVH quedan actualizados.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario ingresa su nombre de usuario y contraseña y presiona "Iniciar".	
	2. El sistema valida que los campos estén completos.	2.1 Campos vacíos: El sistema informa que debe completar todos los campos.
	3. El sistema valida las credenciales del usuario.	3.1 Credenciales incorrectas: El sistema informa "Usuario o contraseña incorrectos" y no continúa.
	4. Por cada tabla protegida, el sistema recalcula el DVH de cada registro y lo compara con el almacenado.	

	5. El sistema determina que la base de datos es íntegra y el usuario ingresa al menú principal.	5.1 Inconsistencias y usuario sin permiso: El sistema registra la verificación en la bitácora e informa que no se puede ingresar hasta que un administrador lo revise. 5.2 Inconsistencias y administrador: El sistema abre la pantalla Verificar integridad con los registros afectados y las opciones "Verificar de nuevo", "Recalcular DVH", "Restaurar backup" y "Cerrar". 5.3 Recalcular DVH: El administrador confirma y el sistema recalcula el DVH de todos los registros y lo registra en la bitácora. 5.4 Restaurar backup: El administrador presiona "Restaurar backup" y se ejecuta el CUS03.

Diagrama de caso de uso

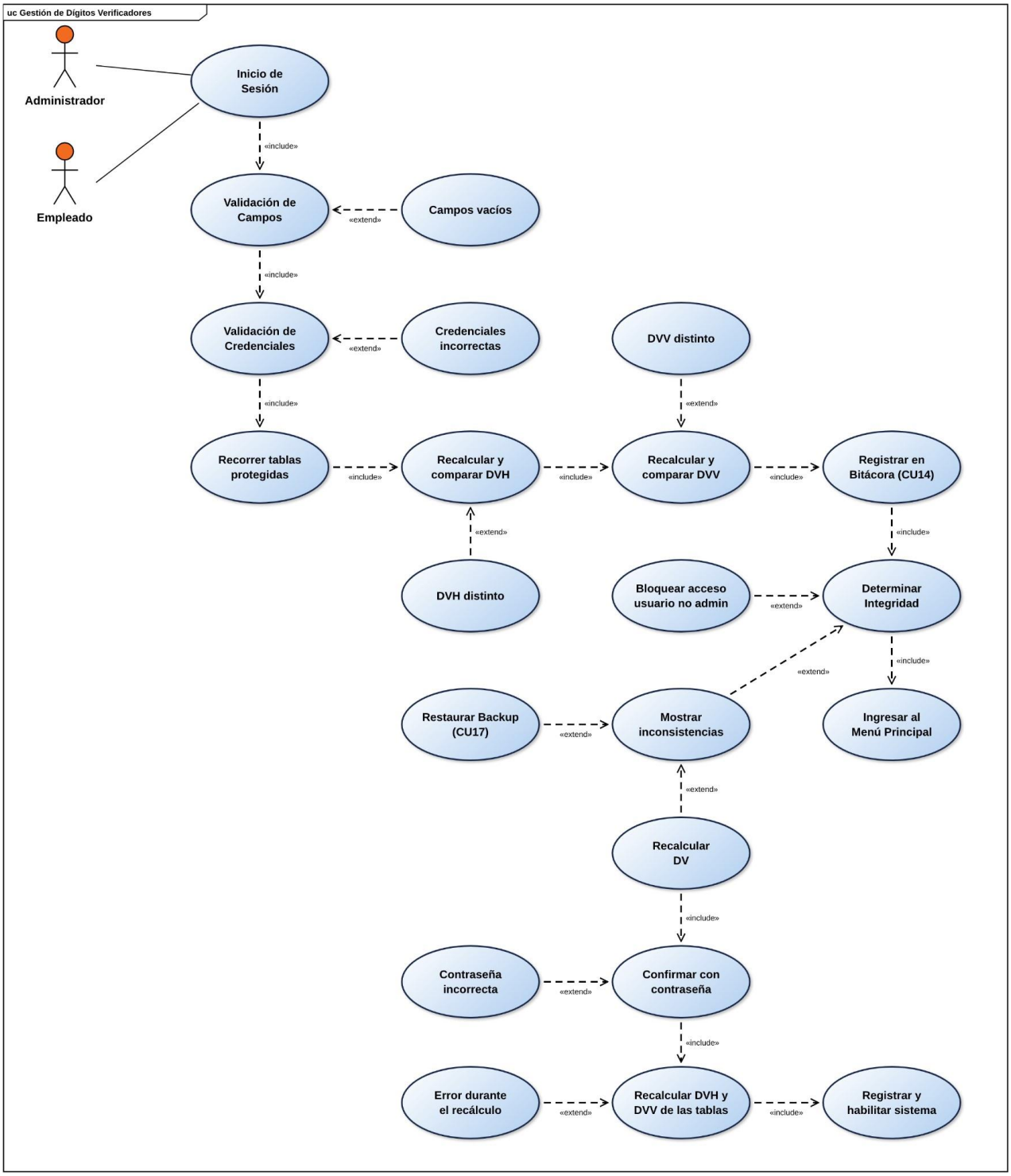


Diagrama de secuencia

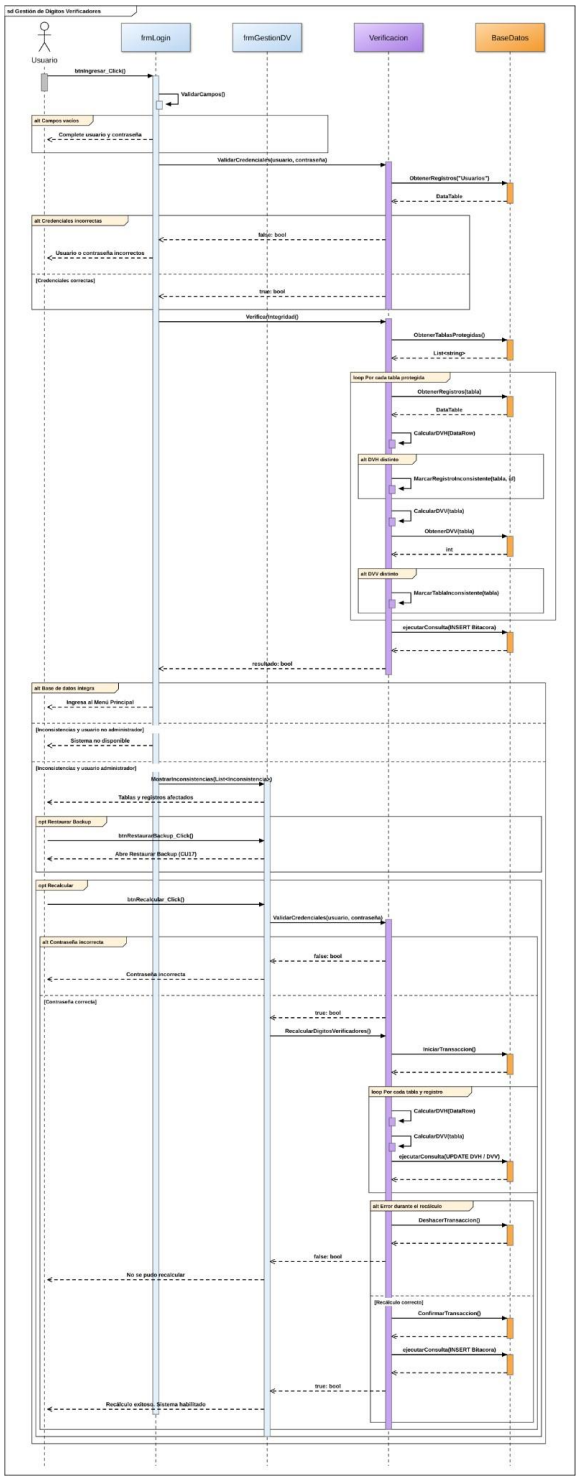
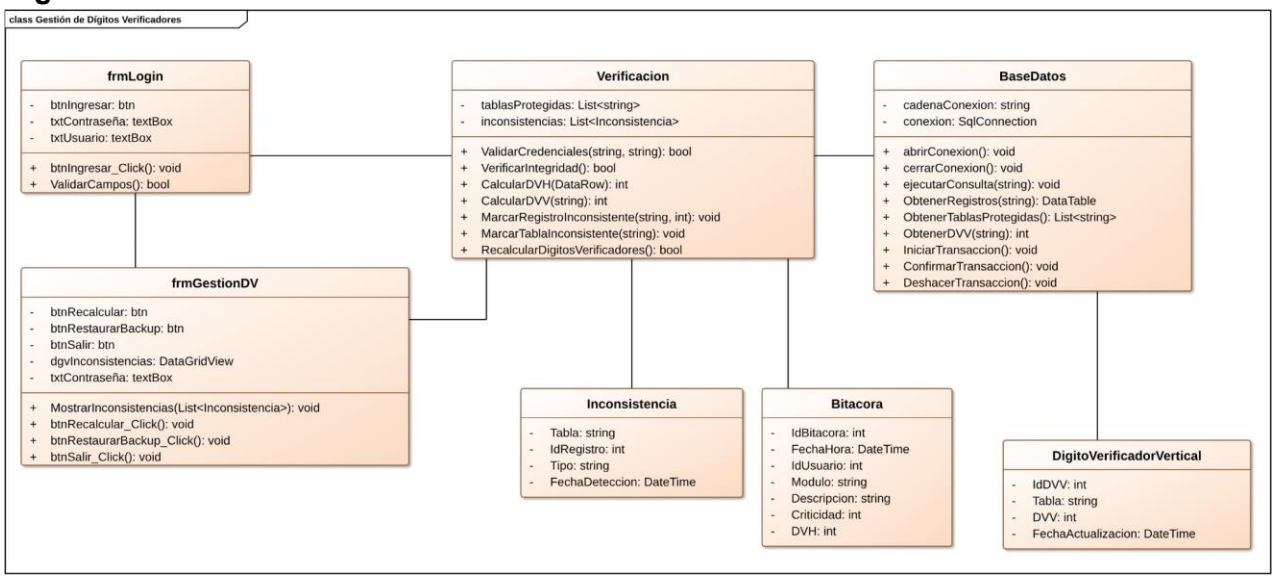


Diagrama de clases



CUS05 - Gestión de Encriptado

ID y Nombre	**CUS05 Gestión de Encriptado**	
Descripción	Permite al sistema validar el acceso de los usuarios y actualizar sus contraseñas utilizando encriptado SHA256. Al iniciar sesión, el sistema encripta la contraseña ingresada, la compara con el hash almacenado y verifica que el usuario esté activo. En el primer ingreso exige reemplazar la contraseña temporal y cargar las preguntas de seguridad. Si el usuario olvidó su contraseña, puede recuperarla validando su usuario y respondiendo su pregunta de seguridad. La contraseña nueva debe tener al menos 8 caracteres, una mayúscula, una minúscula y un número, y no puede ser ninguna de las últimas 3 usadas.	
Actores	Administrador - Empleado.	
Pre-Condición	El usuario debe estar registrado en el sistema. Para la recuperación, el usuario debe tener cargadas sus preguntas de seguridad.	
Post-Condición	El usuario ingresa al menú principal con los módulos habilitados según su rol. Cuando se cambia o recupera la contraseña, en la base de datos queda guardado únicamente su hash SHA256.	
FLUJO	**Flujo Normal**	**Flujo Alternativo**
	1. El usuario ingresa su nombre de usuario y contraseña en la pantalla Login y presiona "Iniciar".	1.1 Olvidé mi contraseña: El usuario presiona el enlace de recuperación. Continúa en el paso 8.
	2. El sistema valida que los campos estén completos.	2.1 Campos vacíos: El sistema solicita completar todos los campos.
	3. El sistema encripta la contraseña ingresada con SHA256.	
	4. El sistema obtiene el hash almacenado del usuario y lo compara con el hash obtenido.	4.1 Credenciales incorrectas: El sistema informa "Usuario o contraseña incorrectos".
	5. El sistema verifica que el usuario se encuentre activo.	5.1 Usuario bloqueado o inactivo: El sistema informa que el usuario está bloqueado y no permite el ingreso.

	6. El sistema verifica si es el primer inicio de sesión del usuario.	6.1 Primer inicio (contraseña temporal): El sistema muestra la pantalla Primer ingreso; el usuario ingresa la contraseña temporal y la nueva dos veces, y luego responde las tres preguntas de seguridad.
	7. El sistema ingresa al menú principal, habilitando solo los módulos permitidos según el rol del usuario.	
	8. (Recuperación) El usuario ingresa su nombre de usuario y presiona "Validar Usuario".	
	9. El sistema muestra la pregunta de seguridad; el usuario responde y presiona "Aceptar".	9.1 Respuesta incorrecta: El sistema informa "Respuesta incorrecta"; tras varios intentos bloquea la recuperación.
	10. El sistema muestra la pantalla Recuperar Contraseña; el usuario ingresa la nueva contraseña y la repite.	
	11. El sistema valida la contraseña, la encripta con SHA256 y guarda el hash.	11.1 Contraseña no válida: El sistema informa que las contraseñas no coinciden, que no cumple los requisitos o que no puede usar ninguna de sus últimas 3 contraseñas.
	12. El sistema informa que la contraseña se cambió correctamente.	

Diagrama de caso de uso

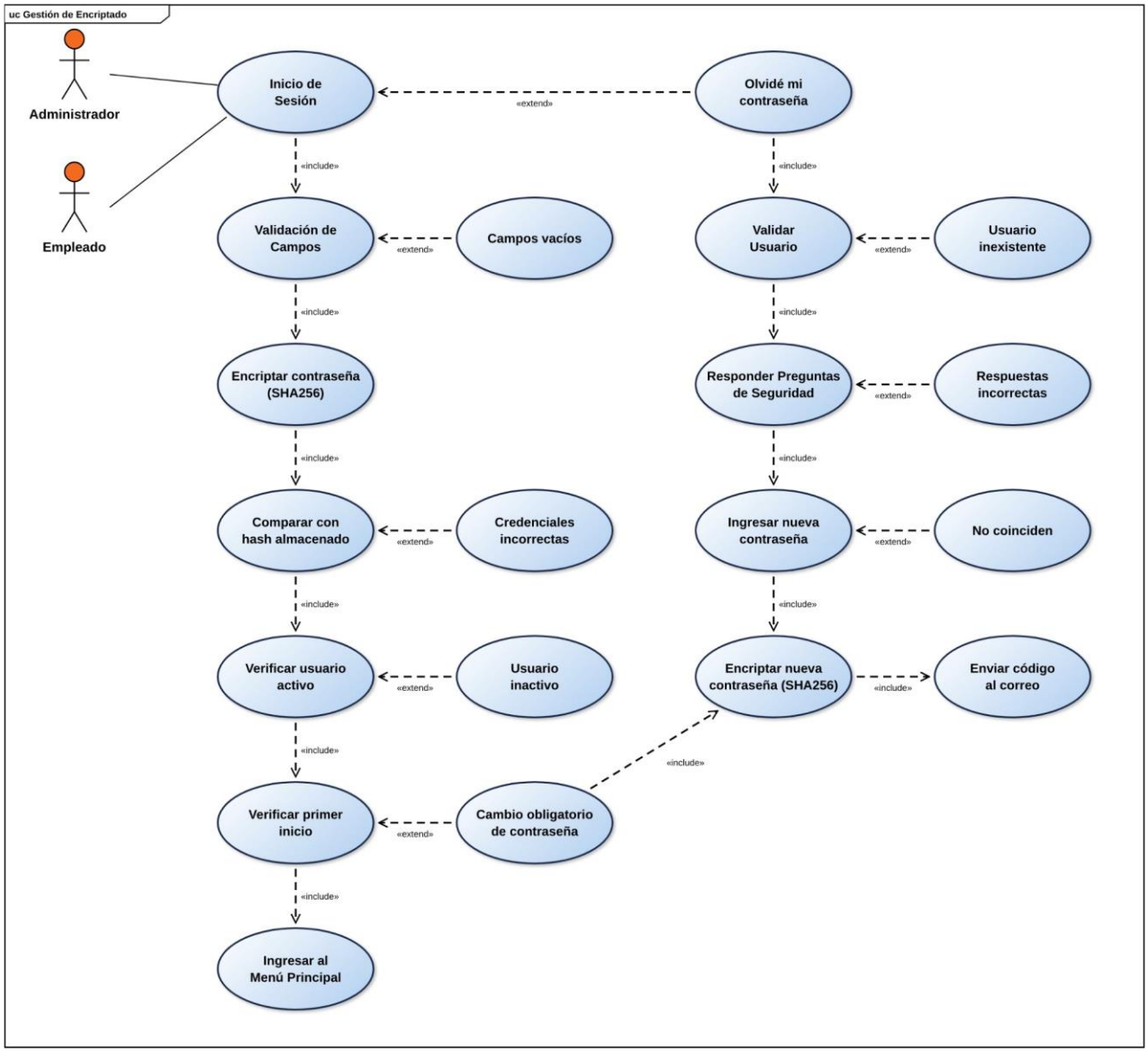
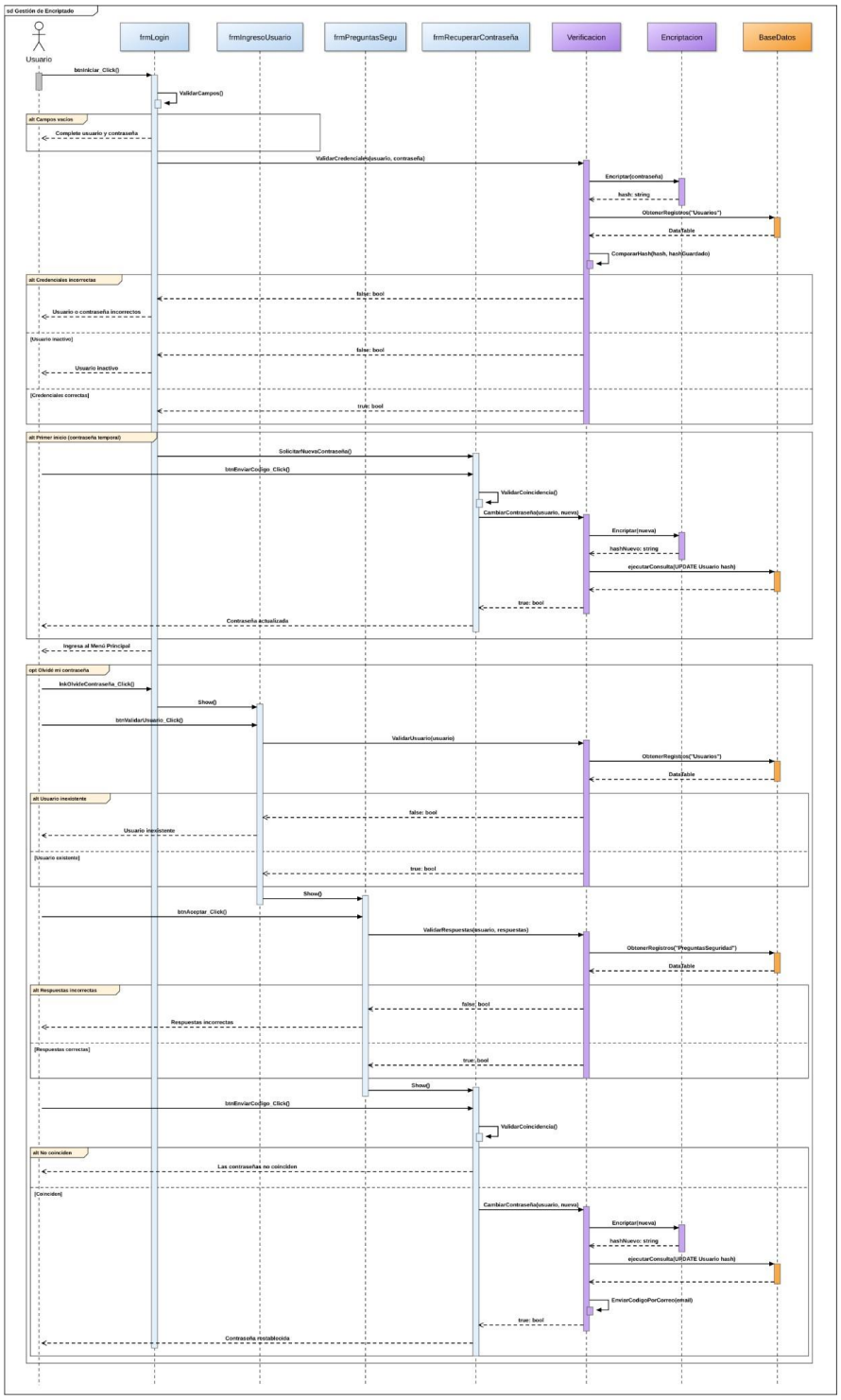


Diagrama de secuencia



3.5.9 Especificaciones Adicionales

En este punto se detallan los requisitos y reglas que complementan a los casos de uso CUS01 a CUS05 y que no forman parte de su flujo paso a paso. Todos se basan en las definiciones del proyecto ya establecidas en los puntos 3.1 a 3.4 y 5.0 de este documento.

Consideraciones generales

- Las funcionalidades de los casos de uso CUS01 a CUS05 se desarrollan respetando los estándares de codificación definidos en 3.4.2: lenguaje C#, .NET 8, Windows Forms y SQL Server, arquitectura en capas (Presentación, Lógica y Acceso a datos), consultas SQL parametrizadas, validación de datos en formularios y manejo de excepciones.

- Funcionan sobre Windows 10 o superior, en la computadora de caja del kiosco, con la base de datos SQL Server instalada en forma local y para una única sucursal (3.4 y 3.4.5).

- Las pantallas mantienen la interfaz simple, clara y fácil de aprender definida en 3.1.6.2, ya que los usuarios poseen conocimientos informáticos básicos.

- Cualquier modificación importante de estas funciones deberá ser aprobada por el administrador (dueño del kiosco), según su perfil de stakeholder (3.1.6.3.1).

Desempeño

- La verificación de integridad (CUS04) y el encriptado de la contraseña (CUS05) se ejecutan durante el inicio de sesión, por lo que deben respetar el tiempo máximo de inicio de sesión de 1,5 segundos (3.4.4).

- La consulta de la bitácora (CUS02) debe responder en un máximo de 3 segundos, igual que las consultas y reportes del sistema (3.4.4).

- Estas funciones deben mantener la estabilidad del sistema durante toda la jornada comercial, de 8:00 a 22:00 hs (3.1.6.2 y 3.4.4).

Múltiples idiomas

- El idioma por defecto del sistema es el Español, idioma en el que está desarrollada la interfaz.

- Los textos de la interfaz (menús, botones, etiquetas, mensajes y encabezados de reportes) se traducen con un diccionario Español / English incluido en el sistema, y el idioma elegido queda guardado en la computadora para el próximo inicio (CUS01).

- El cambio de idioma se aplica a todos los formularios abiertos sin reiniciar el sistema, mediante el patrón Observer (CUS01).

- El cambio de idioma solo modifica los textos de la interfaz; no modifica los datos registrados (artículos, ventas, compras, caja).

Bitácora

- La bitácora amplía el registro automático de movimientos definido en 3.3.5 y el historial completo de movimientos incluido en el alcance (3.1.1, punto 24), registrando además las acciones que realizan los usuarios dentro del sistema.
- Cada evento registra fecha y hora, usuario, acción, tabla afectada y descripción, y se consulta agrupado por categoría: Usuarios, Seguridad, Productos, Compras, Promociones y Errores técnicos (CUS02).
- Los registros no pueden modificarse ni eliminarse desde el sistema, en línea con la restricción de eliminación física de datos (3.3.2) y las buenas prácticas de seguridad de la norma ISO/IEC 27001 (3.4.2).
- La consulta de la bitácora (CUS02) está restringida a los roles con el permiso 12.9.0 Bitácora.

Backup

- El backup se genera con las herramientas propias de SQL Server (archivo .bak), ya que el sistema depende de SQL Server como gestor de base de datos (3.4).
- Su objetivo es resguardar la información frente a las dependencias identificadas en 3.2.3: fallas en el suministro eléctrico o en el equipamiento informático del kiosco.
- Se recomienda realizar el backup al finalizar la jornada comercial y guardar una copia en un dispositivo externo a la computadora de caja.
- Solo los usuarios con el permiso 12.8.0 Gestión de Backup pueden realizar o restaurar un backup; antes de restaurar, el sistema pide confirmar la operación y verifica la firma del archivo (.hash).

Dígitos verificadores

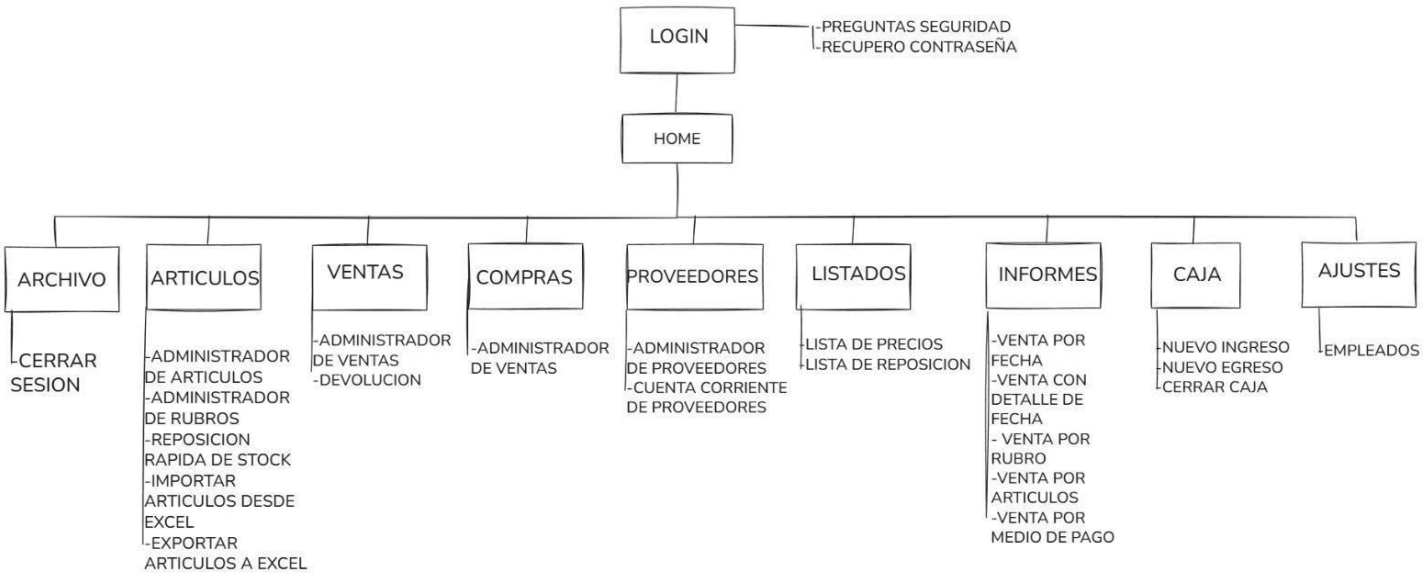
- Responden al requisito de integridad y consistencia de los datos establecido en 3.4.1 y 3.4.3.
- Se calcula un Dígito Verificador Horizontal (DVH) por cada registro de las tablas protegidas: Usuarios, Artículos y Ventas.
- El DVH se calcula automáticamente en cada alta o modificación de un registro protegido y se verifica en cada inicio de sesión (CUS04).

- Si se detectan inconsistencias, solo un usuario con el permiso 12.8.0 Gestión de Backup puede ingresar al sistema para recalcular el DVH o restaurar un backup (CUS04 y CUS03).

Encriptado

- Las contraseñas se almacenan únicamente como hash SHA256 (encriptado irreversible), según lo definido en 3.3.1, 3.3.7 y 3.4.2.
- La contraseña temporal generada en el alta del empleado debe reemplazarse obligatoriamente en el primer inicio de sesión.
- La recuperación de contraseña se realiza validando al usuario y su pregunta de seguridad; después de varios intentos fallidos el sistema bloquea la recuperación.
- La clase ClsSeguridad centraliza el cálculo del hash SHA256 y es utilizada por todos los procesos que manejan contraseñas.

3.5.10 Mapa de Navegación

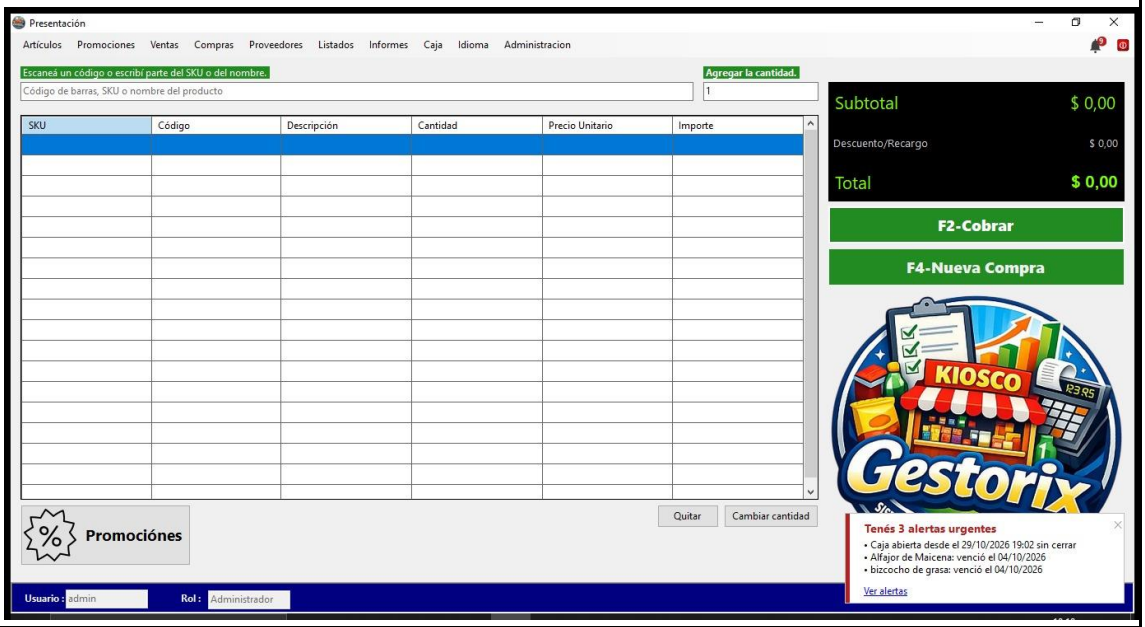


3.6 Prototipos de Interfaces del Usuario

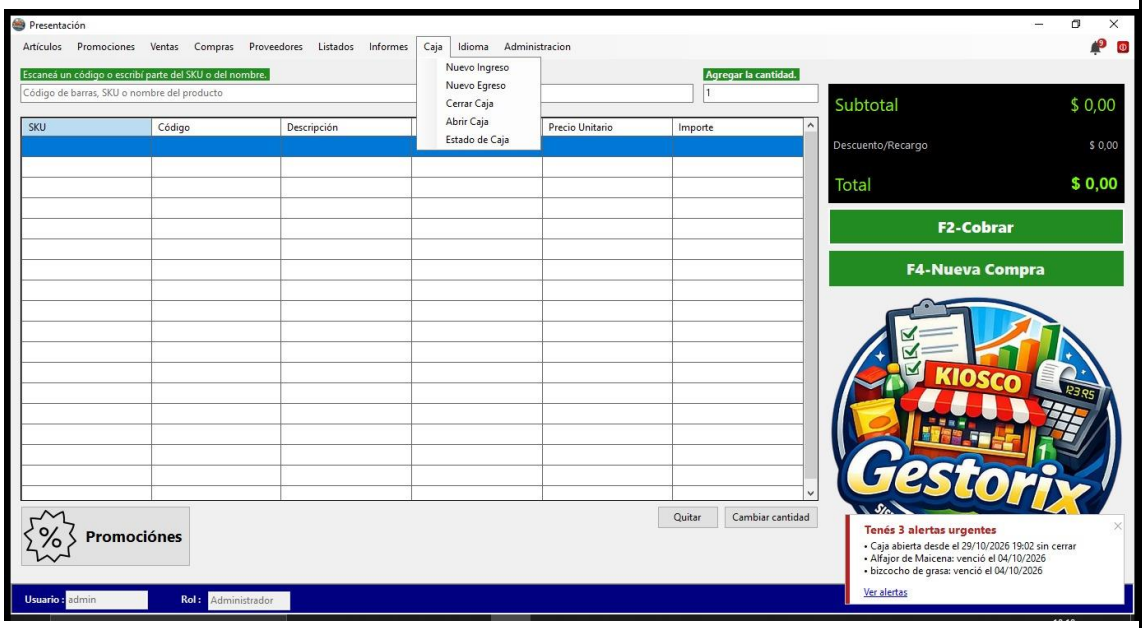
1. Ingreso a la app



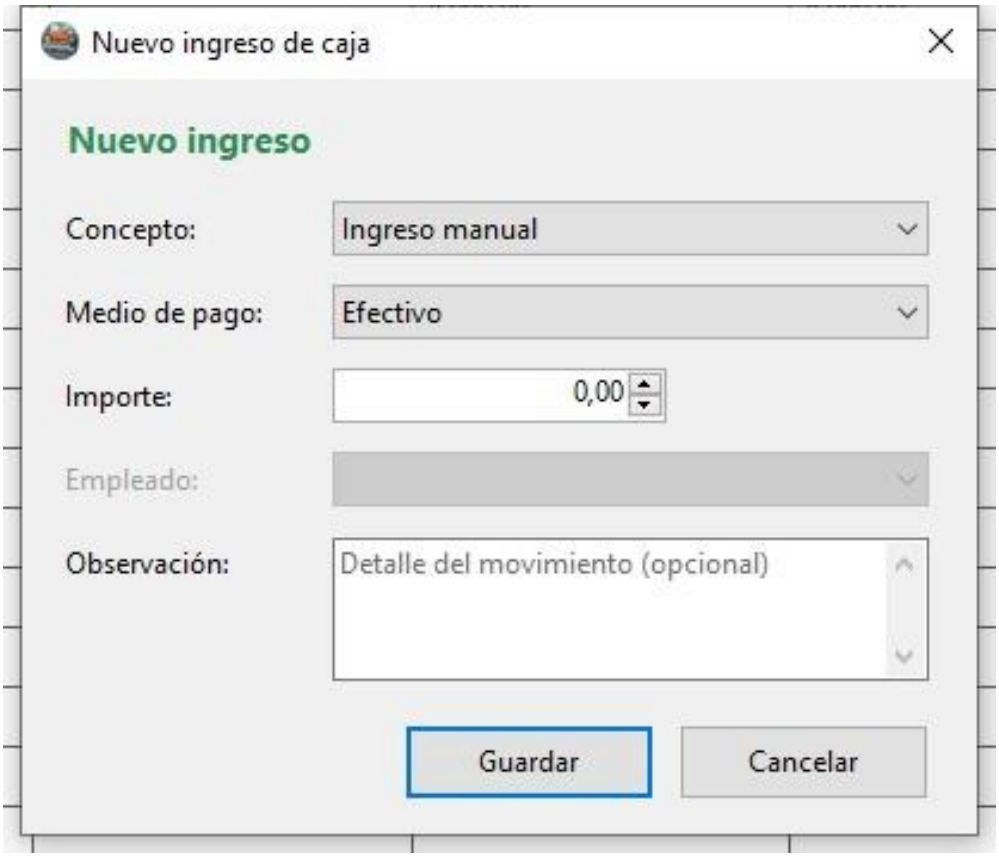
2. Una vez ingresado el usuario, se le mostrará la pantalla Principal.



3. El usuario ya está dentro. Si la caja está cerrada, deberá abrirla desde el menú Caja > Abrir Caja ingresando el fondo inicial.

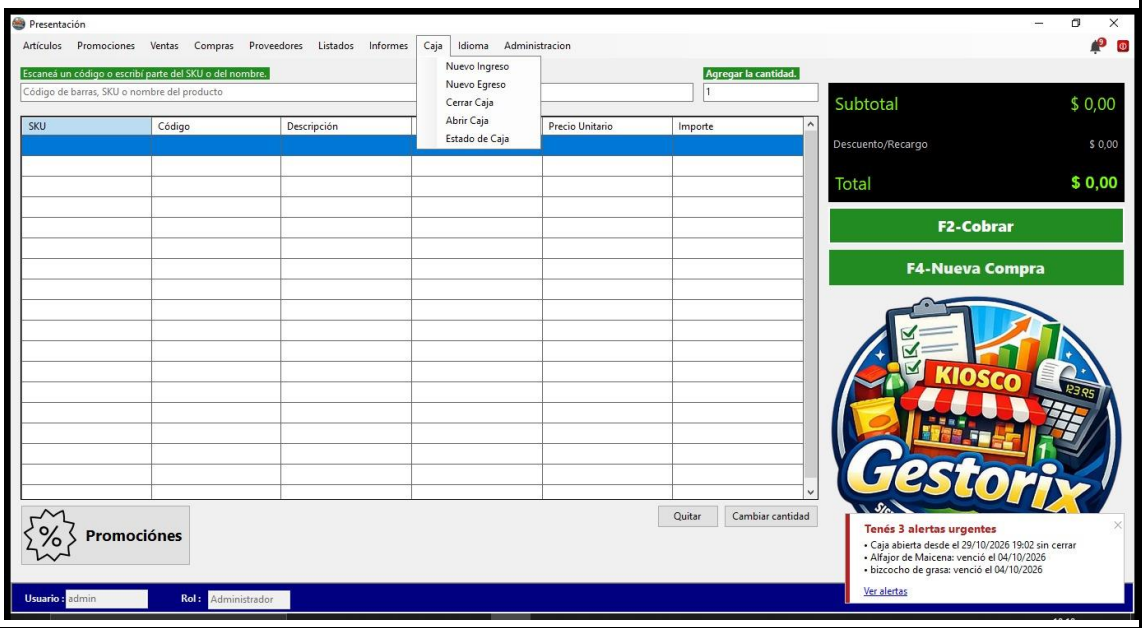


4. En la parte superior hay un menú de navegación donde deberá seleccionar el menú Caja > Nuevo Ingreso.

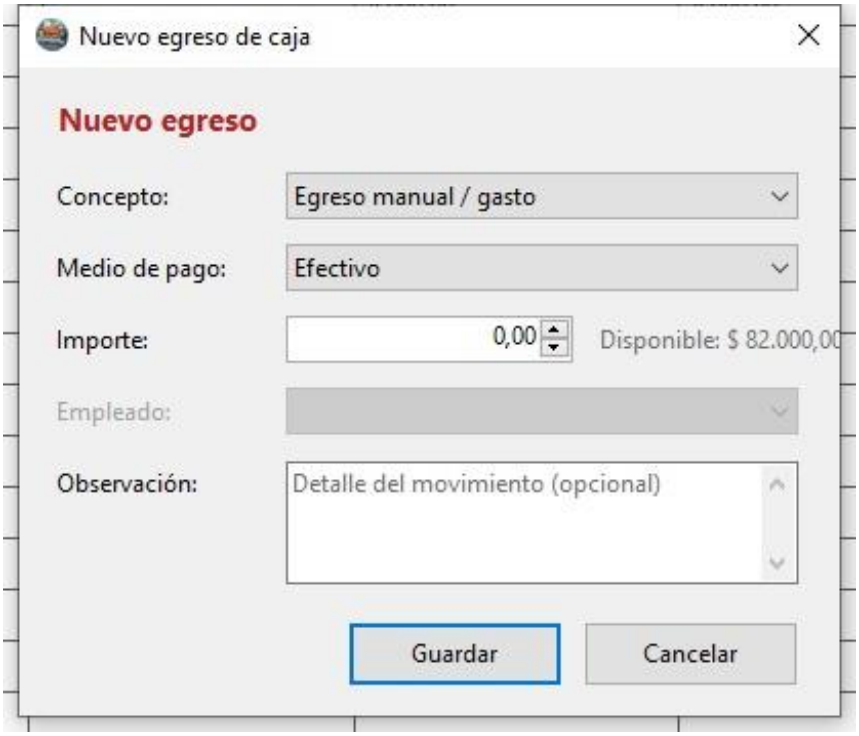


5. Una vez abierto debe seleccionar el concepto y el medio de pago, ingresar el importe y escribir la observación con el motivo del ingreso. Luego debe apretar el botón Guardar y lo va a devolver a la pantalla principal.

6. Para el egreso de plata, por un retiro, un gasto, un vale o el pago de un sueldo, deberá ir al menú y seleccionar Caja.

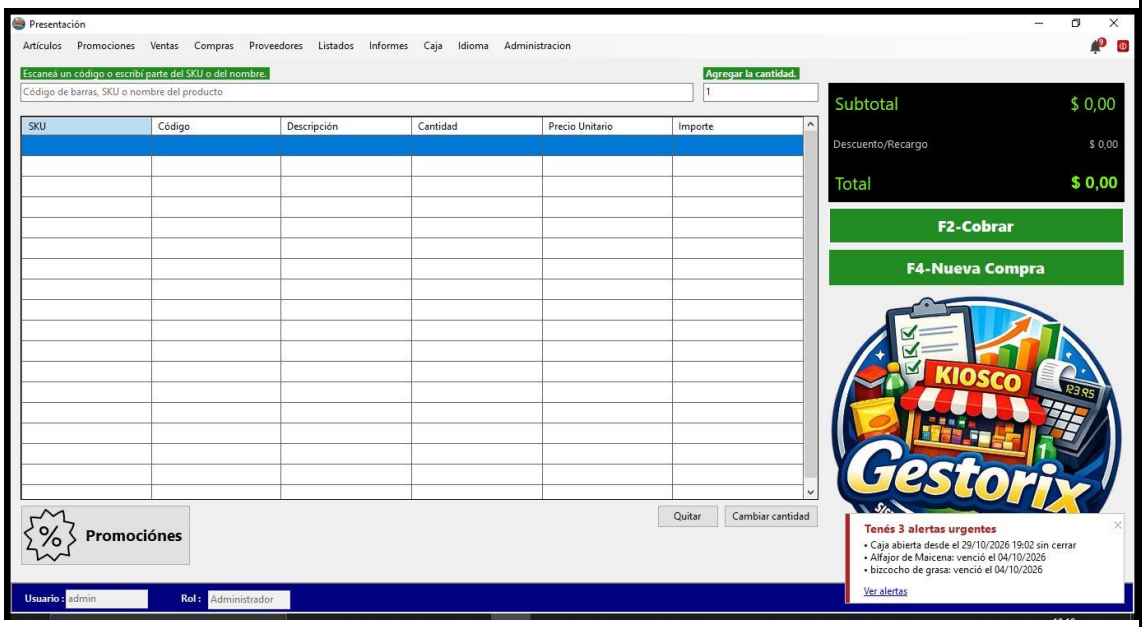


7. Aca debe seleccionar Nuevo Egreso.

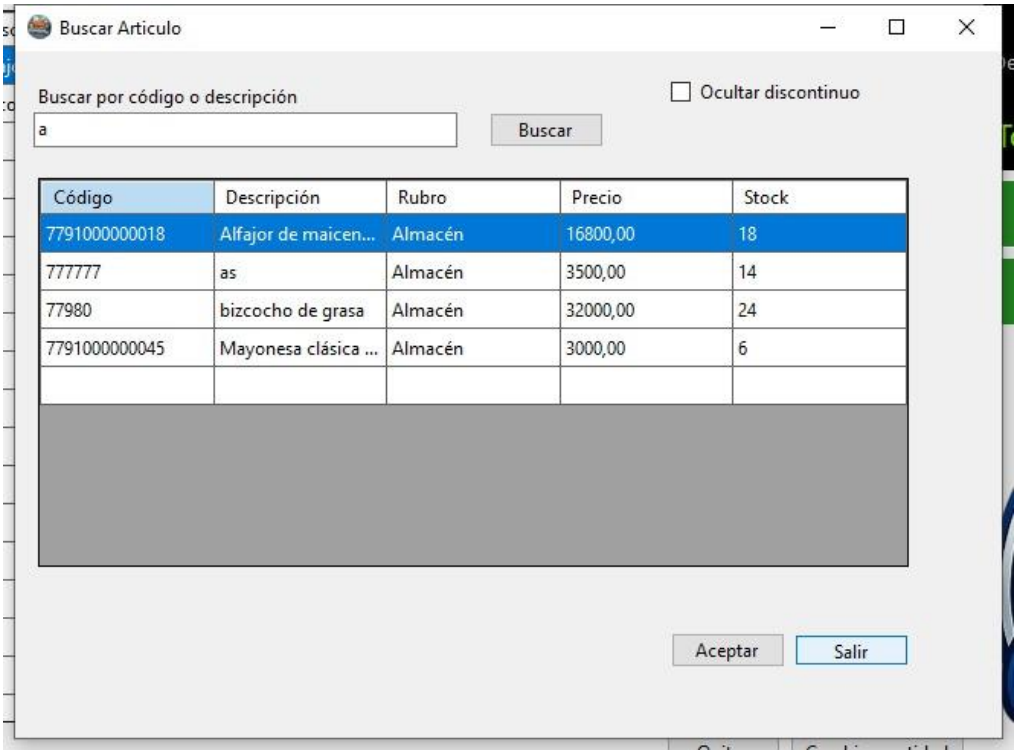


8. Deberá seleccionar el concepto y el medio de pago, ingresar el importe y escribir la observación con el motivo por el cual se retiró la plata (en los vales y sueldos también el empleado). Después debe seleccionar Guardar.

9. Vuelve al menú principal. Para registrar una venta debe escanear el código de barras o escribir parte del SKU o del nombre del producto.

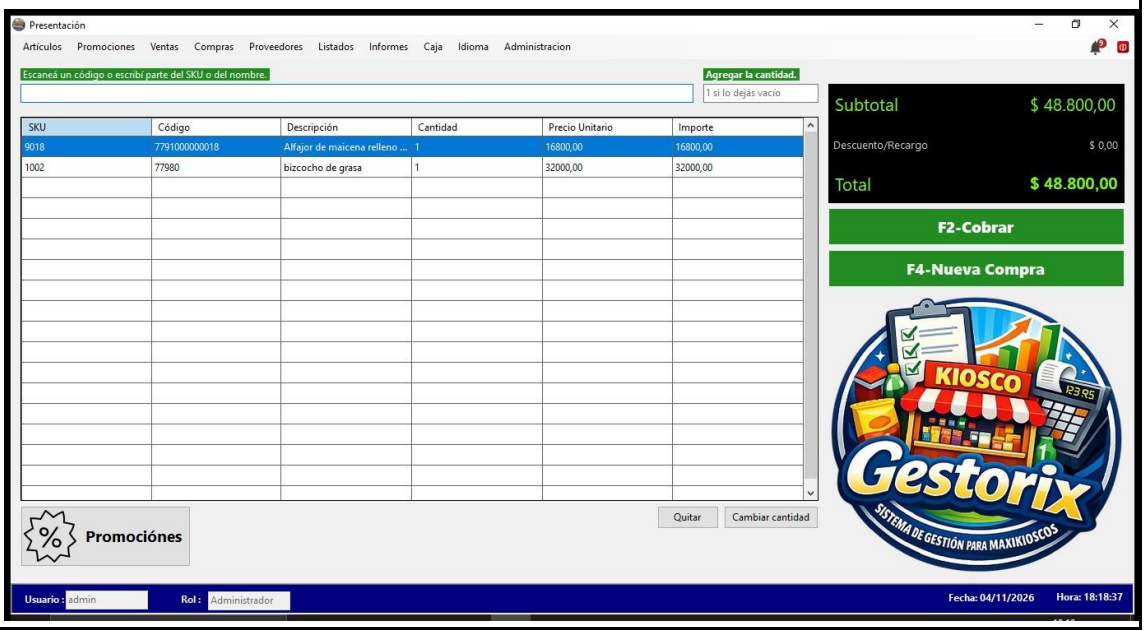


10. Si no encuentra el producto, puede abrir la búsqueda de artículos.



11. Ingresa el código, caso contrario ingresa el nombre o descripción, luego presiona el botón buscar y el o los resultados se mostrarán en las grillas.

12. Una vez seleccionado el producto presiona el botón Aceptar y el producto se agrega a la venta en la pantalla principal.



13. La venta estará cargada en la grilla, para cobrarla debe seleccionar F2-Cobrar o simplemente oprimir la tecla F2.

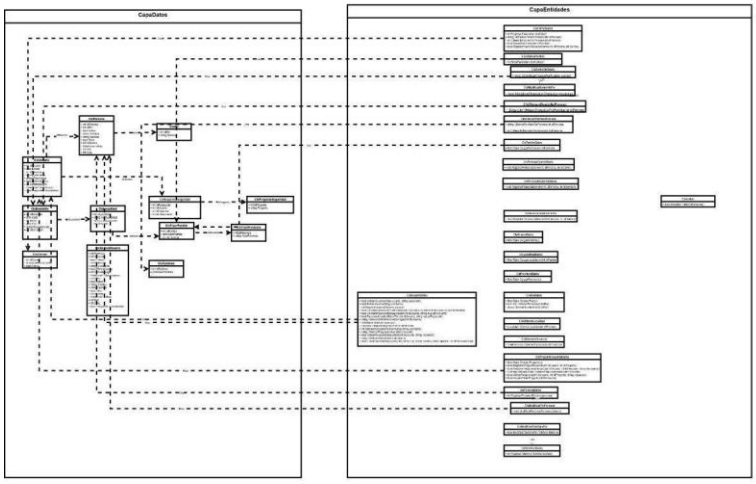


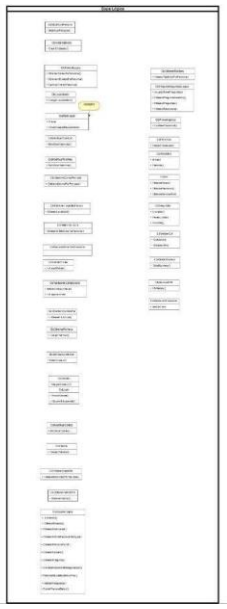
14. Acá debe elegir el medio de pago (Efectivo, Transferencia o Débito, hasta dos por venta) e ingresar el monto. El sistema calcula lo restante y el vuelto.

15. Presiona el botón Confirmar venta; si está todo pagado se registra la venta y lo devuelve a la pantalla principal.

3.7 Diagrama Global de Clases Del Sistema

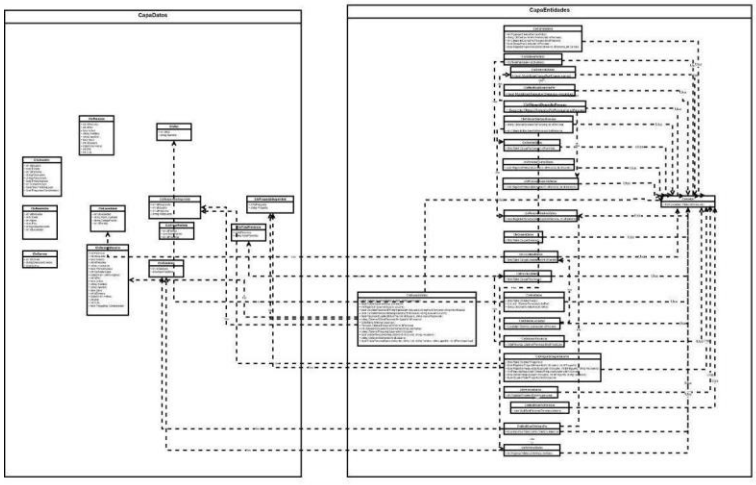
Capa Datos

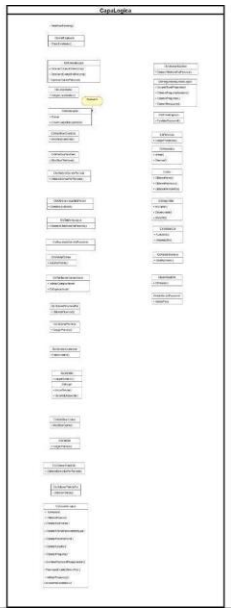






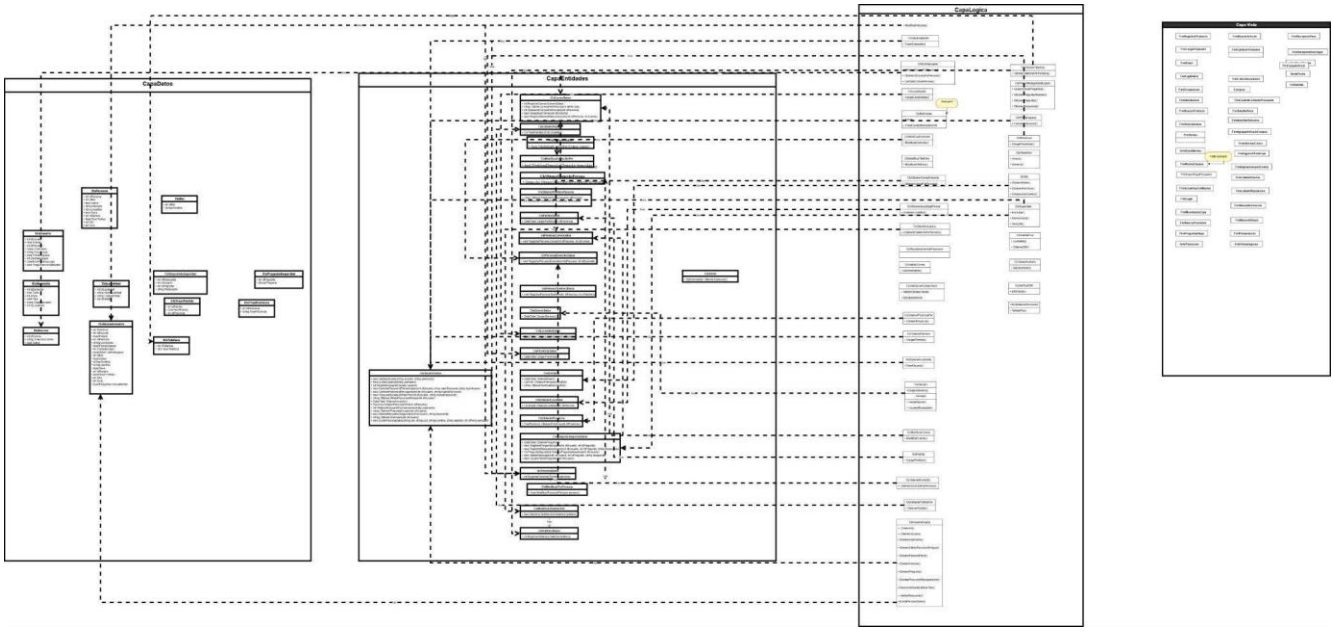
Capa Entidades



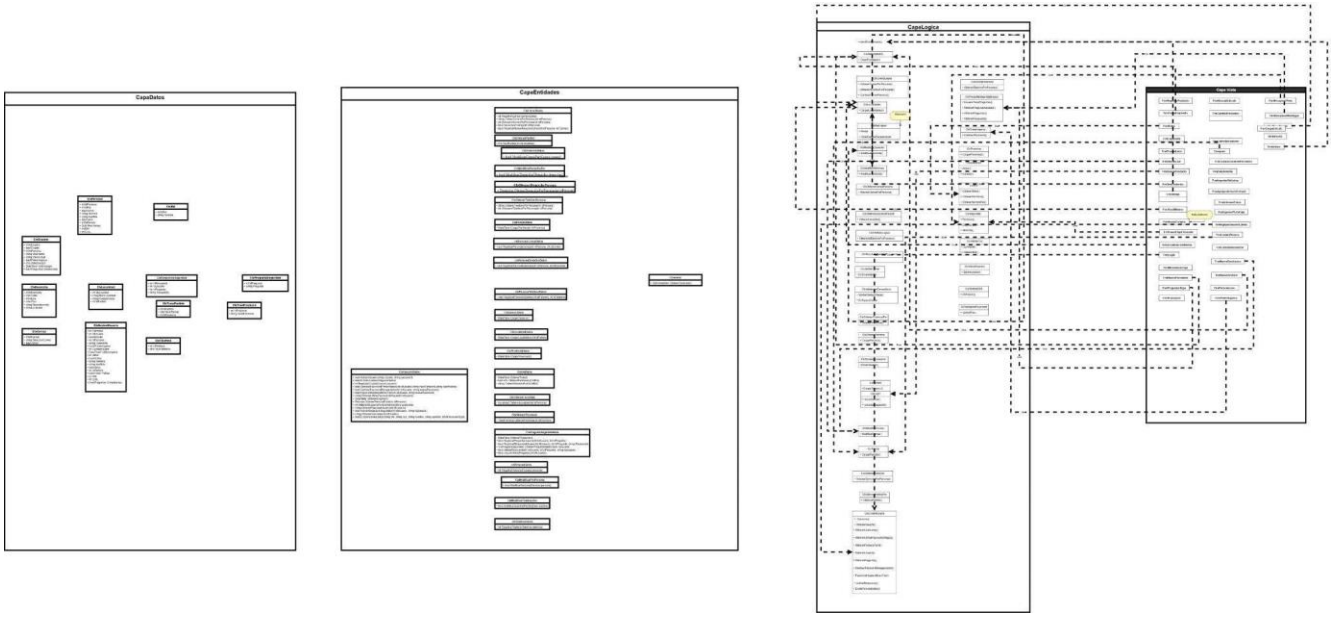




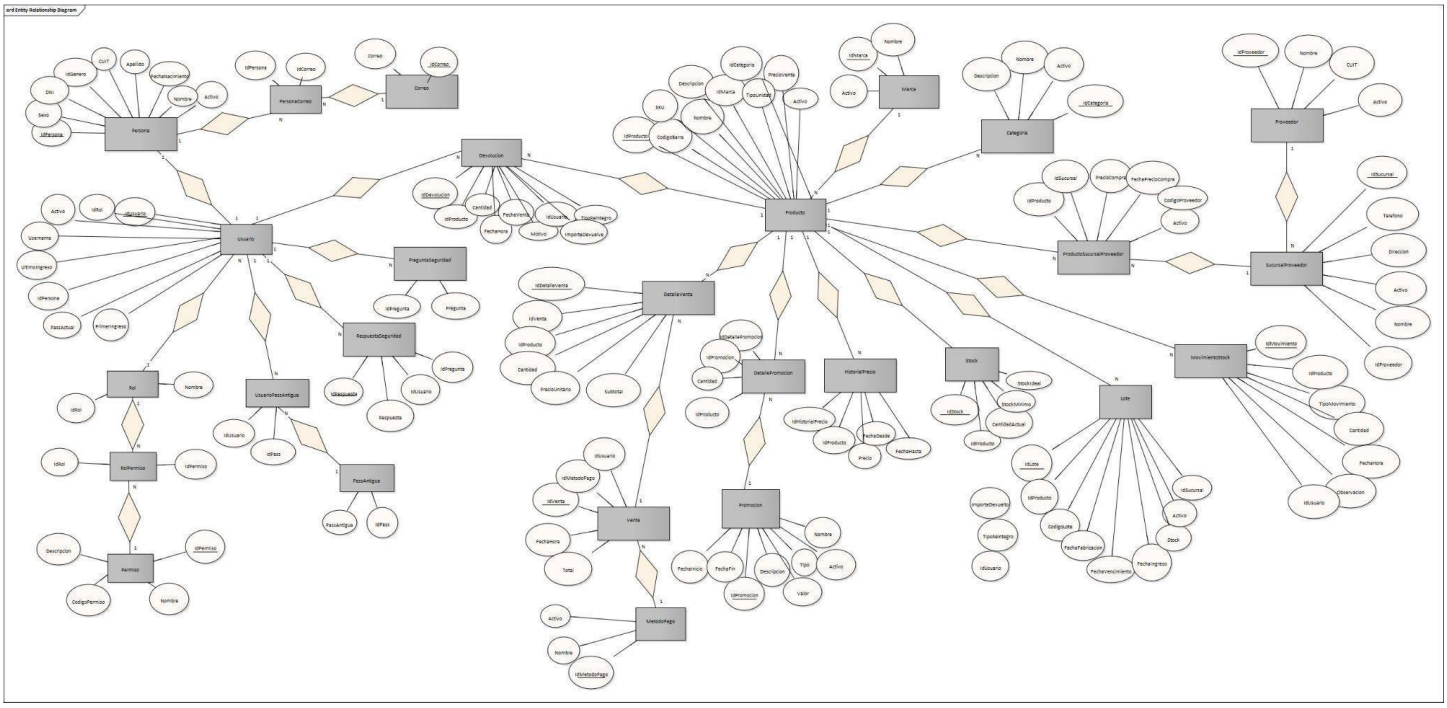
Capa Lógica



Capa Vista

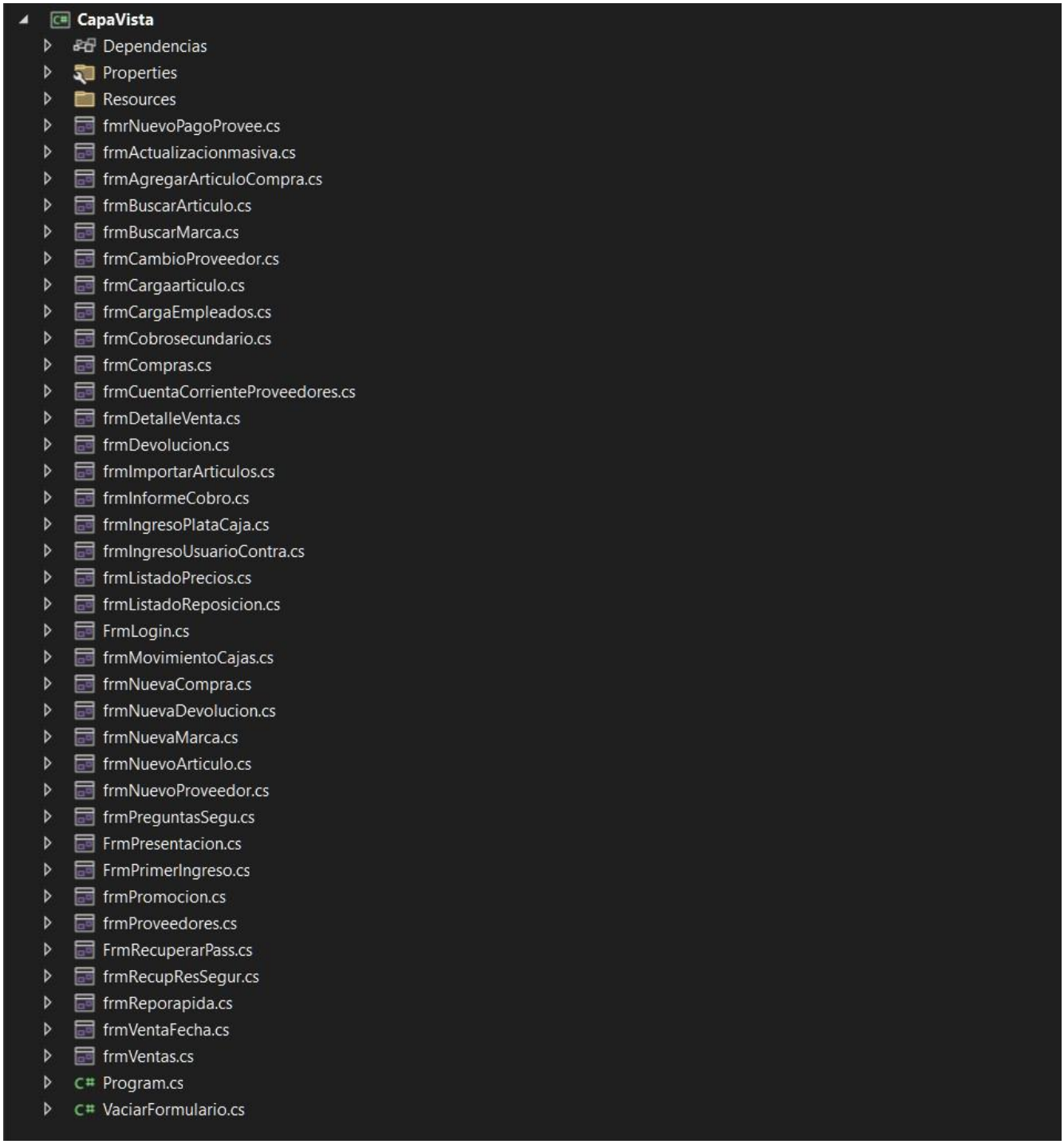


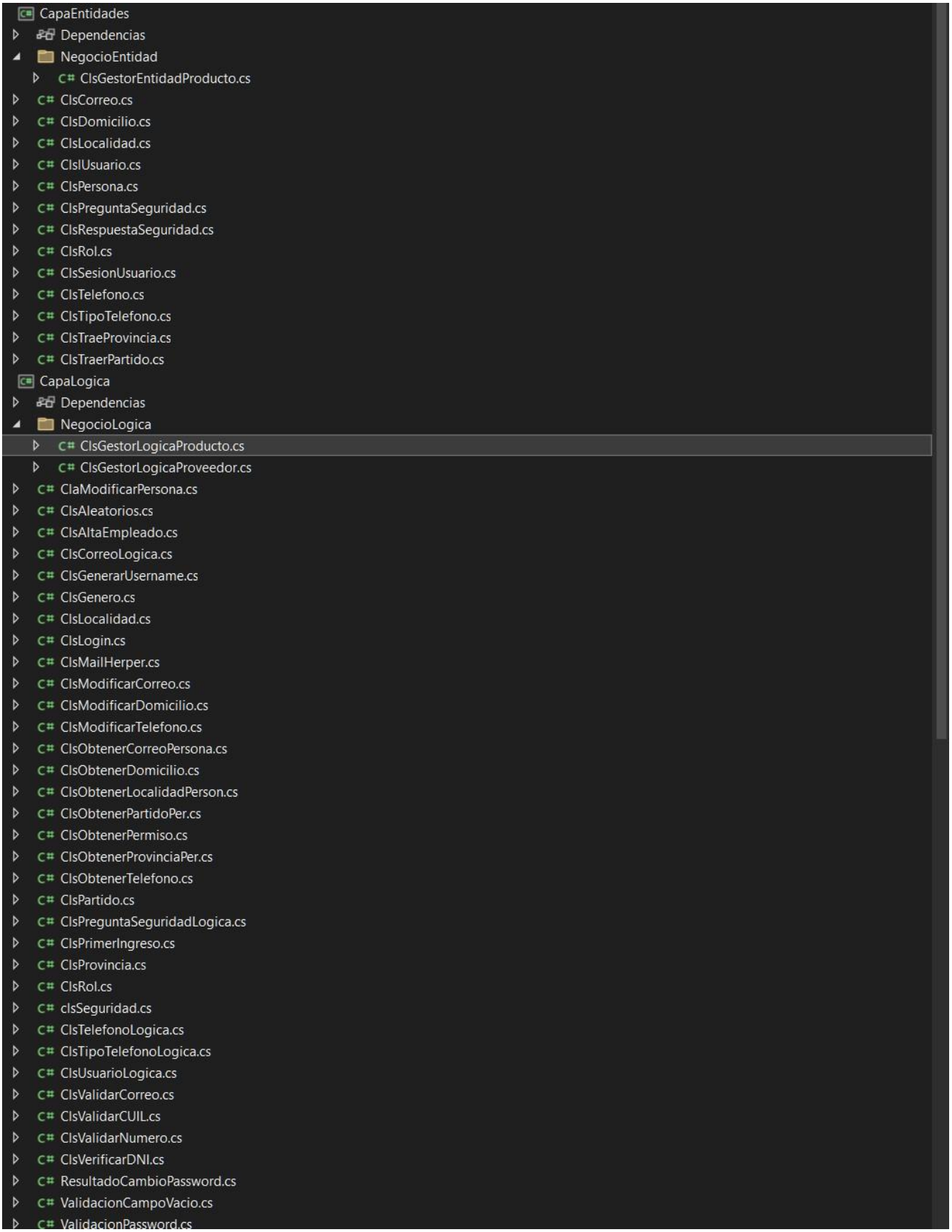
3.9 DER del Sistema

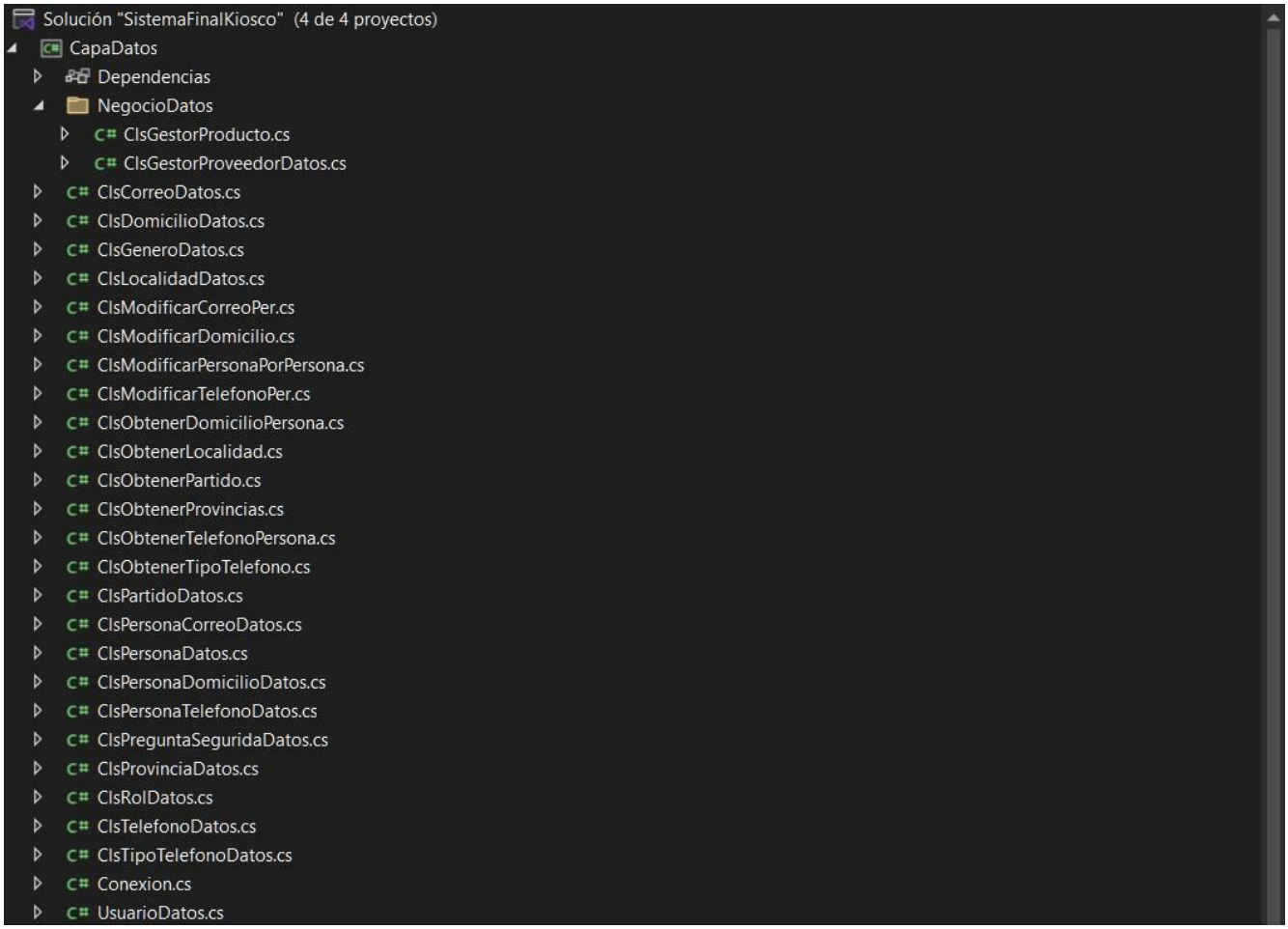


4.0 Estructura principal del software con interfaces

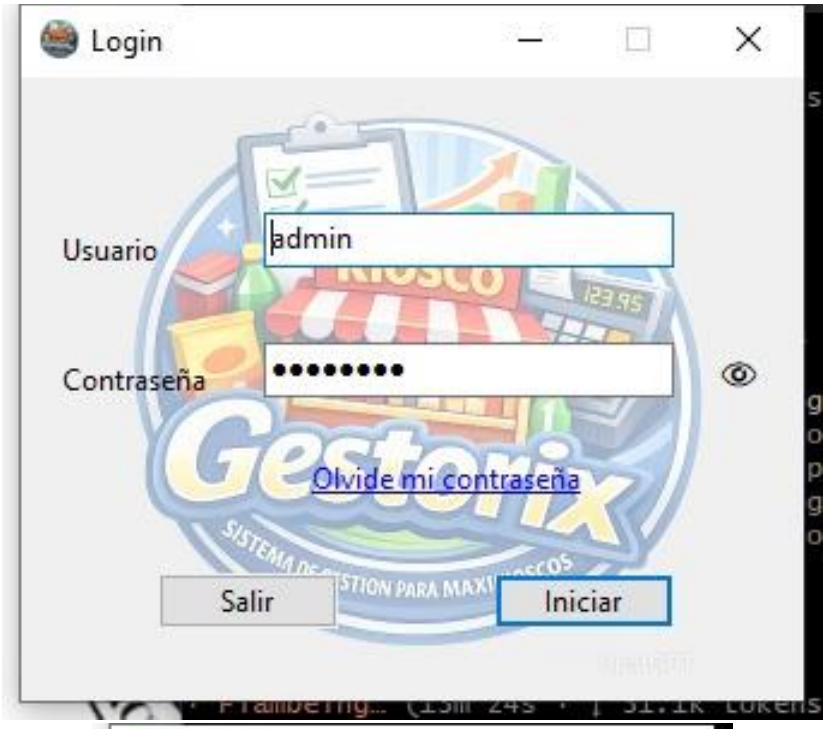
4.1 Estructura de capas del Software

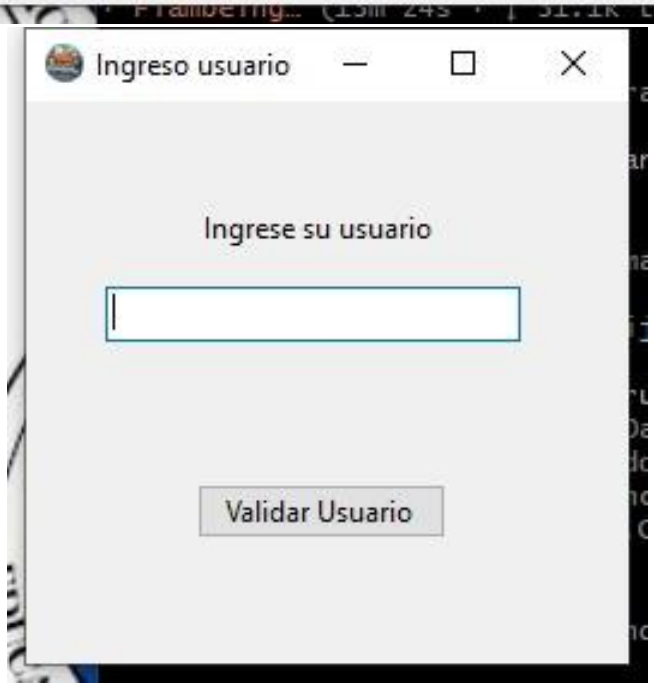

CapaVista
Dependencias
Properties
Resources
fmrNuevoPagoProvee.cs
frmActualizacionmasiva.cs
frmAgregarArticuloCompra.cs
frmBuscarArticulo.cs
frmBuscarMarca.cs
frmCambioProveedor.cs
frmCargaarticulo.cs
frmCargaEmpleados.cs
frmCobrosecundario.cs
frmCompras.cs
frmCuentaCorrienteProveedores.cs
frmDetalleVenta.cs
frmDevolucion.cs
frmImportarArticulos.cs
frmInformeCobro.cs
frmIngresoPlataCaja.cs
frmIngresoUsuarioContra.cs
frmListadoPrecios.cs
frmListadoReposicion.cs
FrmLogin.cs
frmMovimientoCajas.cs
frmNuevaCompra.cs
frmNuevaDevolucion.cs
frmNuevaMarca.cs
frmNuevoArticulo.cs
frmNuevoProveedor.cs
frmPreguntasSegu.cs
FrmPresentacion.cs
FrmPrimerIngreso.cs
frmPromocion.cs
frmProveedores.cs
FrmRecuperarPass.cs
frmRecupResSegur.cs
frmReporapida.cs
frmVentaFecha.cs
frmVentas.cs
Program.cs
VaciarFormulario.cs

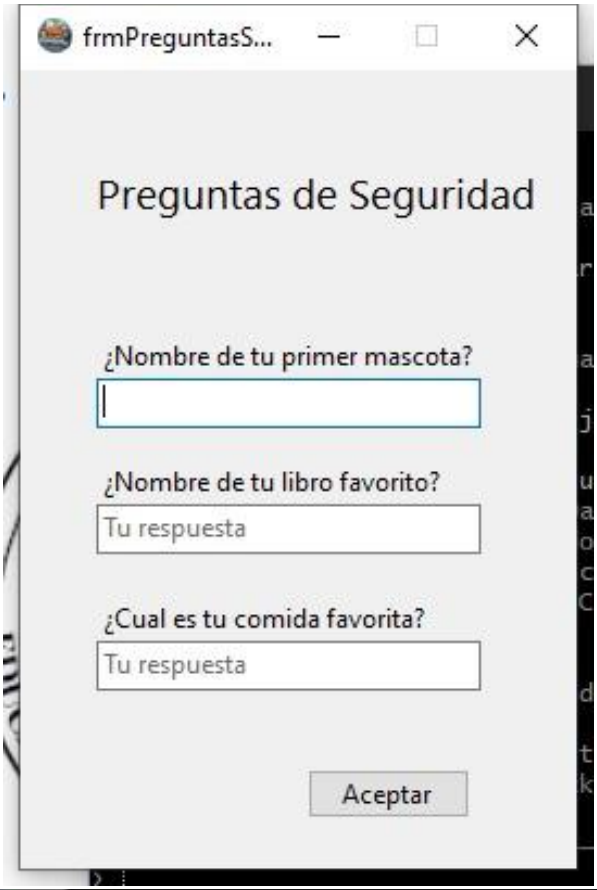

CapaEntidades
Dependencias
NegocioEntidad
ClsGestorEntidadProducto.cs
ClsCorreo.cs
ClsDomicilio.cs
ClsLocalidad.cs
ClsIUsuario.cs
ClsPersona.cs
ClsPreguntaSeguridad.cs
ClsRespuestaSeguridad.cs
ClsRol.cs
ClsSesionUsuario.cs
ClsTelefono.cs
ClsTipoTelefono.cs
ClsTraeProvincia.cs
ClsTraerPartido.cs
CapaLogica
Dependencias
NegocioLogica
ClsGestorLogicaProducto.cs
ClsGestorLogicaProveedor.cs
ClaModificarPersona.cs
ClsAleatorios.cs
ClsAltaEmpleado.cs
ClsCorreoLogica.cs
ClsGenerarUsername.cs
ClsGenero.cs
ClsLocalidad.cs
ClsLogin.cs
ClsMailHerper.cs
ClsModificarCorreo.cs
ClsModificarDomicilio.cs
ClsModificarTelefono.cs
ClsObtenerCorreoPersona.cs
ClsObtenerDomicilio.cs
ClsObtenerLocalidadPerson.cs
ClsObtenerPartidoPer.cs
ClsObtenerPermiso.cs
ClsObtenerProvinciaPer.cs
ClsObtenerTelefono.cs
ClsPartido.cs
ClsPreguntaSeguridadLogica.cs
ClsPrimerIngreso.cs
ClsProvincia.cs
ClsRol.cs
clsSeguridad.cs
ClsTelefonoLogica.cs
ClsTipoTelefonoLogica.cs
ClsUsuarioLogica.cs
ClsValidarCorreo.cs
ClsValidarCUIL.cs
ClsValidarNumero.cs
ClsVerificarDNI.cs
ResultadoCambioPassword.cs
ValidacionCampoVacio.cs
ValidacionPassword.cs

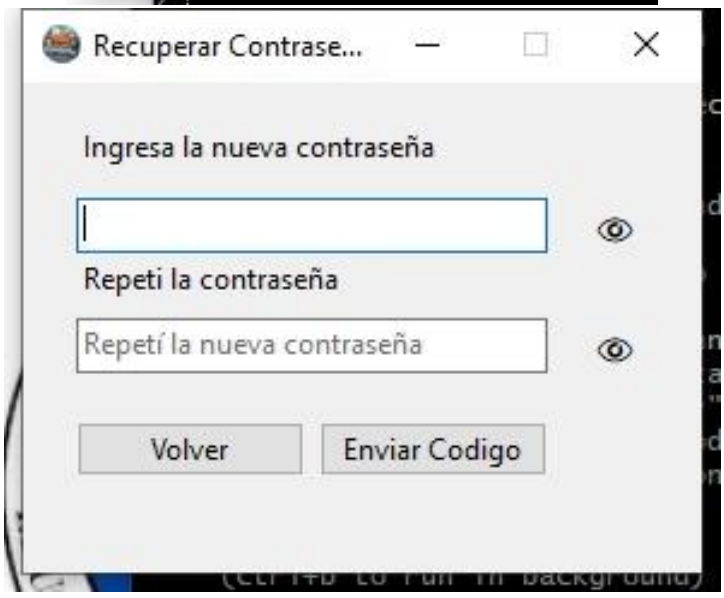


4.2 Estructura principal del Login del software con interfaces

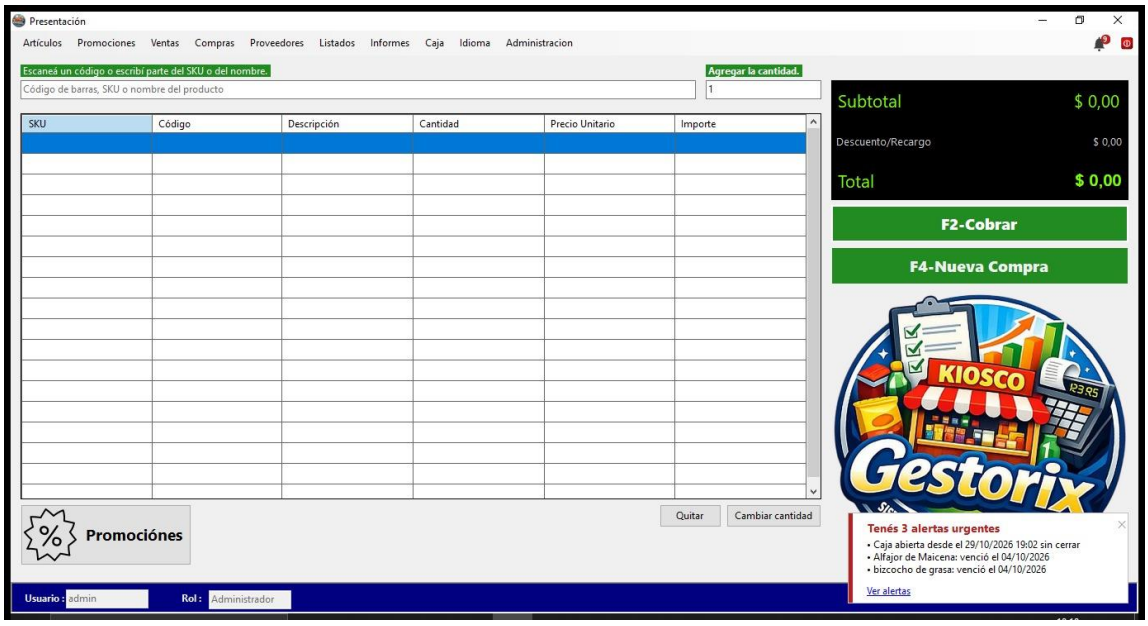

Login
Usuario
admin
Contraseña
Olvide mi contraseña
Salir
Iniciar


Ingreso usuario
Ingrese su usuario
Validar Usuario

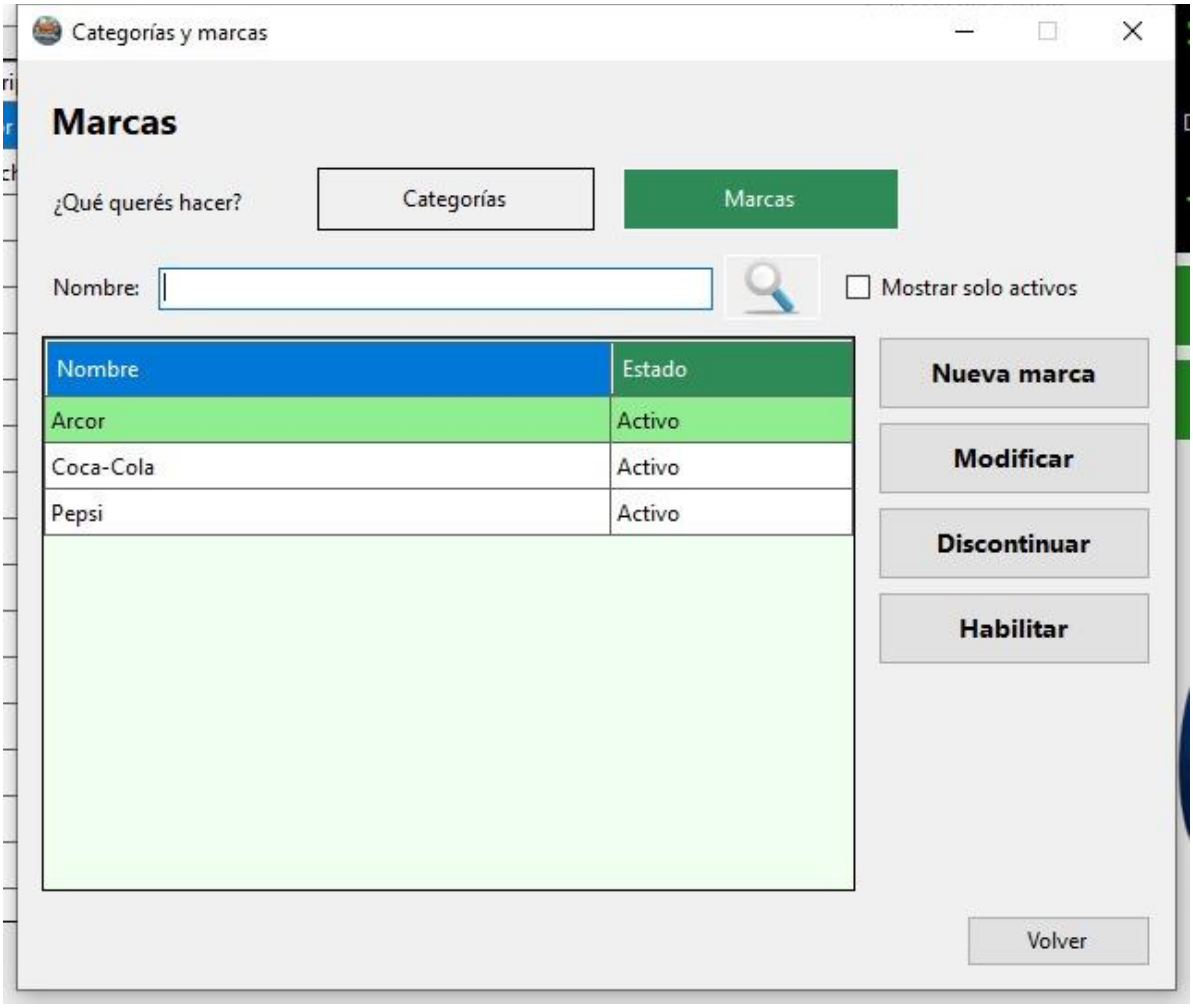

frmPreguntasS...
Preguntas de Seguridad
¿Nombre de tu primer mascota?
¿Nombre de tu libro favorito?
Tu respuesta
¿Cual es tu comida favorita?
Tu respuesta
Aceptar


Recuperar Contrase...
Ingresa la nueva contraseña
Repeti la contraseña
Repetí la nueva contraseña
Volver
Enviar Codigo

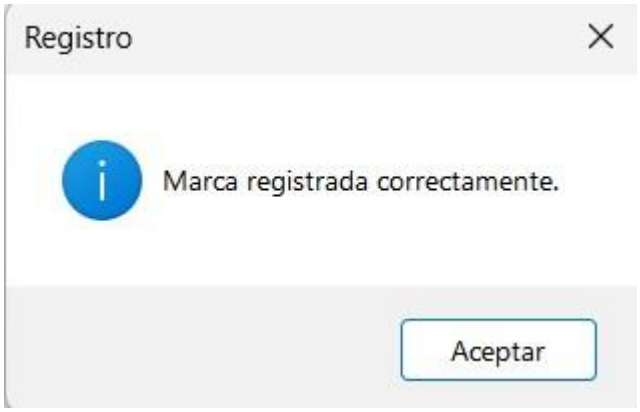
4.3 Estructura principal del menú principal del software con interfaces



4.4 Estructura principal de un caso de uso de negocio del software con interfaces y código: ABM Marca


Categorías y marcas
Marcas
¿Qué querés hacer?
Categorías
Marcas
Nombre:
Mostrar solo activos
Nombre
Estado
Arcor
Activo
Coca-Cola
Activo
Pepsi
Activo
Nueva marca
Modificar
Discontinuar
Habilitar
Volver


Nueva Marca
Nueva Marca
Nombre:
Aceptar
Volver







Capa Vista // frmCategoriaMarca

using CapaEntidades;

```
using CapaLogica.NegocioLogica;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace CapaVista
{
    public partial class frmCategoriaMarca : Form
    {
        private const string MODO_CATEGORIAS = "CATEGORIAS";
        private const string MODO_MARCAS = "MARCAS";

        private ClsGestorProductoLogica logica = new ClsGestorProductoLogica();
        private string modoActual = "";

        public frmCategoriaMarca()
        {
            InitializeComponent();
            ClsIdioma.Registrar(this);

            AplicarModo("");
        }

        #region Metodos
        private bool EsCategorias
        {
            get { return modoActual == MODO_CATEGORIAS; }
        }
        private void AplicarModo(string modo)
        {
            if (modoActual == modo && modo != "")
                modo = "";

            modoActual = modo;

            bool modoActivo = modo != "";

            txtNombre.Clear();
            dgvItems.Rows.Clear();

            txtNombre.Enabled = modoActivo;
            btnBuscar.Enabled = modoActivo;
            chkSoloActivos.Enabled = modoActivo;

            btnEliminar.Visible = EsCategorias;

            // Textos segun el modo
            if (EsCategorias)
            {
                lblTitulo.Text = ClsIdioma.T("Categorías");
                btnNuevo.Text = ClsIdioma.T("Nueva categoría");
            }
            else if (modo == MODO_MARCAS)
            {
                lblTitulo.Text = ClsIdioma.T("Marcas");
```

```
            btnNuevo.Text = ClsIdioma.T("Nueva marca");
        }
        else
        {
            lblTitulo.Text = ClsIdioma.T("Categorías y marcas");
            btnNuevo.Text = ClsIdioma.T("Nuevo");
        }

        // Estilo de los botones
        btnCategorias.FlatStyle = FlatStyle.Flat;
        btnMarcas.FlatStyle = FlatStyle.Flat;

        btnCategorias.UseVisualStyleBackColor = false;
        btnMarcas.UseVisualStyleBackColor = false;

        btnCategorias.BackColor = EsCategorias ? Color.DodgerBlue : SystemColors.Control;
        btnCategorias.ForeColor = EsCategorias ? Color.White : Color.Black;

        btnMarcas.BackColor = modo == MODO_MARCAS ? Color.SeaGreen : SystemColors.Control;
        btnMarcas.ForeColor = modo == MODO_MARCAS ? Color.White : Color.Black;

        // Colores de la grilla
        if (EsCategorias)
        {
            dgvItems.BackgroundColor = Color.AliceBlue;
            dgvItems.DefaultCellStyle.SelectionBackColor = Color.LightSteelBlue;
            dgvItems.ColumnHeadersDefaultCellStyle.BackColor = Color.DodgerBlue;
        }
        else if (modo == MODO_MARCAS)
        {
            dgvItems.BackgroundColor = Color.Honeydew;
            dgvItems.DefaultCellStyle.SelectionBackColor = Color.LightGreen;
            dgvItems.ColumnHeadersDefaultCellStyle.BackColor = Color.SeaGreen;
        }
        else
        {
            dgvItems.BackgroundColor = SystemColors.Window;
            dgvItems.DefaultCellStyle.SelectionBackColor = SystemColors.Highlight;
            dgvItems.ColumnHeadersDefaultCellStyle.BackColor = SystemColors.Control;
        }

        dgvItems.EnableHeadersVisualStyles = false;
        dgvItems.DefaultCellStyle.SelectionForeColor = modoActivo ? Color.Black : SystemColors.HighlightText;
        dgvItems.ColumnHeadersDefaultCellStyle.ForeColor = modoActivo ? Color.White : SystemColors.ControlText;

        AplicarPermisos();

        if (modoActivo)
        {
            CargarDatos();
            txtNombre.Focus();
        }
    }
    private void AplicarPermisos()
    {
        List<int> permisos = SesionUsuario.Permisos;

        btnCategorias.Enabled = permisos.Contains(120) || permisos.Contains(121) || permisos.Contains(122);
        btnMarcas.Enabled = permisos.Contains(130) || permisos.Contains(131) || permisos.Contains(132);

        bool puedeCrear = false;
```

```
    bool puedeModificar = false;

    if (EsCategorias)
    {
        puedeCrear = permisos.Contains(121);
        puedeModificar = permisos.Contains(122);
    }
    else if (modoActual == MODO_MARCAS)
    {
        puedeCrear = permisos.Contains(131);
        puedeModificar = permisos.Contains(132);
    }

    btnNuevo.Enabled = puedeCrear;
    btnModificar.Enabled = puedeModificar;
    btnDiscontinuar.Enabled = puedeModificar;
    btnHabilitar.Enabled = puedeModificar;
    btnEliminar.Enabled = puedeModificar;
}
private void CargarDatos()
{
    if (EsCategorias)
    {
        CargarCategorias();
    }
    else if (modoActual == MODO_MARCAS)
    {
        CargarMarcas();
    }
}
private void CargarCategorias()
{
    DataTable tabla;

    if (chkSoloActivos.Checked)
    {
        tabla = logica.CargarCategoriasActivas();
    }
    else
    {
        tabla = logica.CargarTodasLasCategorias();
    }

    dgvItems.Rows.Clear();

    foreach (DataRow fila in tabla.Rows)
    {
        bool activo = Convert.ToBoolean(fila["Activo"]);
        int indice = dgvItems.Rows.Add(fila["Nombre"], activo ? ClsIdioma.T("Activo") : ClsIdioma.T("Inactivo"));

        dgvItems.Rows[indice].Tag = fila["IdCategoria"];
    }

    FiltrarCategorias();
}
private void FiltrarCategorias()
{
    string texto = txtNombre.Text.Trim().ToLower();

    foreach (DataGridViewRow fila in dgvItems.Rows)
    {
```

```
        if (fila.IsNewRow)
        {
            continue;
        }

        string nombre = fila.Cells["dgvNombre"].Value?.ToString().ToLower() ?? "";

        fila.Visible = string.IsNullOrEmpty(texto) || nombre.Contains(texto);
    }
}
private void CargarMarcas()
{
    DataTable marcas;

    string nombre = txtNombre.Text.Trim();

    if (!string.IsNullOrEmpty(nombre))
    {
        marcas = logica.BuscarMarcas(nombre, chkSoloActivos.Checked);
    }
    else if (chkSoloActivos.Checked)
    {
        marcas = logica.CargarMarcasActivas();
    }
    else
    {
        marcas = logica.CargarTodasLasMarcas();
    }

    dgvItems.Rows.Clear();

    foreach (DataRow fila in marcas.Rows)
    {
        bool activo = Convert.ToBoolean(fila["Activo"]);
        int indice = dgvItems.Rows.Add(fila["Nombre"], activo ? ClsIdioma.T("Activo") : ClsIdioma.T("Discontinuado"));

        dgvItems.Rows[indice].Tag = fila["IdMarca"];
    }
}
private DataGridViewRow ObtenerFilaSeleccionada(string accion)
{
    if (dgvItems.SelectedRows.Count == 0)
    {
        string elemento = EsCategorias ? ClsIdioma.T("una categoría") : ClsIdioma.T("una marca");

        MessageBox.Show(ClsIdioma.T("Seleccione ") + elemento + ClsIdioma.T(" para ") + accion + ".", ClsIdioma.T("Aviso"),
            MessageBoxButtons.OK, MessageBoxIcon.Warning);

        return null;
    }

    return dgvItems.SelectedRows[0];
}
private bool EsCategoriaProtegida(int idCategoria, string mensaje)
{
    if (idCategoria >= 1002 && idCategoria <= 1012)
    {
        MessageBox.Show(mensaje, ClsIdioma.T("Categoría protegida"),
            MessageBoxButtons.OK, MessageBoxIcon.Information);

        return true;
```

```
    }

    return false;
}
#endregion

#region Eventos
private void btnCategorias_Click(object sender, EventArgs e)
{
    AplicarModo(MODO_CATEGORIAS);
}
private void btnMarcas_Click(object sender, EventArgs e)
{
    AplicarModo(MODO_MARCAS);
}
private void btnBuscar_Click(object sender, EventArgs e)
{
    if (EsCategorias)
    {
        FiltrarCategorias();
    }
    else
    {
        CargarMarcas();
    }
}
private void txtNombre_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        e.SuppressKeyPress = true;
        btnBuscar_Click(sender, EventArgs.Empty);
    }
}
private void chkSoloActivos_CheckedChanged(object sender, EventArgs e)
{
    CargarDatos();
}
private void btnNuevo_Click(object sender, EventArgs e)
{
    if (EsCategorias)
    {
        using (frmNuevoRubro formulario = new frmNuevoRubro())
        {
            if (formulario.ShowDialog() == DialogResult.OK)
            {
                CargarCategorias();
            }
        }
    }
    else
    {
        using (frmNuevaMarca formulario = new frmNuevaMarca())
        {
            formulario.ShowDialog();
        }

        CargarMarcas();
    }
}
private void btnModificar_Click(object sender, EventArgs e)
```

```
{
    DataGridViewRow fila = ObtenerFilaSeleccionada(ClsIdioma.T("modificar"));

    if (fila == null)
    {
        return;
    }

    int id = Convert.ToInt32(fila.Tag);
    string nombre = fila.Cells["dgvNombre"].Value?.ToString();

    if (EsCategorias)
    {
        if (EsCategoriaProtegida(id, ClsIdioma.T("Las categorías predeterminadas del sistema no se pueden modificar.")))
        {
            return;
        }

        using (frmNuevoRubro formulario = new frmNuevoRubro(id, nombre))
        {
            if (formulario.ShowDialog() == DialogResult.OK)
            {
                CargarCategorias();
            }
        }
    }
    else
    {
        using (frmNuevaMarca formulario = new frmNuevaMarca())
        {
            formulario.Text = ClsIdioma.T("Modificar Marca");
            formulario.IdMarca = id;
            formulario.NombreMarca = nombre;

            formulario.ShowDialog();
        }

        CargarMarcas();
    }
}
private void btnDiscontinuar_Click(object sender, EventArgs e)
{
    DataGridViewRow fila = ObtenerFilaSeleccionada(ClsIdioma.T("discontinuar"));

    if (fila == null)
    {
        return;
    }

    int id = Convert.ToInt32(fila.Tag);
    string nombre = fila.Cells["dgvNombre"].Value?.ToString();

    if (EsCategorias)
    {
        if (EsCategoriaProtegida(id, ClsIdioma.T("Las categorías predeterminadas del sistema no se pueden desactivar.")))
        {
            return;
        }

        DialogResult respuesta = MessageBox.Show(ClsIdioma.T("¿Está seguro de desactivar la categoría \"") + nombre + "\"?",
ClsIdioma.T("Confirmar"),
```

```
                MessageBoxButtons.YesNo, MessageBoxIcon.Question);

            if (respuesta != DialogResult.Yes)
            {
                return;
            }

            try
            {
                if (logica.DesactivarCategoria(id))
                {
                    CargarCategorias();
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show(ClsIdioma.T("No se pudo desactivar la categoría.\n\n") + ex.Message,
                    ClsIdioma.T("Error"), MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
        }
        else
        {
            DialogResult respuesta = MessageBox.Show(ClsIdioma.T("¿Está seguro de discontinuar la marca \"") + nombre + "\"?",
ClsIdioma.T("Discontinuar marca"),
                MessageBoxButtons.YesNo, MessageBoxIcon.Question);

            if (respuesta != DialogResult.Yes)
            {
                return;
            }

            if (logica.DesactivarMarca(id))
            {
                MessageBox.Show(ClsIdioma.T("La marca fue discontinuada correctamente."), ClsIdioma.T("Discontinuar marca"),
                    MessageBoxButtons.OK, MessageBoxIcon.Information);

                CargarMarcas();
            }
        }
    }
    private void btnHabilitar_Click(object sender, EventArgs e)
    {
        DataGridViewRow fila = ObtenerFilaSeleccionada(ClsIdioma.T("habilitar"));

        if (fila == null)
        {
            return;
        }

        int id = Convert.ToInt32(fila.Tag);
        string nombre = fila.Cells["dgvNombre"].Value?.ToString();

        if (EsCategorias)
        {
            if (EsCategoriaProtegida(id, ClsIdioma.T("Las categorías predeterminadas del sistema no necesitan ser habilitadas.")))
            {
                return;
            }

            DialogResult respuesta = MessageBox.Show(ClsIdioma.T("¿Está seguro de habilitar la categoría \"") + nombre + "\"?",
ClsIdioma.T("Confirmar"),
```

```
                MessageBoxButtons.YesNo, MessageBoxIcon.Question);

            if (respuesta != DialogResult.Yes)
            {
                return;
            }

            try
            {
                if (logica.HabilitarCategoria(id))
                {
                    CargarCategorias();
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show(ClsIdioma.T("No se pudo habilitar la categoría.\n\n") + ex.Message, ClsIdioma.T("Error"),
                    MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
        }
        else
        {
            DialogResult respuesta = MessageBox.Show(ClsIdioma.T("¿Está seguro de habilitar la marca \"") + nombre + "\"?",
ClsIdioma.T("Habilitar marca"),
                MessageBoxButtons.YesNo, MessageBoxIcon.Question);

            if (respuesta != DialogResult.Yes)
            {
                return;
            }

            if (logica.HabilitarMarca(id))
            {
                MessageBox.Show(ClsIdioma.T("La marca fue habilitada correctamente."), ClsIdioma.T("Habilitar marca"),
                    MessageBoxButtons.OK, MessageBoxIcon.Information);

                CargarMarcas();
            }
        }
    }
    private void btnEliminar_Click(object sender, EventArgs e)
    {
        if (!EsCategorias)
        {
            return;
        }

        DataGridViewRow fila = ObtenerFilaSeleccionada(ClsIdioma.T("eliminar"));

        if (fila == null)
        {
            return;
        }

        int idCategoria = Convert.ToInt32(fila.Tag);
        string nombre = fila.Cells["dgvNombre"].Value?.ToString();

        if (EsCategoriaProtegida(idCategoria, ClsIdioma.T("Las categorías predeterminadas del sistema no se pueden eliminar.")))
        {
            return;
        }
```

```
        DialogResult respuesta = MessageBox.Show((ClsIdioma.Ingles ? $"Are you sure you want to delete the category \"{nombre}\"?" :
$"¿Está seguro de eliminar la categoría \"{nombre}\"?"), ClsIdioma.T("Confirmar eliminación"),
            MessageBoxButtons.YesNo, MessageBoxIcon.Warning);

        if (respuesta != DialogResult.Yes)
        {
            return;
        }

        try
        {
            if (logica.EliminarCategoria(idCategoria))
            {
                CargarCategorias();
            }
            else
            {
                MessageBox.Show(ClsIdioma.T("No se pudo eliminar la categoría."), ClsIdioma.T("Aviso"),
                    MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show(ClsIdioma.T("No se pudo eliminar la categoría.\n\n") + ex.Message, ClsIdioma.T("Error"),
                MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
    private void btnVolver_Click(object sender, EventArgs e)
    {
        Close();
    }
    #endregion
  }
}
```

Capa Lógica // ClsGestorProductoLogica

```
using CapaDatos.NegocioDatos;
using CapaEntidades;
using CapaEntidades.NegocioEntidad;
using CapaLogica.BitacoraLogi;
using Microsoft.Data.SqlClient;
using SistemaKiosco;
using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using static CapaEntidades.NegocioEntidad.ClsGestorEntidadProducto;

public int RegistrarMarca(ClsMarca marca)
{
    return datos.RegistrarMarca(marca);
}

public DataTable CargarMarcasActivas()
{
    return datos.CargarMarcasActivas();
}
```

```
public DataTable CargarTodasLasMarcas()
{
    return datos.CargarTodasLasMarcas();
}

public bool ModificarMarca(ClsMarca marca)
{
    return datos.ModificarMarca(marca);
}

public bool DesactivarMarca(int idMarca)
{
    return datos.DesactivarMarca(idMarca);
}

public bool HabilitarMarca(int idMarca)
{
    return datos.HabilitarMarca(idMarca);
}

public DataTable BuscarMarcas(string nombre, bool soloActivos)
{
    return datos.BuscarMarcas(nombre, soloActivos);
}

public int ObtenerIdMarcaPorNombre(string nombre)
{
    DataTable tabla = datos.CargarMarcasActivas();

    foreach (DataRow fila in tabla.Rows)
    {
        string nombreBD = fila["Nombre"].ToString();

        if (string.Equals(nombreBD.Trim(), nombre.Trim(),
            StringComparison.OrdinalIgnoreCase))
        {
            return Convert.ToInt32(fila["IdMarca"]);
        }
    }

    return 0;
}
```

Capa Entidades // ClsGestorEntidadProducto

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

public class ClsMarca
    {
        public int IdMarca { get; set; }
        public string Nombre { get; set; }
        public bool Activo { get; set; }

    }
```

Capa Datos // ClsGestorProductoDatos

```
using CapaDatos.SistemaKiosco;
using CapaEntidades;
using CapaEntidades.NegocioEntidad;
using Microsoft.Data.SqlClient;
using SistemaKiosco;
using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using static CapaEntidades.NegocioEntidad.ClsGestorEntidadProducto;

public int RegistrarMarca(ClsGestorEntidadProducto.ClsMarca marca)
{
    using (SqlConnection conexion = cn.ObtenerConexion())

    using (SqlCommand cmd = new SqlCommand("sp_RegistrarMarca", conexion))
    {
        cmd.CommandType = CommandType.StoredProcedure;
        cmd.Parameters.AddWithValue("@Nombre", marca.Nombre);
        cmd.Parameters.AddWithValue("@Activo", marca.Activo);

        conexion.Open();

        return Convert.ToInt32(cmd.ExecuteScalar());
    }
}

public DataTable CargarMarcasActivas()
{
    using (SqlConnection conexion = cn.ObtenerConexion())

    using (SqlCommand cmd = new SqlCommand("sp_CargarMarcasActivas", conexion))
    {
        cmd.CommandType = CommandType.StoredProcedure;

        using (SqlDataAdapter da = new SqlDataAdapter(cmd))
        {
            DataTable tabla = new DataTable();
            da.Fill(tabla);
            return tabla;
        }
    }
}

public bool ModificarMarca(ClsGestorEntidadProducto.ClsMarca marca)
{
    using (SqlConnection conexion = cn.ObtenerConexion())

    using (SqlCommand cmd = new SqlCommand("sp_ModificarMarca", conexion))
    {
        cmd.CommandType = CommandType.StoredProcedure;
        cmd.Parameters.AddWithValue("@IdMarca", marca.IdMarca);
        cmd.Parameters.AddWithValue("@Nombre", marca.Nombre);

        conexion.Open();

        return Convert.ToInt32(cmd.ExecuteScalar()) > 0;
    }
}
```

```
public bool HabilitarMarca(int idMarca)
{
    using (SqlConnection conexion = cn.ObtenerConexion())

    using (SqlCommand cmd = new SqlCommand("sp_HabilitarMarca", conexion))
    {
        cmd.CommandType = CommandType.StoredProcedure;
        cmd.Parameters.AddWithValue("@IdMarca", idMarca);

        conexion.Open();

        return Convert.ToInt32(cmd.ExecuteScalar()) > 0;
    }
}

public bool DesactivarMarca(int idMarca)
{
    using (SqlConnection conexion = cn.ObtenerConexion())

    using (SqlCommand cmd = new SqlCommand("sp_DesactivarMarca", conexion))
    {
        cmd.CommandType = CommandType.StoredProcedure;
        cmd.Parameters.AddWithValue("@IdMarca", idMarca);

        conexion.Open();

        return Convert.ToInt32(cmd.ExecuteScalar()) > 0;
    }
}

public DataTable CargarTodasLasMarcas()
{
    using (SqlConnection conexion = cn.ObtenerConexion())

    using (SqlCommand cmd = new SqlCommand("sp_CargarTodasLasMarcas", conexion))
    {
        cmd.CommandType = CommandType.StoredProcedure;

        using (SqlDataAdapter da = new SqlDataAdapter(cmd))
        {
            DataTable tabla = new DataTable();
            da.Fill(tabla);
            return tabla;
        }
    }
}

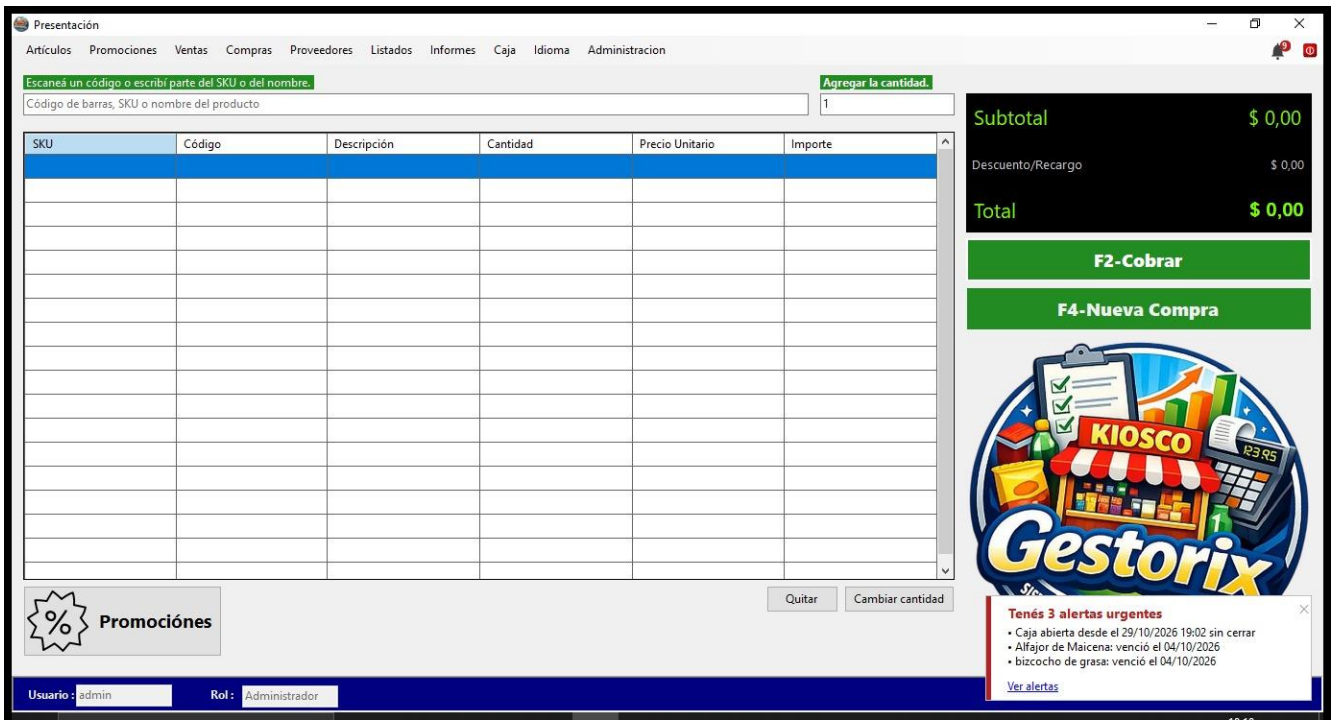
public DataTable BuscarMarcas(string nombre, bool soloActivos)
{
    using (SqlConnection conexion = cn.ObtenerConexion())

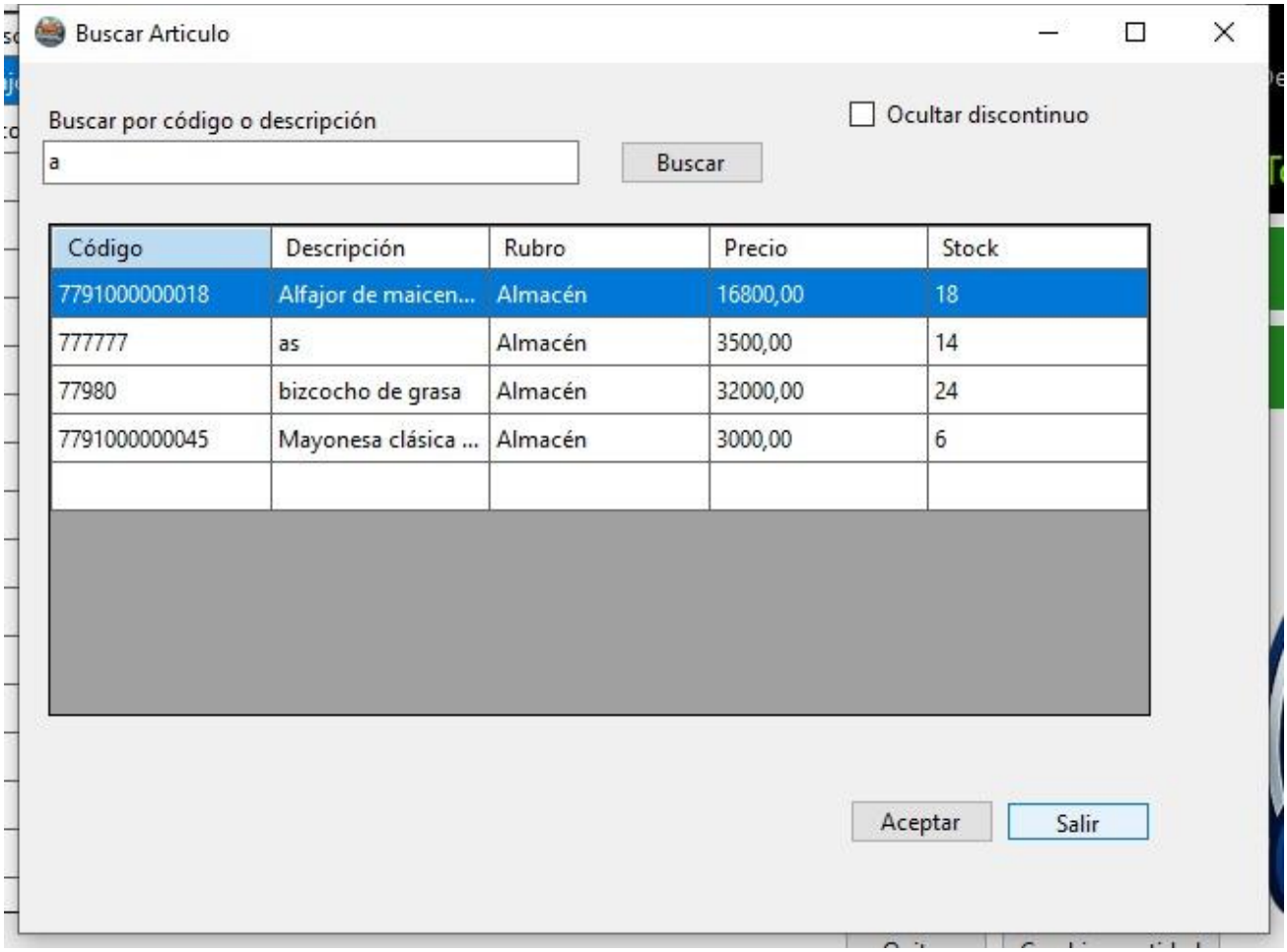
    using (SqlCommand cmd = new SqlCommand("sp_BuscarMarcas", conexion))
    {
        cmd.CommandType = CommandType.StoredProcedure;
        cmd.Parameters.AddWithValue("@Nombre", nombre);
        cmd.Parameters.AddWithValue("@SoloActivos", soloActivos);

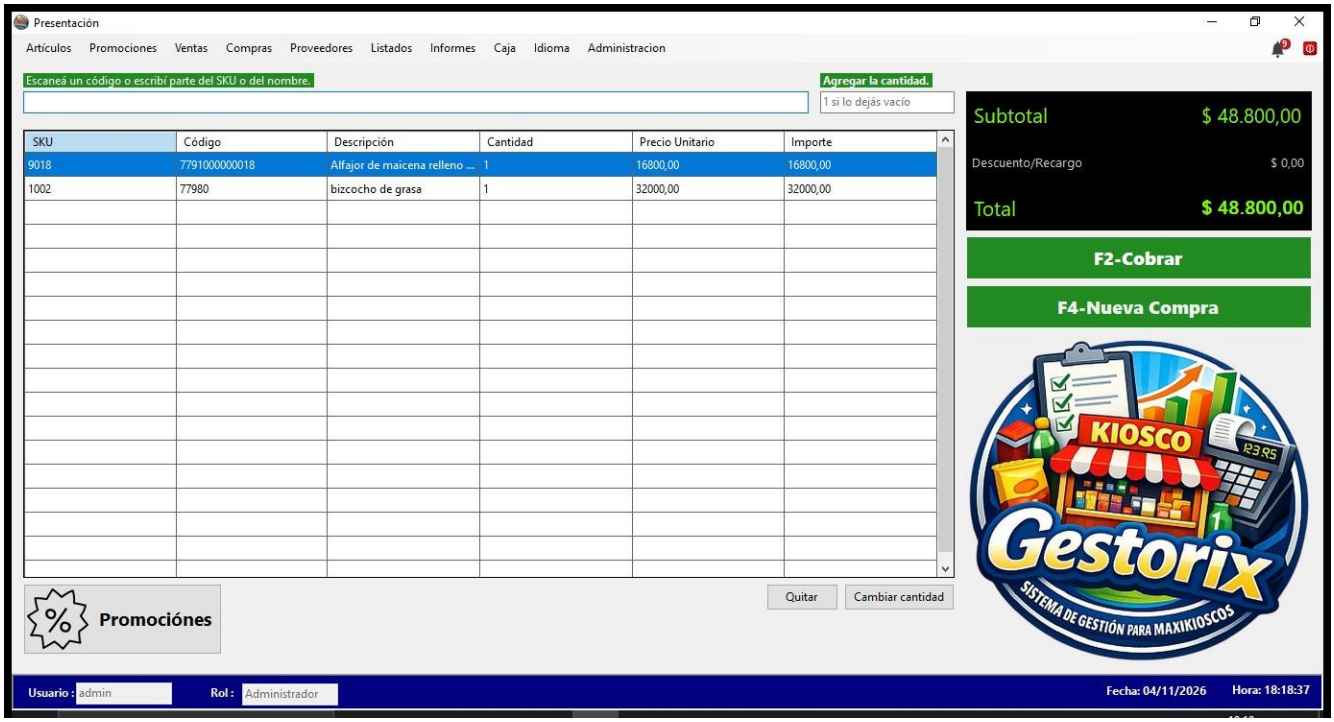
        using (SqlDataAdapter da = new SqlDataAdapter(cmd))
        {
            DataTable tabla = new DataTable();
```

```
            da.Fill(tabla);
            return tabla;
        }
    }
}
```

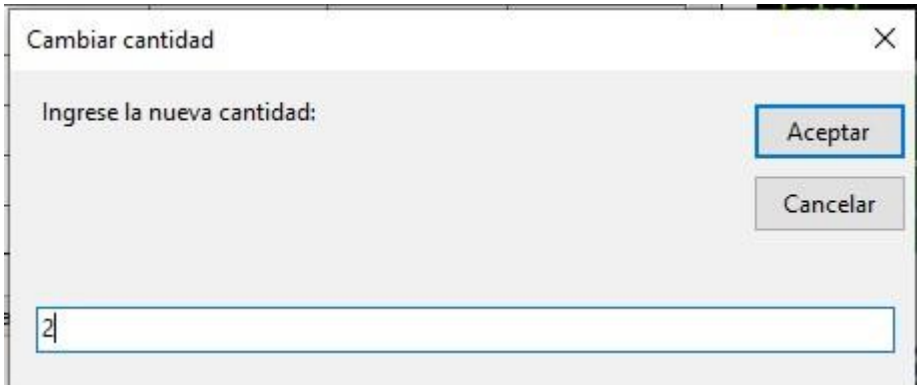
4.5 Estructura principal de un caso de uso de negocio del software con interfaces y código: Venta




Buscar Articulo
Buscar por código o descripción
a
Buscar
Ocultar discontinuo
Código
Descripción
Rubro
Precio
Stock
7791000000018
Alfajor de maicen...
Almacén
16800,00
18
777777
as
Almacén
3500,00
14
77980
bizcocho de grasa
Almacén
32000,00
24
7791000000045
Mayonesa clásica ...
Almacén
3000,00
6
Aceptar
Salir


Presentación
Artículos
Promociones
Ventas
Compras
Proveedores
Listados
Informes
Caja
Idioma
Administracion
Escaneá un código o escribí parte del SKU o del nombre.
Agregar la cantidad.
1 si lo dejás vacío
SKU
Código
Descripción
Cantidad
Precio Unitario
Importe
9018
7791000000018
Alfajor de maicena relleno ...
1
16800,00
16800,00
1002
77980
bizcocho de grasa
1
32000,00
32000,00
Subtotal
$ 48.800,00
Descuento/Recargo
$ 0,00
Total
$ 48.800,00
F2-Cobrar
F4-Nueva Compra
Promociónes
Quitar
Cambiar cantidad
KIOSCO
Gestorix
SISTEMA DE GESTIÓN PARA MAXIKIOSCOS
Usuario :
admin
Rol :
Administrador
Fecha: 04/11/2026
Hora: 18:18:37





Capa Vista // frmmenuPres

```
using CapaEntidades;
using CapaLogica;
using CapaLogica.NegocioLogica;
using DocumentFormat.OpenXml.Office.Word;
using DocumentFormat.OpenXml.Spreadsheet;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
```

```
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using static CapaEntidades.NegocioEntidad.ClsGestorEntidadProducto;
using CapaEntidades.NegocioEntidad;
using CapaEntidades.NegocioEntidades;

namespace CapaVista
{
    public partial class frmmenuPres : Form
    {

        private bool idiomaIngles => ClsIdioma.Ingles;
        private ClsCampanaAlertas campanaAlertas;
        private decimal subtotal = 0.00m;
        private decimal descuentoRecargo = 0.00m;
        private decimal total;
        private ClsGestorProductoLogica logicaProducto = new ClsGestorProductoLogica();
        private DataTable tablaProductosEncontrados;
        private readonly HashSet<DataGridViewRow> filasDePromocion = new HashSet<DataGridViewRow>();

        private frmCobroSecundario formularioSecundario;

        private readonly List<ClsPromocionVentaEntidad> promocionesAplicadas = new List<ClsPromocionVentaEntidad>();


        public frmmenuPres()
        {
            InitializeComponent();
            ClsIdioma.Registrar(this);

            MinimumSize = new Size(900, 620);
            StartPosition = FormStartPosition.CenterScreen;
            KeyPreview = true;

            ActualizarFechaHora();
            tmrFechaHora.Start();

            dtgPresentacion.ReadOnly = true;
            dtgPresentacion.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
            dtgPresentacion.MultiSelect = false;
            dtgPresentacion.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;
            dtgPresentacion.AllowUserToResizeColumns = true;
            dtgPresentacion.AllowUserToResizeRows = false;

            foreach (DataGridViewColumn columna in dtgPresentacion.Columns)
            {
                columna.Resizable = DataGridViewTriState.True;
                columna.MinimumWidth = 50;
            }
            dtgPresentacion.AllowUserToOrderColumns = false;
            dtgPresentacion.RowHeadersVisible = false;

            Load += frmmenuPres_Load;
            Load += FrmPresentacion_Load;

            Resize += (sender, e) => AjustarDiseno();
            Shown += (sender, e) => AjustarDiseno();
        }
```

```
private void frmmenuPres_Load(object sender, EventArgs e)
{
    txtUsuario.Text = SesionUsuario.Username;

    CapaLogica.ClsRol rol = new CapaLogica.ClsRol();

    txtRol.Text = rol.ObtenerNombreRol(SesionUsuario.IdRol);

    txtCantidad.Text = "1";

    this.KeyPreview = true;

    CargarBotones();
    AplicarModoPantalla();

    try
    {
        if (SesionUsuario.Permisos.Contains(730) && !new ClsCajaLogica().HayCajaAbierta())
        {
            DialogResult abrir = MessageBox.Show(ClsIdioma.T("La caja está cerrada.\n\n¿Desea abrirla ahora?"),
                ClsIdioma.T("Caja"), MessageBoxButtons.YesNo, MessageBoxIcon.Question);

            if (abrir == DialogResult.Yes)
            {
                using (frmAbrirCaja apertura = new frmAbrirCaja())
                {
                    apertura.ShowDialog();
                }
            }
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show(ClsIdioma.T("No se pudo verificar la caja.\n\n") + ex.Message, ClsIdioma.T("Error"),
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }

    IniciarAlertas();

    RellenarFilas();
    CalcularYMostrarMontos();

    if (ClsIdioma.Ingles)
    {
        AplicarIdioma();
    }
}

private bool modoBienvenida = false;
private Label? lblBienvenida;
private Label? lblBienvenidaAyuda;

private System.Windows.Forms.Control[] ControlesDeVenta()
{
    return new System.Windows.Forms.Control[]
    {
        label3, txtCodigoBarra, label4, txtCantidad, lstDescripcion, lblDescripcionSku,
        dtgPresentacion, btnPromocion, btnQuitar, btnCambiarcantidad,
        tableLayoutPanel1, btnCobrar, btnNuevaCompra
    };
```

```
}
private void AplicarModoPantalla()
{
    modoBienvenida = !SesionUsuario.Permisos.Contains(210);

    foreach (System.Windows.Forms.Control control in ControlesDeVenta())
    {
        control.Visible = !modoBienvenida;
    }

    lstDescripcion.Visible = false;

    if (!modoBienvenida)
    {
        if (lblBienvenida != null) lblBienvenida.Visible = false;
        if (lblBienvenidaAyuda != null) lblBienvenidaAyuda.Visible = false;

        AjustarDiseno();
        return;
    }

    string nombre = string.IsNullOrWhiteSpace(SesionUsuario.Nombre) ? SesionUsuario.Username : SesionUsuario.Nombre;

    if (lblBienvenida == null)
    {
        lblBienvenida = new Label
        {
            AutoSize = false,
            TextAlign = ContentAlignment.MiddleCenter,
            Font = new System.Drawing.Font("Segoe UI", 18F, System.Drawing.FontStyle.Bold),
            Anchor = AnchorStyles.None
        };

        lblBienvenidaAyuda = new Label
        {
            AutoSize = false,
            TextAlign = ContentAlignment.TopCenter,
            Font = new System.Drawing.Font("Segoe UI", 10.5F),
            ForeColor = SystemColors.GrayText,
            Anchor = AnchorStyles.None
        };

        Controls.Add(lblBienvenida);
        Controls.Add(lblBienvenidaAyuda);
    }

    lblBienvenida.Text = ClsIdioma.Ingles
        ? "Hello, " + nombre + "! Welcome to Gestorix"
        : "¡Hola, " + nombre + "! Te damos la bienvenida a Gestorix";
    lblBienvenidaAyuda!.Text = ClsIdioma.Ingles
        ? "Use the menu above to work with the options of your role."
        : "Usá el menú de arriba para trabajar con las opciones de tu rol.";
    lblBienvenida.Visible = true;
    lblBienvenidaAyuda.Visible = true;

    AjustarDiseno();
}
private void UbicarBienvenida(int ancho, int altoMenu, int abajo)
{
    int margen = 12;
    int disponibleAlto = Math.Max(200, abajo - altoMenu);
```

```
    int anchoLogo = Math.Min(520, (int)(ancho * 0.5));
    int altoLogo = anchoLogo * 169 / 270;

    int altoTextos = 40 + 6 + 30;

    if (altoLogo + altoTextos + 20 > disponibleAlto)
    {
        altoLogo = Math.Max(80, disponibleAlto - altoTextos - 20);
        anchoLogo = altoLogo * 270 / 169;
    }

    int yLogo = altoMenu + (disponibleAlto - altoLogo - altoTextos) / 2;

    pcbLogo.SetBounds((ancho - anchoLogo) / 2, yLogo, anchoLogo, altoLogo);
    pcbLogo.Visible = true;

    lblBienvenida?.SetBounds(margen, pcbLogo.Bottom + 10, ancho - margen * 2, 40);
    lblBienvenidaAyuda?.SetBounds(margen, pcbLogo.Bottom + 56, ancho - margen * 2, 30);

    lblBienvenida?.BringToFront();
    lblBienvenidaAyuda?.BringToFront();
}

private void IniciarAlertas()
{
    if (!SesionUsuario.Permisos.Contains(1110))
    {
        return;
    }

    campanaAlertas = new ClsCampanaAlertas(this, btnConfig, toolClosed, CrearAccionesAlertas());

    campanaAlertas.VerTodas = AbrirAlertas;
    campanaAlertas.PuedeResolver = PuedeResolverAlerta;

    campanaAlertas.Iniciar(mostrarAviso: true);
}
private bool PuedeResolverAlerta(TipoAlerta tipo)
{
    switch (tipo)
    {
        case TipoAlerta.Lote:
        case TipoAlerta.Precio:
            return TieneAlguno(110, 111, 112, 113);        // Modificar articulo
        case TipoAlerta.Stock:
            return TieneAlguno(520, 110, 111, 112, 113);    // Reposicion o Modificar articulo
        case TipoAlerta.Entrega:
            return SesionUsuario.Permisos.Contains(320);     // Recepcion de mercaderia
        case TipoAlerta.Caja:
            return SesionUsuario.Permisos.Contains(740);     // Cerrar caja
        case TipoAlerta.Pago:
            return SesionUsuario.Permisos.Contains(420);     // Cuenta corriente de proveedores
        case TipoAlerta.Promocion:
            return TieneAlguno(230, 240);                  // Administrar promociones
        default:
            return false;
    }
}
private Dictionary<TipoAlerta, Action> CrearAccionesAlertas()
{
```

```
    return new Dictionary<TipoAlerta, Action>
    {
        [TipoAlerta.Lote] = () => menuitemAdmistradorarticulos.PerformClick(),
        [TipoAlerta.Stock] = AbrirReposicion,
        [TipoAlerta.Entrega] = () => recepciónDeMercaderiaToolStripMenuItem.PerformClick(),
        [TipoAlerta.Caja] = () => cerrarSeToolStripMenuItem.PerformClick(),
        [TipoAlerta.Pago] = () => cuentaCorrienteDeProveedoresToolStripMenuItem.PerformClick(),
        [TipoAlerta.Promocion] = () => administrarPromocionesToolStripMenuItem.PerformClick(),
        [TipoAlerta.Precio] = () => menuitemAdmistradorarticulos.PerformClick()
    };
}
private const int ID_ROL_ADMINISTRADOR = 1;

private static bool EsAdministrador
{
    get { return SesionUsuario.IdRol == ID_ROL_ADMINISTRADOR; }
}
private void AbrirAlertas()
{
    if (!SesionUsuario.Permisos.Contains(1110))
    {
        MessageBox.Show(ClsIdioma.T("No tenés permiso para ver las alertas."), ClsIdioma.T("Alertas"),
            MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    using (frmAlertas alertas = new frmAlertas(CrearAccionesAlertas(), PuedeResolverAlerta, () => campanaAlertas?.Actualizar()))
    {
        alertas.Icon = Icon;
        alertas.ShowDialog(this);
    }

    campanaAlertas?.Actualizar();
}
private void AbrirReposicion()
{
    if (!SesionUsuario.Permisos.Contains(520))
    {
        menuitemAdmistradorarticulos.PerformClick();
        return;
    }

    using (frmListados listados = new frmListados(frmListados.Modo.Reposicion))
    {
        listados.ShowDialog();
    }
}

private void AplicarIdioma()
{
    // menu principal
    btnArt.Text = idiomaIngles ? "Products" : ClsIdioma.T("Artículos");
    btnVentas.Text = idiomaIngles ? "Sales" : ClsIdioma.T("Ventas");
    btnCompras.Text = idiomaIngles ? "Purchases" : ClsIdioma.T("Compras");
    btnProve.Text = idiomaIngles ? "Suppliers" : ClsIdioma.T("Proveedores");
    toolStripMenuItem1.Text = idiomaIngles ? "Lists" : ClsIdioma.T("Listados");
    btnInformes.Text = idiomaIngles ? "Reports" : ClsIdioma.T("Informes");
    btnCaja.Text = idiomaIngles ? "Cash Register" : ClsIdioma.T("Caja");
    btnIdioma.Text = idiomaIngles ? "Language" : ClsIdioma.T("Idioma");
    btnEmpleado.Text = idiomaIngles ? "Employees" : ClsIdioma.T("Empleado");
    toolStripMenuItem2.Text = idiomaIngles ? "Promotions" : ClsIdioma.T("Promociones");
```

```
        // articulos
        cargarArticuloNuevoToolStripMenuItem.Text = idiomaIngles ? "Add New Product" : ClsIdioma.T("Cargar artículo nuevo");
        menuitemAdmistradorarticulos.Text = idiomaIngles ? "Edit Product" : ClsIdioma.T("Modificar artículo");
        menuitemAdministradoarticulos.Text = idiomaIngles ? "Categories and Brands" : ClsIdioma.T("Categorías y marcas");
        meniitemReporapida.Text = idiomaIngles ? "Quick Stock Replenishment" : ClsIdioma.T("Reposición rápida de stock");
        menuitemImporexcel.Text = idiomaIngles ? "Import Products from Excel" : ClsIdioma.T("Importar artículos desde Excel");
        menuitemExporexcel.Text = idiomaIngles ? "Export Products to Excel" : ClsIdioma.T("Exportar artículos a Excel");

        // ventas
        menuitemAdministradorventas.Text = idiomaIngles ? "Sales Management" : ClsIdioma.T("Administrador de ventas");
        notaDeCreditoToolStripMenuItem.Text = idiomaIngles ? "Returns" : ClsIdioma.T("Devolución");

        // compras
        menuitemAministradorcompras.Text = idiomaIngles ? "Purchase Management" : ClsIdioma.T("Administrador de compras");

        // proveedores
        administradorDeProveedoresToolStripMenuItem.Text = idiomaIngles ? "Supplier Management" : ClsIdioma.T("Administrador de
proveedores");
        cuentaCorrienteDeProveedoresToolStripMenuItem.Text = idiomaIngles ? "Supplier Account" : ClsIdioma.T("Cuenta corriente de
proveedores");

        // listados
        preciosYReposicionToolStripMenuItem.Text = idiomaIngles ? "Prices and Restocking" : ClsIdioma.T("Precios y reposición");

        // informes
        informesVentasToolStripMenuItem.Text = idiomaIngles ? "Sales" : ClsIdioma.T("Ventas");

        // caja
        nuevoIngresoToolStripMenuItem.Text = idiomaIngles ? "New Deposit" : ClsIdioma.T("Nuevo ingreso");
        nuevoEgresoToolStripMenuItem.Text = idiomaIngles ? "New Withdrawal" : ClsIdioma.T("Nuevo egreso");
        cerrarSeToolStripMenuItem.Text = idiomaIngles ? "Close Cash Register" : ClsIdioma.T("Cerrar caja");
        abrirCajaToolStripMenuItem.Text = idiomaIngles ? "Open Cash Register" : ClsIdioma.T("Abrir caja");

        // idioma
        idiomaToolStripMenuItem.Text = idiomaIngles ? "Language" : ClsIdioma.T("Idioma");
        inglesToolStripMenuItem.Text = "English";

        // empleados
        permisosToolStripMenuItem.Text = idiomaIngles ? "Employees" : ClsIdioma.T("Empleado");
        gestioToolStripMenuItem.Text = idiomaIngles ? "Backup Management" : ClsIdioma.T("Gestión de Backup");
        verificarIntegridadToolStripMenuItem.Text = idiomaIngles ? "Check Integrity" : ClsIdioma.T("Verificar integridad");
        bitacoraToolStripMenuItem.Text = idiomaIngles ? "Activity Log" : ClsIdioma.T("Bitácora");

        // promociones
        administrarPromocionesToolStripMenuItem.Text = idiomaIngles ? "Manage Promotions" : ClsIdioma.T("Administrar promociones");
        btnPromocion.Text = idiomaIngles ? "Promotions" : ClsIdioma.T("Promociones");

        // botones de vernta
        btnCobrar.Text = idiomaIngles ? "F2 - Pay" : ClsIdioma.T("F2 - Cobrar");
        btnNuevaCompra.Text = idiomaIngles ? "F4 - New Purchase" : ClsIdioma.T("F4 - Nueva Compra");
        btnCambiarcantidad.Text = idiomaIngles ? "Change Quantity" : ClsIdioma.T("Cambiar cantidad");
        btnQuitar.Text = idiomaIngles ? "Remove" : ClsIdioma.T("Quitar");

        // etiqueta de productos
        label3.Text = idiomaIngles ? "Scan a code or type part of the SKU or name." : ClsIdioma.T("Escaneá un código o escribí parte del
SKU o del nombre."); label4.Text = idiomaIngles ? "Quantity" : ClsIdioma.T("Cantidad");

        // totales
        label1.Text = idiomaIngles ? ClsIdioma.T("Subtotal") : ClsIdioma.T("Subtotal");
        label2.Text = idiomaIngles ? "Discount/Surcharge" : ClsIdioma.T("Descuento/Recargo");
```

```
    label5.Text = idiomaIngles ? ClsIdioma.T("Total") : ClsIdioma.T("Total");

    // datos del usuario
    label6.Text = idiomaIngles ? "User:" : ClsIdioma.T("Usuario:");
    label7.Text = idiomaIngles ? "Role:" : ClsIdioma.T("Rol:");
    ActualizarFechaHora();

    // columna de las tablas
    ClnSKU.HeaderText = "SKU";
    ClnIdProducto.HeaderText = idiomaIngles ? "Product ID" : "IdProducto";
    ClnCodigo.HeaderText = idiomaIngles ? "Code" : ClsIdioma.T("Código");
    ClnDescripcion.HeaderText = idiomaIngles ? "Description" : ClsIdioma.T("Descripción");
    ClnCantidad.HeaderText = idiomaIngles ? "Quantity" : ClsIdioma.T("Cantidad");
    ClnPrecioUnitario.HeaderText = idiomaIngles ? "Unit Price" : ClsIdioma.T("Precio Unitario");
    ClnImporte.HeaderText = idiomaIngles ? "Amount" : ClsIdioma.T("Importe");

    // Tabla de importes
    label1.Text = idiomaIngles ? ClsIdioma.T("Subtotal") : ClsIdioma.T("Subtotal");
    label2.Text = idiomaIngles ? "Discount/Surcharge" : ClsIdioma.T("Descuento/Recargo");
    label5.Text = idiomaIngles ? ClsIdioma.T("Total") : ClsIdioma.T("Total");

    // fromulario
    this.Text = idiomaIngles ? "Sales Terminal" : ClsIdioma.T("Presentación");

    ActualizarFechaHora();

    if (modoBienvenida)
    {
        AplicarModoPantalla();
    }
}

#region Metodo

private void AjustarDiseno()
{
    if (WindowState == FormWindowState.Minimized || !IsHandleCreated)
        return;

    SuspendLayout();

    try
    {
        int ancho = ClientSize.Width;
        int alto = ClientSize.Height;
        int margen = 12;
        int altoMenu = btnConfig.Bottom;
        int altoPie = pnlUsuario.Height;
        int espacioInferior = 10;

        pnlUsuario.SetBounds(0, alto - altoPie, ancho, altoPie);

        if (modoBienvenida)
        {
            UbicarBienvenida(ancho, altoMenu, pnlUsuario.Top);
            UbicarFechaHora();
            return;
        }

        int anchoDerecha = Math.Clamp((int)(ancho * 0.32), 260, 360);
        int xDerecha = ancho - margen - anchoDerecha;
```

```
        int anchoIzquierda = xDerecha - margen * 2;
        int anchoCantidad = Math.Clamp((int)(anchoIzquierda * 0.25), 95, 140);
        int separacion = 12;
        int anchoCodigo = Math.Max(150, anchoIzquierda - anchoCantidad - separacion);

        label3.SetBounds(margen, altoMenu + 11, anchoCodigo, 18);
        txtCodigoBarra.SetBounds(margen, altoMenu + 30, anchoCodigo, 25);
        label4.SetBounds(margen + anchoCodigo + separacion, altoMenu + 11, anchoCantidad, 18);
        txtCantidad.SetBounds(margen + anchoCodigo + separacion, altoMenu + 30, anchoCantidad, 25);

        int yGrilla = altoMenu + 70;
        int yBotones = alto - altoPie - 94;
        int altoGrilla = Math.Max(150, yBotones - yGrilla - 6);

        dtgPresentacion.SetBounds(margen, yGrilla, anchoIzquierda, altoGrilla);
        yBotones = dtgPresentacion.Bottom + 6;
        btnPromocion.SetBounds(margen, yBotones, 207, 74);
        btnQuitar.SetBounds(Math.Max(margen + 215, xDerecha - 208), yBotones, 75, 28);
        btnCambiarcantidad.SetBounds(Math.Max(margen + 295, xDerecha - 127), yBotones, 115, 28);
        tableLayoutPanel1.SetBounds(xDerecha, altoMenu + 30, anchoDerecha, 145);
        btnCobrar.SetBounds(xDerecha, tableLayoutPanel1.Bottom + 6, anchoDerecha, 45);
        btnNuevaCompra.SetBounds(xDerecha, btnCobrar.Bottom + 6, anchoDerecha, 45);

        int yLogo = btnNuevaCompra.Bottom + 8;
        int altoLogo = Math.Max(60, pnlUsuario.Top - yLogo - espacioInferior);

        pcbLogo.SetBounds(xDerecha, yLogo, anchoDerecha, altoLogo);

        lstDescripcion.Left = txtCodigoBarra.Left;
        lstDescripcion.Width = txtCodigoBarra.Width;
        lstDescripcion.Height = 94;

        if (lstDescripcion.Visible)
        {
            lstDescripcion.BringToFront();
        }

        UbicarFechaHora();
    }
    finally
    {
        ResumeLayout(true);
    }
}
private static bool TieneAlguno(params int[] permisos)
{
    return permisos.Any(SesionUsuario.Permisos.Contains);
}
private void CargarBotones()
{
    // Articulo
    btnArt.Enabled = TieneAlguno(100, 110, 111, 112, 113, 120, 121, 122, 130, 131, 132, 140, 150);

    // Promociones
    toolStripMenuItem2.Enabled = TieneAlguno(230, 240);

    // Ventas
    btnVentas.Enabled = TieneAlguno(200, 210, 220, 221, 222, 250);

    // Compras
    btnCompras.Enabled = TieneAlguno(300, 310, 320, 330);
```

```
// Proveedores
btnProve.Enabled = TieneAlguno(400, 410, 411, 412, 420);

// Listados
toolStripMenuItem1.Enabled = TieneAlguno(500, 510, 520);

// Informes
btnInformes.Enabled = TieneAlguno(600, 610, 620, 630, 640, 650, 660);

// Caja
btnCaja.Enabled = TieneAlguno(700, 710, 720, 730, 740, 750, 760);

// Ajustes
btnIdioma.Enabled = SesionUsuario.Permisos.Contains(800);

btnEmpleado.Enabled = TieneAlguno(1000, 1010, 1020, 1270, 1280, 1290);

// Listados
preciosYReposicionToolStripMenuItem.Enabled = TieneAlguno(510, 520);

// Articulos operaciones

cargarArticuloNuevoToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(111);
// Gestion de Articulos
menuitemAdmistradorarticulos.Enabled = TieneAlguno(110, 111, 112, 113);
menuitemAdministradoarticulos.Enabled = TieneAlguno(120, 121, 122, 130, 131, 132);
// Reposicion rapida de stock
meniitemReporapida.Enabled = SesionUsuario.Permisos.Contains(113);
// Importar articulos desde Excel
menuitemImporexcel.Enabled = SesionUsuario.Permisos.Contains(140);
// Exportar articulos a Excel
menuitemExporexcel.Enabled = SesionUsuario.Permisos.Contains(150);

// Promociones operaciones
administrarPromocionesToolStripMenuItem.Enabled = TieneAlguno(230, 240);

// Ventas

menuitemAdministradorventas.Enabled = TieneAlguno(210, 250);
// Devoluciones
notaDeCreditoToolStripMenuItem.Enabled = TieneAlguno(220, 221, 222);

bool puedeVender = SesionUsuario.Permisos.Contains(210);
txtCodigoBarra.Enabled = puedeVender;
txtCantidad.Enabled = puedeVender;
btnCambiarcantidad.Enabled = puedeVender;
btnQuitar.Enabled = puedeVender;
btnCobrar.Enabled = puedeVender;
btnNuevaCompra.Enabled = puedeVender;
btnPromocion.Enabled = puedeVender;

// Compras operaciones
menuitemAministradorcompras.Enabled = TieneAlguno(310, 330);
recepciónDeMercaderiaToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(320);

// Proveedores

// Gestion de proveedores
administradorDeProveedoresToolStripMenuItem.Enabled = TieneAlguno(410, 411, 412);
// Cuenta corriente
```

```
            cuentaCorrienteDeProveedoresToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(420);
            informesVentasToolStripMenuItem.Enabled = TieneAlguno(610, 620, 630, 640, 650);
            balanceToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(660);
            // Caja operaciones
            nuevoIngresoToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(710);
            nuevoEgresoToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(720);
            abrirCajaToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(730);
            cerrarSeToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(740);
            estadoCajaToolStripMenuItem.Enabled = TieneAlguno(750, 760);

            // Administracion operaciones
            permisosToolStripMenuItem.Enabled = TieneAlguno(1010, 1020, 1270);
            gestioToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(1280);
            verificarIntegridadToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(ClsIntegridadLogica.PermisoIntegridad);
            bitacoraToolStripMenuItem.Enabled = SesionUsuario.Permisos.Contains(1290);        }
        // Ingreso de plata
        public enum TipoMovimiento
        {
            Ingreso,
            Egreso
        }
        // bloqueo menu
        private void BloquearMenu()
        {
            // Menu superior
            btnArt.Enabled = false;
            btnVentas.Enabled = false;
            btnCompras.Enabled = false;
            btnProve.Enabled = false;
            toolStripMenuItem1.Enabled = false;
            btnInformes.Enabled = false;
            btnIdioma.Enabled = false;
            btnEmpleado.Enabled = false;

            // Caja queda disponible
            btnCaja.Enabled = true;

            nuevoIngresoToolStripMenuItem.Enabled = false;
            nuevoEgresoToolStripMenuItem.Enabled = false;
            cerrarSeToolStripMenuItem.Enabled = false;
            estadoCajaToolStripMenuItem.Enabled = false;
            abrirCajaToolStripMenuItem.Enabled = true;

            // Botones del formulario
            btnCambiarcantidad.Enabled = false;
            btnQuitar.Enabled = false;
            btnCobrar.Enabled = false;
            btnNuevaCompra.Enabled = false;
            btnPromocion.Enabled = false;

            // Entrada de datos
            txtCodigoBarra.Enabled = false;
            txtCantidad.Enabled = false;
            dtgPresentacion.Enabled = false;

            txtUsuario.Text = "";
            txtRol.Text = "";
        }
        private bool AgregarProductoAGrilla(int idProducto, string sku, string codigoBarra, string descripcion, decimal precio, decimal cantidad,
decimal stockDisponible)
        {
```

```
        if (cantidad <= 0)
        {
            MessageBox.Show(ClsIdioma.T("La cantidad debe ser mayor a cero."), ClsIdioma.T("Cantidad inválida"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return false;
        }

        if (precio < 0)
        {
            MessageBox.Show(ClsIdioma.T("El precio no puede ser negativo."), ClsIdioma.T("Precio inválido"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return false;
        }

        // Precio 0 = artículo cargado sin precio: no se vende hasta que se le cargue uno.
        if (precio == 0)
        {
            MessageBox.Show(ClsIdioma.T("El artículo no tiene precio de venta. Cárguelo en Artículos antes de venderlo."),
ClsIdioma.T("Precio inválido"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return false;
        }

        decimal cantidadYaCargada = 0m;

        foreach (DataGridViewRow fila in dtgPresentacion.Rows)
        {
            if (fila.IsNewRow || fila.Cells["ClnIdProducto"].Value == null)
                continue;

            if (int.TryParse(Convert.ToString(fila.Cells["ClnIdProducto"].Value), out int idCargado) && idCargado == idProducto)
            {
                if (decimal.TryParse(Convert.ToString(fila.Cells["ClnCantidad"].Value), out decimal cantidadCargada))
                {
                    cantidadYaCargada += cantidadCargada;
                }
            }
        }

        decimal cantidadEnGrillaSecundaria = 0m;

        if (formularioSecundario != null && !formularioSecundario.IsDisposed)
        {
            cantidadEnGrillaSecundaria = formularioSecundario.ObtenerCantidadCargada(idProducto);
        }

        decimal cantidadTotal = cantidadYaCargada + cantidadEnGrillaSecundaria + cantidad;

        if (stockDisponible <= 0 || cantidadTotal > stockDisponible)
        {
            decimal stockRestante = Math.Max(0m, stockDisponible - cantidadYaCargada - cantidadEnGrillaSecundaria);

            MessageBox.Show((ClsIdioma.Ingles ? $"Insufficient stock.\n" : $"Stock insuficiente.\n") + (ClsIdioma.Ingles ? $"Stock in system:
{stockDisponible}\n" : $"Stock en sistema: {stockDisponible}\n") + (ClsIdioma.Ingles ? $"Quantity in main sale: {cantidadYaCargada}\n" :
$"Cantidad en venta principal: {cantidadYaCargada}\n") +
                (ClsIdioma.Ingles ? $"Quantity in secondary sale: {cantidadEnGrillaSecundaria}\n" : $"Cantidad en venta secundaria:
{cantidadEnGrillaSecundaria}\n") + (ClsIdioma.Ingles ? $"Stock available to add: {stockRestante}\n" : $"Stock disponible para agregar:
{stockRestante}\n") + (ClsIdioma.Ingles ? $"Quantity to add: {cantidad}" : $"Cantidad a agregar: {cantidad}"),
                ClsIdioma.T("Stock insuficiente"), MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return false;
```

```
}


if (codigoBarra != "PROMOCION")
{
    foreach (DataGridViewRow fila in dtgPresentacion.Rows)
    {
        if (fila.IsNewRow || Convert.ToString(fila.Cells["ClnCodigo"].Value) == "PROMOCION")
            continue;

        if (int.TryParse(Convert.ToString(fila.Cells["ClnIdProducto"].Value), out int idFila) && idFila == idProducto)
        {
            decimal.TryParse(Convert.ToString(fila.Cells["ClnCantidad"].Value), out decimal cantidadFila);
            decimal.TryParse(Convert.ToString(fila.Cells["ClnPrecioUnitario"].Value), out decimal precioFila);

            decimal nuevaCantidad = cantidadFila + cantidad;

            fila.Cells["ClnCantidad"].Value = nuevaCantidad;
            fila.Cells["ClnImporte"].Value = Math.Round(nuevaCantidad * precioFila, 2);

            dtgPresentacion.ClearSelection();
            dtgPresentacion.CurrentCell = fila.Cells["ClnDescripcion"];
            fila.Selected = true;

            CalcularYMostrarMontos();
            dtgPresentacion.Refresh();

            return true;
        }
    }
}

DataGridViewRow filaDestino = null;

foreach (DataGridViewRow fila in dtgPresentacion.Rows)
{
    if (fila.IsNewRow)
        continue;

    if (fila.Cells["ClnIdProducto"].Value == null || string.IsNullOrWhiteSpace(Convert.ToString(fila.Cells["ClnIdProducto"].Value)))
    {
        filaDestino = fila;
        break;
    }
}

if (filaDestino == null)
{
    int indice = dtgPresentacion.Rows.Add();
    filaDestino = dtgPresentacion.Rows[indice];
}

filaDestino.Cells["ClnIdProducto"].Value = idProducto;
filaDestino.Cells["ClnSKU"].Value = sku;
filaDestino.Cells["ClnCodigo"].Value = codigoBarra;
filaDestino.Cells["ClnDescripcion"].Value = descripcion;
filaDestino.Cells["ClnCantidad"].Value = cantidad;
filaDestino.Cells["ClnPrecioUnitario"].Value = precio;
filaDestino.Cells["ClnImporte"].Value = Math.Round(cantidad * precio, 2);

CalcularYMostrarMontos();
```

```
    dtgPresentacion.Refresh();

    return true;
}
private void IngresarProductoPorCodigo()
{
    string codigoBarra = txtCodigoBarra.Text.Trim();

    if (string.IsNullOrWhiteSpace(codigoBarra))
    {
        txtCodigoBarra.Focus();
        return;
    }

    decimal cantidad = 1;

    if (!string.IsNullOrWhiteSpace(txtCantidad.Text))
    {
        if (!decimal.TryParse(txtCantidad.Text, out cantidad) || cantidad <= 0)
        {
            MessageBox.Show(ClsIdioma.T("Ingrese una cantidad válida mayor a 0."), ClsIdioma.T("Cantidad inválida"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            txtCantidad.SelectAll();
            txtCantidad.Focus();
            return;
        }
    }

    ClsGestorProductoLogica logicaProducto = new ClsGestorProductoLogica();

    DataTable tablaProducto = logicaProducto.ObtenerProductoPorCodigoBarra(codigoBarra);

    if (tablaProducto.Rows.Count == 0)
    {
        MessageBox.Show(ClsIdioma.T("No se encontró un producto con ese código de barras."), ClsIdioma.T("Producto no encontrado"),
            MessageBoxButtons.OK, MessageBoxIcon.Warning);

        txtCodigoBarra.SelectAll();
        txtCodigoBarra.Focus();
        return;
    }

    DataRow producto = tablaProducto.Rows[0];

    int idProducto = Convert.ToInt32(producto["IdProducto"]);
    string sku = producto["SKU"].ToString();
    string codigo = producto["CodigoBarra"].ToString();
    string descripcion = producto["Descripcion"].ToString();
    decimal precio = Convert.ToDecimal(producto["PrecioVenta"]);

    decimal stockDisponible = producto["CantidadActual"] == DBNull.Value ? 0 : Convert.ToDecimal(producto["CantidadActual"]);

    AgregarProductoAGrilla(idProducto, sku, codigo, descripcion, precio, cantidad, stockDisponible);

    txtCodigoBarra.Clear();
    txtCantidad.Clear();

    txtCodigoBarra.Focus();
}
private void tmrFechaHora_Tick(object sender, EventArgs e)
```

```
{
    ActualizarFechaHora();
}
private void ActualizarFechaHora()
{
    DateTime ahora = DateTime.Now;

    lblFecha.Text = (idiomaIngles ? "Date: " : ClsIdioma.T("Fecha: ")) + ahora.ToString("dd/MM/yyyy");
    lblHora.Text = (idiomaIngles ? "Time: " : ClsIdioma.T("Hora: ")) + ahora.ToString("HH:mm:ss");

    UbicarFechaHora();
}
private void UbicarFechaHora()
{
    int margen = 12;

    lblHora.Left = ClientSize.Width - lblHora.Width - margen;
    lblFecha.Left = lblHora.Left - lblFecha.Width - 20;
}
private void LimpiarVenta()
{
    dtgPresentacion.Rows.Clear();

    subtotal = 0.00m;
    descuentoRecargo = 0.00m;
    total = 0.00m;
    promocionesAplicadas.Clear();

    CalcularYMostrarMontos();
    RellenarFilas();

    txtCodigoBarra.Clear();
    txtCodigoBarra.Focus();
}
private void RellenarFilas()
{
    int filasVisibles = dtgPresentacion.Height / dtgPresentacion.RowTemplate.Height;
    while (dtgPresentacion.Rows.Count < filasVisibles)
    {
        dtgPresentacion.Rows.Add();
    }
}
private void CalcularYMostrarMontos()
{
    decimal nuevoSubtotal = 0m;

    foreach (DataGridViewRow fila in dtgPresentacion.Rows)
    {
        if (fila.IsNewRow || fila.Cells["ClnIdProducto"].Value == null)
            continue;

        object valorImporte = fila.Cells["ClnImporte"].Value;

        if (valorImporte != null && valorImporte != DBNull.Value && decimal.TryParse(Convert.ToString(valorImporte), out decimal importe))
        {
            nuevoSubtotal += importe;
        }
    }

    subtotal = Math.Round(nuevoSubtotal, 2);
    total = Math.Max(0m, Math.Round(subtotal + descuentoRecargo, 2));
```

```
    lblSubtotalMonto.Text = subtotal.ToString("C2");
    lblDescuentoMonto.Text = Math.Abs(descuentoRecargo).ToString("C2");

    lblTotalMonto.Visible = true;
    lblTotalMonto.AutoSize = false;
    lblTotalMonto.Dock = System.Windows.Forms.DockStyle.Fill;
    lblTotalMonto.TextAlign = System.Drawing.ContentAlignment.MiddleRight;
    lblTotalMonto.Font = new System.Drawing.Font("Microsoft Sans Serif", 14F, System.Drawing.FontStyle.Bold);

    lblTotalMonto.Text = total.ToString("C2");
}
private void RegistrarPromocionAplicada(int idPromocion, string nombre, decimal importeLista, decimal importeCobrado)
{
    ClsPromocionVentaEntidad existente = promocionesAplicadas.FirstOrDefault(p => p.IdPromocion == idPromocion);

    if (existente == null)
    {
        promocionesAplicadas.Add(new ClsPromocionVentaEntidad
        {
            IdPromocion = idPromocion,
            Nombre = nombre,
            Veces = 1,
            ImporteLista = Math.Round(importeLista, 2),
            ImporteCobrado = Math.Round(importeCobrado, 2)
        });
    }
    else
    {
        existente.Veces += 1;
        existente.ImporteLista += Math.Round(importeLista, 2);
        existente.ImporteCobrado += Math.Round(importeCobrado, 2);
    }
}

internal static bool VerificarCajaAbierta()
{
    try
    {
        ClsCajaLogica logicaCaja = new ClsCajaLogica();

        if (logicaCaja.HayCajaAbierta())
        {
            return true;
        }

        if (!SesionUsuario.Permisos.Contains(730))
        {
            MessageBox.Show(ClsIdioma.T("No hay una caja abierta. Pida a un encargado que abra la caja para poder vender."),
                ClsIdioma.T("Caja cerrada"), MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return false;
        }

        DialogResult respuesta = MessageBox.Show(ClsIdioma.T("No hay una caja abierta.\n\n¿Desea abrirla ahora?"),
            ClsIdioma.T("Caja cerrada"), MessageBoxButtons.YesNo, MessageBoxIcon.Question);

        if (respuesta != DialogResult.Yes)
        {
            return false;
        }
```

```
            using (frmAbrirCaja abrir = new frmAbrirCaja())
            {
                return abrir.ShowDialog() == DialogResult.OK;
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show(ClsIdioma.T("No se pudo verificar la caja.\n\n") + ex.Message, ClsIdioma.T("Error"),
                MessageBoxButtons.OK, MessageBoxIcon.Error);
            return false;
        }
    }
    private void AgregarProductoSeleccionado()
    {
        if (lstDescripcion.SelectedIndex < 0 || tablaProductosEncontrados == null)
            return;

        decimal cantidad = 1;

        if (!string.IsNullOrWhiteSpace(txtCantidad.Text) &&
            (!decimal.TryParse(txtCantidad.Text.Trim(), out cantidad) || cantidad <= 0))
        {
            MessageBox.Show(ClsIdioma.T("Ingresá una cantidad válida mayor que cero."), ClsIdioma.T("Cantidad inválida"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            txtCantidad.Focus();
            txtCantidad.SelectAll();
            return;
        }

        int indice = lstDescripcion.SelectedIndex;

        if (indice >= tablaProductosEncontrados.Rows.Count)
            return;

        DataRow fila = tablaProductosEncontrados.Rows[indice];

        int idProducto = Convert.ToInt32(fila["IdProducto"]);
        string sku = fila["SKU"].ToString();
        string codigoBarra = fila["CodigoBarra"].ToString();
        string descripcion = fila["Descripcion"].ToString();
        decimal precio = Convert.ToDecimal(fila["PrecioVenta"]);

        ClsGestorProductoLogica logica = new ClsGestorProductoLogica();
        DataTable tablaStock = logica.ObtenerStockPorProducto(idProducto);

        decimal stockDisponible = 0m;

        if (tablaStock != null && tablaStock.Rows.Count > 0 && tablaStock.Rows[0]["CantidadActual"] != DBNull.Value)
        {
            stockDisponible = Convert.ToDecimal(tablaStock.Rows[0]["CantidadActual"]);
        }

        AgregarProductoAGrilla(idProducto, sku, codigoBarra, descripcion, precio, cantidad, stockDisponible);

        lstDescripcion.Visible = false;
        lstDescripcion.Items.Clear();
        txtCodigoBarra.Clear();
        txtCantidad.Clear();
        txtCodigoBarra.Focus();
    }
```

```
#endregion

#region Eventos

private void txtCodigoBarra_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        e.SuppressKeyPress = true;
        IngresarProductoPorCodigo();
    }
}
private void txtCodigoBarra_TextChanged(object sender, EventArgs e)
{
    string texto = txtCodigoBarra.Text.Trim();

    lstDescripcion.Items.Clear();
    tablaProductosEncontrados = null;

    if (string.IsNullOrWhiteSpace(texto))
    {
        lstDescripcion.Visible = false;
        return;
    }

    try
    {
        tablaProductosEncontrados = logicaProducto.BuscarArticulos(texto, true);

        if (tablaProductosEncontrados == null || tablaProductosEncontrados.Rows.Count == 0)
        {
            lstDescripcion.Visible = false;
            return;
        }

        foreach (DataRow fila in tablaProductosEncontrados.Rows)
        {
            string descripcion = fila["Descripcion"].ToString();

            lstDescripcion.Items.Add(descripcion);
        }

        lstDescripcion.Visible = lstDescripcion.Items.Count > 0;
    }
    catch (Exception ex)
    {
        lstDescripcion.Visible = false;

        MessageBox.Show(ClsIdioma.T("Error al buscar artículos: ") + ex.Message, ClsIdioma.T("Búsqueda de artículos"),
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
private void txtCantidad_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        e.SuppressKeyPress = true;
        IngresarProductoPorCodigo();
```

```
    }
}
private void lstDescripcion_DoubleClick(object sender, EventArgs e)
{
    AgregarProductoSeleccionado();
}
private void lstDescripcion_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        e.SuppressKeyPress = true;
        AgregarProductoSeleccionado();
    }
}
// Menu Item
private void frmmenuPres_KeyDown(object sender, KeyEventArgs e)
{
    if (modoBienvenida)
        return;

    if (e.KeyCode == Keys.F2)
    {
        e.SuppressKeyPress = true;
        btnCobrar.PerformClick();
    }
    else if (e.KeyCode == Keys.F4)
    {
        e.SuppressKeyPress = true;
        btnNuevaCompra.PerformClick();
    }
}
// Importar y Exportar
private void menuitemImporexcel_Click(object sender, EventArgs e)
{
    frmImportarArticulos frmImportar = new frmImportarArticulos();
    frmImportar.ShowDialog();
}
private void menuitemExporexcel_Click(object sender, EventArgs e)
{
    frmExportarArchivos exportararchivos = new frmExportarArchivos();
    exportararchivos.ShowDialog();
}
private void cuentaCorrienteDeProveedoresToolStripMenuItem_Click(object sender, EventArgs e)
{
    frmCuentaCorrienteProveedores cuentacorr = new frmCuentaCorrienteProveedores();
    cuentacorr.ShowDialog();
}
// Informes
private void informesVentasToolStripMenuItem_Click(object sender, EventArgs e)
{
    using (frmInformesVentas informes = new frmInformesVentas(frmInformesVentas.InformeInicialSegunPermisos()))
    {
        informes.ShowDialog();
    }
}
// Caja
private void nuevoIngresoToolStripMenuItem_Click(object sender, EventArgs e)
{
    frmIngresoPlataCaja ingresoPlata = new frmIngresoPlataCaja(TipoMovimiento.Ingreso);

    ingresoPlata.ShowDialog();
```

```
}
private void nuevoEgresoToolStripMenuItem_Click(object sender, EventArgs e)
{
    frmIngresoPlataCaja egresoPlata = new frmIngresoPlataCaja(TipoMovimiento.Egreso);

    egresoPlata.ShowDialog();
}
private void preciosYReposicionToolStripMenuItem_Click(object sender, EventArgs e)
{
    bool precios = SesionUsuario.Permisos.Contains(510);
    bool reposicion = SesionUsuario.Permisos.Contains(520);

    frmListados.Modo modo = precios && reposicion ? frmListados.Modo.Ninguno
                          : precios ? frmListados.Modo.Precios
                          : frmListados.Modo.Reposicion;

    using (frmListados listados = new frmListados(modo))
    {
        listados.ShowDialog();
    }
}
private void menuitemAministradorcompras_Click(object sender, EventArgs e)
{
    frmCompras compra = new frmCompras();
    compra.ShowDialog();
}
private void menuitemAdmistradorarticulos_Click(object sender, EventArgs e)
{
    frmCargaarticulo cargaarticulo = new frmCargaarticulo();
    cargaarticulo.ShowDialog();
}
private void meniitemReporapida_Click(object sender, EventArgs e)
{
    frmReporapida reporapida = new frmReporapida();
    reporapida.ShowDialog();
}
private void menuitemAdministradorventas_Click(object sender, EventArgs e)
{
    frmVentas ventas = new frmVentas();
    ventas.Show();

}
private void notaDeCreditoToolStripMenuItem_Click(object sender, EventArgs e)
{
    frmDevolucion devolucion = new frmDevolucion();
    devolucion.ShowDialog();
}
private void administradorDeProveedoresToolStripMenuItem_Click(object sender, EventArgs e)
{
    frmProveedores proveedores = new frmProveedores();
    proveedores.ShowDialog();
}
// Cerrar sesion
private void cerrarSeToolStripMenuItem_Click(object sender, EventArgs e)
{

    try
    {
        ClsCajaLogica logicaCaja = new ClsCajaLogica();
        var sesionCaja = logicaCaja.ObtenerSesionAbierta();
```

```
            if (sesionCaja != null)
            {
                DialogResult arqueo = MessageBox.Show(
                    ClsIdioma.T("¿Desea hacer el arqueo y cerrar la caja?\n\n") +
                    ClsIdioma.T("Sí: cuenta el efectivo y cierra la caja.\n") +
                    ClsIdioma.T("No: cierra solo su sesión y la caja sigue abierta para el próximo turno."),
                    ClsIdioma.T("Cerrar caja"), MessageBoxButtons.YesNoCancel, MessageBoxIcon.Question);

                if (arqueo == DialogResult.Cancel)
                {
                    return;
                }

                if (arqueo == DialogResult.Yes)
                {
                    using (frmCerrarCaja cierre = new frmCerrarCaja(sesionCaja.IdSesionCaja))
                    {

                        if (cierre.ShowDialog() != DialogResult.OK)
                        {
                            return;
                        }
                    }
                }
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show(ClsIdioma.T("No se pudo verificar la caja.\n\n") + ex.Message, ClsIdioma.T("Error"),
                MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }


        DialogResult resultado = MessageBox.Show(ClsIdioma.T("Estas seguro de cerrar la sesión?"), ClsIdioma.T("Cerrar sesión"),
MessageBoxButtons.YesNo, MessageBoxIcon.Question);

        if (resultado == DialogResult.Yes)
        {
            ClsLogin login = new ClsLogin();
            login.CerrarSesion();

            BloquearMenu();
        }
    }
    private void abrirCajaToolStripMenuItem_Click(object sender, EventArgs e)
    {
        if (SesionUsuario.SesionActiva)
        {
            MessageBox.Show(
                ClsIdioma.T("Ya hay una sesión abierta. Debe cerrar la sesión actual antes de iniciar otra."),
                ClsIdioma.T("Sesión abierta"),
                MessageBoxButtons.OK,
                MessageBoxIcon.Warning
            );

            return;
        }

        FrmLogin login = Application.OpenForms.OfType<FrmLogin>().FirstOrDefault();
```

```
            if (login != null)
            {
                login.LimpiarCampos();
            }

            this.Close();
        }
        private void tollClosed_Click(object sender, EventArgs e)
        {
            DialogResult resultado = MessageBox.Show(ClsIdioma.T("Estas seguro de cerrar la sesión?"), ClsIdioma.T("Cerrar sesión"),
MessageBoxButtons.YesNo, MessageBoxIcon.Question);
            if (resultado == DialogResult.Yes)
            {
                Application.Exit();

            }
        }
        private void permisosToolStripMenuItem_Click(object sender, EventArgs e)
        {
            frmCargaEmpleados Carga = new frmCargaEmpleados();
            Carga.ShowDialog();
        }
        private void btnNuevaCompra_Click(object sender, EventArgs e)
        {
            bool hayProductos = false;

            foreach (DataGridViewRow fila in dtgPresentacion.Rows)
            {
                if (fila.IsNewRow || fila.Cells["ClnIdProducto"].Value == null)
                    continue;

                if (!string.IsNullOrWhiteSpace(Convert.ToString(fila.Cells["ClnIdProducto"].Value)))
                {
                    hayProductos = true;
                    break;
                }
            }

            if (!hayProductos)
            {
                MessageBox.Show(ClsIdioma.T("Primero debe cargar al menos un producto en la venta principal."), ClsIdioma.T("Venta sin
productos"),
                    MessageBoxButtons.OK, MessageBoxIcon.Warning);

                txtCodigoBarra.Focus();
                return;
            }

            if (formularioSecundario != null && !formularioSecundario.IsDisposed)
            {
                formularioSecundario.Activate();
                return;
            }

            formularioSecundario = new frmCobroSecundario(dtgPresentacion);

            formularioSecundario.FormClosed += (s, arg) =>
            {
                formularioSecundario = null;
            };
```

```
        formularioSecundario.Show();

    }
    private void btnCobrar_Click_1(object sender, EventArgs e)
    {
        if (total <= 0)
        {
            MessageBox.Show(ClsIdioma.T("Debe agregar al menos un producto para realizar el cobro."), ClsIdioma.T("Cobrar venta"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            txtCodigoBarra.Focus();
            return;
        }

        if (!VerificarCajaAbierta())
        {
            return;
        }

        frmInformeCobro cobro = new frmInformeCobro(total, dtgPresentacion, promocionesAplicadas);

        if (cobro.ShowDialog() == DialogResult.OK)
        {
            LimpiarVenta();
        }
    }
    private void menuitemAdministradoarticulos_Click(object sender, EventArgs e)
    {
        frmCategoriaMarca categoriaMarca = new frmCategoriaMarca();
        categoriaMarca.ShowDialog();
    }
    private void btnPromocion_Click(object sender, EventArgs e)
    {
        using (frmBuscarPromo buscarPromo = new frmBuscarPromo())
        {
            if (buscarPromo.ShowDialog() != DialogResult.OK)
                return;

            int idPromocion = buscarPromo.IdPromocionSeleccionada;

            try
            {
                ClsPromocionLogica logicaPromo = new ClsPromocionLogica();
                ClsGestorProductoLogica logicaProducto = new ClsGestorProductoLogica();

                DataSet datos = logicaPromo.ObtenerPromocionPorId(idPromocion);

                if (datos == null || datos.Tables.Count < 2 || datos.Tables[0].Rows.Count == 0)
                {
                    MessageBox.Show(ClsIdioma.T("No se encontraron los datos de la promoción."));
                    return;
                }

                DataRow promo = datos.Tables[0].Rows[0];

                if (promo["Activo"] == DBNull.Value || !Convert.ToBoolean(promo["Activo"]))
                {
                    MessageBox.Show(ClsIdioma.T("La promoción seleccionada está inactiva."));
                    return;
                }
```

```
DataTable detalles = datos.Tables[1];

if (detalles.Rows.Count == 0)
{
    MessageBox.Show(ClsIdioma.T("La promoción no tiene productos asociados."));
    return;
}

int idTipoPromocion = Convert.ToInt32(promo["IdTipoPromocion"]);
decimal valor = Convert.ToDecimal(promo["Valor"]);

DataTable tipos = logicaPromo.ObtenerTiposPromocion();

DataRow[] tiposEncontrados = tipos.Select("IdTipoPromocion = " + idTipoPromocion);

if (tiposEncontrados.Length == 0)
{
    MessageBox.Show(ClsIdioma.T("No se encontró el tipo de promoción."));
    return;
}

DataRow tipoPromocion = tiposEncontrados[0];

bool esVolumen = tipoPromocion["CantidadLleva"] != DBNull.Value && tipoPromocion["CantidadPaga"] != DBNull.Value;

int cantidadLleva = 0;
int cantidadPaga = 0;

if (esVolumen)
{
    cantidadLleva = Convert.ToInt32(tipoPromocion["CantidadLleva"]);
    cantidadPaga = Convert.ToInt32(tipoPromocion["CantidadPaga"]);

    if (cantidadLleva <= 0 || cantidadPaga <= 0 || cantidadPaga >= cantidadLleva)
    {
        MessageBox.Show(ClsIdioma.T("La mecánica de volumen no es válida."));
        return;
    }
}
else if (idTipoPromocion == 4)
{
    if (valor <= 0 || valor > 100)
    {
        MessageBox.Show(ClsIdioma.T("El porcentaje de descuento no es válido."));
        return;
    }
}
else if (idTipoPromocion == 5)
{
    if (valor <= 0)
    {
        MessageBox.Show(ClsIdioma.T("El precio especial no es válido."));
        return;
    }
}
else
{
    MessageBox.Show(ClsIdioma.T("El tipo de promoción no es válido."));
    return;
}
```

```
decimal precioFinal = 0m;

DataTable respaldoGrilla = new DataTable();

foreach (DataGridViewColumn columna in dtgPresentacion.Columns)
{
    respaldoGrilla.Columns.Add(columna.Name, typeof(object));
}

foreach (DataGridViewRow fila in dtgPresentacion.Rows)
{
    if (fila.IsNewRow)
        continue;

    DataRow nuevaFila = respaldoGrilla.NewRow();

    foreach (DataGridViewColumn columna in dtgPresentacion.Columns)
    {
        nuevaFila[columna.Name] = fila.Cells[columna.Name].Value ?? DBNull.Value;
    }

    respaldoGrilla.Rows.Add(nuevaFila);
}

decimal subtotalAnterior = subtotal;
decimal ajusteAnterior = descuentoRecargo;
decimal totalAnterior = total;

foreach (DataRow detalle in detalles.Rows)
{
    decimal cantidadConfigurada = Convert.ToDecimal(detalle["Cantidad"]);
    decimal precioUnitario = Convert.ToDecimal(detalle["PrecioVenta"]);

    if (cantidadConfigurada <= 0 || precioUnitario < 0)
    {
        MessageBox.Show(ClsIdioma.T("La cantidad o el precio de un producto no son válidos."));
        return;
    }

    if (esVolumen)
    {
        precioFinal += cantidadConfigurada * cantidadPaga * precioUnitario;
    }
    else if (idTipoPromocion == 4)
    {
        precioFinal += cantidadConfigurada * precioUnitario * (1m - valor / 100m);
    }
    else if (idTipoPromocion == 5)
    {
        precioFinal += cantidadConfigurada * valor;
    }
}

precioFinal = Math.Round(precioFinal, 2);

var cantidadesPromocion = new Dictionary<int, decimal>();

foreach (DataRow detalle in detalles.Rows)
{
    int idProducto = Convert.ToInt32(detalle["IdProducto"]);
    decimal cantidadConfigurada = Convert.ToDecimal(detalle["Cantidad"]);
```

```
            decimal cantidadAgregar = esVolumen ? cantidadConfigurada * cantidadLleva : cantidadConfigurada;

            if (cantidadesPromocion.ContainsKey(idProducto))
            {
                cantidadesPromocion[idProducto] += cantidadAgregar;
            }
            else
            {
                cantidadesPromocion[idProducto] = cantidadAgregar;
            }
        }

        foreach (var producto in cantidadesPromocion)
        {
            int idProducto = producto.Key;
            decimal cantidadAgregar = producto.Value;

            DataTable tablaStock = logicaProducto.ObtenerStockPorProducto(idProducto);

            decimal stockDisponible = 0m;

            if (tablaStock.Rows.Count > 0 && tablaStock.Rows[0]["CantidadActual"] != DBNull.Value)
            {
                stockDisponible = Convert.ToDecimal(tablaStock.Rows[0]["CantidadActual"]);
            }

            decimal cantidadEnPrincipal = 0m;

            foreach (DataGridViewRow fila in dtgPresentacion.Rows)
            {
                if (fila.IsNewRow || fila.Cells["ClnIdProducto"].Value == null)
                {
                    continue;
                }

                if (int.TryParse(Convert.ToString(fila.Cells["ClnIdProducto"].Value), out int idCargado) && idCargado == idProducto)
                {
                    if (decimal.TryParse(Convert.ToString(fila.Cells["ClnCantidad"].Value), out decimal cantidadCargada))
                    {
                        cantidadEnPrincipal += cantidadCargada;
                    }
                }
            }

            decimal cantidadEnSecundaria = 0m;

            if (formularioSecundario != null && !formularioSecundario.IsDisposed)
            {
                cantidadEnSecundaria = formularioSecundario.ObtenerCantidadCargada(idProducto);
            }

            if (cantidadEnPrincipal + cantidadEnSecundaria + cantidadAgregar > stockDisponible)
            {
                MessageBox.Show(ClsIdioma.T("Stock insuficiente para un producto de la promoción.\n\n") + (ClsIdioma.Ingles ?
$"Product ID: {idProducto}\n" : $"ID del producto: {idProducto}\n") +
                    (ClsIdioma.Ingles ? $"Stock in system: {stockDisponible}\n" : $"Stock en sistema: {stockDisponible}\n") +
(ClsIdioma.Ingles ? $"In main sale: {cantidadEnPrincipal}\n" : $"En venta principal: {cantidadEnPrincipal}\n") + (ClsIdioma.Ingles ? $"In
secondary sale: {cantidadEnSecundaria}\n" : $"En venta secundaria: {cantidadEnSecundaria}\n") +
                    (ClsIdioma.Ingles ? $"Quantity required by the promotion: {cantidadAgregar}" : $"Cantidad requerida por la promoción:
{cantidadAgregar}"),
```

```
                ClsIdioma.T("Promociones"), MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return;
        }
    }


    foreach (DataRow detalle in detalles.Rows)
    {
        int idProducto = Convert.ToInt32(detalle["IdProducto"]);
        string sku = detalle["SKU"].ToString();
        string codigoPromocion = "PROMOCION";
        decimal precio = Convert.ToDecimal(detalle["PrecioVenta"]);
        decimal cantidadConfigurada = Convert.ToDecimal(detalle["Cantidad"]);

        decimal cantidad = esVolumen ? cantidadConfigurada * cantidadLleva : cantidadConfigurada;

        DataTable tablaStock = logicaProducto.ObtenerStockPorProducto(idProducto);

        decimal stockDisponible = 0m;

        if (tablaStock.Rows.Count > 0 && tablaStock.Rows[0]["CantidadActual"] != DBNull.Value)
        {
            stockDisponible = Convert.ToDecimal(tablaStock.Rows[0]["CantidadActual"]);
        }

        bool agregado = AgregarProductoAGrilla(idProducto, sku, codigoPromocion, detalle["NombreProducto"].ToString(), precio,
cantidad, stockDisponible);

        if (agregado && dtgPresentacion.Rows.Count > 0)
        {
            DataGridViewRow filaAgregada = dtgPresentacion.Rows[dtgPresentacion.Rows.Count - 1];

            filasDePromocion.Add(filaAgregada);
        }

        if (!agregado)
        {
            dtgPresentacion.Rows.Clear();

            foreach (DataRow filaRespaldo in respaldoGrilla.Rows)
            {
                int indice = dtgPresentacion.Rows.Add();
                DataGridViewRow filaNueva = dtgPresentacion.Rows[indice];

                foreach (DataGridViewColumn columna in dtgPresentacion.Columns)
                {
                    object valorCelda = filaRespaldo[columna.Name];

                    filaNueva.Cells[columna.Name].Value = valorCelda == DBNull.Value ? null : valorCelda;
                }
            }

            subtotal = subtotalAnterior;
            descuentoRecargo = ajusteAnterior;
            total = totalAnterior;

            RellenarFilas();
            CalcularYMostrarMontos();
            txtCodigoBarra.Focus();
```

```
                    MessageBox.Show(ClsIdioma.T("No se pudo cargar toda la promoción. Se restauró la venta anterior."),
ClsIdioma.T("Promociones"),
                        MessageBoxButtons.OK, MessageBoxIcon.Warning);

                    return;
                }

            }

            decimal importeNormalAgregado = subtotal - subtotalAnterior;
            decimal descuentoPromocion = importeNormalAgregado - precioFinal;

            descuentoRecargo = ajusteAnterior - descuentoPromocion;

            RegistrarPromocionAplicada(
                idPromocion,
                promo.Table.Columns.Contains("Nombre") ? Convert.ToString(promo["Nombre"]) : "",
                importeNormalAgregado,
                precioFinal);

            CalcularYMostrarMontos();


            txtCodigoBarra.Focus();
        }
        catch (Exception ex)
        {
            MessageBox.Show(ClsIdioma.T("Error al cargar la promoción: ") + ex.Message, ClsIdioma.T("Promociones"),
                MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
}
private void btnCambiarcantidad_Click(object sender, EventArgs e)
{
    if (dtgPresentacion.CurrentRow == null || dtgPresentacion.CurrentRow.Cells["ClnIdProducto"].Value == null)
    {
        MessageBox.Show(ClsIdioma.T("Seleccione un producto para cambiar la cantidad."), ClsIdioma.T("Cambiar cantidad"),
            MessageBoxButtons.OK, MessageBoxIcon.Warning);

        return;
    }

    DataGridViewRow fila = dtgPresentacion.CurrentRow;

    string codigoFila = Convert.ToString(fila.Cells["ClnCodigo"].Value);

    if (codigoFila == "PROMOCION")
    {
        MessageBox.Show(
            ClsIdioma.T("No se puede modificar la cantidad de un producto que pertenece a una promoción."),
            ClsIdioma.T("Cambiar cantidad"),
            MessageBoxButtons.OK,
            MessageBoxIcon.Warning);

        txtCodigoBarra.Focus();
        return;
    }


    int idProducto = Convert.ToInt32(fila.Cells["ClnIdProducto"].Value);
    decimal cantidadActual = Convert.ToDecimal(fila.Cells["ClnCantidad"].Value);
```

```
        string entrada = Microsoft.VisualBasic.Interaction.InputBox(ClsIdioma.T("Ingrese la nueva cantidad:"), ClsIdioma.T("Cambiar cantidad"), cantidadActual.ToString());

        if (string.IsNullOrWhiteSpace(entrada))
        {
            return;
        }

        if (!decimal.TryParse(entrada, out decimal nuevaCantidad) || nuevaCantidad <= 0)
        {
            MessageBox.Show(ClsIdioma.T("Ingrese una cantidad válida mayor a 0."), ClsIdioma.T("Cantidad inválida"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return;
        }

        ClsGestorProductoLogica logicaProducto = new ClsGestorProductoLogica();

        DataTable tablaStock = logicaProducto.ObtenerStockPorProducto(idProducto);

        decimal stockDisponible = 0;

        if (tablaStock.Rows.Count > 0 && tablaStock.Rows[0]["CantidadActual"] != DBNull.Value)
        {
            stockDisponible = Convert.ToDecimal(tablaStock.Rows[0]["CantidadActual"]);
        }

        decimal cantidadEnGrillaPrincipalSinFila = 0m;

        foreach (DataGridViewRow otraFila in dtgPresentacion.Rows)
        {
            if (otraFila.IsNewRow || otraFila == fila || otraFila.Cells["ClnIdProducto"].Value == null)
                continue;

            if (int.TryParse(Convert.ToString(otraFila.Cells["ClnIdProducto"].Value), out int idCargado) && idCargado == idProducto)
            {
                if (decimal.TryParse(Convert.ToString(otraFila.Cells["ClnCantidad"].Value), out decimal cantidadCargada))
                {
                    cantidadEnGrillaPrincipalSinFila += cantidadCargada;
                }
            }
        }

        decimal cantidadEnGrillaSecundaria = 0m;

        if (formularioSecundario != null && !formularioSecundario.IsDisposed)
        {
            cantidadEnGrillaSecundaria = formularioSecundario.ObtenerCantidadCargada(idProducto);
        }

        decimal cantidadTotal = cantidadEnGrillaPrincipalSinFila + cantidadEnGrillaSecundaria + nuevaCantidad;

        if (stockDisponible <= 0 || cantidadTotal > stockDisponible)
        {
            decimal stockRestante = Math.Max(0m, stockDisponible - cantidadEnGrillaPrincipalSinFila - cantidadEnGrillaSecundaria);

            MessageBox.Show((ClsIdioma.Ingles ? $"Insufficient stock.\n\n" : $"Stock insuficiente.\n\n") + (ClsIdioma.Ingles ? $"Stock in system: {stockDisponible}\n" : $"Stock en sistema: {stockDisponible}\n") +
                (ClsIdioma.Ingles ? $"Other units in main sale: {cantidadEnGrillaPrincipalSinFila}\n" : $"Otras unidades en venta principal: {cantidadEnGrillaPrincipalSinFila}\n") + (ClsIdioma.Ingles ? $"Units in secondary sale: {cantidadEnGrillaSecundaria}\n" : $"Unidades en venta secundaria: {cantidadEnGrillaSecundaria}\n") +
```

```
            (ClsIdioma.Ingles ? $"Available for this row: {stockRestante}\n" : $"Disponible para esta fila: {stockRestante}\n") +
(ClsIdioma.Ingles ? $"Requested quantity: {nuevaCantidad}" : $"Cantidad solicitada: {nuevaCantidad}"),
            ClsIdioma.T("Stock insuficiente"), MessageBoxButtons.OK, MessageBoxIcon.Warning);

        return;
    }


    decimal precioUnitario = Convert.ToDecimal(fila.Cells["ClnPrecioUnitario"].Value);

    fila.Cells["ClnCantidad"].Value = nuevaCantidad;
    fila.Cells["ClnImporte"].Value = nuevaCantidad * precioUnitario;

    CalcularYMostrarMontos();

    txtCodigoBarra.Focus();
}
private void btnQuitar_Click(object sender, EventArgs e)
{
    if (dtgPresentacion.CurrentRow == null || dtgPresentacion.CurrentRow.Cells["ClnIdProducto"].Value == null)
    {
        MessageBox.Show(ClsIdioma.T("Seleccione un producto para quitar."), ClsIdioma.T("Quitar producto"),
            MessageBoxButtons.OK, MessageBoxIcon.Warning);

        return;
    }

    DataGridViewRow fila = dtgPresentacion.CurrentRow;

    if (Convert.ToString(fila.Cells["ClnCodigo"].Value) == "PROMOCION")
    {
        DialogResult quitarPromos = MessageBox.Show(
            ClsIdioma.T("Ese producto pertenece a una promoción.\n\n") +
            ClsIdioma.T("Se van a quitar todos los productos de promociones y su descuento. ¿Continuar?"),
            ClsIdioma.T("Quitar promoción"), MessageBoxButtons.YesNo, MessageBoxIcon.Question);

        if (quitarPromos != DialogResult.Yes)
        {
            return;
        }

        List<DataGridViewRow> filasPromo = dtgPresentacion.Rows.Cast<DataGridViewRow>()
            .Where(f => !f.IsNewRow && Convert.ToString(f.Cells["ClnCodigo"].Value) == "PROMOCION")
            .ToList();

        foreach (DataGridViewRow filaPromo in filasPromo)
        {
            dtgPresentacion.Rows.Remove(filaPromo);
        }

        filasDePromocion.Clear();
        promocionesAplicadas.Clear();
        descuentoRecargo = 0m;

        RellenarFilas();
        CalcularYMostrarMontos();
        txtCodigoBarra.Focus();
        return;
    }

    dtgPresentacion.Rows.Remove(fila);
```

```
            RellenarFilas();
            CalcularYMostrarMontos();

            txtCodigoBarra.Focus();
        }
        private void administrarPromocionesToolStripMenuItem_Click(object sender, EventArgs e)
        {
            frmPromocion promo = new frmPromocion();
            promo.ShowDialog();
        }
        private void verificarIntegridadToolStripMenuItem_Click(object sender, EventArgs e)
        {
            using (frmVerificarIntegridad frm = new frmVerificarIntegridad())
            {
                frm.ShowDialog(this);
            }
        }
        private void gestioToolStripMenuItem_Click(object sender, EventArgs e)
        {
            frmGestionBackup frm = new frmGestionBackup();
            frm.ShowDialog();
        }
        private async void FrmPresentacion_Load(object sender, EventArgs e)
        {

            try
            {
                ClsBackupLogica backup = new ClsBackupLogica();
                await Task.Run(() => backup.BackupAutomatico());
            }
            catch (Exception ex)
            {
                MessageBox.Show(
                    ClsIdioma.T("No se pudo generar el backup automático: ") + ex.Message, ClsIdioma.T("Error"), MessageBoxButtons.OK,
MessageBoxIcon.Error);
            }
        }
        private void cargarArticuloNuevoToolStripMenuItem_Click(object sender, EventArgs e)
        {
            frmNuevoArticulo nuevoarticulo = new frmNuevoArticulo();
            nuevoarticulo.ShowDialog();

        }


        // idioma
        private void españolToolStripMenuItem_Click(object sender, EventArgs e)
        {
            ClsIdioma.CambiarIdioma(false);
            AplicarIdioma();
        }
        private void inglesToolStripMenuItem_Click(object sender, EventArgs e)
        {
            ClsIdioma.CambiarIdioma(true);
            AplicarIdioma();
        }

        // bitacora
        private void bitacoraToolStripMenuItem_Click(object sender, EventArgs e)
        {
```

```
        frmBitacora bitacora = new frmBitacora();
        bitacora.ShowDialog();
    }
    #endregion

    private void recepciónDeMercaderiaToolStripMenuItem_Click(object sender, EventArgs e)
    {
        frmRecepcionCompra recepcion = new frmRecepcionCompra();
        recepcion.ShowDialog();
    }

    private void estadoCajaToolStripMenuItem_Click(object sender, EventArgs e)
    {
        using (frmCaja caja = new frmCaja())
        {
            caja.ShowDialog();
        }
    }

    private void balanceToolStripMenuItem_Click(object sender, EventArgs e)
    {
        using (frmBalance balance = new frmBalance())
        {
            balance.ShowDialog();
        }
    }
  }

}
```

Capa Vista // frmInformeCobro

```
using CapaEntidades;
```

```
using CapaEntidades.NegocioEntidad;
using CapaLogica.NegocioLogica;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Globalization;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace CapaVista
{
    public partial class frmInformeCobro : Form
    {
        private readonly decimal totalVenta;
        private readonly DataGridView carritoVenta;
        private decimal totalPagado = 0;
        private decimal vueltoVenta = 0;
        private bool cargandoMonto = false;
        private DataTable metodosPago;
        private List<ClsPromocionVentaEntidad> promociones = new List<ClsPromocionVentaEntidad>();
        public frmInformeCobro(decimal total, DataGridView carrito)
        {
            InitializeComponent();
            ClsIdioma.Registrar(this);

            totalVenta = total;
            carritoVenta = carrito;

            lblTotal.Text = $"$ {totalVenta:N2}";
            lblPagado.Text = "$ 0,00";
            lblRestante.Text = $"$ {totalVenta:N2}";
            lblVuelto.Text = "$ 0,00";
            lblRestaP.Text = $"$ {totalVenta:N2}";

            PonerMonto(totalVenta);

            btnConfirmar.Visible = true;
            btnConfirmar.Enabled = false;
            btnConfirmar.BringToFront();

        }

        public frmInformeCobro(decimal total, DataGridView carrito, List<ClsPromocionVentaEntidad> promocionesAplicadas)
            : this(total, carrito)
        {
            if (promocionesAplicadas != null)
            {
                promociones = promocionesAplicadas;
            }
        }

        #region Metodos
        private void AgregarPago(string metodoPago)
        {
            decimal restante = totalVenta - totalPagado;

            if (restante <= 0)
```

```
        {
            MessageBox.Show(ClsIdioma.T("La venta ya está completamente pagada."), ClsIdioma.T("Agregar pago"),
                MessageBoxButtons.OK, MessageBoxIcon.Information);

            return;
        }

        if (!TryLeerMonto(txtMontoRecibido.Text, out decimal monto))
        {
            MessageBox.Show(ClsIdioma.T("Ingrese un monto válido."), ClsIdioma.T("Monto inválido"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            txtMontoRecibido.Focus();
            return;
        }

        if (monto <= 0)
        {
            MessageBox.Show(ClsIdioma.T("El monto debe ser mayor que cero."), ClsIdioma.T("Monto inválido"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            txtMontoRecibido.SelectAll();
            txtMontoRecibido.Focus();
            return;
        }

        bool yaHayPagos = totalPagado > 0;

        if (yaHayPagos && monto < restante)
        {
            MessageBox.Show((ClsIdioma.Ingles ? $"The amount is not enough to complete the payment with this method.\n\n" : $"El importe
no alcanza para completar el pago con este método.\n\n") + (ClsIdioma.Ingles ? $"Outstanding balance: $ {restante:N2}\n" : $"Saldo
pendiente: $ {restante:N2}\n") + (ClsIdioma.Ingles ? $"Amount entered: $ {monto:N2}" : $"Importe ingresado: $ {monto:N2}"),
ClsIdioma.T("Importe insuficiente"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            txtMontoRecibido.SelectAll();
            txtMontoRecibido.Focus();
            return;
        }

        bool esEfectivo = metodoPago.Equals("Efectivo", StringComparison.OrdinalIgnoreCase);

        decimal montoAplicado = monto;
        decimal vuelto = 0;

        if (esEfectivo)
        {
            if (monto > restante)
            {
                montoAplicado = restante;
                vuelto = monto - restante;
            }
        }
        else if (monto > restante)
        {
            MessageBox.Show((ClsIdioma.Ingles ? $"The amount exceeds the remaining balance.\n\n" : $"El monto supera el saldo
restante.\n\n") + (ClsIdioma.Ingles ? $"Outstanding balance: $ {restante:N2}" : $"Saldo pendiente: $ {restante:N2}"), ClsIdioma.T("Monto
inválido"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);
```

```
            txtMontoRecibido.SelectAll();
            txtMontoRecibido.Focus();
            return;
        }

        int idMetodoPago;

        try
        {
            idMetodoPago = ObtenerIdMetodoPago(metodoPago);
        }
        catch (Exception ex)
        {
            MessageBox.Show(ClsIdioma.T("No se pudo agregar el pago.\n\n") + ex.Message, ClsIdioma.T("Error"),
                MessageBoxButtons.OK, MessageBoxIcon.Error);
            return;
        }

        dtgPagos.Rows.Add(idMetodoPago, metodoPago, montoAplicado);

        CalcularPagos();
        ActualizarBotonesMetodo();

        vueltoVenta = vuelto;
        lblVuelto.Text = $"$ {vueltoVenta:N2}";

        PonerMonto(Math.Max(totalVenta - totalPagado, 0));
    }
    private int ObtenerIdMetodoPago(string metodoPago)
    {
        if (metodosPago == null)
        {
            ClsGestorVentaLogica gestorVenta = new ClsGestorVentaLogica();
            metodosPago = gestorVenta.ObtenerMetodosPago();
        }

        foreach (DataRow fila in metodosPago.Rows)
        {
            string nombre = fila["Nombre"].ToString();

            if (nombre.Equals(metodoPago, StringComparison.OrdinalIgnoreCase))
            {
                return Convert.ToInt32(fila["IdMetodoPago"]);
            }
        }

        throw new Exception(
            (ClsIdioma.Ingles ? $"The payment method '{metodoPago}' was not found in the database." : $"No se encontró el método de pago
'{metodoPago}' en la base de datos."));
    }
    private static bool TryLeerMonto(string texto, out decimal monto)
    {
        monto = 0;
        texto = (texto ?? "").Replace("$", "").Replace(" ", "").Trim();

        if (texto.Length == 0)
            return false;

        int ultimoPunto = texto.LastIndexOf('.');
        int ultimaComa = texto.LastIndexOf(',');
        int separador = Math.Max(ultimoPunto, ultimaComa);
```

```
    if (ultimaComa < 0 && ultimoPunto >= 0 && texto.Length - ultimoPunto - 1 == 3)
    {
        texto = texto.Replace(".", "");
        separador = -1;
    }

    if (separador >= 0)
    {
        string entera = texto.Substring(0, separador).Replace(".", "").Replace(",", "");
        string decimales = texto.Substring(separador + 1);
        texto = entera + "." + decimales;
    }

    return decimal.TryParse(texto, NumberStyles.AllowDecimalPoint, CultureInfo.InvariantCulture, out monto);
}
private void PonerMonto(decimal monto)
{
    cargandoMonto = true;
    txtMontoRecibido.Text = monto > 0 ? monto.ToString("0.00") : "";
    cargandoMonto = false;
}
private void CalcularVuelto()
{
    decimal restante = Math.Max(totalVenta - totalPagado, 0);

    if (restante == 0)
        return;

    if (!TryLeerMonto(txtMontoRecibido.Text, out decimal montoRecibido))
    {
        vueltoVenta = 0;
        lblVuelto.Text = "$ 0,00";
        return;
    }

    vueltoVenta = montoRecibido > restante ? montoRecibido - restante : 0;

    lblVuelto.Text = $"$ {vueltoVenta:N2}";
}
private void CalcularPagos()
{
    totalPagado = 0;

    foreach (DataGridViewRow fila in dtgPagos.Rows)
    {
        if (fila.IsNewRow)
            continue;

        if (fila.Cells["ClnMontoPago"].Value != null && decimal.TryParse(fila.Cells["ClnMontoPago"].Value.ToString(), out decimal monto))
        {
            totalPagado += monto;
        }
    }

    decimal restante = Math.Max(totalVenta - totalPagado, 0);

    lblPagado.Text = $"$ {totalPagado:N2}";
    lblRestante.Text = $"$ {restante:N2}";
    lblRestaP.Text = $"$ {restante:N2}";
```

```
    btnConfirmar.Enabled = restante == 0;
}
private HashSet<string> MetodosUtilizados()
{
    HashSet<string> metodos = new HashSet<string>(StringComparer.OrdinalIgnoreCase);

    foreach (DataGridViewRow fila in dtgPagos.Rows)
    {
        if (fila.IsNewRow)
            continue;

        object valor = fila.Cells["ClnMetodoPago"].Value;

        if (valor != null)
            metodos.Add(valor.ToString());
    }

    return metodos;
}
private void ActualizarBotonesMetodo()
{
    HashSet<string> usados = MetodosUtilizados();

    MarcarBoton(btnEfectivo, usados.Contains("Efectivo"));
    MarcarBoton(btnTransferencia, usados.Contains("Transferencia"));
    MarcarBoton(btnDebito, usados.Contains("Débito"));
}
private static void MarcarBoton(Button boton, bool usado)
{
    boton.UseVisualStyleBackColor = !usado;
    boton.BackColor = usado ? Color.LightGreen : SystemColors.Control;
}
private bool SeleccionarMetodoPago(string metodoPago)
{
    HashSet<string> metodosUtilizados = MetodosUtilizados();

    if (metodosUtilizados.Contains(metodoPago))
    {
        for (int i = dtgPagos.Rows.Count - 1; i >= 0; i--)
        {
            DataGridViewRow fila = dtgPagos.Rows[i];

            if (fila.IsNewRow)
                continue;

            string metodoFila = fila.Cells["ClnMetodoPago"].Value?.ToString();

            if (string.Equals(metodoFila, metodoPago, StringComparison.OrdinalIgnoreCase))
            {
                dtgPagos.Rows.RemoveAt(i);
            }
        }

        CalcularPagos();
        ActualizarBotonesMetodo();

        vueltoVenta = 0;
        lblVuelto.Text = "$ 0,00";

        PonerMonto(Math.Max(totalVenta - totalPagado, 0));
```

```
            return false;
        }

        if (metodosUtilizados.Count >= 2)
        {
            MessageBox.Show(ClsIdioma.T("Solo se permiten dos métodos de pago diferentes por venta."), ClsIdioma.T("Límite de métodos
de pago"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return false;
        }

        return true;
    }


    #endregion

    #region Eventos
    private void btnEfectivo_Click(object sender, EventArgs e)
    {
        if (SeleccionarMetodoPago("Efectivo"))
            AgregarPago("Efectivo");
    }
    private void btnTransferencia_Click(object sender, EventArgs e)
    {
        if (SeleccionarMetodoPago("Transferencia"))
            AgregarPago("Transferencia");
    }
    private void btnDebito_Click(object sender, EventArgs e)
    {
        if (SeleccionarMetodoPago("Débito"))
            AgregarPago("Débito");
    }
    private void txtMontoRecibido_TextChanged(object sender, EventArgs e)
    {
        if (cargandoMonto)
            return;

        CalcularVuelto();
    }
    private void btnConfirmar_Click(object sender, EventArgs e)
    {
        btnConfirmar.Enabled = false;

        try
        {

            if (totalPagado != totalVenta)
            {
                MessageBox.Show(ClsIdioma.T("La venta todavía no está completamente pagada."), ClsIdioma.T("Confirmar venta"),
                    MessageBoxButtons.OK, MessageBoxIcon.Warning);

                return;
            }

            List<DetalleVentaEntidad> detalles = new List<DetalleVentaEntidad>();

            foreach (DataGridViewRow fila in carritoVenta.Rows)
            {
                if (fila.IsNewRow || fila.Cells["ClnIdProducto"].Value == null)
```

```
            {
                continue;
            }

            int idProducto = Convert.ToInt32(fila.Cells["ClnIdProducto"].Value);
            decimal cantidad = Convert.ToDecimal(fila.Cells["ClnCantidad"].Value);
            decimal precioUnitario = Convert.ToDecimal(fila.Cells["ClnPrecioUnitario"].Value);
            decimal subtotal = Convert.ToDecimal(fila.Cells["ClnImporte"].Value);

            detalles.Add(
                new DetalleVentaEntidad
                {
                    IdProducto = idProducto,
                    Cantidad = cantidad,
                    PrecioUnitario = precioUnitario,
                    Subtotal = subtotal
                });
        }

        List<PagoVentaEntidad> pagos = new List<PagoVentaEntidad>();

        foreach (DataGridViewRow fila in dtgPagos.Rows)
        {
            if (fila.IsNewRow || fila.Cells["ClnIdMetodoPago"].Value == null)
            {
                continue;
            }

            int idMetodoPago = Convert.ToInt32(fila.Cells["ClnIdMetodoPago"].Value);
            decimal monto = Convert.ToDecimal(fila.Cells["ClnMontoPago"].Value);

            pagos.Add(
                new PagoVentaEntidad
                {
                    IdMetodoPago = idMetodoPago,
                    Monto = monto
                });
        }

        ClsGestorVentaLogica gestorVenta = new ClsGestorVentaLogica();

        int idVenta = gestorVenta.RegistrarVenta(totalVenta, SesionUsuario.IdUsuario, null, detalles, pagos, promociones);

        MessageBox.Show((ClsIdioma.Ingles ? $"Sale #{idVenta} registered successfully." : $"Venta #{idVenta} registrada
correctamente."), ClsIdioma.T("Venta registrada"),
            MessageBoxButtons.OK, MessageBoxIcon.Information);

        DialogResult = DialogResult.OK;

        Close();
    }
    catch (Exception ex)
    {
        MessageBox.Show((ClsIdioma.Ingles ? $"The sale could not be registered.\n\n{ex.Message}" : $"No se pudo registrar la
venta.\n\n{ex.Message}"), ClsIdioma.T("Error"),
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    finally
    {
        if (!IsDisposed && DialogResult != DialogResult.OK)
        {
```

```
                btnConfirmar.Enabled = totalPagado == totalVenta;
            }
        }
    }
    private void btnCancelar_Click(object sender, EventArgs e)
    {
        this.Close();
    }

    #endregion
  }
}
```

Capa Lógica // ClsGestorProductoLogica

```
using CapaDatos.NegocioDatos;
using CapaEntidades;
using CapaEntidades.NegocioEntidad;
using CapaLogica.BitacoraLogi;
using Microsoft.Data.SqlClient;
using SistemaKiosco;
using System;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using static CapaEntidades.NegocioEntidad.ClsGestorEntidadProducto;

namespace CapaLogica.NegocioLogica
{
  public partial class ClsGestorProductoLogica
  {
    private clsBitacoraLogica bitacora = new clsBitacoraLogica();
    CapaDatos.NegocioDatos.ClsGestorProductoDatos datos = new CapaDatos.NegocioDatos.ClsGestorProductoDatos();



    public int RegistrarProducto(ClsGestorEntidadProducto.ClsProducto producto)
    {
        int idProducto = this.datos.RegistrarProducto(producto);

        if (idProducto > 0)
        {
            this.RegistrarEventoBitacora(
                "CREAR",
                "Producto",
                idProducto,
                "Se creó el producto: " + producto.Nombre
            );
        }

        return idProducto;
    }
    public bool ModificarProducto(ClsGestorEntidadProducto.ClsProducto producto)
    {
        bool modificado = this.datos.ModificarProducto(producto);

        if (modificado)
        {
            this.RegistrarEventoBitacora(
                "MODIFICAR",
```

```
            "Producto",
            producto.IdProducto,
            "Se modificó el producto: " + producto.Nombre
        );
    }

    return modificado;
}
public bool EliminarProducto(int idProducto)
{
    bool eliminado = this.datos.EliminarProducto(idProducto);

    if (eliminado)
    {
        this.RegistrarEventoBitacora(
            "ELIMINAR",
            "Producto",
            idProducto,
            "Se eliminó el producto con ID " + idProducto
        );
    }

    return eliminado;
}
public bool DescontinuarProducto(int idProducto)
{
    bool descontinuado = this.datos.DescontinuarProducto(idProducto);

    if (descontinuado)
    {
        this.RegistrarEventoBitacora(
            "DISCONTINUAR",
            "Producto",
            idProducto,
            "Se discontinuó el producto con ID " + idProducto
        );
    }

    return descontinuado;
}
public bool ActivarProducto(int idProducto)
{
    bool activado = this.datos.ActivarProducto(idProducto);

    if (activado)
    {
        this.RegistrarEventoBitacora(
            "ACTIVAR",
            "Producto",
            idProducto,
            "Se reactivó el producto con ID " + idProducto
        );
    }

    return activado;
}
public void RecalcularDVHProductos()
{
    datos.RecalcularDVHProductos();
}
public int ObtenerProximoSKU()
```

```
{
    return datos.ObtenerProximoSKU();
}
public bool ExisteCodigoBarra(string codigoBarra, int idProductoExcluir = 0)
{
    return datos.ExisteCodigoBarra(codigoBarra, idProductoExcluir);
}
public DataTable ObtenerProductoPorCodigoBarra(string codigoBarra)
{
    return datos.ObtenerProductoPorCodigoBarra(codigoBarra);
}
public DataTable CargarIVAsActivos()
{
    return datos.CargarIVAsActivos();
}
public DataTable CargarUnidades()
{
    return datos.CargarUnidades();
}
public int RegistrarMarca(ClsMarca marca)
{
    return datos.RegistrarMarca(marca);
}
public DataTable CargarMarcasActivas()
{
    return datos.CargarMarcasActivas();
}
public DataTable CargarTodasLasMarcas()
{
    return datos.CargarTodasLasMarcas();
}
public bool ModificarMarca(ClsMarca marca)
{
    return datos.ModificarMarca(marca);
}
public bool DesactivarMarca(int idMarca)
{
    return datos.DesactivarMarca(idMarca);
}
public bool HabilitarMarca(int idMarca)
{
    return datos.HabilitarMarca(idMarca);
}
public DataTable BuscarMarcas(string nombre, bool soloActivos)
{
    return datos.BuscarMarcas(nombre, soloActivos);
}
public DataTable CargarCategoriasActivas()
{
    return datos.CargarCategoriasActivas();
}
public DataTable CargarTodasLasCategorias()
{
    return datos.CargarTodasLasCategorias();
}
public int RegistrarCategoria(string nombre, string descripcion)
{
    return datos.RegistrarCategoria(nombre, descripcion);
}
public bool ModificarCategoria(int idCategoria, string nombre, string descripcion)
{
```

```
        return datos.ModificarCategoria(idCategoria, nombre, descripcion);
    }
    public bool DesactivarCategoria(int idCategoria)
    {
        return datos.DesactivarCategoria(idCategoria);
    }
    public bool HabilitarCategoria(int idCategoria)
    {
        return datos.HabilitarCategoria(idCategoria);
    }
    public bool EliminarCategoria(int idCategoria)
    {
        return datos.EliminarCategoria(idCategoria);
    }
    public void RegistrarStock(int idProducto,decimal cantidadActual,decimal stockMinimo, decimal stockIdeal)
    {
        datos.RegistrarStock(idProducto,cantidadActual,stockMinimo,stockIdeal);
    }
    public bool RegistrarStock(ClsStock stock)
    {
        return datos.RegistrarStock(stock);
    }
    public DataTable ObtenerProveedoresPorProducto(int idProducto)
    {
        return datos.ObtenerProveedoresPorProducto(idProducto);
    }
    public DataTable ObtenerProductoPorId(int idProducto)
    {
        return datos.ObtenerProductoPorId(idProducto);
    }
    public DataTable CargarArticulos()
    {
        return datos.CargarArticulos();
    }
    public DataTable CargarArticulosParaExportar(int idProveedor, int idCategoria, int idMarca, int idSucursal, int estado, bool soloStock,
bool soloVencimiento, bool soloLote)
    {
        return datos.CargarArticulosParaExportar(idProveedor, idCategoria, idMarca, idSucursal, estado, soloStock, soloVencimiento,
soloLote);
    }
    public DataTable BuscarArticulos(string texto, bool ocultarDiscontinuos)
    {
        return datos.BuscarArticulos(texto, ocultarDiscontinuos);
    }
    public DataTable ObtenerStockPorProducto(int idProducto)
    {
        return datos.ObtenerStockPorProducto(idProducto);
    }
    public void RegistrarProductoSucursalProveedor(ClsProductoSucursalProveedor relacion)
    {
        datos.RegistrarProductoSucursalProveedor(relacion);
    }
    public bool RegistrarLote(ClsLote lote)
    {
        return datos.RegistrarLote(lote);
    }
    public bool ModificarLote(ClsLote lote)
    {
        return datos.ModificarLote(lote);
    }
    public bool ModificarProductoSucursalProveedor(ClsProductoSucursalProveedor relacion)
```

```
    {
        return datos.ModificarProductoSucursalProveedor(relacion);
    }
    public DataTable BuscarProductoPorCodigoOSku(string texto)
    {
        return datos.BuscarProductoPorCodigoOSku(texto);
    }
    public bool AgregarStock(int idProducto, int cantidad)
    {
        return datos.AgregarStock(idProducto, cantidad);
    }
    public bool ModificarStock(int idProducto, decimal cantidadActual, decimal stockMinimo, decimal stockIdeal)
    {
        return datos.ModificarStock(idProducto, cantidadActual, stockMinimo, stockIdeal);
    }
    private void RegistrarEventoBitacora(string accion, string tablaAfectada, int? idRegistroAfectado, string descripcion)
    {
        try
        {
            this.bitacora.RegistrarBitacora(
                SesionUsuario.IdUsuario,
                accion,
                tablaAfectada,
                idRegistroAfectado,
                descripcion
            );
        }
        catch (Exception ex)
        {
            System.Diagnostics.Debug.WriteLine("Error al registrar la bitácora: " + ex.ToString()
            );
        }
    }
    public DataTable BuscarProductosConFiltrosRapida( string texto, int? idProveedor, int? idSucursal, int? idCategoria, int? idMarca)
    {
        return datos.BuscarProductosConFiltrosRapida(
            texto, idProveedor, idSucursal, idCategoria, idMarca);
    }
    public DataTable ObtenerListadoArticulos(string texto, int? idProducto, int? idProveedor, int? idSucursal, int? idCategoria, int? idMarca,
bool soloOfertas, bool soloReponer)
    {
        return datos.ObtenerListadoArticulos(texto, idProducto, idProveedor, idSucursal, idCategoria, idMarca, soloOfertas, soloReponer);
    }


    // Metodos para excel
    public int ObtenerIdMarcaPorNombre(string nombre)
    {
        DataTable tabla = datos.CargarMarcasActivas();

        foreach (DataRow fila in tabla.Rows)
        {
            string nombreBD = fila["Nombre"].ToString();

            if (string.Equals(nombreBD.Trim(), nombre.Trim(),
                StringComparison.OrdinalIgnoreCase))
            {
                return Convert.ToInt32(fila["IdMarca"]);
            }
        }
```

```
    return 0;
}
public int ObtenerIdCategoriaPorNombre(string nombre)
{
    DataTable tabla = datos.CargarCategoriasActivas();

    foreach (DataRow fila in tabla.Rows)
    {
        string nombreBD = fila["Nombre"].ToString();

        if (string.Equals(nombreBD.Trim(), nombre.Trim(),
            StringComparison.OrdinalIgnoreCase))
        {
            return Convert.ToInt32(fila["IdCategoria"]);
        }
    }

    return 0;
}
public int ObtenerIdUnidadPorNombre(string nombre)
{
    DataTable tabla = datos.CargarUnidades();

    foreach (DataRow fila in tabla.Rows)
    {
        string nombreBD = fila["Nombre"].ToString();

        if (string.Equals(nombreBD.Trim(), nombre.Trim(),
            StringComparison.OrdinalIgnoreCase))
        {
            return Convert.ToInt32(fila["IdUnidad"]);
        }
    }

    return 0;
}
public int ObtenerIdIVAPorAlicuota(string alicuota)
{
    DataTable tabla = datos.CargarIVAsActivos();

    foreach (DataRow fila in tabla.Rows)
    {
        string valorBD = fila["Alicuota"].ToString();

        if (string.Equals(valorBD.Trim(), alicuota.Trim(),
            StringComparison.OrdinalIgnoreCase))
        {
            return Convert.ToInt32(fila["IdIVA"]);
        }
    }

    return 0;
}
public int ObtenerIdIVAPorAlicuota(decimal alicuota)
{
    DataTable tabla = datos.CargarIVAsActivos();

    foreach (DataRow fila in tabla.Rows)
    {
        decimal alicuotaBD = Convert.ToDecimal(fila["Alicuota"]);
```

```
            if (alicuotaBD == alicuota)
            {
                return Convert.ToInt32(fila["IdIVA"]);
            }
        }

        return 0;
    }
    public int ObtenerIdSucursalImportacion(string nombreProveedor, string nombreSucursal)
    {
        return datos.ObtenerIdSucursalImportacion(nombreProveedor, nombreSucursal);
    }
    public void ImportarProductos(List<ClsGestorEntidadProducto.ClsProducto> productos,
List<ClsGestorEntidadProducto.ClsImportacionProveedorCosto> proveedoresCostos, Dictionary<string, DateTime?> fechasVencimiento)
    {
        this.datos.ImportarProductos(
            productos,
            proveedoresCostos,
            fechasVencimiento
        );

        if (productos != null && productos.Count > 0)
        {
            this.RegistrarEventoBitacora(
                "IMPORTAR",
                "Producto",
                null,
                "Se importaron " + productos.Count + " productos."
            );
        }
    }


  }
}
```

Capa Datos // ClsGestorVentaDatos

```
using CapaDatos.SistemaKiosco;
using CapaEntidades;
using CapaEntidades.NegocioEntidad;
using Microsoft.Data.SqlClient;
using SistemaKiosco;
using System;
using System.Collections.Generic;
using System.Collections.Generic;
using System.Data;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace CapaDatos.NegocioDatos
{
  public class ClsGestorVentaDatos
  {
    Conexion cn = new Conexion();

    public DataTable ObtenerMetodosPago()
    {
        using (SqlConnection conexion = cn.ObtenerConexion())
        using (SqlCommand cmd = new SqlCommand("sp_ObtenerMetodosPago", conexion))
        {
```

```
            cmd.CommandType = CommandType.StoredProcedure;

            using (SqlDataAdapter adaptador = new SqlDataAdapter(cmd))
            {
                DataTable tabla = new DataTable();

                adaptador.Fill(tabla);

                return tabla;
            }
        }
    }
    public int RegistrarVenta(decimal total, int idUsuario, string observaciones, List<DetalleVentaEntidad> detalles, List<PagoVentaEntidad>
pagos,
                              List<ClsPromocionVentaEntidad> promociones = null)
    {
        string dvh = GenerarDVHVenta(total, idUsuario);
        using (SqlConnection conexion = cn.ObtenerConexion())
        using (SqlCommand cmd = new SqlCommand("sp_RegistrarVenta", conexion))
        {
            cmd.CommandType = CommandType.StoredProcedure;
            cmd.Parameters.Add("@Total", SqlDbType.Decimal).Value = total;
            cmd.Parameters["@Total"].Precision = 18;
            cmd.Parameters["@Total"].Scale = 2;
            cmd.Parameters.Add("@IdUsuario", SqlDbType.Int).Value = idUsuario;
            cmd.Parameters.AddWithValue("@DVH", dvh);

            SqlParameter parametroObservaciones = cmd.Parameters.Add("@Observaciones", SqlDbType.NVarChar, 1000);

            parametroObservaciones.Value = string.IsNullOrWhiteSpace(observaciones) ? DBNull.Value : observaciones.Trim();

            DataTable tablaDetalles = new DataTable();

            tablaDetalles.Columns.Add("IdProducto", typeof(int));
            tablaDetalles.Columns.Add("Cantidad", typeof(decimal));
            tablaDetalles.Columns.Add("PrecioUnitario", typeof(decimal));
            tablaDetalles.Columns.Add("Subtotal", typeof(decimal));

            foreach (DetalleVentaEntidad detalle in detalles)
            {
                tablaDetalles.Rows.Add(
                    detalle.IdProducto,
                    detalle.Cantidad,
                    detalle.PrecioUnitario,
                    detalle.Subtotal);
            }

            SqlParameter parametroDetalles = cmd.Parameters.AddWithValue("@Detalles", tablaDetalles);

            parametroDetalles.SqlDbType = SqlDbType.Structured;
            parametroDetalles.TypeName = "dbo.TipoDetalleVenta";

            DataTable tablaPagos = new DataTable();

            tablaPagos.Columns.Add("IdMetodoPago", typeof(int));
            tablaPagos.Columns.Add("Monto", typeof(decimal));

            foreach (PagoVentaEntidad pago in pagos)
            {
                tablaPagos.Rows.Add(pago.IdMetodoPago, pago.Monto);
            }
```

```
        SqlParameter parametroPagos = cmd.Parameters.AddWithValue("@Pagos", tablaPagos);

        parametroPagos.SqlDbType = SqlDbType.Structured;
        parametroPagos.TypeName = "dbo.TipoPagoVenta";

        ClsPromocionVentaDatos.AgregarParametroPromociones(cmd, promociones);

        conexion.Open();

        return Convert.ToInt32(cmd.ExecuteScalar());
    }
}
public DataTable ObtenerVentas(DateTime desde, DateTime hasta)
{
    using (SqlConnection conexion = cn.ObtenerConexion())
    using (SqlCommand cmd = new SqlCommand("sp_ObtenerVentas", conexion))
    {
        cmd.CommandType = CommandType.StoredProcedure;
        cmd.Parameters.Add("@Desde", SqlDbType.DateTime).Value = desde;
        cmd.Parameters.Add("@Hasta", SqlDbType.DateTime).Value = hasta;

        using (SqlDataAdapter adaptador = new SqlDataAdapter(cmd))
        {
            DataTable tabla = new DataTable();
            adaptador.Fill(tabla);
            return tabla;
        }
    }
}
public DataTable ObtenerVentasListado(DateTime desde, DateTime hasta)
{
    string query = @"
        SELECT
            v.IdVenta,
            v.FechaHora,
            ISNULL(u.Username, '') AS Cajero,
            v.IdSesionCaja,
            v.Total,
            v.Anulada,
            v.Observaciones,
            v.MotivoAnulacion,
            ISNULL((SELECT SUM(d.Cantidad) FROM dbo.DetalleVenta AS d WHERE d.IdVenta = v.IdVenta), 0) AS Articulos,
            ISNULL(
                (SELECT STRING_AGG(mp.Nombre, ', ')
                 FROM dbo.VentaPago AS vp
                 INNER JOIN dbo.MetodoPago AS mp ON mp.IdMetodoPago = vp.IdMetodoPago
                 WHERE vp.IdVenta = v.IdVenta),
                mpv.Nombre) AS MediosPago
        FROM dbo.Venta AS v
        LEFT JOIN dbo.Usuario AS u ON u.IdUsuario = v.IdUsuario
        LEFT JOIN dbo.MetodoPago AS mpv ON mpv.IdMetodoPago = v.IdMetodoPago
        WHERE v.FechaHora >= @Desde
          AND v.FechaHora < DATEADD(DAY, 1, @Hasta)
        ORDER BY v.FechaHora DESC;";

    using (SqlConnection conexion = cn.ObtenerConexion())
    using (SqlCommand cmd = new SqlCommand(query, conexion))
    {
        cmd.Parameters.Add("@Desde", SqlDbType.Date).Value = desde.Date;
        cmd.Parameters.Add("@Hasta", SqlDbType.Date).Value = hasta.Date;
```

```
        using (SqlDataAdapter adaptador = new SqlDataAdapter(cmd))
        {
            DataTable tabla = new DataTable();
            adaptador.Fill(tabla);
            return tabla;
        }
    }
}
public DataTable ObtenerProductosVenta(int idVenta)
{
    string query = @"
        SELECT
            p.CodigoBarra,
            p.Nombre AS Producto,
            d.Cantidad,
            d.PrecioUnitario,
            d.Subtotal
        FROM dbo.DetalleVenta AS d
        INNER JOIN dbo.Producto AS p ON p.IdProducto = d.IdProducto
        WHERE d.IdVenta = @IdVenta
        ORDER BY d.IdDetalleVenta;";

    using (SqlConnection conexion = cn.ObtenerConexion())
    using (SqlCommand cmd = new SqlCommand(query, conexion))
    {
        cmd.Parameters.Add("@IdVenta", SqlDbType.Int).Value = idVenta;

        using (SqlDataAdapter adaptador = new SqlDataAdapter(cmd))
        {
            DataTable tabla = new DataTable();
            adaptador.Fill(tabla);
            return tabla;
        }
    }
}
public DataTable ObtenerPagosVenta(int idVenta)
{
    string query = @"
        SELECT mp.Nombre AS Medio, vp.Monto
        FROM dbo.VentaPago AS vp
        INNER JOIN dbo.MetodoPago AS mp ON mp.IdMetodoPago = vp.IdMetodoPago
        WHERE vp.IdVenta = @IdVenta

        UNION ALL

        SELECT mp.Nombre, v.Total
        FROM dbo.Venta AS v
        INNER JOIN dbo.MetodoPago AS mp ON mp.IdMetodoPago = v.IdMetodoPago
        WHERE v.IdVenta = @IdVenta
          AND NOT EXISTS (SELECT 1 FROM dbo.VentaPago AS vp WHERE vp.IdVenta = v.IdVenta);";

    using (SqlConnection conexion = cn.ObtenerConexion())
    using (SqlCommand cmd = new SqlCommand(query, conexion))
    {
        cmd.Parameters.Add("@IdVenta", SqlDbType.Int).Value = idVenta;

        using (SqlDataAdapter adaptador = new SqlDataAdapter(cmd))
        {
            DataTable tabla = new DataTable();
            adaptador.Fill(tabla);
```

```
            return tabla;
        }
    }
}
public void CambiarEstadoVenta(int idVenta, bool anulada, string motivo, int idUsuario)
{
    using (SqlConnection conexion = cn.ObtenerConexion())
    {
        conexion.Open();

        bool aceptaUsuario;

        using (SqlCommand consulta = new SqlCommand(
            @"SELECT COUNT(*) FROM sys.parameters
              WHERE object_id = OBJECT_ID('dbo.sp_CambiarEstadoVenta') AND name = '@IdUsuario'", conexion))
        {
            aceptaUsuario = Convert.ToInt32(consulta.ExecuteScalar()) > 0;
        }

        using (SqlCommand cmd = new SqlCommand("sp_CambiarEstadoVenta", conexion))
        {
            cmd.CommandType = CommandType.StoredProcedure;

            cmd.Parameters.Add("@IdVenta", SqlDbType.Int).Value = idVenta;
            cmd.Parameters.Add("@Anulada", SqlDbType.Bit).Value = anulada;

            SqlParameter parametroMotivo =
                cmd.Parameters.Add("@MotivoAnulacion", SqlDbType.VarChar, 500);

            parametroMotivo.Value = string.IsNullOrWhiteSpace(motivo)
                ? DBNull.Value
                : motivo.Trim();

            if (aceptaUsuario && idUsuario > 0)
            {
                cmd.Parameters.Add("@IdUsuario", SqlDbType.Int).Value = idUsuario;
            }

            cmd.ExecuteNonQuery();
        }
    }
}
public void ActualizarObservaciones(int idVenta, string observaciones)
{
    using (SqlConnection conexion = cn.ObtenerConexion())
    using (SqlCommand cmd = new SqlCommand(
        @"UPDATE dbo.Venta
          SET Observaciones = @Observaciones
          WHERE IdVenta = @IdVenta", conexion))
    {
        cmd.Parameters.Add("@IdVenta", SqlDbType.Int).Value = idVenta;

        cmd.Parameters.Add("@Observaciones", SqlDbType.NVarChar, 1000).Value = string.IsNullOrWhiteSpace(observaciones) ?
(object)DBNull.Value : observaciones.Trim();

        conexion.Open();

        int filas = cmd.ExecuteNonQuery();

        if (filas == 0)
            throw new Exception("No se encontró la venta seleccionada.");
```

```
    }
}
private string GenerarDVHVenta(decimal total, int idUsuario)
{
    string cadena = ClsDigitoVerificador.FormatoImporte(total) + idUsuario.ToString();

    return ClsDigitoVerificador.GenerarHash(cadena);
}
// Recalcula el DVH de todas las ventas (lo usa el administrador desde Verificar integridad).
public void RecalcularDVHTodasLasVentas()
{
    using (SqlConnection conexion = cn.ObtenerConexion())
    {
        conexion.Open();

        foreach (var (idVenta, _, total, idUsuario, _) in LeerVentasParaDVH(conexion))
        {
            using (SqlCommand cmd = new SqlCommand("UPDATE Venta SET DVH = @DVH WHERE IdVenta = @IdVenta", conexion))
            {
                cmd.Parameters.AddWithValue("@DVH", GenerarDVHVenta(total, idUsuario));
                cmd.Parameters.AddWithValue("@IdVenta", idVenta);

                cmd.ExecuteNonQuery();
            }
        }
    }
}
// Ventas cuyo DVH guardado no coincide con el que dan sus datos actuales.
public List<ClsInconsistenciaDVH> VerificarDVHVentas()
{
    List<ClsInconsistenciaDVH> inconsistencias = new List<ClsInconsistenciaDVH>();

    using (SqlConnection conexion = cn.ObtenerConexion())
    {
        conexion.Open();

        foreach (var (idVenta, fechaHora, total, idUsuario, dvhGuardado) in LeerVentasParaDVH(conexion))
        {
            string problema = ClsDigitoVerificador.CompararDVH(dvhGuardado, GenerarDVHVenta(total, idUsuario));

            if (problema != null)
            {
                inconsistencias.Add(new ClsInconsistenciaDVH
                {
                    Tabla = "Venta",
                    IdRegistro = idVenta,
                    Registro = "Venta #" + idVenta + " - " + fechaHora.ToString("dd/MM/yyyy HH:mm") +
                               " - $" + ClsDigitoVerificador.FormatoImporte(total),
                    Problema = problema
                });
            }
        }
    }

    return inconsistencias;
}
private List<(int IdVenta, DateTime FechaHora, decimal Total, int IdUsuario, string DVH)> LeerVentasParaDVH(SqlConnection conexion)
{
    List<(int, DateTime, decimal, int, string)> ventas = new List<(int, DateTime, decimal, int, string)>();

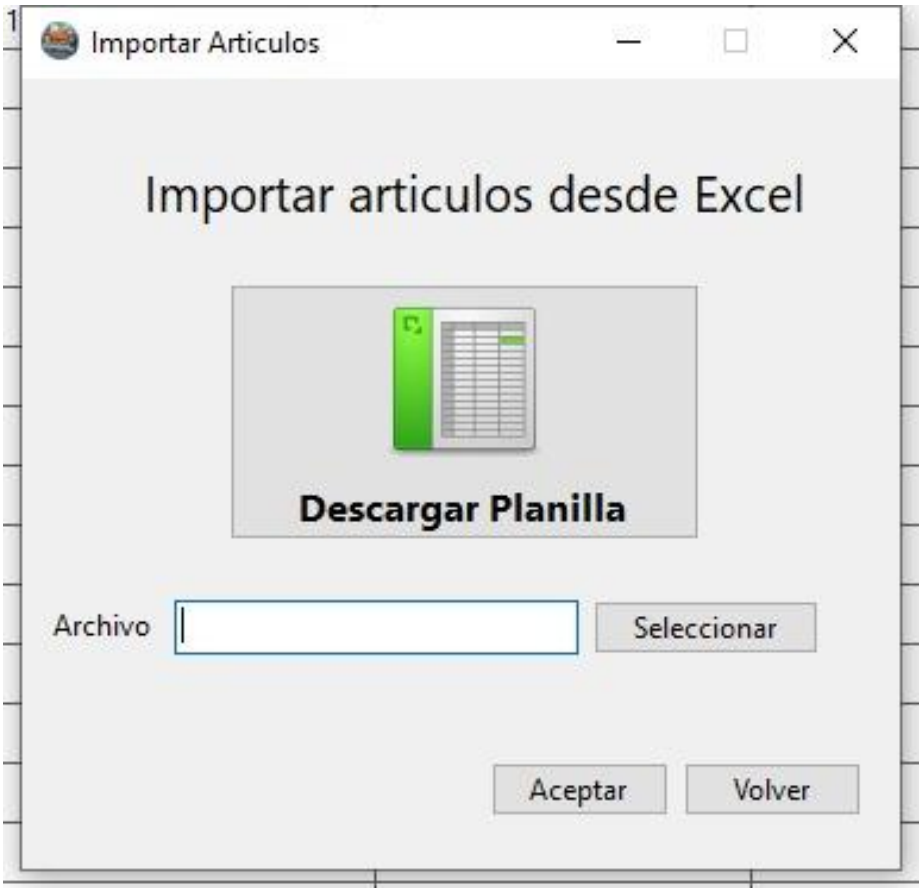
    using (SqlCommand cmd = new SqlCommand("SELECT IdVenta, FechaHora, Total, IdUsuario, DVH FROM Venta", conexion))
```

```
        using (SqlDataReader dr = cmd.ExecuteReader())
        {
            while (dr.Read())
            {
                ventas.Add((
                    Convert.ToInt32(dr["IdVenta"]),
                    Convert.ToDateTime(dr["FechaHora"]),
                    Convert.ToDecimal(dr["Total"]),
                    Convert.ToInt32(dr["IdUsuario"]),
                    dr["DVH"] == DBNull.Value ? null : dr["DVH"].ToString()));
            }
        }

        return ventas;
    }


  }
}
```

4.6 Estructura principal de un caso de uso de negocio del software con interfaces y código: Importar y Exportar Excel





Capa Vista // frmExportarArchivos

```
using CapaLogica;
using CapaLogica.NegocioLogica;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using ClosedXML.Excel;
using System.IO;

namespace CapaVista
{
    public partial class frmExportarArchivos : Form
    {
        private ClsGestorProductoLogica logicaProducto;
        private ClsLogicaProveedor logicaProveedor;

        public frmExportarArchivos()
        {
            InitializeComponent();
            ClsIdioma.Registrar(this);

            logicaProducto = new ClsGestorProductoLogica();
            logicaProveedor = new ClsLogicaProveedor();
```

```
}

#region Metodos
private void frmExportarArchivos_Load_1(object sender, EventArgs e)
{
    CargarProveedores();
    CargarRubros();
    CargarMarcas();
    CargarEstados();
}

private void CargarProveedores()
{
    DataTable tablaProveedores = logicaProveedor.ObtenerProveedores();

    cmbProveedor.DataSource = tablaProveedores;
    cmbProveedor.DisplayMember = "RazonSocial";
    cmbProveedor.ValueMember = "IdProveedor";

    cmbProveedor.SelectedIndex = -1;
}
private void CargarRubros()
{
    DataTable tablaRubros = logicaProducto.CargarCategoriasActivas();

    cmbRubro.DataSource = tablaRubros;
    cmbRubro.DisplayMember = "Nombre";
    cmbRubro.ValueMember = "IdCategoria";

    cmbRubro.SelectedIndex = -1;
}
private void CargarMarcas()
{
    DataTable tablaMarcas = logicaProducto.CargarMarcasActivas();

    DataRow filaTodos = tablaMarcas.NewRow();
    filaTodos["IdMarca"] = 0;
    filaTodos["Nombre"] = ClsIdioma.T("Todos");

    tablaMarcas.Rows.InsertAt(filaTodos, 0);

    cmbMarca.DataSource = tablaMarcas;
    cmbMarca.DisplayMember = "Nombre";
    cmbMarca.ValueMember = "IdMarca";

    cmbMarca.SelectedIndex = 0;
}
private void CargarSucursales(int idProveedor)
{
    if (idProveedor <= 0)
    {
        cmbSucursal.DataSource = null;
        return;
    }

    DataTable tablaSucursales = logicaProveedor.ObtenerSucursalesPorProveedor(idProveedor);

    cmbSucursal.DataSource = tablaSucursales;
    cmbSucursal.DisplayMember = "Sucursal";
    cmbSucursal.ValueMember = "IdSucursal";
```

```
    cmbSucursal.SelectedIndex = -1;
}
private void CargarEstados()
{
    cmbEstado.Items.Clear();

    cmbEstado.Items.Add("Todos");
    cmbEstado.Items.Add("Activo");
    cmbEstado.Items.Add("Inactivo");

    cmbEstado.SelectedIndex = 0;
}
private DataTable ObtenerArticulosFiltrados()
{
    int idProveedor = 0;
    int idRubro = 0;
    int idMarca = 0;
    int idSucursal = 0;
    int estado = -1;

    if (cmbProveedor.SelectedValue != null)
    {
        int.TryParse(cmbProveedor.SelectedValue.ToString(), out idProveedor);
    }

    if (cmbRubro.SelectedValue != null)
    {
        int.TryParse(cmbRubro.SelectedValue.ToString(), out idRubro);
    }

    if (cmbMarca.SelectedValue != null)
    {
        int.TryParse(cmbMarca.SelectedValue.ToString(), out idMarca);
    }

    if (cmbSucursal.SelectedValue != null)
    {
        int.TryParse(cmbSucursal.SelectedValue.ToString(), out idSucursal);
    }

    if (cmbEstado.SelectedIndex == 1)
    {
        estado = 1;
    }
    else if (cmbEstado.SelectedIndex == 2)
    {
        estado = 0;
    }

    return logicaProducto.CargarArticulosParaExportar(idProveedor, idRubro, idMarca, idSucursal, estado,
        chkArticulosStock.Checked, chkArticulosVencimiento.Checked, chkArticuloLote.Checked);
}
private void CrearExcel(DataTable tabla, string rutaArchivo)
{
    using (XLWorkbook libro = new XLWorkbook())
    {
        IXLWorksheet hoja = libro.Worksheets.Add("Articulos");

        string[] encabezados =
        {
            "Código de Barra",
```

```
            "Descripción",
            "Rubro",
            "Marca",
            "Proveedor",
            "Sucursal",
            "Precio Costo",
            "Precio Venta",
            "IVA",
            "Alicuota",
            "Cantidad Actual",
            "Stock Mínimo",
            "Stock Ideal",
            "Fecha Vencimiento",
            "Estado"
        };

        for (int columna = 0; columna < encabezados.Length; columna++)
        {
            hoja.Cell(1, columna + 1).Value = encabezados[columna];
        }

        for (int fila = 0; fila < tabla.Rows.Count; fila++)
        {
            DataRow registro = tabla.Rows[fila];

            hoja.Cell(fila + 2, 1).Value = registro["CodigoBarra"]?.ToString() ?? "";
            hoja.Cell(fila + 2, 2).Value = registro["Descripcion"]?.ToString() ?? "";
            hoja.Cell(fila + 2, 3).Value = registro["Rubro"]?.ToString() ?? "";
            hoja.Cell(fila + 2, 4).Value = registro["Marca"]?.ToString() ?? "";
            hoja.Cell(fila + 2, 5).Value = registro["Proveedor"]?.ToString() ?? "";
            hoja.Cell(fila + 2, 6).Value = registro["Sucursal"]?.ToString() ?? "";
            hoja.Cell(fila + 2, 7).Value = registro["PrecioCosto"] == DBNull.Value ? "" : Convert.ToDecimal(registro["PrecioCosto"]);
            hoja.Cell(fila + 2, 8).Value = registro["PrecioVenta"] == DBNull.Value ? "" : Convert.ToDecimal(registro["PrecioVenta"]);
            hoja.Cell(fila + 2, 9).Value = registro["IVA"]?.ToString() ?? "";
            hoja.Cell(fila + 2, 10).Value = registro["Alicuota"] == DBNull.Value ? "" : Convert.ToDecimal(registro["Alicuota"]);
            hoja.Cell(fila + 2, 11).Value = registro["CantidadActual"] == DBNull.Value ? 0 : Convert.ToDecimal(registro["CantidadActual"]);
            hoja.Cell(fila + 2, 12).Value = registro["StockMinimo"] == DBNull.Value ? 0: Convert.ToDecimal(registro["StockMinimo"]);
            hoja.Cell(fila + 2, 13).Value = registro["StockIdeal"] == DBNull.Value ? 0 : Convert.ToDecimal(registro["StockIdeal"]);


            if (registro["FechaVencimiento"] != DBNull.Value)
            {
                hoja.Cell(fila + 2, 13).Value = Convert.ToDateTime(registro["FechaVencimiento"]);
                hoja.Cell(fila + 2, 13).Style.DateFormat.Format = "dd/MM/yyyy";
            }
            else
            {
                hoja.Cell(fila + 2, 13).Value = "";
            }

            hoja.Cell(fila + 2, 14).Value = Convert.ToBoolean(registro["Activo"]) ? "Activo" : "Inactivo";
        }

        hoja.RangeUsed().SetAutoFilter();
        hoja.Columns().AdjustToContents();

        libro.SaveAs(rutaArchivo);


    }
}
```

```
    #endregion


    #region Eventos
    private void btnSalir_Click(object sender, EventArgs e)
    {
        Close();
    }
    private void cmbProveedor_SelectedIndexChanged(object sender, EventArgs e)
    {
        if (cmbProveedor.SelectedValue == null)
        {
            cmbSucursal.DataSource = null;
            return;
        }

        if (!int.TryParse(cmbProveedor.SelectedValue.ToString(), out int idProveedor))
        {
            cmbSucursal.DataSource = null;
            return;
        }

        CargarSucursales(idProveedor);
    }
    private void btnExportar_Click(object sender, EventArgs e)
    {
        DataTable tabla = ObtenerArticulosFiltrados();

        if (tabla.Rows.Count == 0)
        {
            MessageBox.Show(ClsIdioma.T("No se encontraron artículos con los filtros seleccionados."),
                ClsIdioma.T("Exportar artículos"), MessageBoxButtons.OK, MessageBoxIcon.Information);

            return;
        }

        using (SaveFileDialog guardar = new SaveFileDialog())
        {
            guardar.Title = ClsIdioma.T("Guardar archivo de artículos");
            guardar.Filter = "Archivo Excel (*.xlsx)|*.xlsx";
            guardar.DefaultExt = "xlsx";
            guardar.AddExtension = true;
            guardar.FileName = "Articulos.xlsx";

            if (guardar.ShowDialog() != DialogResult.OK)
            {
                return;
            }


            CrearExcel(tabla, guardar.FileName);

            MessageBox.Show((ClsIdioma.Ingles ? $"The file was exported successfully.\n\nExported products: {tabla.Rows.Count}" : $"El
archivo se exportó correctamente.\n\nArtículos exportados: {tabla.Rows.Count}"),
                ClsIdioma.T("Exportación completa"), MessageBoxButtons.OK, MessageBoxIcon.Information);
        }

    }
```

```
        #endregion



    }
}
```

Capa Vista // frmImportarArticulos

```
using CapaEntidades.NegocioEntidad;
using CapaLogica.NegocioLogica;
using ClosedXML.Excel;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using static CapaEntidades.NegocioEntidad.ClsGestorEntidadProducto;

namespace CapaVista
{
    public partial class frmImportarArticulos : Form
    {
        public frmImportarArticulos()
        {
            InitializeComponent();
            ClsIdioma.Registrar(this);
        }

        #region Metodos
        private void ImportarArticulosDesdeExcel(string rutaArchivo)
        {
            try
            {
                ClsGestorProductoLogica logica = new ClsGestorProductoLogica();

                using (XLWorkbook libro = new XLWorkbook(rutaArchivo))
                {
                    if (!libro.Worksheets.Contains("ARTICULOS"))
                    {
                        MessageBox.Show(ClsIdioma.T("El archivo no contiene la hoja 'ARTICULOS'."), ClsIdioma.T("Importación de artículos"),
                            MessageBoxButtons.OK, MessageBoxIcon.Error);

                        return;
                    }

                    IXLWorksheet hoja = libro.Worksheet("ARTICULOS");

                    int ultimaFila = hoja.LastRowUsed()?.RowNumber() ?? 0;

                    if (ultimaFila < 3)
                    {
                        MessageBox.Show(ClsIdioma.T("No se encontraron artículos para importar."),ClsIdioma.T("Importación de artículos"),
                            MessageBoxButtons.OK, MessageBoxIcon.Warning);

                        return;
                    }
```

```
            // Listas temporalres
            List<string> errores = new List<string>();
            List<ClsGestorEntidadProducto.ClsProducto> productos = new List<ClsGestorEntidadProducto.ClsProducto>();
            List<(string CodigoBarra, string Proveedor, string Sucursal, decimal PrecioCompra)> proveedoresCostos = new List<(string
CodigoBarra, string Proveedor, string Sucursal, decimal PrecioCompra)>();
            Dictionary<string, DateTime?> fechasVencimiento = new Dictionary<string, DateTime?>();

            int cantidadArticulos = 0;

            // Leer articuos
            for (int fila = 3; fila <= ultimaFila; fila++)
            {
               string codigoBarra = hoja.Cell(fila, 1).GetString().Trim();
               string nombre = hoja.Cell(fila, 2).GetString().Trim();
               string descripcion = hoja.Cell(fila, 3).GetString().Trim();
               string marca = hoja.Cell(fila, 4).GetString().Trim();
               string categoria = hoja.Cell(fila, 5).GetString().Trim();
               string unidad = hoja.Cell(fila, 6).GetString().Trim();
               string ivaTexto = hoja.Cell(fila, 7).GetString().Trim();


               // Fecha vencimineto
               DateTime? fechaVencimiento = null;

               IXLCell celdaFecha = hoja.Cell(fila, 8);

               if (!celdaFecha.IsEmpty())
               {
                  // Reconoce fecha como celda
                  if (celdaFecha.DataType == XLDataType.DateTime)
                  {
                     fechaVencimiento = celdaFecha.GetDateTime().Date;
                  }
                  else
                  {
                     string fechaTexto = celdaFecha.GetString().Trim();

                     // fecha escritacomo dd/mm/aaaa
                     if (DateTime.TryParseExact(fechaTexto,new string[]
                        {
                           "dd/MM/yyyy",
                           "d/M/yyyy",
                           "dd/M/yyyy",
                           "d/MM/yyyy"
                        },
                        System.Globalization.CultureInfo.InvariantCulture,
                        System.Globalization.DateTimeStyles.None,
                        out DateTime fechaExacta))
                     {
                        fechaVencimiento = fechaExacta.Date;
                     }

                     // Fecha como ocho digitos
                     if (!fechaVencimiento.HasValue)
                     {
                        string numeroTexto = fechaTexto.Replace("/", "").Replace("-", "").Trim();

                        if (numeroTexto.All(char.IsDigit))
                        {
```

```
                        numeroTexto = numeroTexto.PadLeft(8, '0');

                        if (numeroTexto.Length == 8 && DateTime.TryParseExact( numeroTexto,
                            "ddMMyyyy", System.Globalization.CultureInfo.InvariantCulture, System.Globalization.DateTimeStyles.None,
out DateTime fechaNumerica))
                        {
                            fechaVencimiento = fechaNumerica.Date;
                        }
                    }
                }

                if (!fechaVencimiento.HasValue && DateTime.TryParse(fechaTexto,
                    System.Globalization.CultureInfo.CurrentCulture, System.Globalization.DateTimeStyles.None, out DateTime
fechaConvertida))
                {
                    fechaVencimiento =
                        fechaConvertida.Date;
                }

                if (!fechaVencimiento.HasValue)
                {
                    errores.Add( (ClsIdioma.Ingles ? $"Row {fila}: the expiration date '{fechaTexto}' does not have a valid format. Use
dd/MM/yyyy or 8 digits, for example 25092026." : $"Fila {fila}: la Fecha Vencimiento '{fechaTexto}' no tiene un formato válido. Use
dd/MM/yyyy o 8 dígitos, por ejemplo 25092026."));

                    continue;
                }
            }
        }

        // Fila vacia
        if (string.IsNullOrWhiteSpace(codigoBarra) && string.IsNullOrWhiteSpace(nombre) && string.IsNullOrWhiteSpace(descripcion)
&& string.IsNullOrWhiteSpace(marca) &&
            string.IsNullOrWhiteSpace(categoria) && string.IsNullOrWhiteSpace(unidad) && string.IsNullOrWhiteSpace(ivaTexto) &&
            fechaVencimiento == null)
        {
            continue;
        }

        // Codigo de barra
        if (!string.IsNullOrWhiteSpace(codigoBarra))
        {
            if (logica.ExisteCodigoBarra(codigoBarra))
            {
                errores.Add((ClsIdioma.Ingles ? $"Row {fila}: the barcode '{codigoBarra}' already exists in the database." : $"Fila {fila}: el
código de barra '{codigoBarra}' ya existe en la base de datos."));

                continue;
            }
        }

        // Campos obligatorio
        if (string.IsNullOrWhiteSpace(nombre))
        {
            errores.Add((ClsIdioma.Ingles ? $"Row {fila}: the 'Nombre' field is required." : $"Fila {fila}: el campo 'Nombre' es
obligatorio."));

            continue;
        }

        if (string.IsNullOrWhiteSpace(marca))
```

```
                {
                    errores.Add((ClsIdioma.Ingles ? $"Row {fila}: the 'Marca' field is required." : $"Fila {fila}: el campo 'Marca' es obligatorio."));

                    continue;
                }

                int idMarca = logica.ObtenerIdMarcaPorNombre(marca);

                if (idMarca == 0)
                {
                    errores.Add((ClsIdioma.Ingles ? $"Row {fila}: the brand '{marca}' does not exist or is not active." : $"Fila {fila}: la marca
'{marca}' no existe o no está activa."));

                    continue;
                }

                if (string.IsNullOrWhiteSpace(categoria))
                {
                    errores.Add((ClsIdioma.Ingles ? $"Row {fila}: the 'Categoría' field is required." : $"Fila {fila}: el campo 'Categoría' es
obligatorio."));

                    continue;
                }

                int idCategoria = logica.ObtenerIdCategoriaPorNombre(categoria);

                if (idCategoria == 0)
                {
                    errores.Add((ClsIdioma.Ingles ? $"Row {fila}: the category '{categoria}' does not exist or is not active." : $"Fila {fila}: la
categoría '{categoria}' no existe o no está activa."));

                    continue;
                }

                if (string.IsNullOrWhiteSpace(unidad))
                {
                    errores.Add((ClsIdioma.Ingles ? $"Row {fila}: the 'Unidad' field is required." : $"Fila {fila}: el campo 'Unidad' es obligatorio."));

                    continue;
                }

                int idUnidad = logica.ObtenerIdUnidadPorNombre(unidad);

                if (idUnidad == 0)
                {
                    errores.Add((ClsIdioma.Ingles ? $"Row {fila}: the unit '{unidad}' does not exist." : $"Fila {fila}: la unidad '{unidad}' no
existe."));

                    continue;
                }

                if (string.IsNullOrWhiteSpace(ivaTexto))
                {
                    errores.Add((ClsIdioma.Ingles ? $"Row {fila}: the 'IVA' field is required." : $"Fila {fila}: el campo 'IVA' es obligatorio."));

                    continue;
                }

                decimal alicuota;
```

```
                if (!decimal.TryParse(ivaTexto, System.Globalization.NumberStyles.Number, System.Globalization.CultureInfo.CurrentCulture,
out alicuota))
                {
                    errores.Add((ClsIdioma.Ingles ? $"Row {fila}: the VAT '{ivaTexto}' does not have a valid format." : $"Fila {fila}: el IVA
'{ivaTexto}' no tiene un formato válido."));

                    continue;
                }

                int idIVA = logica.ObtenerIdIVAPorAlicuota(alicuota);

                if (idIVA == 0)
                {
                    errores.Add((ClsIdioma.Ingles ? $"Row {fila}: there is no active VAT with rate {alicuota}." : $"Fila {fila}: no existe un IVA
activo con alícuota {alicuota}."));

                    continue;
                }

                // Crear Producto
                ClsGestorEntidadProducto.ClsProducto producto = new ClsGestorEntidadProducto.ClsProducto();

                producto.CodigoBarra = codigoBarra;
                producto.Nombre = nombre;
                producto.Descripcion = descripcion;
                producto.IdMarca = idMarca;
                producto.IdCategoria = idCategoria;
                producto.IdUnidad = idUnidad;
                producto.IdIVA = idIVA;
                producto.Activo = true;
                producto.PrecioVenta = 0;

                productos.Add(producto);

                if (!string.IsNullOrWhiteSpace(codigoBarra))
                {
                    fechasVencimiento[codigoBarra] = fechaVencimiento;
                }

                cantidadArticulos++;
            }

            // Validar articulo
            if (cantidadArticulos == 0 && errores.Count == 0)
            {
                MessageBox.Show(ClsIdioma.T("No se encontraron artículos para importar."),ClsIdioma.T("Importación de artículos"),
                    MessageBoxButtons.OK, MessageBoxIcon.Warning);

                return;
            }

            if (errores.Count > 0)
            {
                string mensaje = ClsIdioma.T("Se encontraron los siguientes errores:\n\n");

                mensaje += string.Join(Environment.NewLine, errores);

                mensaje += ClsIdioma.T("\n\nNo se importó ningún artículo. Corrija el archivo y vuelva a intentarlo.");

                MessageBox.Show(mensaje,ClsIdioma.T("Errores de validación"), MessageBoxButtons.OK, MessageBoxIcon.Error);
```

```
                return;
            }

            // Verificacion proveedores costo
            if (!libro.Worksheets.Contains("PROVEEDORES_COSTOS"))
            {
                MessageBox.Show(ClsIdioma.T("El archivo no contiene la hoja 'PROVEEDORES_COSTOS'."), ClsIdioma.T("Importación de
artículos"),
                    MessageBoxButtons.OK, MessageBoxIcon.Error);

                return;
            }

            IXLWorksheet hojaProveedoresCostos = libro.Worksheet("PROVEEDORES_COSTOS");

            int ultimaFilaProveedores = hojaProveedoresCostos.LastRowUsed()?.RowNumber() ?? 0;

            // Leer proveedores costo
            if (ultimaFilaProveedores >= 3)
            {
                for (int fila = 3; fila <= ultimaFilaProveedores; fila++)
                {
                    string codigoBarra = hojaProveedoresCostos.Cell(fila, 1).GetString().Trim();
                    string proveedor = hojaProveedoresCostos.Cell(fila, 2).GetString().Trim();
                    string sucursal = hojaProveedoresCostos.Cell(fila, 3).GetString().Trim();
                    string precioTexto = hojaProveedoresCostos.Cell(fila, 4).GetString().Trim();

                    // Fila vacia
                    if (string.IsNullOrWhiteSpace(codigoBarra) && string.IsNullOrWhiteSpace(proveedor) &&
string.IsNullOrWhiteSpace(sucursal) && string.IsNullOrWhiteSpace(precioTexto))
                    {
                        continue;
                    }

                    if (string.IsNullOrWhiteSpace(codigoBarra))
                    {
                        errores.Add((ClsIdioma.Ingles ? $"PROVEEDORES_COSTOS - Row {fila}: the barcode is required." :
$"PROVEEDORES_COSTOS - Fila {fila}: el Código de Barra es obligatorio."));

                        continue;
                    }

                    if (string.IsNullOrWhiteSpace(proveedor))
                    {
                        errores.Add((ClsIdioma.Ingles ? $"PROVEEDORES_COSTOS - Row {fila}: the supplier is required." :
$"PROVEEDORES_COSTOS - Fila {fila}: el Proveedor es obligatorio."));

                        continue;
                    }

                    if (string.IsNullOrWhiteSpace(sucursal))
                    {
                        errores.Add((ClsIdioma.Ingles ? $"PROVEEDORES_COSTOS - Row {fila}: the branch is required." :
$"PROVEEDORES_COSTOS - Fila {fila}: la Sucursal es obligatoria."));

                        continue;
                    }

                    if (string.IsNullOrWhiteSpace(precioTexto))
                    {
```

```
                    errores.Add((ClsIdioma.Ingles ? $"PROVEEDORES_COSTOS - Row {fila}: the purchase price is required." :
$"PROVEEDORES_COSTOS - Fila {fila}: el Precio de Compra es obligatorio."));

                    continue;
                }

                decimal precioCompra;

                if (!decimal.TryParse(precioTexto, System.Globalization.NumberStyles.Number,
System.Globalization.CultureInfo.CurrentCulture, out precioCompra))
                {
                    errores.Add((ClsIdioma.Ingles ? $"PROVEEDORES_COSTOS - Row {fila}: the purchase price '{precioTexto}' does not
have a valid format." : $"PROVEEDORES_COSTOS - Fila {fila}: el Precio de Compra '{precioTexto}' no tiene un formato válido."));

                    continue;
                }

                if (precioCompra < 0)
                {
                    errores.Add( (ClsIdioma.Ingles ? $"PROVEEDORES_COSTOS - Row {fila}: the purchase price cannot be negative." :
$"PROVEEDORES_COSTOS - Fila {fila}: el Precio de Compra no puede ser negativo."));

                    continue;
                }

                proveedoresCostos.Add((codigoBarra, proveedor, sucursal, precioCompra));
            }
        }

        // Validar codigo de barra
        foreach (var proveedorCosto in proveedoresCostos)
        {
            bool existeArticuloEnImportacion = productos.Any(p => p.CodigoBarra == proveedorCosto.CodigoBarra);

            if (!existeArticuloEnImportacion)
            {
                errores.Add((ClsIdioma.Ingles ? $"PROVEEDORES_COSTOS: the barcode '{proveedorCosto.CodigoBarra}'" :
$"PROVEEDORES_COSTOS: el código de barra '{proveedorCosto.CodigoBarra}' ") + (ClsIdioma.Ingles ? $"does not match any product in
the ARTICULOS sheet." : $"no corresponde a ningún artículo de la hoja ARTICULOS."));
            }
        }

        // Validar proveedores y sucursales
        if (errores.Count == 0)
        {
            foreach (var proveedorCosto in proveedoresCostos)
            {
                int idSucursal = logica.ObtenerIdSucursalImportacion(proveedorCosto.Proveedor, proveedorCosto.Sucursal);

                if (idSucursal == 0)
                {
                    errores.Add((ClsIdioma.Ingles ? $"PROVEEDORES_COSTOS: the supplier '{proveedorCosto.Proveedor}'" :
$"PROVEEDORES_COSTOS: el proveedor '{proveedorCosto.Proveedor}' ") +
                        (ClsIdioma.Ingles ? $"does not have an active branch named '{proveedorCosto.Sucursal}'." : $"no tiene una sucursal
activa llamada '{proveedorCosto.Sucursal}'."));
                }
            }
        }

        if (errores.Count > 0)
        {
```

```
                string mensaje = ClsIdioma.T("Se encontraron los siguientes errores:\n\n");

                mensaje += string.Join(Environment.NewLine,errores);
                mensaje += ClsIdioma.T("\n\nNo se importó ningún artículo. Corrija el archivo y vuelva a intentarlo.");

                MessageBox.Show(mensaje,ClsIdioma.T("Errores de validación"),MessageBoxButtons.OK, MessageBoxIcon.Error);

                return;
            }

            List<ClsGestorEntidadProducto.ClsImportacionProveedorCosto> costosImportacion = proveedoresCostos
                .Select(pc => new ClsGestorEntidadProducto.ClsImportacionProveedorCosto
                {
                    CodigoBarra = pc.CodigoBarra,
                    Proveedor = pc.Proveedor,
                    Sucursal = pc.Sucursal,
                    PrecioCompra = pc.PrecioCompra
                })
                .ToList();

            try
            {
                logica.ImportarProductos(productos, costosImportacion, fechasVencimiento);

                MessageBox.Show((ClsIdioma.Ingles ? $"Import completed successfully.\n\n" : $"Importación realizada correctamente.\n\n")
+(ClsIdioma.Ingles ? $"Imported products: {productos.Count}\n" : $"Artículos importados: {productos.Count}\n") +
                    (ClsIdioma.Ingles ? $"Supplier/branch relations: {costosImportacion.Count}" : $"Relaciones proveedor/sucursal:
{costosImportacion.Count}"), ClsIdioma.T("Importación de artículos"),
                    MessageBoxButtons.OK, MessageBoxIcon.Information);
            }
            catch (Exception ex)
            {
                MessageBox.Show(ClsIdioma.T("No se pudo completar la importación.\n\n") +
                    ClsIdioma.T("No se guardó ningún artículo, proveedor, costo ni lote.\n\n") +ex.Message, ClsIdioma.T("Error de
importación"),
                    MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show(ClsIdioma.T("Ocurrió un error al leer o procesar el archivo de Excel.\n\n") +ex.Message,
ClsIdioma.T("Importación de artículos"),
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
private void GenerarPlantillaImportacion()
{
    try
    {
        using (XLWorkbook libro = new XLWorkbook())
        {
            // Colores
            XLColor colorTitulo = XLColor.FromHtml("#1F4E78");
            XLColor colorEncabezado = XLColor.FromHtml("#4472C4");
            XLColor colorCarga = XLColor.FromHtml("#F3F8FC");
            XLColor colorCargaAlterna = XLColor.FromHtml("#EAF2F8");
            XLColor colorObligatorio = XLColor.FromHtml("#FCE4D6");
            XLColor colorBorde = XLColor.FromHtml("#B4C7E7");
            XLColor colorInstrucciones = XLColor.FromHtml("#FFF2CC");
```

```
// Constantes de filas
int filaEncabezado = 2;
int primeraFilaDatos = 3;
int ultimaFilaDatos = 502;

// Hoja de los articulos
IXLWorksheet hojaArticulos = libro.Worksheets.Add("ARTICULOS");

hojaArticulos.Cell(1, 1).Value = "PLANTILLA DE IMPORTACIÓN DE ARTÍCULOS";
hojaArticulos.Range("A1:H1").Merge();

// Encabezado
hojaArticulos.Cell(filaEncabezado, 1).Value = "Código de Barra";
hojaArticulos.Cell(filaEncabezado, 2).Value = "Nombre (*)";
hojaArticulos.Cell(filaEncabezado, 3).Value = "Descripción";
hojaArticulos.Cell(filaEncabezado, 4).Value = "Marca (*)";
hojaArticulos.Cell(filaEncabezado, 5).Value = "Categoría (*)";
hojaArticulos.Cell(filaEncabezado, 6).Value = "Unidad (*)";
hojaArticulos.Cell(filaEncabezado, 7).Value = "IVA (*)";
hojaArticulos.Cell(filaEncabezado, 8).Value = "Fecha Vencimiento";

// Estilo del titulo
hojaArticulos.Range("A1:H1").Style.Fill.SetBackgroundColor(colorTitulo);
hojaArticulos.Range("A1:H1").Style.Font.Bold = true;
hojaArticulos.Range("A1:H1").Style.Font.FontColor = XLColor.White;
hojaArticulos.Range("A1:H1").Style.Font.FontSize = 16;
hojaArticulos.Range("A1:H1").Style.Alignment.Horizontal = XLAlignmentHorizontalValues.Center;
hojaArticulos.Range("A1:H1").Style.Alignment.Vertical = XLAlignmentVerticalValues.Center;
hojaArticulos.Row(1).Height = 30;

// Estilo del encabedo
hojaArticulos.Range("A2:H2").Style.Fill.SetBackgroundColor(colorEncabezado);
hojaArticulos.Range("A2:H2").Style.Font.Bold = true;
hojaArticulos.Range("A2:H2").Style.Font.FontColor = XLColor.White;
hojaArticulos.Range("A2:H2").Style.Alignment.Horizontal = XLAlignmentHorizontalValues.Center;
hojaArticulos.Range("A2:H2").Style.Alignment.Vertical = XLAlignmentVerticalValues.Center;
hojaArticulos.Range("A2:H2").Style.Alignment.WrapText = true;
hojaArticulos.Row(2).Height = 30;

// Bordes
hojaArticulos.Range($"A2:H{ultimaFilaDatos}").Style.Border.OutsideBorder = XLBorderStyleValues.Thin;
hojaArticulos.Range($"A2:H{ultimaFilaDatos}").Style.Border.InsideBorder = XLBorderStyleValues.Thin;
hojaArticulos.Range($"A2:H{ultimaFilaDatos}").Style.Border.OutsideBorderColor = colorBorde;
hojaArticulos.Range($"A2:H{ultimaFilaDatos}").Style.Border.InsideBorderColor = colorBorde;

// Colo de las filas
hojaArticulos.Range($"A{primeraFilaDatos}:H{ultimaFilaDatos}").Style.Fill.SetBackgroundColor(colorCarga);

for (int fila = primeraFilaDatos; fila <= ultimaFilaDatos; fila++)
{
   if (fila % 2 == 0)
   {
      hojaArticulos.Range($"A{fila}:H{fila}").Style.Fill.SetBackgroundColor(colorCargaAlterna);
   }
}

// Campo oblogatorios
hojaArticulos.Range($"B{primeraFilaDatos}:B{ultimaFilaDatos}").Style.Fill.SetBackgroundColor(colorObligatorio);
hojaArticulos.Range($"D{primeraFilaDatos}:G{ultimaFilaDatos}").Style.Fill.SetBackgroundColor(colorObligatorio);

// Codigo de barras
```

```
            hojaArticulos.Range($"A{primeraFilaDatos}:A{ultimaFilaDatos}").Style.NumberFormat.Format = "@";
            hojaArticulos.Range($"A{primeraFilaDatos}:A{ultimaFilaDatos}").Style.Alignment.Horizontal =
XLAlignmentHorizontalValues.Center;

            // Fecha
            IXLRange rangoFechas = hojaArticulos.Range($"H{primeraFilaDatos}:H{ultimaFilaDatos}");

            rangoFechas.Style.NumberFormat.Format = "00\\/00\\/0000";
            rangoFechas.Style.Alignment.Horizontal = XLAlignmentHorizontalValues.Center;

            // Alineacion
            hojaArticulos.Range($"D{primeraFilaDatos}:G{ultimaFilaDatos}").Style.Alignment.Horizontal =
XLAlignmentHorizontalValues.Center;

            // Anchos
            hojaArticulos.Column(1).Width = 18;
            hojaArticulos.Column(2).Width = 28;
            hojaArticulos.Column(3).Width = 35;
            hojaArticulos.Column(4).Width = 22;
            hojaArticulos.Column(5).Width = 22;
            hojaArticulos.Column(6).Width = 18;
            hojaArticulos.Column(7).Width = 15;
            hojaArticulos.Column(8).Width = 20;

            // Hojas auxiliares
            IXLWorksheet hojaMarcas = libro.Worksheets.Add("_MARCAS");
            IXLWorksheet hojaCategorias = libro.Worksheets.Add("_CATEGORIAS");
            IXLWorksheet hojaUnidades = libro.Worksheets.Add("_UNIDADES");
            IXLWorksheet hojaIVA = libro.Worksheets.Add("_IVA");
            IXLWorksheet hojaProveedores = libro.Worksheets.Add("_PROVEEDORES");
            IXLWorksheet hojaSucursales = libro.Worksheets.Add("_SUCURSALES");

            // Marcas
            DataTable tablaMarcas = new ClsGestorProductoLogica().CargarMarcasActivas();

            hojaMarcas.Cell(1, 1).Value = "Marca";

            int filaMarca = 2;

            foreach (DataRow fila in tablaMarcas.Rows)
            {
                hojaMarcas.Cell(filaMarca, 1).Value = fila["Nombre"].ToString();

                filaMarca++;
            }

            // Categorias
            DataTable tablaCategorias = new ClsGestorProductoLogica().CargarCategoriasActivas();

            hojaCategorias.Cell(1, 1).Value = "Categoría";

            int filaCategoria = 2;

            foreach (DataRow fila in tablaCategorias.Rows)
            {
                hojaCategorias.Cell(filaCategoria, 1).Value = fila["Nombre"].ToString();

                filaCategoria++;
            }

            // Unidades
```

```
DataTable tablaUnidades = new ClsGestorProductoLogica().CargarUnidades();

hojaUnidades.Cell(1, 1).Value = "Unidad";

int filaUnidad = 2;

foreach (DataRow fila in tablaUnidades.Rows)
{
    hojaUnidades.Cell(filaUnidad, 1).Value = fila["Nombre"].ToString();

    filaUnidad++;
}

// Iva
DataTable tablaIVA = new ClsGestorProductoLogica().CargarIVAsActivos();

hojaIVA.Cell(1, 1).Value = "Alicuota";

int filaIVA = 2;

foreach (DataRow fila in tablaIVA.Rows)
{
    hojaIVA.Cell(filaIVA, 1).Value = Convert.ToDecimal(fila["Alicuota"]);

    filaIVA++;
}

// Proveedores
ClsLogicaProveedor logicaProveedor = new ClsLogicaProveedor();
DataTable tablaProveedores = logicaProveedor.ObtenerProveedores();

hojaProveedores.Cell(1, 1).Value = "IdProveedor";
hojaProveedores.Cell(1, 2).Value = "Proveedor";

int filaProveedor = 2;

foreach (DataRow fila in tablaProveedores.Rows)
{
    bool proveedorActivo = false;

    if (fila["Estado"] != DBNull.Value)
    {
        string estado = fila["Estado"].ToString().Trim();

        proveedorActivo = estado.Equals("Activo", StringComparison.OrdinalIgnoreCase) ||
        estado.Equals("1", StringComparison.OrdinalIgnoreCase) || estado.Equals("True", StringComparison.OrdinalIgnoreCase);
    }

    if (proveedorActivo)
    {
        int idProveedor;

        if (!int.TryParse(fila["IdProveedor"].ToString(), out idProveedor))
        {
            continue;
        }

        hojaProveedores.Cell(filaProveedor, 1).Value = idProveedor;
        hojaProveedores.Cell(filaProveedor, 2).Value = fila["RazonSocial"].ToString();

        filaProveedor++;
```

```
    }
}

// Sucursales
hojaSucursales.Cell(1, 1).Value = "IdProveedor";
hojaSucursales.Cell(1, 2).Value = "Proveedor";
hojaSucursales.Cell(1, 3).Value = "IdSucursal";
hojaSucursales.Cell(1, 4).Value = "Sucursal";

int filaSucursalAuxiliar = 2;

for (int fila = 2; fila < filaProveedor; fila++)
{
    string valorIdProveedor = hojaProveedores.Cell(fila, 1).GetString().Trim();

    if (!int.TryParse(valorIdProveedor, out int idProveedor))
    {
        continue;
    }

    string nombreProveedor = hojaProveedores.Cell(fila, 2).GetString().Trim();

    if (idProveedor <= 0 || string.IsNullOrWhiteSpace(nombreProveedor))
    {
        continue;
    }

    DataTable tablaSucursales = logicaProveedor.ObtenerSucursalesPorProveedor(idProveedor);

    if (tablaSucursales == null || tablaSucursales.Rows.Count == 0)
    {
        continue;
    }

    foreach (DataRow sucursal in tablaSucursales.Rows)
    {
        if (!tablaSucursales.Columns.Contains("Sucursal"))
        {
            continue;
        }

        if (sucursal["Sucursal"] == DBNull.Value)
        {
            continue;
        }

        string nombreSucursal = sucursal["Sucursal"].ToString().Trim();

        if (string.IsNullOrWhiteSpace(nombreSucursal))
        {
            continue;
        }

        int idSucursal;

        if (!int.TryParse(sucursal["IdSucursal"].ToString(), out idSucursal))
        {
            continue;
        }

        hojaSucursales.Cell(filaSucursalAuxiliar, 1).Value = idProveedor;
```

```
            hojaSucursales.Cell(filaSucursalAuxiliar, 2).Value = nombreProveedor;
            hojaSucursales.Cell(filaSucursalAuxiliar, 3).Value = idSucursal;
            hojaSucursales.Cell(filaSucursalAuxiliar, 4).Value = nombreSucursal;

            filaSucursalAuxiliar++;
        }
    }

    // Rango nombrados
    if (filaMarca > 2)
    {
        hojaMarcas.Range($"A2:A{filaMarca - 1}").AddToNamed("ListaMarcas");
    }

    if (filaCategoria > 2)
    {
        hojaCategorias.Range($"A2:A{filaCategoria - 1}").AddToNamed("ListaCategorias");
    }

    if (filaUnidad > 2)
    {
        hojaUnidades.Range($"A2:A{filaUnidad - 1}").AddToNamed("ListaUnidades");
    }

    if (filaIVA > 2)
    {
        hojaIVA.Range($"A2:A{filaIVA - 1}").AddToNamed("ListaIVA");
    }

    hojaArticulos.Range($"A{primeraFilaDatos}:A{ultimaFilaDatos}").AddToNamed("ListaCodigosBarra");

    if (filaProveedor > 2)
    {
        hojaProveedores.Range($"B2:B{filaProveedor - 1}").AddToNamed("ListaProveedores");
    }

    // Validacionde los los articulos
    if (filaMarca > 2)
    {
        hojaArticulos.Range($"D{primeraFilaDatos}:D{ultimaFilaDatos}").CreateDataValidation().List("=ListaMarcas");
    }

    if (filaCategoria > 2)
    {
        hojaArticulos.Range($"E{primeraFilaDatos}:E{ultimaFilaDatos}").CreateDataValidation().List("=ListaCategorias");
    }

    if (filaUnidad > 2)
    {
        hojaArticulos.Range($"F{primeraFilaDatos}:F{ultimaFilaDatos}").CreateDataValidation().List("=ListaUnidades");
    }

    if (filaIVA > 2)
    {
        hojaArticulos.Range($"G{primeraFilaDatos}:G{ultimaFilaDatos}").CreateDataValidation().List("=ListaIVA");
    }

    // Validacion de la fecha
    var validacionFecha = rangoFechas.CreateDataValidation();

    validacionFecha.IgnoreBlanks = true;
```

```
            IXLWorksheet hojaProveedoresCostos = libro.Worksheets.Add("PROVEEDORES_COSTOS");

            hojaProveedoresCostos.Cell(1, 1).Value = "PROVEEDORES Y COSTOS";
            hojaProveedoresCostos.Range("A1:D1").Merge();

            // Encabezados
            hojaProveedoresCostos.Cell(2, 1).Value = "Código de Barra";
            hojaProveedoresCostos.Cell(2, 2).Value = "Proveedor (*)";
            hojaProveedoresCostos.Cell(2, 3).Value = "Sucursal (*)";
            hojaProveedoresCostos.Cell(2, 4).Value = "Precio Compra (*)";

            // Titulo pagina 2


            hojaProveedoresCostos.Range("A1:D1").Style.Fill.SetBackgroundColor(colorTitulo);
            hojaProveedoresCostos.Range("A1:D1").Style.Font.Bold = true;
            hojaProveedoresCostos.Range("A1:D1").Style.Font.FontColor = XLColor.White;
            hojaProveedoresCostos.Range("A1:D1").Style.Font.FontSize = 16;
            hojaProveedoresCostos.Range("A1:D1").Style.Alignment.Horizontal = XLAlignmentHorizontalValues.Center;
            hojaProveedoresCostos.Range("A1:D1").Style.Alignment.Vertical = XLAlignmentVerticalValues.Center;
            hojaProveedoresCostos.Row(1).Height = 30;

            // Encabezados
            hojaProveedoresCostos.Range("A2:D2").Style.Fill.SetBackgroundColor(colorEncabezado);
            hojaProveedoresCostos.Range("A2:D2").Style.Font.Bold = true;
            hojaProveedoresCostos.Range("A2:D2").Style.Font.FontColor = XLColor.White;
            hojaProveedoresCostos.Range("A2:D2").Style.Alignment.Horizontal = XLAlignmentHorizontalValues.Center;
            hojaProveedoresCostos.Range("A2:D2").Style.Alignment.Vertical = XLAlignmentVerticalValues.Center;
            hojaProveedoresCostos.Range("A2:D2").Style.Alignment.WrapText = true;
            hojaProveedoresCostos.Row(2).Height = 30;

            // Borde
            hojaProveedoresCostos.Range($"A2:D{ultimaFilaDatos}").Style.Border.OutsideBorder = XLBorderStyleValues.Thin;
            hojaProveedoresCostos.Range($"A2:D{ultimaFilaDatos}").Style.Border.InsideBorder = XLBorderStyleValues.Thin;
            hojaProveedoresCostos.Range($"A2:D{ultimaFilaDatos}").Style.Border.OutsideBorderColor = colorBorde;
            hojaProveedoresCostos.Range($"A2:D{ultimaFilaDatos}").Style.Border.InsideBorderColor = colorBorde;

            // Colo de las filad
            hojaProveedoresCostos.Range($"A{primeraFilaDatos}:D{ultimaFilaDatos}").Style.Fill.SetBackgroundColor(colorCarga);

            for (int fila = primeraFilaDatos; fila <= ultimaFilaDatos; fila++)
            {
               if (fila % 2 == 0)
               {
                  hojaProveedoresCostos.Range($"A{fila}:D{fila}").Style.Fill.SetBackgroundColor(colorCargaAlterna);
               }
            }

            // Obligatorios
            hojaProveedoresCostos.Range($"B{primeraFilaDatos}:D{ultimaFilaDatos}").Style.Fill.SetBackgroundColor(colorObligatorio);

            // Codigo de barra
            hojaProveedoresCostos.Range($"A{primeraFilaDatos}:A{ultimaFilaDatos}").Style.NumberFormat.Format = "@";
            hojaProveedoresCostos.Range($"A{primeraFilaDatos}:A{ultimaFilaDatos}").Style.Alignment.Horizontal =
XLAlignmentHorizontalValues.Center;

            // Precio
            hojaProveedoresCostos.Range($"D{primeraFilaDatos}:D{ultimaFilaDatos}").Style.NumberFormat.Format = "#,##0.00";
            hojaProveedoresCostos.Range($"D{primeraFilaDatos}:D{ultimaFilaDatos}").Style.Alignment.Horizontal =
XLAlignmentHorizontalValues.Right;
```

```
            // Sucursal
            hojaProveedoresCostos.Range($"C{primeraFilaDatos}:C{ultimaFilaDatos}").Style.Alignment.Horizontal =
XLAlignmentHorizontalValues.Center;

            // Anchos
            hojaProveedoresCostos.Column(1).Width = 20;
            hojaProveedoresCostos.Column(2).Width = 30;
            hojaProveedoresCostos.Column(3).Width = 30;
            hojaProveedoresCostos.Column(4).Width = 20;

            var validacionCodigo = hojaProveedoresCostos.Range($"A{primeraFilaDatos}:A{ultimaFilaDatos}").CreateDataValidation();

            validacionCodigo.List("=ListaCodigosBarra");
            validacionCodigo.IgnoreBlanks = true;

            // Validacion de los proveedores
            if (filaProveedor > 2)
            {
                hojaProveedoresCostos.Range($"B{primeraFilaDatos}:B{ultimaFilaDatos}").CreateDataValidation().List("=ListaProveedores");
            }

            // Rango de sucursales
            int ultimaFilaSucursales = 0;

            if (hojaSucursales.LastRowUsed() != null)
            {
                ultimaFilaSucursales = hojaSucursales.LastRowUsed().RowNumber();
            }

            for (int fila = 2; fila < filaProveedor; fila++)
            {
                string valorIdProveedor = hojaProveedores.Cell(fila, 1).GetString().Trim();

                if (!int.TryParse(valorIdProveedor, out int idProveedor))
                {
                    continue;
                }

                string nombreRango = "Proveedor_" + idProveedor;

                List<string> sucursales = new List<string>();

                for (int filaS = 2; filaS <= ultimaFilaSucursales; filaS++)
                {
                    string valorIdProveedorSucursal = hojaSucursales.Cell(filaS, 1).GetString().Trim();

                    if (!int.TryParse(valorIdProveedorSucursal, out int idProveedorSucursal))
                    {
                        continue;
                    }

                    if (idProveedorSucursal == idProveedor)
                    {
                        string nombreSucursal = hojaSucursales.Cell(filaS, 4).GetString().Trim();

                        if (!string.IsNullOrWhiteSpace(nombreSucursal))
                        {
                            sucursales.Add(nombreSucursal);
                        }
                    }
```

```
            }

            if (sucursales.Count > 0)
            {
                int columna = 5 + idProveedor;

                for (int i = 0; i < sucursales.Count; i++)
                {
                    hojaSucursales.Cell(i + 2, columna).Value = sucursales[i];
                }

                IXLRange rangoSucursal = hojaSucursales.Range(hojaSucursales.Cell(2, columna), hojaSucursales.Cell( sucursales.Count
+ 1, columna));

                libro.NamedRanges.Add(nombreRango, rangoSucursal);
            }
        }

        // Sucursal dependiente
        if (filaProveedor > 2)
        {
            string ultimaFilaProveedor = (filaProveedor - 1).ToString();

            for (int fila = primeraFilaDatos; fila <= ultimaFilaDatos; fila++)
            {
                string formulaSucursal = "=INDIRECT(" +
                    "\"Proveedor_\"&" +
                    "INDEX(" +
                    "_PROVEEDORES!$A$2:$A$" +
                    ultimaFilaProveedor +
                    ",MATCH(B" +
                    fila +
                    ",_PROVEEDORES!$B$2:$B$" +
                    ultimaFilaProveedor +
                    ",0)" +
                    ")" +
                    ")";

                var validacionSucursal = hojaProveedoresCostos.Cell(fila, 3).CreateDataValidation();
                validacionSucursal.List(formulaSucursal);
                validacionSucursal.IgnoreBlanks = true;
                validacionSucursal.InCellDropdown = true;
            }
        }

        IXLWorksheet hojaInstrucciones = libro.Worksheets.Add("INSTRUCCIONES");

        hojaInstrucciones.Cell(1, 1).Value = "INSTRUCCIONES PARA IMPORTAR ARTÍCULOS";
        hojaInstrucciones.Range("A1:D1").Merge();

        // Instrucciones
        hojaInstrucciones.Cell(2, 1).Value = "IMPORTACIÓN DE ARTÍCULOS";
        hojaInstrucciones.Range("A2:D2").Merge();
        hojaInstrucciones.Cell(3, 1).Value = "1. Complete la hoja ARTICULOS.";
        hojaInstrucciones.Cell(4, 1).Value = "2. Los campos marcados con (*) son obligatorios.";
        hojaInstrucciones.Cell(5, 1).Value = "3. Seleccione Marca, Categoría, Unidad e IVA mediante los desplegables.";
        hojaInstrucciones.Cell(6, 1).Value = "4. La Fecha Vencimiento puede cargarse como dd/MM/yyyy o como 8 dígitos, por ejemplo
25092026.";
        hojaInstrucciones.Cell(7, 1).Value = "5. En PROVEEDORES_COSTOS puede cargar más de una fila para el mismo artículo.";
        hojaInstrucciones.Cell(8, 1).Value = "6. Seleccione primero el Proveedor y luego la Sucursal correspondiente.";
        hojaInstrucciones.Cell(9, 1).Value = "7. La Sucursal se filtra automáticamente según el Proveedor seleccionado.";
```

```
            hojaInstrucciones.Cell(10, 1).Value = "8. El Código de Barra de PROVEEDORES_COSTOS se obtiene de los códigos cargados
en ARTICULOS.";
            hojaInstrucciones.Cell(11, 1).Value = "9. El Precio de Compra debe ser mayor o igual a cero.";
            hojaInstrucciones.Cell(13, 1).Value = "IMPORTANTE";
            hojaInstrucciones.Range("A13:D13").Merge();
            hojaInstrucciones.Cell(14, 1).Value = "No modifique ni elimine las hojas auxiliares ocultas.";
            hojaInstrucciones.Cell(15, 1).Value = "Los campos con (*) son necesarios para realizar la importación.";
            hojaInstrucciones.Column(1).Width = 100;

            // Estilo instrucciones
            hojaInstrucciones.Range("A1:D1").Style.Fill.SetBackgroundColor(colorTitulo);
            hojaInstrucciones.Range("A1:D1").Style.Font.Bold = true;
            hojaInstrucciones.Range("A1:D1").Style.Font.FontColor = XLColor.White;
            hojaInstrucciones.Range("A1:D1").Style.Font.FontSize = 16;
            hojaInstrucciones.Range("A1:D1").Style.Alignment.Horizontal = XLAlignmentHorizontalValues.Center;
            hojaInstrucciones.Range("A2:D2").Style.Fill.SetBackgroundColor(colorEncabezado);
            hojaInstrucciones.Range("A2:D2").Style.Font.Bold = true;
            hojaInstrucciones.Range("A2:D2").Style.Font.FontColor = XLColor.White;
            hojaInstrucciones.Range("A2:D2").Style.Font.FontSize = 13;
            hojaInstrucciones.Range("A13:D13").Style.Fill.SetBackgroundColor(colorObligatorio);
            hojaInstrucciones.Range("A13:D13").Style.Font.Bold = true;
            hojaInstrucciones.Range("A14:D15").Style.Fill.SetBackgroundColor(colorInstrucciones);
            hojaInstrucciones.Range("A3:A15").Style.Alignment.WrapText = true;

            // Filtros
            hojaArticulos.Range($"A2:H{ultimaFilaDatos}").SetAutoFilter();
            hojaProveedoresCostos.Range($"A2:D{ultimaFilaDatos}").SetAutoFilter();

            // Congelar los encabezados
            hojaArticulos.SheetView.FreezeRows(2);
            hojaProveedoresCostos.SheetView.FreezeRows(2);

            // Oculta las hojas auxiliares
            hojaMarcas.Visibility = XLWorksheetVisibility.VeryHidden;
            hojaCategorias.Visibility = XLWorksheetVisibility.VeryHidden;
            hojaUnidades.Visibility = XLWorksheetVisibility.VeryHidden;
            hojaIVA.Visibility = XLWorksheetVisibility.VeryHidden;
            hojaProveedores.Visibility = XLWorksheetVisibility.VeryHidden;
            hojaSucursales.Visibility = XLWorksheetVisibility.VeryHidden;

            // Guardar
            SaveFileDialog guardar = new SaveFileDialog();

            guardar.Title = ClsIdioma.T("Guardar plantilla de importación");
            guardar.Filter = "Archivo de Excel (*.xlsx)|*.xlsx";
            guardar.FileName = "Plantilla_Importacion_Articulos.xlsx";
            guardar.InitialDirectory = Environment.GetFolderPath(Environment.SpecialFolder.Desktop);

            if (guardar.ShowDialog() == DialogResult.OK)
            {
                libro.SaveAs(guardar.FileName);

                MessageBox.Show(ClsIdioma.T("La plantilla fue generada correctamente."),ClsIdioma.T("Plantilla de importación"),
                    MessageBoxButtons.OK, MessageBoxIcon.Information);
            }
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show(ClsIdioma.T("No se pudo generar la plantilla.\n\n") + ex.Message, ClsIdioma.T("Plantilla de importación"),
            MessageBoxButtons.OK, MessageBoxIcon.Error);
```

```
        }
    }


    #endregion

    #region Eventos
    private void btnVolver_Click(object sender, EventArgs e)
    {
        Close();
    }
    private void btnSeleccionar_Click(object sender, EventArgs e)
    {
        OpenFileDialog abrirArchivo = new OpenFileDialog();

        abrirArchivo.Title = ClsIdioma.T("Seleccionar plantilla de artículos");
        abrirArchivo.Filter = "Archivo de Excel (*.xlsx)|*.xlsx";
        abrirArchivo.InitialDirectory = Environment.GetFolderPath(Environment.SpecialFolder.Desktop);
        abrirArchivo.Multiselect = false;

        if (abrirArchivo.ShowDialog() == DialogResult.OK)
        {
            txtTraerArchivo.Text = abrirArchivo.FileName;
        }
    }
    private void btnDescargarPlanilla_Click(object sender, EventArgs e)
    {
        GenerarPlantillaImportacion();
    }
    private void btnAceptar_Click(object sender, EventArgs e)
    {
        if (string.IsNullOrWhiteSpace(txtTraerArchivo.Text))
        {
            MessageBox.Show(ClsIdioma.T("Debe seleccionar un archivo de Excel."), ClsIdioma.T("Importación de artículos"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return;
        }

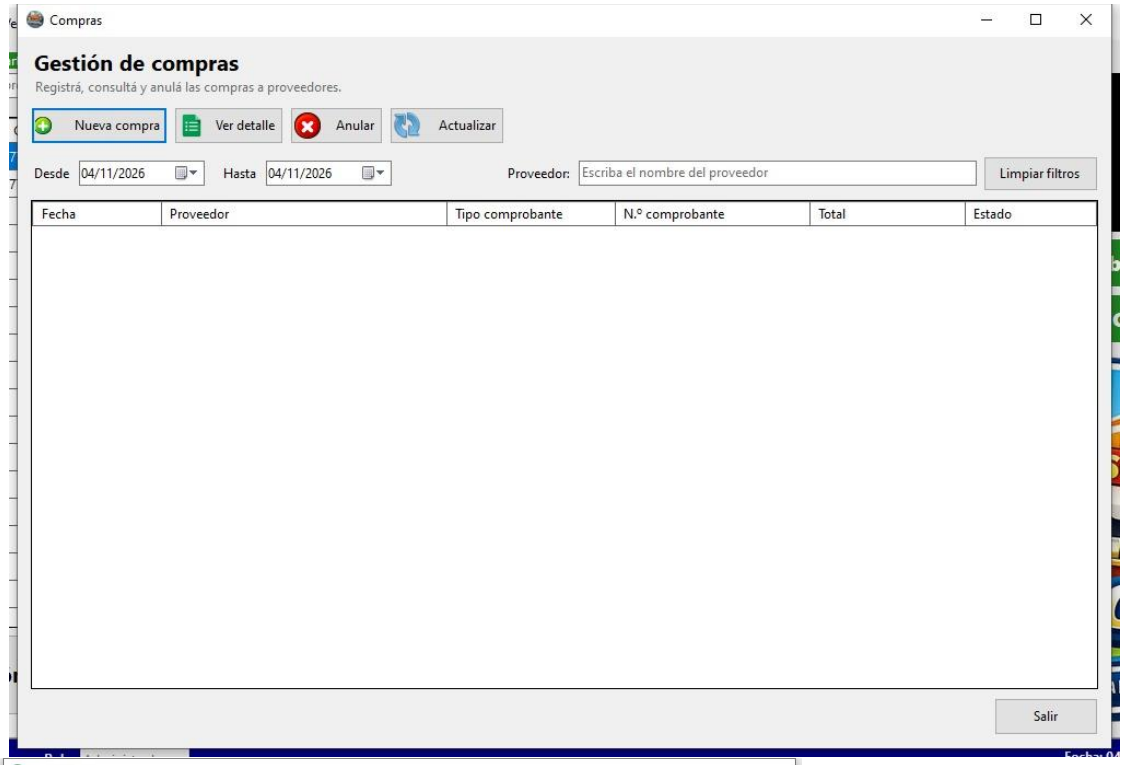
        if (!File.Exists(txtTraerArchivo.Text))
        {
            MessageBox.Show(ClsIdioma.T("El archivo seleccionado no existe."), ClsIdioma.T("Importación de artículos"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

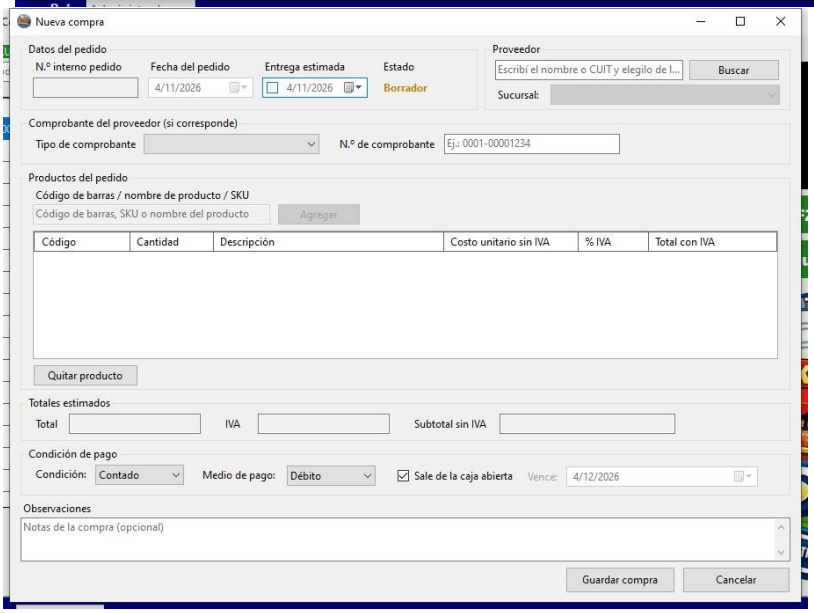
            return;
        }

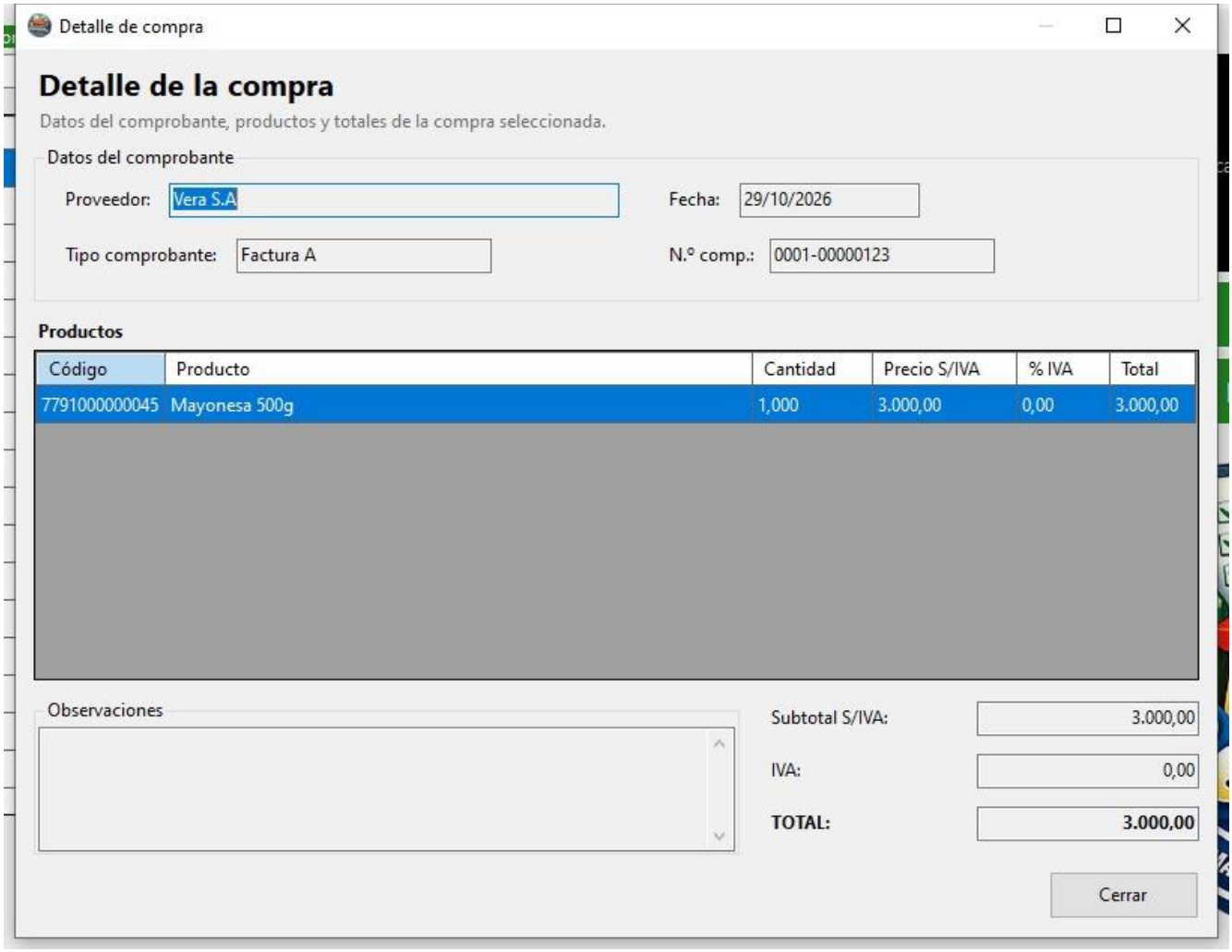
        ImportarArticulosDesdeExcel(txtTraerArchivo.Text);
    }
    #endregion

  }
}
```

4.7 Estructura principal de un caso de uso de negocio del software con interfaces y código: Compras


Compras
Gestión de compras
Registrá, consultá y anulá las compras a proveedores.
Nueva compra
Ver detalle
Anular
Actualizar
Desde 04/11/2026
Hasta 04/11/2026
Proveedor: Escriba el nombre del proveedor
Limpiar filtros
Fecha
Proveedor
Tipo comprobante
N.º comprobante
Total
Estado
Salir


Nueva compra
Datos del pedido
N.º interno pedido
Fecha del pedido
4/11/2026
Entrega estimada
4/11/2026
Estado
Borrador
Proveedor
Escribí el nombre o CUIT y elegilo de l...
Buscar
Sucursal:
Comprobante del proveedor (si corresponde)
Tipo de comprobante
N.º de comprobante
Ej.: 0001-00001234
Productos del pedido
Código de barras / nombre de producto / SKU
Código de barras, SKU o nombre del producto
Agregar
Código
Cantidad
Descripción
Costo unitario sin IVA
% IVA
Total con IVA
Quitar producto
Totales estimados
Total
IVA
Subtotal sin IVA
Condición de pago
Condición: Contado
Medio de pago: Débito
Sale de la caja abierta
Vence: 4/12/2026
Observaciones
Notas de la compra (opcional)
Guardar compra
Cancelar



Capa Vista // frmCompras

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using CapaLogica.NegocioLogica;
using CapaEntidades;


namespace CapaVista
{
    public partial class frmCompras : Form
    {
        private ClsCompraLogica logica = new ClsCompraLogica();
        private ClsLogicaProveedor logicaProveedor = new ClsLogicaProveedor();
        private List<string> nombresProveedores = new List<string>();
        private DataTable? comprasCargadas;
```

```
private string? proveedorSeleccionado = null;
private bool cambiandoTexto = false;

public frmCompras()
{
    InitializeComponent();
    ClsIdioma.Registrar(this);

    ConfigurarGrid();
    CargarNombresProveedores();
    CargarCompras();
    AplicarPermisos();

}

#region Metodos
private void AplicarPermisos()
{
    btnNuevacompra.Enabled = SesionUsuario.Permisos.Contains(310);
    btnAnular.Enabled = SesionUsuario.Permisos.Contains(330);
}
private void ConfigurarGrid()
{
    dataGridView1.AutoGenerateColumns = false;
    dataGridView1.ReadOnly = true;
    dataGridView1.SelectionMode = DataGridViewSelectionMode.FullRowSelect;
    dataGridView1.MultiSelect = false;
    dataGridView1.AllowUserToAddRows = false;
    dataGridView1.AllowUserToResizeColumns = false;
    dataGridView1.AllowUserToOrderColumns = false;
    dataGridView1.RowHeadersVisible = false;

    dataGridView1.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill;

    clnFecha.Width = 100;
    clnProveedor.Width = 220;
    clnTipoFactura.Width = 130;
    clnNCoprobante.Width = 150;
    clnTotal.Width = 120;
    clnEstado.Width = 100;
}
private void CargarCompras()
{
    try
    {
        DateTime desde = dateTimePicker1.Value.Date;
        DateTime hasta = dateTimePicker2.Value.Date;

        comprasCargadas = logica.ObtenerCompras(desde, hasta);
    }
    catch (Exception ex)
    {
        comprasCargadas = null;

        MessageBox.Show(ClsIdioma.T("No se pudieron cargar las compras.\n\n") + ex.Message, ClsIdioma.T("Error"),
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }

    MostrarCompras();
}
private void MostrarCompras()
```

```
    {
        dataGridView1.Rows.Clear();

        if (comprasCargadas == null)
        {
            return;
        }

        try
        {
            foreach (DataRow fila in comprasCargadas.Rows)
            {
                if (proveedorSeleccionado != null &&
                    !string.Equals(fila["Proveedor"].ToString(), proveedorSeleccionado, StringComparison.CurrentCultureIgnoreCase))
                {
                    continue;
                }

                int indice = dataGridView1.Rows.Add();

                dataGridView1.Rows[indice].Cells["clnFecha"].Value = Convert.ToDateTime(fila["Fecha"]).ToString("dd/MM/yyyy");
                dataGridView1.Rows[indice].Cells["clnProveedor"].Value = fila["Proveedor"].ToString();
                dataGridView1.Rows[indice].Cells["clnTipoFactura"].Value = fila["TipoComprobante"].ToString();
                dataGridView1.Rows[indice].Cells["clnNCoprobante"].Value = fila["NumeroComprobante"].ToString();
dataGridView1.Rows[indice].Cells["clnTotal"].Value = Convert.ToDecimal(fila["Total"]).ToString("N2");
                dataGridView1.Rows[indice].Cells["clnEstado"].Value = fila["Estado"].ToString();
                dataGridView1.Rows[indice].Tag = Convert.ToInt32(fila["IdCompra"]);
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show(ClsIdioma.T("No se pudieron mostrar las compras.\n\n") + ex.Message, ClsIdioma.T("Error"),
                MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
    private void CargarNombresProveedores()
    {
        try
        {
            nombresProveedores = logicaProveedor.ObtenerProveedores().AsEnumerable()
                .Select(f => f["RazonSocial"].ToString() ?? "")
                .Where(n => n != "")
                .Distinct(StringComparer.CurrentCultureIgnoreCase)
                .OrderBy(n => n)
                .ToList();
        }
        catch (Exception ex)
        {
            nombresProveedores = new List<string>();

            MessageBox.Show(ClsIdioma.T("No se pudieron cargar los proveedores para el buscador.\n\n") + ex.Message,
ClsIdioma.T("Error"),
                MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
    private void MostrarSugerencias(string texto)
    {
        lstProveedores.Items.Clear();

        foreach (string nombre in nombresProveedores.Where(n => n.StartsWith(texto, StringComparison.CurrentCultureIgnoreCase)))
        {
```

```
            lstProveedores.Items.Add(nombre);
        }

        lstProveedores.Visible = lstProveedores.Items.Count > 0;

        if (lstProveedores.Visible)
        {
            lstProveedores.SelectedIndex = 0;
            lstProveedores.BringToFront();
        }
    }
    private void SeleccionarProveedor()
    {
        if (!lstProveedores.Visible || lstProveedores.SelectedItem == null)
        {
            return;
        }

        proveedorSeleccionado = lstProveedores.SelectedItem.ToString() ?? "";

        cambiandoTexto = true;

        try
        {
            txtBuscarCompra.Text = proveedorSeleccionado;
            txtBuscarCompra.SelectionStart = txtBuscarCompra.Text.Length;
        }
        finally
        {
            cambiandoTexto = false;
        }

        lstProveedores.Visible = false;
        MostrarCompras();

        if (dataGridView1.Rows.Count == 0)
        {
            MessageBox.Show("\"" + proveedorSeleccionado + ClsIdioma.T("\" no tiene compras entre el ") +
                dateTimePicker1.Value.ToString("dd/MM/yyyy") + ClsIdioma.T(" y el ") + dateTimePicker2.Value.ToString("dd/MM/yyyy") +
                ClsIdioma.T(".\n\nAmplíe las fechas Desde y Hasta para ver sus compras."), ClsIdioma.T("Sin compras"),
                MessageBoxButtons.OK, MessageBoxIcon.Information);

            dateTimePicker1.Focus();
            return;
        }

        dataGridView1.Focus();
    }
    private void OcultarSugerencias()
    {
        lstProveedores.Items.Clear();
        lstProveedores.Visible = false;
    }


    #endregion



    #region Eventos
    private void btnNuevacompra_Click(object sender, EventArgs e)
```

```
{
    frmNuevaCompra Nuevacomp = new frmNuevaCompra(false);
    Nuevacomp.ShowDialog();

    CargarCompras();
}
private void btnDetalle_Click(object sender, EventArgs e)
{
    if (dataGridView1.SelectedRows.Count == 0)
    {
        MessageBox.Show(ClsIdioma.T("Seleccione una compra para ver el detalle."), ClsIdioma.T("Aviso"),
            MessageBoxButtons.OK, MessageBoxIcon.Warning);

        return;
    }

    DataGridViewRow fila = dataGridView1.SelectedRows[0];

    if (fila.Tag == null)
    {
        MessageBox.Show(ClsIdioma.T("No se pudo obtener el identificador de la compra."), ClsIdioma.T("Error"),
            MessageBoxButtons.OK, MessageBoxIcon.Error);

        return;
    }

    int idCompra = Convert.ToInt32(fila.Tag);

    using (frmDetalleCompra formulario = new frmDetalleCompra(idCompra))
    {
        formulario.ShowDialog();
    }

}
private void button5_Click(object sender, EventArgs e)
{
    CargarNombresProveedores();
    CargarCompras();
}
private void dateTimePicker1_ValueChanged(object sender, EventArgs e)
{
    if (dateTimePicker1.Value.Date > dateTimePicker2.Value.Date)
    {
        dateTimePicker2.Value = dateTimePicker1.Value.Date;
        return;
    }

    CargarCompras();
}
private void dateTimePicker2_ValueChanged(object sender, EventArgs e)
{
    if (dateTimePicker2.Value.Date < dateTimePicker1.Value.Date)
    {
        dateTimePicker1.Value = dateTimePicker2.Value.Date;
        return;
    }

    CargarCompras();
}
private void txtBuscarCompra_TextChanged(object sender, EventArgs e)
{
```

```
    if (cambiandoTexto)
    {
        return;
    }

    string texto = txtBuscarCompra.Text.Trim();

    if (proveedorSeleccionado != null)
    {
        proveedorSeleccionado = null;
        MostrarCompras();
    }

    if (texto == string.Empty)
    {
        OcultarSugerencias();
        return;
    }

    MostrarSugerencias(texto);
}
private void txtBuscarCompra_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        e.SuppressKeyPress = true;
        SeleccionarProveedor();
        return;
    }

    if (!lstProveedores.Visible || lstProveedores.Items.Count == 0)
    {
        return;
    }

    if (e.KeyCode == Keys.Down)
    {
        e.SuppressKeyPress = true;

        if (lstProveedores.SelectedIndex < lstProveedores.Items.Count - 1)
        {
            lstProveedores.SelectedIndex++;
        }
    }
    else if (e.KeyCode == Keys.Up)
    {
        e.SuppressKeyPress = true;

        if (lstProveedores.SelectedIndex > 0)
        {
            lstProveedores.SelectedIndex--;
        }
    }
    else if (e.KeyCode == Keys.Escape)
    {
        e.SuppressKeyPress = true;
        OcultarSugerencias();
    }
}
private void lstProveedores_DoubleClick(object sender, EventArgs e)
{
```

```
        SeleccionarProveedor();
    }
    private void lstProveedores_KeyDown(object sender, KeyEventArgs e)
    {
        if (e.KeyCode == Keys.Enter)
        {
            e.SuppressKeyPress = true;
            SeleccionarProveedor();
        }
        else if (e.KeyCode == Keys.Escape)
        {
            e.SuppressKeyPress = true;
            OcultarSugerencias();
            txtBuscarCompra.Focus();
        }
    }
    private void btnLimpiarFiltros_Click(object sender, EventArgs e)
    {
        proveedorSeleccionado = null;

        cambiandoTexto = true;

        try
        {
            txtBuscarCompra.Clear();
        }
        finally
        {
            cambiandoTexto = false;
        }

        OcultarSugerencias();

        dateTimePicker1.Value = DateTime.Today;
        dateTimePicker2.Value = DateTime.Today;

        CargarCompras();

        txtBuscarCompra.Focus();
    }
    private void btnAnular_Click(object sender, EventArgs e)
    {
        if (dataGridView1.SelectedRows.Count == 0)
        {
            MessageBox.Show(ClsIdioma.T("Seleccione una compra para anular."), ClsIdioma.T("Aviso"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return;
        }

        DataGridViewRow fila = dataGridView1.SelectedRows[0];

        if (fila.Tag == null)
        {
            MessageBox.Show(ClsIdioma.T("No se pudo obtener el identificador de la compra."), ClsIdioma.T("Error"),
                MessageBoxButtons.OK, MessageBoxIcon.Error);

            return;
        }

        string estado = fila.Cells["clnEstado"].Value?.ToString();
```

```
        if (estado == "Anulada")
        {
            MessageBox.Show(ClsIdioma.T("La compra seleccionada ya se encuentra anulada."), ClsIdioma.T("Aviso"),
                MessageBoxButtons.OK, MessageBoxIcon.Information);

            return;
        }

        int idCompra = Convert.ToInt32(fila.Tag);

        string motivo = Microsoft.VisualBasic.Interaction.InputBox(ClsIdioma.T("Ingrese el motivo de la anulación:"), ClsIdioma.T("Anular
compra"), "");

        if (string.IsNullOrWhiteSpace(motivo))
        {
            MessageBox.Show(ClsIdioma.T("Debe ingresar un motivo para anular la compra."), ClsIdioma.T("Aviso"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return;
        }

        DialogResult respuesta = MessageBox.Show(ClsIdioma.T("¿Está seguro de anular esta compra?\n\nMotivo: ") + motivo,
ClsIdioma.T("Confirmar anulación"),
            MessageBoxButtons.YesNo, MessageBoxIcon.Question);

        if (respuesta != DialogResult.Yes)
        {
            return;
        }

        try
        {
            logica.AnularCompra(idCompra, motivo);

            MessageBox.Show(ClsIdioma.T("La compra fue anulada correctamente."), ClsIdioma.T("Compra anulada"),
                MessageBoxButtons.OK, MessageBoxIcon.Information);

            CargarCompras();
        }
        catch (Exception ex)
        {
            MessageBox.Show(ClsIdioma.T("No se pudo anular la compra.\n\n") + ex.Message, ClsIdioma.T("Error"),
                MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }

    private void btnSalir_Click(object sender, EventArgs e)
    {
        Close();
    }


    #endregion


  }
}
```

Capa Vista // frmNuevaCompra

```
using CapaEntidades;
using CapaEntidades.NegocioEntidad;
using CapaEntidades.NegocioEntidades;
using CapaLogica.NegocioLogica;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Globalization;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace CapaVista
{
    public partial class frmNuevaCompra : Form
    {
        private ClsCompraLogica logica = new ClsCompraLogica();
        private ClsLogicaProveedor logicaProveedor = new ClsLogicaProveedor();
        private int idProveedor = 0;
        private int idSucursalActual = 0;
        private DataTable productosEncontrados;
        private DataTable proveedoresEncontrados;
        private ClsCajaLogica logicaCaja = new ClsCajaLogica();
        public frmNuevaCompra(bool mostrarChk)
        {
            InitializeComponent();
            ClsIdioma.Registrar(this);

            dtgNuevaCompra.Columns["Column1"].ReadOnly = true;
            dtgNuevaCompra.Columns["Column2"].ReadOnly = false;
            dtgNuevaCompra.Columns["Column3"].ReadOnly = true;
            dtgNuevaCompra.Columns["Column4"].ReadOnly = false;
            dtgNuevaCompra.Columns["Column4"].DefaultCellStyle.BackColor = Color.LightYellow;
            dtgNuevaCompra.Columns["Column5"].ReadOnly = true;
            dtgNuevaCompra.Columns["Column6"].ReadOnly = true;

            cmbSucursal.SelectionChangeCommitted += cmbSucursal_SelectionChangeCommitted;

            dtpFechaPedido.Value = DateTime.Today;
            dtpFechaPedido.Enabled = false;
            dtpEntrega.MinDate = DateTime.Today;
            dtpEntrega.Value = DateTime.Today;
            dtpEntrega.Checked = false;

            ConfigurarAutocompletadoProveedores();
            CargarOpcionesPago();
            ActualizarSucursalSeleccionada();
        }


        #region Metodos

        private void CargarOpcionesPago()
        {
            cmbCondicionPago.Items.Clear();
            cmbCondicionPago.Items.Add("Contado");
            cmbCondicionPago.Items.Add("Cuenta corriente");
            cmbCondicionPago.SelectedIndex = 0;
```

```
    try
    {
        DataTable medios = logicaCaja.ObtenerMetodosPago();

        foreach (DataRow fila in medios.Select("Nombre = 'Cuenta Corriente'"))
        {
            medios.Rows.Remove(fila);
        }

        cmbMedioPagoCompra.DataSource = medios;
        cmbMedioPagoCompra.DisplayMember = "Nombre";
        cmbMedioPagoCompra.ValueMember = "IdMetodoPago";
    ClsIdioma.TraducirDatosAlMostrar(cmbMedioPagoCompra);

        chkPagaDesdeCaja.Enabled = logicaCaja.HayCajaAbierta();
        chkPagaDesdeCaja.Checked = chkPagaDesdeCaja.Enabled;
    }
    catch (Exception ex)
    {
        MessageBox.Show(ClsIdioma.T("No se pudieron cargar los medios de pago.\n\n") + ex.Message, ClsIdioma.T("Error"),
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }

    dtpVencimiento.Value = DateTime.Today.AddDays(30);
    ActualizarOpcionesPago();
}

private bool EsCuentaCorriente()
{
    return cmbCondicionPago.SelectedIndex == 1;
}

private void ActualizarOpcionesPago()
{
    bool ctaCte = EsCuentaCorriente();

    lblMedioPagoCompra.Enabled = !ctaCte;
    cmbMedioPagoCompra.Enabled = !ctaCte;
    chkPagaDesdeCaja.Visible = !ctaCte;
    lblVencimiento.Enabled = ctaCte;
    dtpVencimiento.Enabled = ctaCte;
}


private void CalcularTotales()
{
    decimal subtotalSinIVA = 0;
    decimal ivaTotal = 0;

    foreach (DataGridViewRow fila in dtgNuevaCompra.Rows)
    {
        if (fila.IsNewRow)
        {
            continue;
        }

        if (fila.Cells["Column2"].Value == null || fila.Cells["Column4"].Value == null || fila.Cells["Column5"].Value == null)
        {
            continue;
        }
```

```
        decimal cantidad = Convert.ToDecimal(fila.Cells["Column2"].Value);
        decimal precio = Convert.ToDecimal(fila.Cells["Column4"].Value);
        decimal porcentajeIVA = Convert.ToDecimal(fila.Cells["Column5"].Value);

        decimal subtotal = Math.Round(cantidad * precio, 2, MidpointRounding.AwayFromZero);
        decimal iva = Math.Round(subtotal * porcentajeIVA / 100, 2, MidpointRounding.AwayFromZero);

        subtotalSinIVA += subtotal;
        ivaTotal += iva;
    }

    decimal total = subtotalSinIVA + ivaTotal;

    txtSubsinImpuesto.Text = subtotalSinIVA.ToString("N2");
    txtIva.Text = ivaTotal.ToString("N2");
    txtTotal.Text = total.ToString("N2");
}
private void ConfigurarAutocompletadoProveedores()
{
    txtProveedor.AutoCompleteMode = AutoCompleteMode.None;
    txtProveedor.AutoCompleteSource = AutoCompleteSource.None;

    lstProveedores.Visible = false;
}
private void SeleccionarProveedor()
{
    if (proveedoresEncontrados == null || proveedoresEncontrados.Rows.Count == 0)
    {
        return;
    }

    int indice = lstProveedores.SelectedIndex;

    if (indice < 0 || indice >= proveedoresEncontrados.Rows.Count)
    {
        indice = 0;
    }

    DataRow proveedor = proveedoresEncontrados.Rows[indice];

    string nombreProveedor = proveedor["Nombre"].ToString();
    int idSeleccionado = Convert.ToInt32(proveedor["IdProveedor"]);

    txtProveedor.Text = nombreProveedor;
    idProveedor = idSeleccionado;

    lstProveedores.Visible = false;
    lstProveedores.Items.Clear();
    proveedoresEncontrados = null;

    txtProveedor.SelectionStart = txtProveedor.Text.Length;
    CargarSucursales();
    txtCodigobarra.Focus();
}
private void CargarSucursales()
{
    cmbSucursal.DataSource = null;
    cmbSucursal.Enabled = false;
```

```
        if (idProveedor == 0)
        {
            return;
        }

        try
        {
            DataTable sucursales = logicaProveedor.ObtenerSucursalesPorProveedor(idProveedor);

            cmbSucursal.DisplayMember = "Sucursal";
            cmbSucursal.ValueMember = "IdSucursal";
            cmbSucursal.DataSource = sucursales;
            cmbSucursal.Enabled = sucursales.Rows.Count > 1;
            cmbSucursal.SelectedIndex = sucursales.Rows.Count > 0 ? 0 : -1;

            if (sucursales.Rows.Count == 0)
            {
                MessageBox.Show(ClsIdioma.T("El proveedor no tiene sucursales activas. Cargue una sucursal para poder comprarle."),
ClsIdioma.T("Aviso"),
                    MessageBoxButtons.OK, MessageBoxIcon.Warning);
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show(ClsIdioma.T("No se pudieron cargar las sucursales del proveedor.\n\n") + ex.Message, ClsIdioma.T("Error"),
                MessageBoxButtons.OK, MessageBoxIcon.Error);
        }

        ActualizarSucursalSeleccionada();
    }
    private int ObtenerIdSucursal()
    {
        return cmbSucursal.SelectedValue is int idSucursal ? idSucursal : 0;
    }
    private void ActualizarSucursalSeleccionada()
    {
        int idSucursal = ObtenerIdSucursal();

        txtCodigobarra.Enabled = idSucursal > 0;
        btnCargarProducto.Enabled = idSucursal > 0;

        if (idSucursal == 0 || idSucursal == idSucursalActual)
        {
            return;
        }

        bool hayProductos = dtgNuevaCompra.Rows.Cast<DataGridViewRow>().Any(f => !f.IsNewRow);

        if (hayProductos && idSucursalActual != 0)
        {
            dtgNuevaCompra.Rows.Clear();
            CalcularTotales();

            MessageBox.Show(ClsIdioma.T("Cambió el proveedor o la sucursal: se quitaron los productos cargados, porque sus costos
dependen de la sucursal."),
                ClsIdioma.T("Aviso"), MessageBoxButtons.OK, MessageBoxIcon.Information);
        }

        idSucursalActual = idSucursal;
        txtCodigobarra.Clear();
    }
```

```
        private static bool TryLeerDecimal(object valor, out decimal numero)
        {
            if (valor is decimal valorDecimal)
            {
                numero = valorDecimal;
                return true;
            }

            return decimal.TryParse(Convert.ToString(valor), NumberStyles.Number, CultureInfo.CurrentCulture, out numero);
        }

        #endregion

        #region Eventos
        private void btnCacelar_Click(object sender, EventArgs e)
        {
            bool hayProductos = dtgNuevaCompra.Rows.Cast<DataGridViewRow>().Any(f => !f.IsNewRow);

            if (hayProductos && MessageBox.Show(ClsIdioma.T("Hay productos cargados que no se guardaron.\n\n¿Querés salir igual?"),
ClsIdioma.T("Salir sin guardar"),
                MessageBoxButtons.YesNo, MessageBoxIcon.Question) != DialogResult.Yes)
            {
                return;
            }

            Close();
        }
        private void btnCargarProducto_Click(object sender, EventArgs e)
        {
            if (productosEncontrados == null || lstProductos.SelectedIndex < 0 || lstProductos.SelectedIndex >=
productosEncontrados.Rows.Count)
            {
                MessageBox.Show(ClsIdioma.T("Seleccione un producto de la lista."), ClsIdioma.T("Aviso"),
                    MessageBoxButtons.OK, MessageBoxIcon.Warning);

                txtCodigobarra.Focus();
                return;
            }

            DataRow producto = productosEncontrados.Rows[lstProductos.SelectedIndex];

            int idProducto = Convert.ToInt32(producto["IdProducto"]);
            string codigoBarra = producto["CodigoBarra"].ToString();
            string nombre = producto["Nombre"].ToString();
            decimal? precio = producto["PrecioCompra"] == DBNull.Value ? (decimal?)null : Convert.ToDecimal(producto["PrecioCompra"]);
            decimal iva = Convert.ToDecimal(producto["Alicuota"]);

            DataGridViewRow? filaExistente = dtgNuevaCompra.Rows.Cast<DataGridViewRow>()
                .FirstOrDefault(f => !f.IsNewRow && f.Tag is int id && id == idProducto);

            if (filaExistente != null)
            {
                TryLeerDecimal(filaExistente.Cells["Column2"].Value, out decimal cantidadActual);
                filaExistente.Cells["Column2"].Value = cantidadActual + 1;

                dataGridView1_CellEndEdit_1(dtgNuevaCompra, new DataGridViewCellEventArgs(filaExistente.Cells["Column2"].ColumnIndex,
filaExistente.Index));

                txtCodigobarra.Clear();
                lstProductos.Items.Clear();
                lstProductos.Visible = false;
```

```
            productosEncontrados = null;

            dtgNuevaCompra.ClearSelection();
            filaExistente.Selected = true;

            txtCodigobarra.Focus();
            return;
        }

        int indice = dtgNuevaCompra.Rows.Add();

        DataGridViewRow fila = dtgNuevaCompra.Rows[indice];

        fila.Cells["Column1"].Value = codigoBarra;
        fila.Cells["Column2"].Value = 1;
        fila.Cells["Column3"].Value = nombre;
        fila.Cells["Column4"].Value = precio;
        fila.Cells["Column5"].Value = iva;

        if (precio.HasValue)
        {
            decimal subtotal = Math.Round(precio.Value, 2, MidpointRounding.AwayFromZero);
            decimal ivaCalculado = Math.Round(subtotal * iva / 100, 2, MidpointRounding.AwayFromZero);
            fila.Cells["Column6"].Value = subtotal + ivaCalculado;
        }

        fila.Tag = idProducto;

        CalcularTotales();

        txtCodigobarra.Clear();
        lstProductos.Items.Clear();
        lstProductos.Visible = false;
        productosEncontrados = null;

        if (!precio.HasValue)
        {
            MessageBox.Show("\"" + nombre + ClsIdioma.T("\" no tiene costo cargado para esta sucursal.\n\nIngréselo en la columna \"Costo
unitario sin IVA\"; al guardar la compra quedará registrado en la sucursal."),
                ClsIdioma.T("Costo pendiente"), MessageBoxButtons.OK, MessageBoxIcon.Information);

            dtgNuevaCompra.Focus();
            dtgNuevaCompra.CurrentCell = fila.Cells["Column4"];
            dtgNuevaCompra.BeginEdit(true);
            return;
        }

        txtCodigobarra.Focus();
    }
    private void dataGridView1_CellEndEdit_1(object sender, DataGridViewCellEventArgs e)
    {
        if (e.RowIndex < 0)
        {
            return;
        }

        DataGridViewRow fila = dtgNuevaCompra.Rows[e.RowIndex];

        if (fila.Cells["Column5"].Value == null)
        {
            return;
```

```
        }

        if (!TryLeerDecimal(fila.Cells["Column2"].Value, out decimal cantidad))
        {
            cantidad = 0;
            fila.Cells["Column2"].Value = 0;
        }

        if (!TryLeerDecimal(fila.Cells["Column4"].Value, out decimal precio) || precio < 0)
        {
            fila.Cells["Column4"].Value = null;
            fila.Cells["Column6"].Value = null;
            CalcularTotales();
            return;
        }

        fila.Cells["Column4"].Value = precio;
        decimal iva = Convert.ToDecimal(fila.Cells["Column5"].Value);

        decimal subtotal = Math.Round(cantidad * precio, 2, MidpointRounding.AwayFromZero);
        decimal ivaCalculado = Math.Round(subtotal * iva / 100, 2, MidpointRounding.AwayFromZero);
        decimal total = subtotal + ivaCalculado;

        fila.Cells["Column6"].Value = total;

        CalcularTotales();
    }
    private void btnEliminarCompra_Click(object sender, EventArgs e)
    {
        if (dtgNuevaCompra.SelectedRows.Count == 0)
        {
            MessageBox.Show(ClsIdioma.T("Seleccione un producto para eliminar."), ClsIdioma.T("Aviso"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return;
        }

        DialogResult respuesta = MessageBox.Show(ClsIdioma.T("¿Está seguro de eliminar el producto seleccionado?"),
ClsIdioma.T("Confirmar eliminación"),
            MessageBoxButtons.YesNo, MessageBoxIcon.Question);

        if (respuesta != DialogResult.Yes)
        {
            return;
        }

        dtgNuevaCompra.Rows.RemoveAt(dtgNuevaCompra.SelectedRows[0].Index);

        CalcularTotales();
    }
    private void btnGuardar_Click(object sender, EventArgs e)
    {
        if (idProveedor == 0)
        {
            MessageBox.Show(ClsIdioma.T("Debe seleccionar un proveedor."), ClsIdioma.T("Aviso"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            txtProveedor.Focus();
            return;
        }
```

```
        if (ObtenerIdSucursal() == 0 || ObtenerIdSucursal() != idSucursalActual)
        {
            MessageBox.Show(ClsIdioma.T("Debe seleccionar la sucursal del proveedor."), ClsIdioma.T("Aviso"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            cmbSucursal.Focus();
            return;
        }

        if (string.IsNullOrWhiteSpace(cmbTipoComprobante.Text))
        {
            MessageBox.Show(ClsIdioma.T("Debe seleccionar un tipo de comprobante."), ClsIdioma.T("Aviso"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            cmbTipoComprobante.Focus();
            return;
        }

        dtpFechaPedido.Value = DateTime.Today;

        if (dtpEntrega.Checked && dtpEntrega.Value.Date < DateTime.Today)
        {
            MessageBox.Show(ClsIdioma.T("La entrega estimada no puede ser anterior a hoy."), ClsIdioma.T("Aviso"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            dtpEntrega.Focus();
            return;
        }

        if (dtgNuevaCompra.Rows.Cast<DataGridViewRow>().All(fila => fila.IsNewRow))
        {
            MessageBox.Show(ClsIdioma.T("Debe agregar al menos un producto a la compra."), ClsIdioma.T("Aviso"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            txtCodigobarra.Focus();
            return;
        }

        try
        {
            if (logica.ExisteComprobante(idProveedor, cmbTipoComprobante.Text, txtNumeroComprobante.Text))
            {
                MessageBox.Show(ClsIdioma.T("Ya está cargada la ") + cmbTipoComprobante.Text.Trim() + " N.º " +
txtNumeroComprobante.Text.Trim() +
                    ClsIdioma.T(" de ") + txtProveedor.Text.Trim() + ClsIdioma.T(".\n\nRevise el número o anule la compra anterior si estaba mal
cargada."),
                    ClsIdioma.T("Comprobante repetido"), MessageBoxButtons.OK, MessageBoxIcon.Warning);

                txtNumeroComprobante.Focus();
                return;
            }

            ClsCompraEntidad compra = new ClsCompraEntidad();

            compra.IdProveedor = idProveedor;
            compra.IdSucursal = ObtenerIdSucursal();
            compra.Fecha = DateTime.Now;
            compra.FechaEntregaEstimada = dtpEntrega.Checked ? dtpEntrega.Value.Date : (DateTime?)null;
            compra.TipoComprobante = cmbTipoComprobante.Text.Trim();
            compra.NumeroComprobante = string.IsNullOrWhiteSpace(txtNumeroComprobante.Text) ? null :
txtNumeroComprobante.Text.Trim();
```

```
compra.SubtotalSinIVA = Convert.ToDecimal(txtSubsinImpuesto.Text);
compra.IVA = Convert.ToDecimal(txtIva.Text);
compra.Total = Convert.ToDecimal(txtTotal.Text);
compra.Observaciones = txtObservaciones.Text.Trim();


compra.IdUsuario = SesionUsuario.IdUsuario;

if (EsCuentaCorriente())
{
    if (dtpVencimiento.Value.Date < DateTime.Today)
    {
        MessageBox.Show(ClsIdioma.T("La fecha de vencimiento no puede ser anterior a la fecha de la compra."),
            ClsIdioma.T("Aviso"), MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    compra.CondicionPago = "CTA_CTE";
    compra.FechaVencimiento = dtpVencimiento.Value.Date;
    compra.IdMetodoPago = null;
    compra.IdSesionCaja = null;
}
else
{
    if (cmbMedioPagoCompra.SelectedValue == null)
    {
        MessageBox.Show(ClsIdioma.T("Seleccione el medio de pago de la compra."), ClsIdioma.T("Aviso"),
            MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    compra.CondicionPago = "CONTADO";
    compra.FechaVencimiento = null;
    compra.IdMetodoPago = Convert.ToInt32(cmbMedioPagoCompra.SelectedValue);

    if (chkPagaDesdeCaja.Enabled && chkPagaDesdeCaja.Checked)
    {
        ClsSesionCajaEntidad sesion = logicaCaja.ObtenerSesionAbierta();
        compra.IdSesionCaja = sesion != null ? sesion.IdSesionCaja : (int?)null;
    }
}

compra.Detalles = new List<ClsDetalleCompraEntidad>();
compra.Detalles = new List<ClsDetalleCompraEntidad>();

foreach (DataGridViewRow fila in dtgNuevaCompra.Rows)
{
    if (fila.IsNewRow)
    {
        continue;
    }

    if (fila.Tag == null)
    {
        MessageBox.Show(ClsIdioma.T("Uno de los productos no tiene un identificador válido."), ClsIdioma.T("Error"),
            MessageBoxButtons.OK, MessageBoxIcon.Error);

        return;
    }

    if (fila.Cells["Column4"].Value == null)
```

```
        {
            MessageBox.Show(ClsIdioma.T("Ingrese el costo de \"") + fila.Cells["Column3"].Value + "\".", ClsIdioma.T("Aviso"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            dtgNuevaCompra.CurrentCell = fila.Cells["Column4"];
            return;
        }

        if (fila.Cells["Column2"].Value == null || fila.Cells["Column5"].Value == null)
        {
            MessageBox.Show(ClsIdioma.T("Complete la cantidad, el precio y el IVA del producto."), ClsIdioma.T("Aviso"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return;
        }

        if (!decimal.TryParse(fila.Cells["Column2"].Value.ToString(), out decimal cantidad) || cantidad <= 0)
        {
            MessageBox.Show(ClsIdioma.T("La cantidad debe ser mayor que cero."), ClsIdioma.T("Aviso"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return;
        }

        if (!decimal.TryParse(fila.Cells["Column4"].Value.ToString(), out decimal precio) || precio < 0)
        {
            MessageBox.Show(ClsIdioma.T("El precio sin IVA no puede ser negativo."), ClsIdioma.T("Aviso"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return;
        }

        if (!decimal.TryParse(fila.Cells["Column5"].Value.ToString(), out decimal porcentajeIVA) || porcentajeIVA < 0 || porcentajeIVA >
100)
        {
            MessageBox.Show(ClsIdioma.T("El porcentaje de IVA debe estar entre 0 y 100."), ClsIdioma.T("Aviso"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            return;
        }

        ClsDetalleCompraEntidad detalle = new ClsDetalleCompraEntidad();

        detalle.IdProducto = Convert.ToInt32(fila.Tag);
        detalle.Cantidad = decimal.Round(cantidad, 3, MidpointRounding.AwayFromZero);
        detalle.PrecioSinIVA = decimal.Round(precio, 2, MidpointRounding.AwayFromZero);
        detalle.PorcentajeIVA = decimal.Round(porcentajeIVA, 2, MidpointRounding.AwayFromZero);

        decimal subtotalDetalle = Math.Round(detalle.Cantidad * detalle.PrecioSinIVA, 2, MidpointRounding.AwayFromZero);
        decimal ivaDetalle = Math.Round(subtotalDetalle * detalle.PorcentajeIVA / 100, 2, MidpointRounding.AwayFromZero);

        detalle.Total = subtotalDetalle + ivaDetalle;

        compra.Detalles.Add(detalle);
    }

    int idCompra = logica.GuardarCompra(compra);

    MessageBox.Show(ClsIdioma.T("La compra se guardó correctamente.\n\nN.º de compra: ") + idCompra,
        ClsIdioma.T("Compra guardada"), MessageBoxButtons.OK, MessageBoxIcon.Information);
```

```
            this.DialogResult = DialogResult.OK;
            this.Close();
        }
        catch (Exception ex)
        {
            MessageBox.Show(ClsIdioma.T("No se pudo guardar la compra.\n\n") + ex.Message, ClsIdioma.T("Error"),
                MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
    private void btnBuscar_Click(object sender, EventArgs e)
    {
        string texto = txtProveedor.Text.Trim();

        if (string.IsNullOrWhiteSpace(texto))
        {
            MessageBox.Show(ClsIdioma.T("Ingrese el nombre o CUIT del proveedor."), ClsIdioma.T("Aviso"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            txtProveedor.Focus();
            return;
        }

        DataTable tabla = logicaProveedor.BuscarProveedor(texto);

        if (tabla.Rows.Count == 0)
        {
            MessageBox.Show(ClsIdioma.T("No se encontró ningún proveedor activo."), ClsIdioma.T("Proveedor no encontrado"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            txtProveedor.Focus();
            return;
        }

        if (tabla.Rows.Count > 1)
        {
            MessageBox.Show(ClsIdioma.T("Se encontraron varios proveedores. Ingrese un nombre o CUIT más específico."),
                ClsIdioma.T("Varios proveedores"), MessageBoxButtons.OK, MessageBoxIcon.Information);

            return;
        }

        int idEncontrado = Convert.ToInt32(tabla.Rows[0]["IdProveedor"]);
        txtProveedor.Text = tabla.Rows[0]["Nombre"].ToString();

        idProveedor = idEncontrado;
        lstProveedores.Visible = false;
        CargarSucursales();

    }
    private void txtCodigobarra_TextChanged(object sender, EventArgs e)
    {
        string texto = txtCodigobarra.Text.Trim();

        lstProductos.Items.Clear();
        lstProductos.Visible = false;

        if (string.IsNullOrWhiteSpace(texto))
        {
            productosEncontrados = null;
            return;
        }
```

```
    if (ObtenerIdSucursal() == 0)
    {
        productosEncontrados = null;
        return;
    }

    try
    {
        productosEncontrados = logica.BuscarProductoCompra(texto, ObtenerIdSucursal());
    }
    catch (Exception ex)
    {
        productosEncontrados = null;
        MessageBox.Show(ClsIdioma.T("No se pudieron buscar los productos.\n\n") + ex.Message, ClsIdioma.T("Error"),
            MessageBoxButtons.OK, MessageBoxIcon.Error);
        return;
    }

    if (productosEncontrados.Rows.Count == 0)
    {
        return;
    }

    foreach (DataRow fila in productosEncontrados.Rows)
    {
        lstProductos.Items.Add(fila["CodigoBarra"].ToString() + " - " + fila["Nombre"].ToString());
    }

    lstProductos.Visible = true;
    lstProductos.BringToFront();

    if (lstProductos.Items.Count > 0)
    {
        lstProductos.SelectedIndex = 0;
    }

    txtCodigobarra.Focus();
    txtCodigobarra.SelectionStart = txtCodigobarra.Text.Length;
}
private void txtCodigobarra_KeyDown(object sender, KeyEventArgs e)
{
    if (!lstProductos.Visible || lstProductos.Items.Count == 0)
    {
        return;
    }

    if (e.KeyCode == Keys.Down)
    {
        e.SuppressKeyPress = true;

        if (lstProductos.SelectedIndex < lstProductos.Items.Count - 1)
        {
            lstProductos.SelectedIndex++;
        }
    }
    else if (e.KeyCode == Keys.Up)
    {
        e.SuppressKeyPress = true;

        if (lstProductos.SelectedIndex > 0)
```

```
        {
            lstProductos.SelectedIndex--;
        }
    }
    else if (e.KeyCode == Keys.Enter)
    {
        e.SuppressKeyPress = true;
        btnCargarProducto_Click(sender, e);
    }
    else if (e.KeyCode == Keys.Escape)
    {
        e.SuppressKeyPress = true;

        lstProductos.Items.Clear();
        lstProductos.Visible = false;
        productosEncontrados = null;
    }
}
private void lstProductos_DoubleClick(object sender, EventArgs e)
{
    btnCargarProducto_Click(sender, e);
}
private void txtProveedor_TextChanged(object sender, EventArgs e)
{
    string texto = txtProveedor.Text.Trim();

    idProveedor = 0;
    cmbSucursal.DataSource = null;
    cmbSucursal.Enabled = false;
    ActualizarSucursalSeleccionada();
    lstProveedores.Items.Clear();
    lstProveedores.Visible = false;

    if (string.IsNullOrWhiteSpace(texto))
    {
        proveedoresEncontrados = null;
        return;
    }

    proveedoresEncontrados = logicaProveedor.BuscarProveedor(texto);

    if (proveedoresEncontrados == null || proveedoresEncontrados.Rows.Count == 0)
    {
        proveedoresEncontrados = null;
        return;
    }

    foreach (DataRow fila in proveedoresEncontrados.Rows)
    {
        lstProveedores.Items.Add(fila["Nombre"].ToString() + " - " + fila["IdProveedor"].ToString());
    }

    lstProveedores.Visible = true;
    lstProveedores.BringToFront();

    if (lstProveedores.Items.Count > 0)
        lstProveedores.SelectedIndex = 0;

}
private void txtProveedor_KeyDown(object sender, KeyEventArgs e)
{
```

```
    if (e.KeyCode == Keys.Enter)
    {
        e.SuppressKeyPress = true;
        e.Handled = true;

        if (proveedoresEncontrados != null && proveedoresEncontrados.Rows.Count > 0)
        {
            SeleccionarProveedor();
        }
        else
        {
            btnBuscar_Click(sender, e);
        }

        return;
    }

    if (!lstProveedores.Visible || lstProveedores.Items.Count == 0)
    {
        return;
    }

    if (e.KeyCode == Keys.Down)
    {
        e.SuppressKeyPress = true;

        if (lstProveedores.SelectedIndex < lstProveedores.Items.Count - 1)
        {
            lstProveedores.SelectedIndex++;
        }
    }
    else if (e.KeyCode == Keys.Up)
    {
        e.SuppressKeyPress = true;

        if (lstProveedores.SelectedIndex > 0)
        {
            lstProveedores.SelectedIndex--;
        }
    }
    else if (e.KeyCode == Keys.Escape)
    {
        e.SuppressKeyPress = true;
        lstProveedores.Visible = false;
    }
}
private void txtProveedor_DoubleClick(object sender, EventArgs e)
{
    SeleccionarProveedor();
}
private void lstProveedores_DoubleClick(object sender, EventArgs e)
{
    SeleccionarProveedor();
}
private void lstProveedores_KeyDown(object sender, KeyEventArgs e)
{
    if (e.KeyCode == Keys.Enter)
    {
        e.SuppressKeyPress = true;
        SeleccionarProveedor();
    }
```

```
            else if (e.KeyCode == Keys.Escape)
            {
                e.SuppressKeyPress = true;
                lstProveedores.Visible = false;
                txtProveedor.Focus();
            }
        }

        #endregion

        private void cmbSucursal_SelectionChangeCommitted(object sender, EventArgs e)
        {
            ActualizarSucursalSeleccionada();
        }
        private void cmbCondicionPago_SelectedIndexChanged(object sender, EventArgs e)
        {
            ActualizarOpcionesPago();
        }
    }
}
```

Capa Vista // frmDetalleCompra

```
using System;
using System.Data;
using System.Windows.Forms;
using CapaLogica.NegocioLogica;

namespace CapaVista
{
    public partial class frmDetalleCompra : Form
    {
        private ClsCompraLogica logica = new ClsCompraLogica();

        private int idCompra;
        private bool sinDetalle = false;

        public frmDetalleCompra(int idCompra)
        {
            InitializeComponent();
            ClsIdioma.Registrar(this);
            ClsGrilla.AjustarAlContenido(dgvDetalle, "clnProducto");

            this.idCompra = idCompra;

            CargarDetalle();
```

```
    this.Load += frmDetalleCompra_Load;
}
private void frmDetalleCompra_Load(object? sender, EventArgs e)
{
    if (sinDetalle)
    {
        BeginInvoke(new Action(Close));
    }
}

#region Metodos
private void CargarDetalle()
{
    try
    {
        DataTable tabla = logica.ObtenerDetalleCompra(idCompra);

        if (tabla.Rows.Count == 0)
        {
            MessageBox.Show(ClsIdioma.T("No se encontró el detalle de la compra."), ClsIdioma.T("Aviso"),
                MessageBoxButtons.OK, MessageBoxIcon.Warning);

            sinDetalle = true;
            return;
        }

        DataRow primeraFila = tabla.Rows[0];

        txtProveedor.Text = primeraFila["Proveedor"].ToString();
        txtFecha.Text = Convert.ToDateTime(primeraFila["Fecha"]).ToString("dd/MM/yyyy");
        txtTipoComprobante.Text = primeraFila["TipoComprobante"].ToString();
        txtNumeroComprobante.Text = primeraFila["NumeroComprobante"].ToString();
        txtSubtotal.Text = Convert.ToDecimal(primeraFila["SubtotalSinIVA"]).ToString("N2");
        txtIVA.Text = Convert.ToDecimal(primeraFila["IVA"]).ToString("N2");
        txtTotal.Text = Convert.ToDecimal(primeraFila["Total"]).ToString("N2");
        txtObservaciones.Text = primeraFila["Observaciones"] == DBNull.Value ? "" : primeraFila["Observaciones"].ToString();
        dgvDetalle.Rows.Clear();

        foreach (DataRow fila in tabla.Rows)
        {
            int indice = dgvDetalle.Rows.Add();

            dgvDetalle.Rows[indice].Cells["clnCodigo"].Value = fila["CodigoBarra"].ToString();
            dgvDetalle.Rows[indice].Cells["clnProducto"].Value = fila["Producto"].ToString();
            dgvDetalle.Rows[indice].Cells["clnCantidad"].Value = Convert.ToDecimal(fila["Cantidad"]).ToString("N3");
            dgvDetalle.Rows[indice].Cells["clnPrecio"].Value = Convert.ToDecimal(fila["PrecioSinIVA"]).ToString("N2");
            dgvDetalle.Rows[indice].Cells["clnIVA"].Value = Convert.ToDecimal(fila["PorcentajeIVA"]).ToString("N2");
            dgvDetalle.Rows[indice].Cells["clnTotal"].Value = Convert.ToDecimal(fila["TotalDetalle"]).ToString("N2");
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show(ClsIdioma.T("No se pudo cargar el detalle de la compra.\n\n") + ex.Message, ClsIdioma.T("Error"),
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

#endregion
```

```
        #region Eventos
        private void btnCerrar_Click(object sender, EventArgs e)
        {
            Close();
        }
        #endregion



    }
}
```

Capa Lógica // ClsCompraLogica

```
using CapaDatos.NegocioDatos;
using CapaEntidades;
using CapaEntidades.NegocioEntidades;
using System;
using System.Data;
using System.Linq;

namespace CapaLogica.NegocioLogica
{
    public class ClsCompraLogica
    {
        private ClsCompraDatos datos = new ClsCompraDatos();

        public DataTable BuscarProducto(string texto)
        {
            return datos.BuscarProducto(texto);
        }
        public DataTable BuscarProductoCompra(string texto, int idSucursal)
        {
            if (idSucursal <= 0)
            {
                throw new InvalidOperationException("Seleccione la sucursal del proveedor antes de agregar productos.");
            }

            return datos.BuscarProductoCompra(texto, idSucursal);
        }
        public int GuardarCompra(ClsCompraEntidad compra)
        {
            int idUsuario = SesionUsuario.IdUsuario;

            if (idUsuario <= 0)
            {
                throw new InvalidOperationException("No hay una sesión de usuario válida. Volvé a iniciar sesión.");
            }

            return datos.GuardarCompra(compra, idUsuario);
        }
        public bool ExisteComprobante(int idProveedor, string tipoComprobante, string? numeroComprobante)
        {
            if (string.IsNullOrWhiteSpace(numeroComprobante))
            {
                return false;
            }

            return datos.ExisteComprobante(idProveedor, tipoComprobante, numeroComprobante);
        }
        public DataTable ObtenerCompras(DateTime desde, DateTime hasta)
        {
```

```
        return datos.ObtenerCompras(desde, hasta);
    }
    public DataTable ObtenerDetalleCompra(int idCompra)
    {
        return datos.ObtenerDetalleCompra(idCompra);
    }
    public void AnularCompra(int idCompra, string motivo)
    {
        int idUsuario = SesionUsuario.IdUsuario;

        if (idUsuario <= 0)
        {
            throw new InvalidOperationException("No hay una sesión de usuario válida. Volvé a iniciar sesión.");
        }

        datos.AnularCompra(idCompra, motivo, idUsuario);
    }
    public DataTable ObtenerComprasPendientesRecepcion(string buscar, DateTime? fecha)
    {
        return datos.ObtenerComprasPendientesRecepcion(buscar, fecha);
    }
    public int ConfirmarRecepcion(ClsRecepcionCompraEntidad recepcion)
    {
        int idUsuario = SesionUsuario.IdUsuario;

        if (idUsuario <= 0)
        {
            throw new InvalidOperationException("No hay una sesión de usuario válida. Volvé a iniciar sesión.");
        }

        if (recepcion == null || recepcion.IdCompra <= 0)
        {
            throw new InvalidOperationException("Seleccione una compra para recibir.");
        }

        if (recepcion.Detalles == null || recepcion.Detalles.Count == 0)
        {
            throw new InvalidOperationException("La recepción debe tener al menos un producto.");
        }

        if (recepcion.Detalles.Any(d => d.IdProducto <= 0 || d.CantidadRecibida < 0))
        {
            throw new InvalidOperationException("Las cantidades recibidas no pueden ser negativas.");
        }

        if (recepcion.Detalles.GroupBy(d => d.IdProducto).Any(g => g.Count() > 1))
        {
            throw new InvalidOperationException("Hay productos repetidos en la recepción.");
        }

        if (recepcion.TotalBoleta < 0)
        {
            throw new InvalidOperationException("El total de la boleta no puede ser negativo.");
        }

        bool hayDiferencias = recepcion.TotalBoleta != recepcion.TotalCompra || recepcion.Detalles.Any(d => d.CantidadRecibida !=
d.CantidadPedida);

        if (hayDiferencias && string.IsNullOrWhiteSpace(recepcion.Motivos))
        {
            throw new InvalidOperationException("Hay diferencias con la compra: seleccione al menos un motivo.");
```

```
        }

        if (recepcion.Motivos != null && recepcion.Motivos.Contains("Otro motivo") && string.IsNullOrWhiteSpace(recepcion.Observaciones))
        {
            throw new InvalidOperationException("Si elige \"Otro motivo\", detalle el motivo en las observaciones.");
        }

        if (recepcion.Observaciones != null && recepcion.Observaciones.Length > 500)
        {
            throw new InvalidOperationException("Las observaciones no pueden superar los 500 caracteres.");
        }

        return datos.ConfirmarRecepcion(recepcion, idUsuario);
    }
  }
}
```

Capa Datos // ClsCompraDatos

```
using CapaEntidades.NegocioEntidades;
using Microsoft.Data.SqlClient;
using SistemaKiosco;
using System;
using System.Data;
using System.Linq;

namespace CapaDatos.NegocioDatos
{
   public class ClsCompraDatos
   {
      private Conexion cn = new Conexion();

      public DataTable BuscarProducto(string texto)
      {
         using (SqlConnection conexion = cn.ObtenerConexion())
         using (SqlCommand cmd = new SqlCommand("sp_BuscarProducto", conexion))
         {
            cmd.CommandType = CommandType.StoredProcedure;
            cmd.Parameters.AddWithValue("@Texto", texto);

            using (SqlDataAdapter adaptador = new SqlDataAdapter(cmd))
            {
               DataTable tabla = new DataTable();
               adaptador.Fill(tabla);
               return tabla;
            }
         }
      }
      public DataTable BuscarProductoCompra(string texto, int idSucursal)
      {
         using (SqlConnection conexion = cn.ObtenerConexion())
         using (SqlCommand cmd = new SqlCommand("sp_BuscarProductoCompra", conexion))
         {
            cmd.CommandType = CommandType.StoredProcedure;
            cmd.Parameters.Add("@Texto", SqlDbType.NVarChar, 200).Value = texto;
            cmd.Parameters.Add("@IdSucursal", SqlDbType.Int).Value = idSucursal;

            using (SqlDataAdapter adaptador = new SqlDataAdapter(cmd))
            {
               DataTable tabla = new DataTable();
               adaptador.Fill(tabla);
               return tabla;
```

```
            }
        }
    }
    public bool ExisteComprobante(int idProveedor, string tipoComprobante, string numeroComprobante)
    {
        string query = @"SELECT COUNT(*)
            FROM dbo.Compra
            WHERE IdProveedor = @IdProveedor
              AND TipoComprobante = @TipoComprobante
              AND LTRIM(RTRIM(NumeroComprobante)) = @NumeroComprobante
              AND Anulada = 0";

        using (SqlConnection conexion = cn.ObtenerConexion())
        using (SqlCommand cmd = new SqlCommand(query, conexion))
        {
            cmd.Parameters.Add("@IdProveedor", SqlDbType.Int).Value = idProveedor;
            cmd.Parameters.Add("@TipoComprobante", SqlDbType.NVarChar, 50).Value = tipoComprobante.Trim();
            cmd.Parameters.Add("@NumeroComprobante", SqlDbType.NVarChar, 50).Value = numeroComprobante.Trim();

            conexion.Open();

            return Convert.ToInt32(cmd.ExecuteScalar()) > 0;
        }
    }
    public int GuardarCompra(ClsCompraEntidad compra, int idUsuario)
    {
        if (compra == null)
            throw new ArgumentNullException(nameof(compra));

        if (compra.Detalles == null || compra.Detalles.Count == 0)
            throw new InvalidOperationException("La compra debe tener al menos un producto.");

        if (idUsuario <= 0)
            throw new InvalidOperationException("No hay un usuario válido para registrar la compra.");

        if (compra.Detalles.Any(d => d.IdProducto <= 0 || d.Cantidad <= 0))
        {
            throw new InvalidOperationException("La compra contiene productos o cantidades inválidas.");
        }

        using (SqlConnection conexion = cn.ObtenerConexion())
        {
            conexion.Open();

            using (SqlTransaction transaccion = conexion.BeginTransaction())
            {
                try
                {
                    int idCompra;

                    using (SqlCommand cmd = new SqlCommand("sp_GuardarCompra", conexion, transaccion))
                    {
                        cmd.CommandType = CommandType.StoredProcedure;
                        cmd.Parameters.AddWithValue("@IdProveedor", compra.IdProveedor);
                        cmd.Parameters.AddWithValue("@Fecha", compra.Fecha);
                        cmd.Parameters.AddWithValue("@TipoComprobante", compra.TipoComprobante);
                        cmd.Parameters.AddWithValue("@NumeroComprobante", string.IsNullOrWhiteSpace(compra.NumeroComprobante) ?
(object)DBNull.Value : compra.NumeroComprobante);

                        SqlParameter parametroSubtotal = cmd.Parameters.Add("@SubtotalSinIVA", SqlDbType.Decimal);
                        parametroSubtotal.Precision = 18;
```

```
                parametroSubtotal.Scale = 2;
                parametroSubtotal.Value = decimal.Round(compra.SubtotalSinIVA, 2, MidpointRounding.AwayFromZero);

                SqlParameter parametroIVA = cmd.Parameters.Add("@IVA", SqlDbType.Decimal);
                parametroIVA.Precision = 18;
                parametroIVA.Scale = 2;
                parametroIVA.Value = decimal.Round(compra.IVA, 2, MidpointRounding.AwayFromZero);

                SqlParameter parametroTotal = cmd.Parameters.Add("@Total", SqlDbType.Decimal);
                parametroTotal.Precision = 18;
                parametroTotal.Scale = 2;
                parametroTotal.Value = decimal.Round(compra.Total, 2, MidpointRounding.AwayFromZero);

                cmd.Parameters.AddWithValue("@Observaciones", string.IsNullOrWhiteSpace(compra.Observaciones) ?
(object)DBNull.Value : compra.Observaciones);

                cmd.Parameters.AddWithValue("@IdUsuario",
                    compra.IdUsuario.HasValue ? (object)compra.IdUsuario.Value : DBNull.Value);
                cmd.Parameters.AddWithValue("@CondicionPago",
                    string.IsNullOrWhiteSpace(compra.CondicionPago) ? "CONTADO" : compra.CondicionPago);
                cmd.Parameters.AddWithValue("@FechaVencimiento",
                    compra.FechaVencimiento.HasValue ? (object)compra.FechaVencimiento.Value.Date : DBNull.Value);
                cmd.Parameters.AddWithValue("@IdMetodoPago",
                    compra.IdMetodoPago.HasValue ? (object)compra.IdMetodoPago.Value : DBNull.Value);
                cmd.Parameters.AddWithValue("@IdSesionCaja",
                    compra.IdSesionCaja.HasValue ? (object)compra.IdSesionCaja.Value : DBNull.Value);
                cmd.Parameters.AddWithValue("@IdSucursal",
                    compra.IdSucursal.HasValue ? (object)compra.IdSucursal.Value : DBNull.Value);
                cmd.Parameters.Add("@FechaEntregaEstimada", SqlDbType.Date).Value =
                    compra.FechaEntregaEstimada.HasValue ? (object)compra.FechaEntregaEstimada.Value.Date : DBNull.Value;

                DataTable tablaDetalles = new DataTable();
                tablaDetalles.Columns.Add("IdProducto", typeof(int));
                tablaDetalles.Columns.Add("Cantidad", typeof(decimal));
                tablaDetalles.Columns.Add("PrecioSinIVA", typeof(decimal));
                tablaDetalles.Columns.Add("PorcentajeIVA", typeof(decimal));
                tablaDetalles.Columns.Add("Total", typeof(decimal));

                foreach (ClsDetalleCompraEntidad detalle in compra.Detalles)
                {
                    decimal cantidadDetalle = decimal.Round(detalle.Cantidad, 3, MidpointRounding.AwayFromZero);
                    decimal precioDetalle = decimal.Round(detalle.PrecioSinIVA, 2, MidpointRounding.AwayFromZero);
                    decimal porcentajeIVA = decimal.Round(detalle.PorcentajeIVA, 2, MidpointRounding.AwayFromZero);
                    decimal subtotalDetalle = decimal.Round(cantidadDetalle * precioDetalle, 2, MidpointRounding.AwayFromZero);
                    decimal ivaDetalle = decimal.Round(subtotalDetalle * porcentajeIVA / 100, 2, MidpointRounding.AwayFromZero);
                    decimal totalDetalle = subtotalDetalle + ivaDetalle;

                    tablaDetalles.Rows.Add(detalle.IdProducto, cantidadDetalle, precioDetalle, porcentajeIVA, totalDetalle);
                }

                SqlParameter parametroDetalles = cmd.Parameters.AddWithValue("@Detalles", tablaDetalles);
                parametroDetalles.SqlDbType = SqlDbType.Structured;
                parametroDetalles.TypeName = "dbo.TipoDetalleCompra";

                idCompra = Convert.ToInt32(cmd.ExecuteScalar());
            }

            foreach (var grupo in compra.Detalles.GroupBy(d => d.IdProducto))
            {
                decimal cantidad = decimal.Round(grupo.Sum(d => d.Cantidad), 3, MidpointRounding.AwayFromZero);
```

```
                using (SqlCommand cmdStock = new SqlCommand(@"
                    UPDATE dbo.Stock WITH (UPDLOCK, HOLDLOCK)
                    SET CantidadActual = CantidadActual + @Cantidad
                    WHERE IdProducto = @IdProducto;

                    IF @@ROWCOUNT = 0
                    BEGIN
                        INSERT INTO dbo.Stock
                            (IdProducto, CantidadActual,
                             StockMinimo, StockIdeal)
                        VALUES
                            (@IdProducto, @Cantidad, 0, 0);
                    END;", conexion, transaccion))
                {
                    cmdStock.Parameters.Add("@IdProducto", SqlDbType.Int).Value = grupo.Key;
                    cmdStock.Parameters.Add("@Cantidad", SqlDbType.Decimal).Value = cantidad;
                    cmdStock.Parameters["@Cantidad"].Precision = 10;
                    cmdStock.Parameters["@Cantidad"].Scale = 3;

                    cmdStock.ExecuteNonQuery();
                }

                using (SqlCommand cmdMovimiento = new SqlCommand(@"
                    INSERT INTO dbo.MovimientoStock
                        (IdProducto, TipoMovimiento, Cantidad,
                         FechaHora, Observacion, IdUsuario)
                    VALUES
                        (@IdProducto, N'INGRESO', @Cantidad,
                         SYSDATETIME(), @Observacion, @IdUsuario);",
                    conexion, transaccion))
                {
                    cmdMovimiento.Parameters.Add("@IdProducto", SqlDbType.Int).Value = grupo.Key;
                    cmdMovimiento.Parameters.Add("@Cantidad", SqlDbType.Decimal).Value = cantidad;
                    cmdMovimiento.Parameters["@Cantidad"].Precision = 10;
                    cmdMovimiento.Parameters["@Cantidad"].Scale = 3;
                    cmdMovimiento.Parameters.Add("@Observacion", SqlDbType.NVarChar, 255).Value = "Compra #" + idCompra;
                    cmdMovimiento.Parameters.Add("@IdUsuario", SqlDbType.Int).Value = idUsuario;
                    cmdMovimiento.ExecuteNonQuery();
                }
            }

            transaccion.Commit();
            return idCompra;
        }
        catch
        {
            try
            {
                transaccion.Rollback();
            }
            catch
            {

            }

            throw;
        }
    }
  }
}
public DataTable ObtenerCompras(DateTime desde, DateTime hasta)
```

```
    {
        using (SqlConnection conexion = cn.ObtenerConexion())
        using (SqlCommand cmd = new SqlCommand("sp_ObtenerCompras", conexion))
        {
            cmd.CommandType = CommandType.StoredProcedure;
            cmd.Parameters.AddWithValue("@Desde", desde.Date);
            cmd.Parameters.AddWithValue("@Hasta", hasta.Date);

            using (SqlDataAdapter adaptador = new SqlDataAdapter(cmd))
            {
                DataTable tabla = new DataTable();
                adaptador.Fill(tabla);
                return tabla;
            }
        }
    }
    public DataTable ObtenerDetalleCompra(int idCompra)
    {
        using (SqlConnection conexion = cn.ObtenerConexion())
        using (SqlCommand cmd = new SqlCommand("sp_ObtenerDetalleCompra", conexion))
        {
            cmd.CommandType = CommandType.StoredProcedure;
            cmd.Parameters.AddWithValue("@IdCompra", idCompra);

            using (SqlDataAdapter adaptador = new SqlDataAdapter(cmd))
            {
                DataTable tabla = new DataTable();
                adaptador.Fill(tabla);
                return tabla;
            }
        }
    }
    public void AnularCompra(int idCompra, string motivo, int idUsuario)
    {
        using (SqlConnection conexion = cn.ObtenerConexion())
        {
            conexion.Open();

            using (SqlCommand cmd = new SqlCommand("sp_AnularCompra", conexion))
            {
                cmd.CommandType = CommandType.StoredProcedure;
                cmd.Parameters.AddWithValue("@IdCompra", idCompra);
                cmd.Parameters.AddWithValue("@Motivo", motivo);
                cmd.Parameters.AddWithValue("@IdUsuario", idUsuario);
                cmd.ExecuteNonQuery();
            }
        }
    }
    public DataTable ObtenerComprasPendientesRecepcion(string buscar, DateTime? fecha)
    {
        using (SqlConnection conexion = cn.ObtenerConexion())
        using (SqlCommand cmd = new SqlCommand("sp_ObtenerComprasPendientesRecepcion", conexion))
        {
            cmd.CommandType = CommandType.StoredProcedure;
            cmd.Parameters.Add("@Buscar", SqlDbType.NVarChar, 100).Value = string.IsNullOrWhiteSpace(buscar) ? (object)DBNull.Value :
buscar.Trim();
            cmd.Parameters.Add("@Fecha", SqlDbType.Date).Value = fecha.HasValue ? (object)fecha.Value.Date : DBNull.Value;

            using (SqlDataAdapter adaptador = new SqlDataAdapter(cmd))
            {
                DataTable tabla = new DataTable();
```

```
                adaptador.Fill(tabla);
                return tabla;
            }
        }
    }
    public int ConfirmarRecepcion(ClsRecepcionCompraEntidad recepcion, int idUsuario)
    {
        using (SqlConnection conexion = cn.ObtenerConexion())
        using (SqlCommand cmd = new SqlCommand("sp_ConfirmarRecepcionCompra", conexion))
        {
            cmd.CommandType = CommandType.StoredProcedure;
            cmd.Parameters.Add("@IdCompra", SqlDbType.Int).Value = recepcion.IdCompra;
            cmd.Parameters.Add("@IdUsuario", SqlDbType.Int).Value = idUsuario;

            SqlParameter parametroTotalBoleta = cmd.Parameters.Add("@TotalBoleta", SqlDbType.Decimal);
            parametroTotalBoleta.Precision = 10;
            parametroTotalBoleta.Scale = 2;
            parametroTotalBoleta.Value = decimal.Round(recepcion.TotalBoleta, 2, MidpointRounding.AwayFromZero);

            cmd.Parameters.Add("@Motivos", SqlDbType.NVarChar, 500).Value = string.IsNullOrWhiteSpace(recepcion.Motivos) ?
(object)DBNull.Value : recepcion.Motivos;
            cmd.Parameters.Add("@Observaciones", SqlDbType.NVarChar, 500).Value =
string.IsNullOrWhiteSpace(recepcion.Observaciones) ? (object)DBNull.Value : recepcion.Observaciones;

            DataTable tablaDetalles = new DataTable();
            tablaDetalles.Columns.Add("IdProducto", typeof(int));
            tablaDetalles.Columns.Add("CantidadRecibida", typeof(decimal));

            foreach (ClsDetalleRecepcionEntidad detalle in recepcion.Detalles)
            {
                tablaDetalles.Rows.Add(detalle.IdProducto, decimal.Round(detalle.CantidadRecibida, 3, MidpointRounding.AwayFromZero));
            }

            SqlParameter parametroDetalles = cmd.Parameters.AddWithValue("@Detalles", tablaDetalles);
            parametroDetalles.SqlDbType = SqlDbType.Structured;
            parametroDetalles.TypeName = "dbo.TipoDetalleRecepcion";

            conexion.Open();

            return Convert.ToInt32(cmd.ExecuteScalar());
        }
    }
  }
}
```

5.0 Roles y Permisos

Gestorix funciona con un sistema de roles personalizables por parte del cliente, permitiendo crear, modificar y eliminar roles en base a los siguientes permisos:

ID_Permiso	CodigoPermiso	Permiso	SubPermiso	Descripción
100	1.0.0	Artículos	Total	Permite usar todas las funciones de la sección Artículos.
110	1.1.0		Gestión de Artículos	Permite gestionar el Alta, Baja, Modificación y Carga de Artículos.

111	1.1.1		Alta de Artículos	Permite gestionar el Alta de nuevos Artículos.
112	1.1.2		Baja/Modificación de Artículos	Permite gestionar la Baja y Modificación de los Artículos.
113	1.1.3		Carga de Artículos	Permite Cargar Artículos Existentes al Stock.
120	1.2.0		Gestión de Rubros	Permite administrar la Alta, Baja y Modificación de los Rubros/Categorías de los Productos.
121	1.2.1		Alta de Rubros	Permite administrar la Alta de los Rubros/Categorías de los Productos.
122	1.2.2		Baja/Modificación de Rubros	Permite administrar la Baja y Modificación de los Rubros/Categorías de los Productos.
130	1.3.0		Gestión de Marcas	Permite administrar la Alta, Baja y Modificación de las Marcas de los Productos.
131	1.3.1		Alta de Marcas	Permite administrar la Alta de las Marcas de los Productos.
132	1.3.2		Baja/Modificación de Marcas	Permite administrar la Baja y Modificación de las Marcas de los Productos.
140	1.4.0		Importar Articulos desde Excel	Permite importar artículos desde archivos Excel externos al sistema.
150	1.5.0		Exportar Artículos a Excel	Permite exportar información de artículos a archivos Excel.
200	2.0.0	Ventas	Total	Permite usar todas las funciones de la sección Ventas.
210	2.1.0		Realizar Ventas	Permite realizar ventas.
220	2.2.0		Devoluciones	Permite realizar Devoluciones y ver las realizadas previamente.
221	2.2.1		Realizar Devoluciones	Permite realizar Devoluciones.
222	2.2.2		Historial Devoluciones	Permite acceder al Historial de Devoluciones.
230	2.3.0		Crear Promociones	Permite crear nuevas Promociones.
240	2.4.0		Baja Promociones	Permite dar de Baja Promociones.
250	2.5.0		Anular Ventas	Permite anular ventas realizadas.

300	3.0.0	Compras	Total	Permite usar todas las funciones de la sección Compras.
310	3.1.0		Nueva Compra	Permite acceder al Administrador de Compras y registrar nuevas compras.
320	3.2.0		Recepción de Mercadería	Permite recibir la mercadería de las compras pendientes.
330	3.3.0		Anular Compras	Permite anular compras registradas.
400	4.0.0	Proveedores	Total	Permite usar todas las funciones de la sección Proveedores.
410	4.1.0		Gestión de Proveedores	Permite administrar la Alta, Baja y Modificación de los Proveedores.
411	4.1.1		Alta de Proveedores	Permite administrar la Alta de Proveedores.
412	4.1.2		Baja/ Modificación de Proveedores	Permite administrar la Baja y Modificación de Proveedores.
420	4.2.0		Cuenta Corriente de Proveedores	Permite generar un próximo pago a Proveedores.
500	5.0.0	Listados	Total	Permite usar todas las funciones de la sección Listados.
510	5.1.0		Listado de Precios	Permite Visualizar la Lista de Precios.
520	5.2.0		Listado de Reposición	Permite Visualizar el Listado de Reposición.
600	6.0.0	Informes	Total	Permite usar todas las funciones de la sección Informes.
610	6.1.0		Ventas por Fecha	Permite Realizar, Visualizar y Exportar Informes de Ventas por Fecha.
620	6.2.0		Ventas con Detalle de Fecha	Permite Realizar, Visualizar y Exportar Informes de Ventas por Detalle de Fecha.
630	6.3.0		Venta por Rubro	Permite Realizar, Visualizar y Exportar Informes de Ventas por Rubro.
640	6.4.0		Venta por Artículos	Permite Realizar, Visualizar y Exportar Informes de Ventas por Artículos.
650	6.5.0		Venta por Medio de Pago	Permite Realizar, Visualizar y Exportar Informes de Ventas por Medio de Pago.
660	6.6.0		Balance	Permite consultar y exportar el Balance.

700	7.0.0	Caja	Total	Permite usar todas las funciones de la sección Caja.
710	7.1.0		Nuevo Ingreso	Permite realizar un Nuevo Ingreso Manualmente.
720	7.2.0		Nuevo Egreso	Permite realizar un Nuevo Egreso Manualmente.
730	7.3.0		Abrir Caja	Permite Abrir la Caja.
740	7.4.0		Cerrar Caja	Permite Cerrar la Caja.
750	7.5.0		Estado e Historial de Caja	Permite ver el estado de la caja y el historial de cajas.
760	7.6.0		Anular Movimiento de Caja	Permite anular ingresos y egresos de la caja.
800	8.0.0	Ajustes	Total	Permite Acceder a los ajustes del Sistema.
900	9.0.0	Ayuda	Total	Permite Acceder a la sección de Ayuda.
1000	10.0.0	Empleado	Total	Permite usar todas las funciones de la sección Empleado.
1010	10.1.0		Alta Empleado	Permite realizar una alta en el sistema de un nuevo empleado.
1020	10.2.0		Modificación/Baja Empleado	Permite modificar los datos o dar de baja al empleado.
1030	10.3.0		Modificación de Datos Propios	Permite modificar datos propios por parte del empleado.
1100	11.0.0	Alarmas	Total	Permite usar todas las funciones de la sección Alarmas.
1110	11.1.0		Visualizar Alarma	Permite Visualizar las Alarmas.
1120	11.2.0		Dar de Baja Alarma	Permite Dar de Baja una Alarma.
1200	12.0.0	Datos Sensibles	Total	Permite usar todas las funciones de la sección Datos Sensibles.
1210	12.1.0		Datos Sensibles Articulo	Permite modificar datos sensibles de los Artículos.
1220	12.2.0		Datos Sensibles Rubro	Permite modificar datos sensibles de los Rubros/Categorías.

1230	12.3.0		Datos Sensibles Marca	Permite modificar datos sensibles de la Marcas.
1240	12.4.0		Datos Sensibles Proveedor	Permite modificar datos sensibles de los Proveedores.
1250	12.5.0		Datos Sensibles Empleados	Permite modificar datos sensibles de los Empleados.
1260	12.6.0		Datos Sensibles Promociones	Permite modificar datos sensibles de las Promociones.
1270	12.7.0		Roles y Permisos	Permite gestionar los Roles y Permisos.
1280	12.8.0		Gestión de Backup	Permite generar y restaurar copias de seguridad de la base de datos.
1290	12.9.0		Bitácora	Permite consultar la bitácora del sistema.

Funcionando con una lógica en cascada, si se le otorga un permiso X.0.0, se le otorgara por defecto sus concatenados X.1.0, X.2.0, etc. lo mismo pasaría con un X.1.0 que otorga el X.1.1, X.1.2.
Si bien le damos la opción al cliente de otorgar permisos sobre datos que normalmente solo deberían tener acceso los dueños/administradores de la empresa, también nos aseguramos por medio advertencias tanto en la guía como en el programa al intentar activarlos que no es recomendable por la seguridad y guardado de datos.

Roles Base

Para este proyecto, Gestorix entrega 4 roles principales ya creados, Administrador, Empleado General, Empleado Ventas y Empleado Compras.

Administrador

Tiene acceso a todos los permisos, solo debería ser utilizado por el Dueño del Maxikiosco MTK.

1.0.0 - Artículos (Total)
2.0.0 - Ventas (Total)
3.0.0 - Compras (Total)
4.0.0 - Proveedores (Total)
5.0.0 - Listados (Total)
6.0.0 - Informes (Total)
7.0.0 - Caja (Total)
8.0.0 - Ajustes (Total)
9.0.0 - Ayuda (Total)
10.0.0 - Empleado (Total)

11.0.0 - Alarmas (Total)
12.0.0 - Datos Sensibles (Total)

Empleado General

Rol pensado para el Empleado normal del Maxikiosco MTK, donde los trabajadores son pocos y no hay una distribución clásica de tareas, todos tienen que poder encargarse de todo si lo requiere en el ámbito de las tareas normales de Compra y Venta.

1.1.0 - Gestión de Artículos

1.2.1 - Alta de Rubros

1.3.0 - Gestión de Marcas

1.4.0 - Importar Articulos desde Excel

2.1.0 - Realizar Ventas

2.2.0 - Devoluciones

2.3.0 - Crear Promociones

2.4.0 - Baja Promociones

3.1.0 - Nueva Compra

3.2.0 - Recepción de Mercadería

4.2.0 - Cuenta Corriente de Proveedores

5.1.0 - Listado de Precios

5.2.0 - Listado de Reposición

7.1.0 - Nuevo Ingreso

7.2.0 - Nuevo Egreso

7.3.0 - Abrir Caja

7.4.0 - Cerrar Caja

7.5.0 - Estado e Historial de Caja

9.0.0 - Ayuda (Total)

10.3.0 - Modificación de Datos Propios

11.0.0 - Alarmas (Total)

Empleado Compras

Rol planeado para el clásico empleado del ámbito de Compras.

1.1.0 - Gestión de Artículos
1.2.1 - Alta de Rubros

1.3.0 - Gestión de Marcas
1.4.0 - Importar Articulos desde Excel
3.1.0 - Nueva Compra
3.2.0 - Recepción de Mercadería
4.2.0 - Cuenta Corriente de Proveedores
5.2.0 - Listado de Reposición
7.1.0 - Nuevo Ingreso
7.2.0 - Nuevo Egreso
9.0.0 - Ayuda (Total)
10.3.0 - Modificación de Datos Propios
11.0.0 - Alarmas (Total)

Empleado Ventas

Rol planeado para el clásico empleado del ámbito de Ventas.

2.1.0 - Realizar Ventas
2.2.0 - Devoluciones
5.1.0 - Listado de Precios
7.1.0 - Nuevo Ingreso
7.2.0 - Nuevo Egreso
7.3.0 - Abrir Caja
7.4.0 - Cerrar Caja
7.5.0 - Estado e Historial de Caja
9.0.0 - Ayuda (Total)
10.3.0 - Modificación de Datos Propios
11.1.0 - Visualizar Alarma